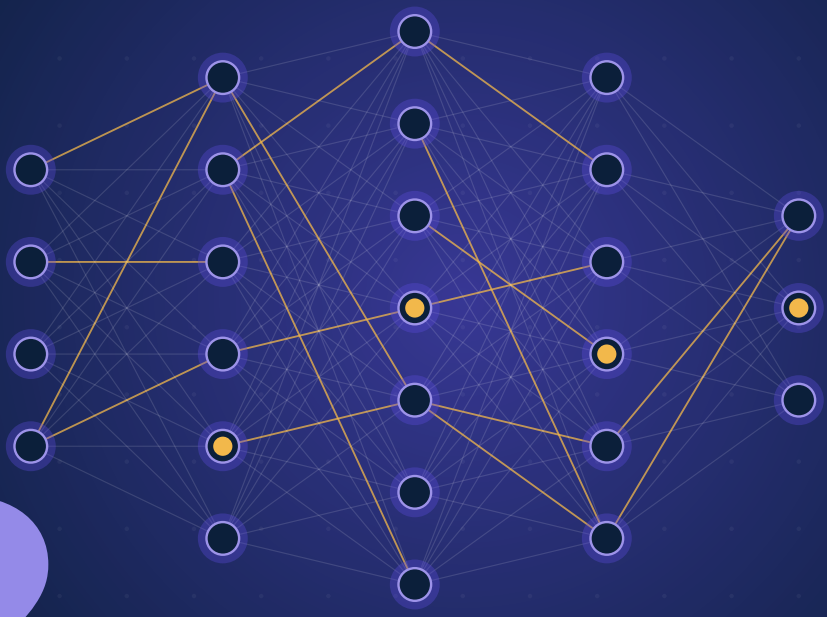
MACHINE LEARNING AND DEEP LEARNING

*Foundations, Methods and Practice*



CHAPTER

# 02

# Deep Learning

61 sections · 74 figures · 61 tables · 29 algorithms · 144 numbered equations · 59 code listings

**LUXSHI KARUNAKARAN**

B.Sc. (Hons) Computer Science and Technology · M.Sc. Data Science and AI (Reading)
Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka

*Foundations, Methods and Practice*

CHAPTER 2

# Deep Learning

61 sections · 74 figures · 61 tables · 29 algorithms · 144 numbered equations · 59 code listings

**Luxshi Karunakaran**

B.Sc. (Hons) in Computer Science and Technology
M.Sc. in Data Science and Artificial Intelligence (Reading)

Department of Social Statistics
University of Sri Jayewardenepura, Sri Lanka

**Machine Learning and Deep Learning: Foundations, Methods and Practice**
Chapter 2 – Deep Learning

**Author.** Luxshi Karunakaran, B.Sc. (Hons) in Computer Science and Technology; M.Sc. in Data Science and Artificial Intelligence (Reading). Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka.

**Copyright.** © 2026 Luxshi Karunakaran. All rights reserved.

**Suggested citation.** Karunakaran, L. (2026). Deep learning. In *Machine Learning and Deep Learning: Foundations, Methods and Practice* (Chapter 2). Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka.

**Edition.** Version 1.0, October 2026. Distributed as a book-chapter preprint via ResearchGate.

**Conventions.** Sections, figures, tables, equations, algorithms, code listings and highlight panels are numbered in the form *chapter.sequence*; see *How to Read This Chapter* for details. Code listings are written in Python 3.

**Typesetting.** Typeset with XeLaTeX in TeX Gyre Pagella, Inter and DejaVu Sans Mono.

# Contents


**How to Read This Chapter** xix

**Chapter Overview** 1

**2.1 Introduction to Deep Learning** 3

2.1.1 Learning Representations 3

2.1.2 Why Deep Learning Works Now 5

2.1.3 Strengths and Limitations 6

**2.2 History and Evolution of Deep Learning** 7

2.2.1 The Cybernetic Era (1943–1969) 7

2.2.2 Connectionism and Backpropagation (1970–1995) 8

2.2.3 The Deep Learning Revival (2006–2016) 8

2.2.4 The Transformer and Foundation-Model Era (2017–present) 9

**2.3 Relationship Between Artificial Intelligence, Machine Learning, and Deep Learning** 10

2.3.1 Three Nested Fields 11

2.3.2 Comparing the Three Levels 13

**2.4 Fundamentals of Neural Networks** 14

2.4.1 The Artificial Neuron 14

2.4.2 Layers and Networks 14

2.4.3 Why Non-linearity and Depth Matter 16

2.4.4 A First Implementation 16

**2.5 Biological Neurons and Artificial Neurons** 17

2.5.1 The Biological Neuron 17

2.5.2 From Biology to Mathematics 18

2.5.3 Where the Analogy Breaks Down 18

**2.6 Perceptrons and Multilayer Perceptrons (MLPs)** 19

2.6.1 The Perceptron Model 20

2.6.2 The Perceptron Learning Rule 20

2.6.3 The XOR Problem 21

2.6.4 Multilayer Perceptrons 22

**2.7 Neural Network Architecture** 24

2.7.1 Depth, Width and Connectivity 24

2.7.2 Designing the Output Layer 25

2.7.3 Choosing an Architecture in Practice 25

**2.8 Activation Functions** 26

2.8.1 Sigmoid and Hyperbolic Tangent . . . 26
2.8.2 Rectified Linear Units and Variants . . . 27
2.8.3 Smooth Modern Activations: GELU and Swish . . . 28
2.8.4 Softmax for Multi-class Outputs . . . 28

**2.9 Forward Propagation 30**
2.9.1 Layer-by-Layer Computation . . . 30
2.9.2 Vectorised Mini-Batch Form . . . 30
2.9.3 Worked Numerical Example . . . 31

**2.10 Loss Functions and Cost Functions 32**
2.10.1 Losses for Regression . . . 33
2.10.2 Losses for Classification . . . 33
2.10.3 Divergence-Based Losses . . . 35

**2.11 Backpropagation 36**
2.11.1 Computational Graphs . . . 36
2.11.2 Derivation for Fully Connected Networks . . . 37
2.11.3 Worked Example: Two-Layer Network . . . 38
2.11.4 Gradient Checking and Automatic Differentiation . . . 39

**2.12 Gradient Descent and Optimization in Deep Learning 40**
2.12.1 The Gradient Descent Update . . . 40
2.12.2 The Geometry of Loss Landscapes . . . 41
2.12.3 Practical Optimisation Challenges . . . 42

**2.13 Stochastic Gradient Descent, Mini-Batch Gradient Descent, and Batch Gradient Descent 43**
2.13.1 Three Ways to Estimate the Gradient . . . 44
2.13.2 Comparing the Variants . . . 45
2.13.3 Choosing the Batch Size . . . 45

**2.14 Optimizers: SGD, Momentum, RMSprop, and Adam 47**
2.14.1 SGD with Momentum and Nesterov Acceleration . . . 47
2.14.2 Adaptive Learning Rates: AdaGrad and RMSprop . . . 47
2.14.3 Adam and AdamW . . . 48
2.14.4 Choosing an Optimiser . . . 51

**2.15 Weight Initialization Techniques 52**
2.15.1 Why Not Zeros? Symmetry Breaking . . . 52
2.15.2 Variance-Preserving Initialisation . . . 53
2.15.3 Practice and Beyond . . . 54

**2.16 Learning Rate Scheduling 55**
2.16.1 Decay Schedules . . . 55

2.16.2 Warm-up and the One-Cycle Policy 56
2.16.3 Schedules in Practice 57
**2.17 Vanishing and Exploding Gradients 58**
2.17.1 Mathematical Origin 58
2.17.2 Symptoms and Diagnosis 59
2.17.3 Remedies 59
**2.18 Regularization in Deep Learning 61**
2.18.1 Parameter Norm Penalties and Weight Decay 61
2.18.2 Early Stopping 62
2.18.3 Data Augmentation 63
2.18.4 Label Smoothing and Other Techniques 64
**2.19 Dropout and Batch Normalization 65**
2.19.1 Dropout 66
2.19.2 Batch Normalisation 67
2.19.3 Layer Normalisation and Other Variants 68
**2.20 Training, Validation, and Testing Deep Neural Networks 70**
2.20.1 Data Splits 70
2.20.2 The Training Loop 71
2.20.3 Reading Learning Curves 72
2.20.4 Debugging and Reproducibility 73
**2.21 Hyperparameter Tuning in Deep Learning 74**
2.21.1 What to Tune First 75
2.21.2 Search Strategies 75
2.21.3 Tools and Best Practices 77
**2.22 Deep Feedforward Neural Networks 79**
2.22.1 Layered Composition of Functions 79
2.22.2 Why Depth Helps 80
2.22.3 Design Choices 81
2.22.4 Strengths and Limitations 82
**2.23 Convolutional Neural Networks (CNNs) 83**
2.23.1 The Convolution Operation 83
2.23.2 Padding, Stride and Output Size 84
2.23.3 Worked Example of a 2D Convolution 85
2.23.4 Pooling 86
2.23.5 Receptive Field 86
2.23.6 The CNN Pipeline 87
**2.24 CNN Architectures: LeNet, AlexNet, VGG, Inception, and ResNet 88**

2.24.1 LeNet-5 (1998) 89
2.24.2 AlexNet (2012) 89
2.24.3 VGG (2014) 90
2.24.4 Inception / GoogLeNet (2014) 90
2.24.5 ResNet and Residual Learning (2015) 90
2.24.6 Beyond ResNet 92

**2.25 Image Classification and Object Recognition 93**
2.25.1 Problem Formulation and Pipeline 93
2.25.2 Training Recipes that Matter 94
2.25.3 Evaluation 95
2.25.4 Interpreting Classifiers 95

**2.26 Image Segmentation and Object Detection 96**
2.26.1 Intersection over Union 96
2.26.2 Non-Maximum Suppression 97
2.26.3 Evaluating Detectors: Mean Average Precision 98
2.26.4 Two-Stage Detectors: the R-CNN Family 98
2.26.5 One-Stage Detectors: YOLO and SSD 99
2.26.6 Semantic Segmentation: FCN and U-Net 100
2.26.7 Instance Segmentation: Mask R-CNN 100

**2.27 Recurrent Neural Networks (RNNs) 102**
2.27.1 The Recurrence 102
2.27.2 Input–Output Configurations 103
2.27.3 Backpropagation Through Time 103
2.27.4 Vanishing and Exploding Gradients 104

**2.28 Long Short-Term Memory Networks (LSTMs) 106**
2.28.1 The Cell State and Gating Idea 106
2.28.2 Gate Equations 106
2.28.3 Why the LSTM Preserves Gradients 107
2.28.4 Parameters, Variants and Practice 108

**2.29 Gated Recurrent Units (GRUs) 109**
2.29.1 GRU Equations 109
2.29.2 LSTM versus GRU 110

**2.30 Sequence Modeling and Time-Series Forecasting 111**
2.30.1 Autoregressive Factorisation 111
2.30.2 Sequence-to-Sequence Models 112
2.30.3 Time-Series Forecasting with Deep Learning 112

**2.31 Natural Language Processing (NLP) with Deep Learning 115**
2.31.1 From Text to Tensors: Tokenisation 116

2.31.2 The Deep NLP Pipeline . . . 117
2.31.3 Core NLP Tasks . . . 117

**2.32 Word Embeddings and Representation Learning 119**
2.32.1 One-Hot versus Dense Representations . . . 119
2.32.2 Word2Vec: CBOW and Skip-Gram . . . 119
2.32.3 GloVe and fastText . . . 121
2.32.4 Properties, Analogies and Limitations . . . 122

**2.33 Attention Mechanisms 123**
2.33.1 Bahdanau (Additive) Attention . . . 123
2.33.2 Luong (Multiplicative) Attention . . . 125
2.33.3 Queries, Keys and Values; Scaled Dot-Product Attention . . . 125

**2.34 Transformer Architecture 127**
2.34.1 Overall Architecture . . . 127
2.34.2 Multi-Head Attention . . . 128
2.34.3 Position-Wise Feedforward Network . . . 129
2.34.4 Positional Encoding . . . 129
2.34.5 Residual Connections and Layer Normalisation . . . 130
2.34.6 Computational Complexity . . . 131

**2.35 BERT, GPT, and Other Transformer Models 132**
2.35.1 BERT and Masked Language Modelling . . . 132
2.35.2 Improving BERT: RoBERTa and Friends . . . 133
2.35.3 GPT and Causal Language Modelling . . . 134
2.35.4 Encoder–Decoder Models: T5 and BART . . . 134
2.35.5 Beyond Text: Vision Transformers . . . 135

**2.36 Large Language Models (LLMs) 136**
2.36.1 Pretraining and Scaling Laws . . . 137
2.36.2 In-Context Learning and Prompting . . . 137
2.36.3 Instruction Tuning and RLHF . . . 138
2.36.4 Decoding Strategies . . . 140
2.36.5 Hallucination and Other Limitations . . . 141

**2.37 Generative Adversarial Networks (GANs) 143**
2.37.1 The Adversarial Game . . . 143
2.37.2 Theory: the Optimal Discriminator . . . 144
2.37.3 Training Algorithm . . . 144
2.37.4 Training Difficulties: Mode Collapse and Instability . . . 145
2.37.5 Notable GAN Architectures . . . 146

**2.38 Variational Autoencoders (VAEs) 148**
2.38.1 Latent-Variable Generative Model . . . 148

2.38.2 Deriving the Evidence Lower Bound . . . 148
2.38.3 The Reparameterisation Trick . . . 149
2.38.4 Latent Space, Variants and Limitations . . . 151

**2.39 Autoencoders and Representation Learning 152**
2.39.1 Undercomplete Autoencoders . . . 152
2.39.2 Regularised Autoencoders . . . 153
2.39.3 Applications: Anomaly Detection and Beyond . . . 155
2.39.4 From Autoencoders to Self-Supervised Learning . . . 156

**2.40 Diffusion Models and Generative AI 156**
2.40.1 The Forward (Noising) Process . . . 157
2.40.2 The Reverse (Denoising) Process . . . 157
2.40.3 Training Objective and Algorithms . . . 158
2.40.4 Latent Diffusion . . . 160
2.40.5 Conditioning and Classifier-Free Guidance . . . 160

**2.41 Deep Reinforcement Learning 162**
2.41.1 Policy-Based Learning . . . 162
2.41.2 The Policy Gradient Theorem and REINFORCE . . . 162
2.41.3 Actor–Critic Methods . . . 163
2.41.4 Proximal Policy Optimisation . . . 164
2.41.5 Landmarks, Challenges and Outlook . . . 166

**2.42 Graph Neural Networks (GNNs) 168**
2.42.1 Message Passing: The Unifying Framework . . . 168
2.42.2 Graph Convolutional Networks (GCN) . . . 169
2.42.3 Graph Attention Networks (GAT) and GraphSAGE . . . 170
2.42.4 Tasks, Readout and Practical Issues . . . 171

**2.43 Self-Supervised and Contrastive Learning 172**
2.43.1 Contrastive Learning and the InfoNCE Loss . . . 172
2.43.2 SimCLR . . . 173
2.43.3 MoCo, BYOL and Masked Autoencoders . . . 174

**2.44 Transfer Learning and Fine-Tuning 176**
2.44.1 Why Pre-Trained Features Transfer . . . 176
2.44.2 Feature Extraction versus Fine-Tuning . . . 177
2.44.3 Parameter-Efficient Fine-Tuning: Adapters and LoRA . . . 178

**2.45 Foundation Models and Pretrained Models 179**
2.45.1 From Task-Specific Models to Pre-Train-then-Adapt . . . 180
2.45.2 Families of Pretrained Models . . . 180
2.45.3 Adaptation, Emergence and Homogenisation . . . 182

**2.46 Multimodal Deep Learning** 183
2.46.1 Core Challenges and Fusion Strategies 183
2.46.2 CLIP: Contrastive Language–Image Pre-training 184
2.46.3 Visual Question Answering and Image Captioning 185
**2.47 Speech Recognition and Audio Processing** 186
2.47.1 From Waveform to Features: Spectrograms and MFCCs 187
2.47.2 Connectionist Temporal Classification (CTC) 188
2.47.3 Self-Supervised Speech: wav2vec 2.0 and Whisper 189
**2.48 Deep Learning for Medical Imaging and Healthcare** 190
2.48.1 Imaging Modalities and Their Characteristics 191
2.48.2 Segmentation and the Dice Loss 191
2.48.3 Beyond Imaging: Clinical Data and Healthcare AI 193
2.48.4 Clinical, Regulatory and Ethical Issues 193
**2.49 Deep Learning for Computer Vision** 194
2.49.1 A Taxonomy of Vision Tasks 194
2.49.2 Human Pose Estimation 195
2.49.3 Object Tracking 196
2.49.4 Vision Transformers: A Recap and Outlook 197
**2.50 Deep Learning for Recommendation Systems** 198
2.50.1 Matrix Factorisation as a Shallow Embedding Model 198
2.50.2 Deep Recommendation Architectures 198
2.50.3 Evaluating Recommenders: HR@k and NDCG@k 201
**2.51 Explainable AI for Deep Learning** 202
2.51.1 A Taxonomy of Explanation Methods 202
2.51.2 Gradient-Based Saliency and Grad-CAM 202
2.51.3 Integrated Gradients 204
2.51.4 Attention Visualisation and SHAP for Deep Networks 205
**2.52 Adversarial Attacks and Model Robustness** 206
2.52.1 Threat Models 206
2.52.2 FGSM and PGD 207
2.52.3 Defences: Adversarial Training and Certified Robustness 208
**2.53 Ethical Issues, Bias, Privacy, and Security** 210
2.53.1 Sources of Bias 210
2.53.2 Formal Fairness Criteria 210
2.53.3 Privacy Risks and Privacy-Preserving Learning 211
2.53.4 Security: Model Stealing, Poisoning and Backdoors 212
2.53.5 Governance and Responsible Practice 213

**2.54 Model Compression, Quantization, and Pruning** 214
2.54.1 Knowledge Distillation . . . 214
2.54.2 Quantisation . . . 215
2.54.3 Pruning . . . 216
**2.55 Distributed Training and Parallel Computing** 217
2.55.1 Data Parallelism and All-Reduce . . . 218
2.55.2 Model Parallelism: Pipeline and Tensor . . . 219
2.55.3 ZeRO and Fully Sharded Data Parallelism . . . 220
2.55.4 Mixed-Precision Training . . . 221
**2.56 GPU and TPU Computing for Deep Learning** 222
2.56.1 CPUs, GPUs and TPUs . . . 222
2.56.2 CUDA and the GPU Programming Model . . . 223
2.56.3 Memory Hierarchy and the Roofline Model . . . 224
**2.57 Deep Learning Frameworks: TensorFlow, Keras, and PyTorch** 226
2.57.1 Static and Dynamic Computation Graphs . . . 226
2.57.2 The Same Model in Keras and PyTorch . . . 227
2.57.3 Choosing a Framework . . . 228
**2.58 Deploying Deep Learning Models** 229
2.58.1 Exporting Models: TorchScript, SavedModel and ONNX . . . 229
2.58.2 Serving Patterns: Cloud and Edge . . . 230
2.58.3 Serving a Model with FastAPI . . . 230
**2.59 MLOps, Model Monitoring, and Model Maintenance** 232
2.59.1 The ML Lifecycle . . . 232
2.59.2 Data Drift, Concept Drift and Their Detection . . . 233
2.59.3 CI/CD/CT Pipelines and Tooling . . . 235
2.59.4 MLOps Maturity Levels . . . 235
**2.60 Practical Deep Learning Projects Using Python and Google Colab** 236
2.60.1 Step 1: Environment and GPU Setup . . . 237
2.60.2 Step 2: Data Loading and Augmentation . . . 238
2.60.3 Step 3: Defining the Model . . . 238
2.60.4 Step 4: Training with Good Practices . . . 239
2.60.5 Step 5: Evaluation and Error Analysis . . . 241
2.60.6 Step 6: Saving, Exporting and Reusing the Model . . . 242
**2.61 Challenges, Limitations, and Future Directions of Deep Learning** 243
2.61.1 Data, Compute and Efficiency . . . 243
2.61.2 Robustness, Generalisation and Reasoning . . . 243
2.61.3 Interpretability, Trust and Alignment . . . 244
2.61.4 Emerging Directions . . . 244

Chapter Summary 245
Key Terms 247
Review Questions 250
Exercises 251
Further Reading 252

# List of Figures


2.1 Classical machine learning relies on hand-engineered features fed to a shallow learner, whereas deep learning learns a hierarchy of features and the predictor jointly, end to end . . . 5

2.2 Timeline of major milestones in neural network and deep learning research . . . 10

2.3 Deep learning is a subfield of machine learning, which is itself one approach to artificial intelligence . . . 12

2.4 A fully connected feed-forward network with three inputs, two hidden layers of four neurons each, and two outputs . . . 15

2.5 Left: schematic biological neuron with dendrites, soma, axon and synaptic terminals 18

2.6 Left: no single line separates the XOR classes . . . 22

2.7 Connectivity patterns of four architectural families: fully connected (every unit to every unit), convolutional (local, shared weights), recurrent (feedback through time), and attention-based (input-dependent all-to-all weights) . . . 24

2.8 Common activation functions (left) and their derivatives (right) . . . 28

2.9 Data flow of the forward pass in a two-layer network: each layer applies an affine map followed by an activation, and the output enters the loss together with the target 31

2.10 Left: regression losses as a function of the residual . . . 34

2.11 Computational graph of a single sigmoid neuron with squared-error loss, for $w = (0.5, -1)$, $x = (2, 1)$, $b = 0.5$, $y = 1$ . . . 37

2.12 Gradient descent on $J(\theta) = \theta^2$ (curvature 2, so the stability limit is $\eta < 1$) from $\theta_0 = 2.5$ with three learning rates: too small converges slowly, a good value converges quickly, and too large diverges . . . 41

2.13 Contour plots of three two-dimensional loss surfaces: a convex, ill-conditioned bowl (single global minimum), a saddle point (minimum along one axis, maximum along the other), and a non-convex surface with several local minima . . . 42

2.14 Parameter trajectories of batch, mini-batch ($m = 16$) and stochastic ($m = 1$) gradient descent on a least-squares problem, each run for 60 updates from the same start . . . 45

2.15 Trajectories of four optimisers for 40 steps on the ill-conditioned quadratic $J = 0.5(0.1{\theta_1}^2 + 2{\theta_2}^2)$ from $\theta_0 = (-4.5, 1.8)$ . . . 50

2.16 Standard deviation of activations across ten 512-unit layers for different initialisations: with tanh, a too-small scale vanishes and $N(0,1)$ saturates while Xavier stays stable . . . 53

2.17 Common learning-rate schedules over 100 epochs with maximum rate 0.1: step decay, exponential decay, cosine annealing, linear warm-up followed by cosine decay, and the one-cycle policy . . . 56

2.18 Norm of the backpropagated error signal $\delta^{(1)}$ at each layer of a 20-layer, 256-unit MLP after one backward pass: with sigmoid and Xavier initialisation the gradient vanishes towards the input . . . 59
2.19 Typical training and validation loss curves: the validation loss reaches its minimum and then rises as the model overfits . . . 63
2.20 Dropout: the full network (left) and two random thinned networks sampled on different training iterations with $p = 0.5$ (centre, right) . . . 66
2.21 Normalisation methods differ in the set of values (shaded) over which the mean and variance are computed . . . 69
2.22 Diagnostic learning-curve patterns: underfitting (both losses high), overfitting (validation loss diverges from training loss), and a good fit (both low with a small gap) . . . 73
2.23 Grid search versus random search with nine trials when one hyperparameter is important and one is not . . . 76
2.24 A deep feedforward network with four inputs, two hidden layers of five units and three softmax outputs . . . 80
2.25 A $3 \times 3$ kernel slides over a $5 \times 5$ input (stride 1, no padding) . . . 84
2.26 The canonical CNN pipeline: stacked convolution–ReLU–pooling stages reduce spatial size while increasing channel depth, followed by global pooling and a dense classifier . . . 87
2.27 The Inception module (GoogLeNet) . . . 91
2.28 A residual block. The input $x$ bypasses two weight layers through an identity shortcut and is added to their output, so the layers need only learn the residual $F(x) = H(x) - x$ . . . 92
2.29 End-to-end image classification workflow, from labelled images through augmentation, a pretrained backbone and fine-tuning to evaluation and deployment . . . 94
2.30 Left: IoU is the area of overlap divided by the area of union of the predicted and ground-truth boxes . . . 98
2.31 U-Net encoder–decoder: the contracting path (left) halves resolution and doubles channels . . . 100
2.32 An RNN shown as a compact loop (left) and unrolled through time (right) . . . 103
2.33 Norm of the back-propagated gradient $\|\partial h_t / \partial h_k\|$ versus time gap $t - k$ for a simple RNN whose recurrent matrix has spectral radius 0.9, 1.0 or 1.1 (log scale): gradients vanish or explode exponentially . . . 105
2.34 The LSTM cell. The cell state $c_t$ (top line) is updated additively: the forget gate $f_t$ scales the old memory, the input gate $i_t$ admits the candidate $\tilde{c}_t$, and the output gate $o_t$ filters $\tanh(c_t)$ to form the hidden state $h_t$ . . . 107
2.35 The GRU cell. The reset gate $r_t$ modulates the previous state when forming the candidate $\tilde{h}_t$ . . . 110

2.36 Encoder–decoder (seq2seq) model translating a three-word sentence . . . 113
2.37 Forecasting a seasonal series: the model sees a lookback window of history (blue) and predicts the next 24 steps (orange) with an 80% prediction interval from quantile outputs . . . 114
2.38 The deep learning NLP pipeline: text is tokenised into subword ids, embedded, contextualised by an encoder and mapped to task outputs by a small head . . . 117
2.39 Two-dimensional projection of word embeddings: the gender offset (man→woman) and the capital-city offset (country→capital) are approximately constant vectors, enabling analogy arithmetic . . . 122
2.40 Attention weights $\alpha_{ti}$ of an English→French translation model (illustrative) . . . 124
2.41 The transformer encoder (left) and decoder (right) blocks . . . 128
2.42 Sinusoidal positional encodings (d_model = 128) . . . 130
2.43 Attention masks of the three transformer families (shaded = allowed) . . . 133
2.44 Illustrative scaling law: test loss of language models decreases as a power law in training compute (straight lines on log–log axes) . . . 138
2.45 The three-stage LLM training pipeline: self-supervised pretraining, supervised instruction tuning, and preference optimisation via a reward model and reinforcement learning (RLHF) . . . 139
2.46 Effect of decoding settings on a next-token distribution . . . 141
2.47 GAN architecture. The generator turns noise z into fake samples . . . 144
2.48 Variational autoencoder. The encoder outputs the mean and log-variance of $q(z \mid x)$ . . . 148
2.49 Left: two-dimensional VAE latent space of a three-class dataset . . . 151
2.50 An undercomplete autoencoder: the encoder compresses a 784-dimensional input to a 32-dimensional code, and the decoder reconstructs the input from the code . . . 153
2.51 Forward diffusion on a two-dimensional 'two moons' dataset: as t increases the data are progressively blurred into an isotropic Gaussian . . . 157
2.52 Latent diffusion (Stable Diffusion) at inference: a text encoder conditions a denoising network via cross-attention . . . 160
2.53 Actor–critic architecture. The actor selects actions . . . 164
2.54 The PPO clipped objective for a single time step as a function of the probability ratio r ($\varepsilon = 0.2$) . . . 165
2.55 One round of message passing: the target node v (orange) receives messages from its neighbours (blue), aggregates them and updates its state . . . 169
2.56 The SimCLR framework: two augmented views of one image pass through a shared encoder f and projection head g . . . 174
2.57 LoRA: the frozen weight $W_0$ is augmented by a trainable low-rank path BA . . . 179

2.58 Illustrative scaling-law curves computed from Equation (2.120): loss decreases as a power law in training tokens, and larger models achieve lower asymptotic loss . . . 181
2.59 Fusion strategies: early fusion concatenates raw or low-level features . . . 184
2.60 Feature-extraction pipeline for speech: framing and windowing, STFT, mel filter bank and logarithm produce a log-mel spectrogram . . . 187
2.61 Left: the mel scale compresses high frequencies, mirroring human hearing . . . 188
2.62 Lifecycle of a clinical deep-learning system, from data governance to post-market surveillance . . . 193
2.63 Taxonomy of computer-vision tasks grouped by the granularity of the output, from image-level recognition to dense, temporal and 3-D understanding . . . 195
2.64 Two-tower retrieval model: user and item towers map features to a shared embedding space, the score is a dot product, and item embeddings are pre-computed in an ANN index for fast candidate retrieval . . . 199
2.65 Illustration of Grad-CAM on a synthetic image: the class-weighted combination of last-layer feature maps yields a coarse heatmap that is upsampled and overlaid on the input, highlighting the object region . . . 204
2.66 Illustrative accuracy under PGD attack as the $\ell\infty$ budget $\varepsilon$ grows: a standard model collapses almost immediately, while an adversarially trained model trades some clean accuracy for much greater robustness . . . 209
2.67 Illustrative accuracy versus sparsity for magnitude pruning of an image classifier: iterative pruning with fine-tuning preserves accuracy to much higher sparsity than one-shot pruning without retraining . . . 217
2.68 Four forms of parallelism: data parallelism replicates the model and splits the batch . 219
2.69 GPU memory hierarchy: capacity increases and bandwidth decreases from registers to host memory . . . 225
2.70 A typical cloud serving architecture: requests pass through a gateway and load balancer to replicated model servers, which load versioned models from a registry . . . 231
2.71 The MLOps lifecycle: a continuous loop in which production monitoring feeds back into data collection and retraining . . . 233
2.72 Workflow of the end-to-end Colab project, with the listing that implements each stage 237
2.73 Typical learning curves for the CIFAR-10 project under a one-cycle schedule (illustrative): validation accuracy rises steadily as the learning rate anneals, and the small train–validation gap indicates well-controlled over-fitting . . . 241
2.74 Map of current limitations of deep learning (left) and the research directions that address them (right) . . . 245

# List of Tables


2.1 Chapter roadmap: the twelve themes of Chapter 2 and the topics they cover . . . 4
2.2 Classical machine learning compared with deep learning . . . 6
2.3 Selected milestones in the history of deep learning . . . 11
2.4 Characteristics of artificial intelligence, machine learning and deep learning . . . 13
2.5 Notation for fully connected networks used throughout this chapter . . . 15
2.6 Correspondence between biological and artificial neurons . . . 19
2.7 Major neural network architecture families and their inductive biases . . . 25
2.8 Output layer design for common tasks . . . 25
2.9 Comparison of activation functions . . . 29
2.10 Summary of common loss functions . . . 35
2.11 Types of critical points and their effect on gradient descent . . . 43
2.12 Comparison of batch, stochastic and mini-batch gradient descent . . . 46
2.13 Comparison of gradient-based optimisers . . . 50
2.14 Weight initialisation schemes and when to use them . . . 54
2.15 Learning-rate schedules and their typical uses . . . 57
2.16 Remedies for vanishing and exploding gradients . . . 60
2.17 Summary of regularisation techniques for deep networks . . . 65
2.18 Comparison of normalisation layers . . . 69
2.19 Diagnosing training problems from learning curves . . . 73
2.20 Important hyperparameters, their typical ranges and search scales . . . 75
2.21 Hyperparameter search strategies compared . . . 77
2.22 Key design decisions for a deep feedforward network and common defaults . . . 82
2.23 Building blocks of a convolutional neural network . . . 87
2.24 Landmark CNN architectures . . . 89
2.25 Widely used image classification benchmarks . . . 95
2.26 Summary of major detection and segmentation methods . . . 101
2.27 Input–output configurations of recurrent networks . . . 103
2.28 Comparison of LSTM and GRU cells (input size d, hidden size n) . . . 110
2.29 Deep learning architectures for time-series forecasting . . . 115
2.30 Core NLP tasks, their output structure, typical metrics and benchmark datasets . . . 118
2.31 One-hot versus dense word representations . . . 120
2.32 Attention scoring functions and variants . . . 126

2.33 Per-layer complexity, sequential operations and maximum path length for sequence length n, representation dimension d, kernel size k (after Vaswani et al. 2017) . . . 131
2.34 Representative pretrained transformer models . . . 135
2.35 Stages of building an LLM assistant . . . 139
2.36 Decoding strategies for language models . . . 142
2.37 Influential GAN variants . . . 147
2.38 Types of autoencoder and how each prevents trivial identity mappings . . . 155
2.39 Comparison of deep generative model families . . . 161
2.40 Families of deep reinforcement learning algorithms . . . 167
2.41 Typical tasks solved with graph neural networks . . . 171
2.42 Comparison of representative self-supervised methods for vision . . . 175
2.43 Choosing a transfer strategy by target-data size and domain similarity . . . 176
2.44 Representative pretrained and foundation models by modality . . . 181
2.45 Comparison of multimodal fusion strategies . . . 184
2.46 Evolution of speech-recognition approaches . . . 190
2.47 Medical imaging modalities and typical deep-learning tasks . . . 191
2.48 Computer-vision tasks, their outputs and representative deep models . . . 195
2.49 Comparison of recommendation model families . . . 200
2.50 Common explanation methods for deep networks . . . 203
2.51 Sources of bias in the machine-learning pipeline . . . 210
2.52 Attacks on deep-learning systems and representative defences . . . 213
2.53 Comparison of model-compression techniques . . . 217
2.54 Comparison of parallelism strategies for deep-learning training . . . 220
2.55 Comparison of CPU, GPU and TPU for deep-learning workloads . . . 223
2.56 Comparison of the main deep-learning frameworks . . . 226
2.57 Edge versus cloud deployment . . . 230
2.58 MLOps maturity levels (after Google Cloud's MLOps guidance) . . . 236
2.59 Common Colab pitfalls and remedies . . . 242
2.60 Key challenges of deep learning and promising responses . . . 246
2.61 Key terms introduced in Chapter 2 . . . 248

# List of Algorithms


2.1 Perceptron Learning Algorithm . . . 20
2.2 Forward Propagation for a Fully Connected Network . . . 30
2.3 Backpropagation for a Fully Connected Network (one example) . . . 38
2.4 Mini-Batch Stochastic Gradient Descent . . . 44
2.5 Adam Optimiser . . . 48
2.6 Linear Warm-up with Cosine Decay . . . 56
2.7 Training with Early Stopping . . . 62
2.8 Batch Normalisation Layer (training and inference) . . . 67
2.9 Complete Training Loop with Validation and Checkpointing . . . 71
2.10 Successive Halving for Hyperparameter Search . . . 76
2.11 Greedy Non-Maximum Suppression (per class) . . . 97
2.12 Truncated Backpropagation Through Time . . . 104
2.13 Byte-Pair Encoding (BPE) Vocabulary Learning . . . 116
2.14 Skip-Gram with Negative Sampling (SGNS) . . . 120
2.15 Beam Search Decoding . . . 140
2.16 GAN Training with Alternating Mini-batch SGD (non-saturating loss) . . . 144
2.17 VAE Training with the Reparameterisation Trick . . . 150
2.18 DDPM Training (Ho et al. 2020) . . . 158
2.19 DDPM Sampling (Ancestral) . . . 158
2.20 REINFORCE with Baseline . . . 163
2.21 Proximal Policy Optimisation (PPO-Clip), actor–critic style . . . 165
2.22 GraphSAGE Forward Pass with Neighbour Sampling . . . 171
2.23 Two-Stage Fine-Tuning with Gradual Unfreezing . . . 177
2.24 Tracking-by-Detection (SORT / DeepSORT style) . . . 196
2.25 Integrated Gradients (Riemann approximation) . . . 204
2.26 Projected Gradient Descent (PGD) $\ell\infty$ Attack . . . 207
2.27 Iterative Magnitude Pruning with Fine-Tuning . . . 216
2.28 Ring All-Reduce . . . 218
2.29 Continuous Training with Drift-Triggered Retraining . . . 235

# List of Code Listings


2.1 A dense layer and a two-layer network forward pass in NumPy. 16
2.2 Perceptron learning from scratch in NumPy, applied to AND and XOR. 22
2.3 A two-layer MLP learning XOR in PyTorch. 23
2.4 Specifying a funnel-shaped MLP for 10-class classification in Keras. 26
2.5 The forward pass of the worked example in NumPy. 32
2.6 Manual backpropagation in NumPy verified against PyTorch autograd. 39
2.7 The Adam update implemented from scratch in NumPy. 51
2.8 Creating optimisers in PyTorch and Keras. 51
2.9 Applying explicit initialisation schemes in PyTorch. 54
2.10 Learning-rate schedulers in PyTorch. 57
2.11 Monitoring gradient norms and applying gradient clipping in PyTorch. 60
2.12 An image data-augmentation pipeline with torchvision and its Keras equivalent. 64
2.13 Inverted dropout and batch normalisation from scratch in NumPy, and the equivalent PyTorch block. 69
2.14 A complete training loop with validation, early stopping and checkpointing in PyTorch. 72
2.15 Hyperparameter tuning with Optuna (TPE sampler and median pruning). 78
2.16 A deep feedforward classifier and training loop in PyTorch. 81
2.17 A direct NumPy implementation of 2D convolution (cross-correlation) with stride and padding. 85
2.18 A small CNN for 32 × 32 colour images (e.g. CIFAR-10) in Keras. 88
2.19 A ResNet basic block with optional projection shortcut (PyTorch). 92
2.20 Fine-tuning a pretrained ResNet-50 for a new 5-class problem with torchvision. 94
2.21 Running a pretrained Faster R-CNN detector with torchvision. 101
2.22 A many-to-one RNN text classifier in PyTorch, with gradient clipping. 105
2.23 A stacked bidirectional LSTM sentiment classifier in Keras. 108
2.24 Sliding-window dataset and a direct multi-step LSTM forecaster in PyTorch. 114
2.25 Tokenisation and inference with pretrained models using Hugging Face Transformers. 118
2.26 Training skip-gram Word2Vec with negative sampling using gensim. 123
2.27 Scaled dot-product attention with an optional mask (PyTorch). 126
2.28 A Pre-LN transformer encoder block (PyTorch). 131
2.29 Fine-tuning a pretrained BERT-style encoder for sentence classification with Hugging Face. 135

2.30 Text generation with different decoding strategies using Hugging Face Transformers. 142
2.31 One GAN training step in PyTorch using the non-saturating loss. 145
2.32 A fully connected VAE for 28 × 28 images in PyTorch. 150
2.33 A convolutional denoising autoencoder in Keras used for anomaly detection. 155
2.34 Core DDPM training step and sampling loop in PyTorch (ε_θ is any network taking (x, t)). 159
2.35 The PPO clipped loss with value and entropy terms (PyTorch). 166
2.36 A compact PyTorch implementation of the NT-Xent loss used by SimCLR. 175
2.37 Feature extraction followed by fine-tuning of an ImageNet-pre-trained EfficientNet in Keras. 177
2.38 Using pretrained foundation models through Hugging Face pipelines. 182
2.39 Zero-shot image classification with CLIP via Hugging Face Transformers. 185
2.40 Computing the CTC loss in PyTorch for a batch of acoustic-model outputs. 189
2.41 Soft Dice loss combined with binary cross-entropy for segmentation in PyTorch. 192
2.42 Loading a pretrained vision transformer from the timm library and classifying an image. 197
2.43 A compact NeuMF-style neural collaborative filtering model in PyTorch. 200
2.44 Computing a Grad-CAM heatmap for a pretrained ResNet-50 with PyTorch hooks. 203
2.45 FGSM and PGD attacks in PyTorch (inputs scaled to $[0, 1]$). 207
2.46 Knowledge-distillation loss in PyTorch. 214
2.47 Data-parallel training with PyTorch DDP and automatic mixed precision (launch with torchrun). 221
2.48 A CUDA kernel written in Python with Numba, and GPU tensors in PyTorch. 223
2.49 An MLP classifier for MNIST in Keras. 227
2.50 The same MLP classifier for MNIST in PyTorch, with an explicit training loop. 227
2.51 Exporting a PyTorch model to TorchScript and ONNX and running it with ONNX Runtime. 229
2.52 A minimal FastAPI inference service for an image classifier. 231
2.53 Computing PSI for a feature with NumPy, using reference-data quantile bins. 234
2.54 Colab environment setup: GPU check, Google Drive mount and seeding. 237
2.55 Loading, splitting and augmenting CIFAR-10 with torchvision. 238
2.56 A compact residual CNN for CIFAR-10. 239
2.57 Training loop with one-cycle schedule, mixed precision, clipping and best-model checkpointing. 240
2.58 Final test evaluation, classification report and confusion matrix. 241
2.59 Saving, exporting and reloading the trained model. 242

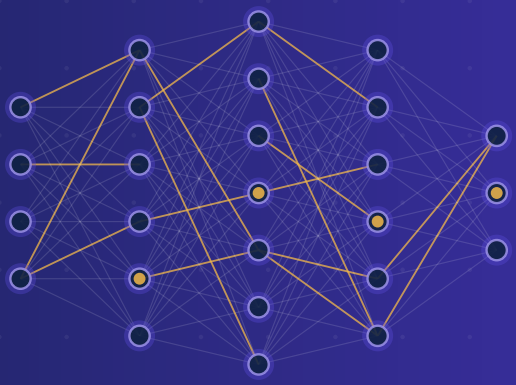
# How to Read This Chapter

Every element in this chapter carries a label and a number so that it can be identified, cited and cross-referenced precisely. Numbers restart in each chapter and take the form *chapter.sequence*. The conventions are summarised in the table below.

| Element | Label format | Purpose |
|---|---|---|
| Section / Subsection | 2.4 / 2.4.2 | Numbered heading; subsections are nested inside sections. |
| Figure | Figure 2.1 | Diagram or chart; caption appears below the figure. |
| Table | Table 2.1 | Structured comparison or summary; caption appears above the table. |
| Equation | (2.1) | Display equation; number at the right margin; symbols explained beneath. |
| Algorithm | Algorithm 2.1 | Pseudocode with inputs, outputs and numbered steps. |
| Listing | Listing 2.1 | Runnable Python code in a shaded panel. |
| Definition | Definition 2.1 | Formal definition of a key term (blue panel). |
| Worked Example | Worked Example 2.1 | Step-by-step numerical illustration (green panel). |
| Case Study | Case Study 2.1 | Real-world application (orange panel). |
| Key Point / Note / Practice Tip | — | Unnumbered highlight panels: takeaways, caveats and practical advice. |

CHAPTER 2

# Deep Learning



61 sections · 74 figures · 61 tables · 29 algorithms · 144 numbered equations · 59 code listings

## Chapter Overview

Deep learning is the branch of machine learning that builds models from many stacked layers of simple, differentiable processing units and trains them end to end from data. In little more than a decade it has moved from a niche research topic to the engine behind image recognition, machine translation, speech assistants, protein-structure prediction and large language models. This chapter develops the subject from first principles: what a neural network computes, how it is trained, why training sometimes fails, and how the major architectural families (convolutional, recurrent, attention-based and generative networks) extend the basic ideas to images, sequences, language and data synthesis.

The chapter is written for readers who already understand the core ideas of machine learning covered in Chapter 1: supervised learning, loss functions, gradient-based optimisation, overfitting and model evaluation. Deep learning reuses all of these concepts, but changes their emphasis. Instead of carefully engineering features and then fitting a relatively simple model, the deep learning practitioner designs an architecture, chooses a loss and an optimiser, and lets the network discover useful intermediate representations by itself. Most of the craft therefore lies in making optimisation of very large, non-convex models reliable and in controlling their capacity so that they generalise.

The first part of the chapter lays the mathematical and practical foundations. It begins with history and terminology, introduces artificial neurons, perceptrons and multi-

layer perceptrons, and then derives forward propagation, loss functions and backpropagation in matrix form. It then treats the optimisation toolkit in detail (gradient descent variants, adaptive optimisers, weight initialisation and learning-rate schedules), explains the vanishing and exploding gradient problems, and closes with regularisation, normalisation, the training–validation–testing workflow and hyperparameter tuning. Later parts build on these foundations to cover convolutional and recurrent networks, transformers and large language models, generative models, advanced learning paradigms, applications, trustworthy and efficient deep learning, and deployment.

Throughout, every idea is presented in three layers: intuition (what the method is trying to do), formalism (equations and algorithms), and practice (worked numerical examples and runnable NumPy, PyTorch or Keras code). Readers are encouraged to run the listings, change their settings and observe the effect, because much of deep learning is learned by experiment.

## Learning Objectives

After studying this chapter you should be able to:

- ✓ Explain what distinguishes deep learning from classical machine learning and place it within the wider field of artificial intelligence.
- ✓ Describe the key historical milestones of neural network research and the factors behind the deep learning revival after 2006–2012.
- ✓ Compute the output of an artificial neuron, a perceptron and a multilayer perceptron, and explain why hidden layers with non-linear activations are needed.
- ✓ Select appropriate activation and loss functions for regression, binary, multi-class and multi-label problems.
- ✓ Derive and implement forward propagation and backpropagation for a fully connected network in matrix form.
- ✓ Compare batch, mini-batch and stochastic gradient descent and the SGD, momentum, RMSprop, Adam and AdamW optimisers.
- ✓ Choose weight initialisation schemes and learning-rate schedules that keep training stable, and diagnose vanishing or exploding gradients.
- ✓ Apply regularisation techniques such as weight decay, dropout, data augmentation, early stopping and normalisation layers.
- ✓ Explain the architectural principles of convolutional, recurrent, transformer and generative models and match them to data modalities.
- ✓ Design a sound training, validation and testing protocol and tune hyperparameters systematically.
- ✓ Discuss the trustworthiness, efficiency and deployment issues that arise when deep models are used in practice.
- ✓ Implement and train deep learning models in PyTorch or Keras and interpret their learning curves.

### Prerequisites

This chapter assumes familiarity with the material of Chapter 1, in particular the supervised learning set-up, linear and logistic regression, loss minimisation by gradient descent, the bias–variance trade-off, cross-validation and standard evaluation metrics. On the mathematical side you will need linear algebra (vectors, matrices, matrix products, transposes), multivariate calculus (partial derivatives, gradients and the chain rule) and elementary probability (expectations, the Gaussian and categorical distributions, and maximum likelihood). Programming examples use Python 3 with NumPy, and either PyTorch or TensorFlow/Keras; basic fluency with arrays and functions is sufficient.

### Chapter Roadmap

The sections of this chapter are grouped into twelve themes, summarised in Table 2.1. The themes are ordered so that each builds on the previous ones: the foundations and training themes are prerequisites for everything that follows, while the later themes can be read selectively according to interest.

## 2.1 Introduction to Deep Learning

Deep learning is a family of machine learning methods in which the model is a composition of many parameterised, differentiable transformations, called **layers**, whose parameters are learned jointly from data by gradient-based optimisation. The word *deep* refers to the number of layers that sit between raw input and final output: a modern image classifier may contain more than a hundred, and a large language model contains dozens of transformer blocks, each with several sub-layers. The word *learning* emphasises that almost nothing about the internal representation is designed by hand; the network discovers it in the course of minimising a loss function.

> **D Definition 2.1 | Deep Learning**
>
> **Deep learning** is the study and application of models built as a composition of multiple trainable, typically non-linear, layers, trained end to end to minimise a task loss so that each layer learns a progressively more abstract representation of the input.

### 2.1.1 Learning Representations

Classical machine learning, as described in Chapter 1, separates a problem into two stages. A domain expert first converts raw data into a vector of **features** (for example,

**Table 2.1** Chapter roadmap: the twelve themes of Chapter 2 and the topics they cover.

| Theme | Topics covered | Main question addressed |
|---|---|---|
| Foundations | Introduction, history, AI/ML/DL relationship, neural network fundamentals, biological and artificial neurons, perceptrons and MLPs, architecture, activation functions | What does a neural network compute? |
| Training neural networks | Forward propagation, loss functions, backpropagation, gradient descent, SGD variants, optimisers, initialisation, learning-rate schedules, vanishing and exploding gradients | How are the parameters learned? |
| Regularisation & tuning | Regularisation, dropout and normalisation, training/validation/testing, hyperparameter tuning | How do we make networks generalise? |
| CNNs & vision | Convolution and pooling, classic CNN architectures, transfer learning, detection and segmentation | How do we exploit spatial structure? |
| Sequence models & NLP | Recurrent networks, LSTM and GRU, sequence-to-sequence models, word embeddings | How do we model ordered data? |
| Transformers & LLMs | Attention, the transformer, pre-trained language models, large language models and prompting | How does attention replace recurrence? |
| Generative models | Autoencoders, variational autoencoders, GANs, diffusion models | How do we learn to synthesise data? |
| Advanced learning paradigms | Transfer, self-supervised, few-shot, reinforcement, graph and multimodal learning | How do we learn with less or richer supervision? |
| Applications | Computer vision, NLP, speech, healthcare, science and industry case studies | Where is deep learning used? |
| Trustworthy DL | Interpretability, robustness, fairness, privacy and uncertainty | Can we trust the model? |
| Efficiency & systems | Hardware, distributed training, mixed precision, compression, pruning and quantisation | How do we train and run models efficiently? |
| Deployment & practice | Frameworks, MLOps, serving, monitoring, and best practices | How do we put models into production? |

edge histograms of an image, word counts of a document, or Mel-frequency cepstral coefficients of an audio clip), and a comparatively simple learner such as logistic regression, a support vector machine or a random forest is then fitted to those features. The quality of the system is bounded by the quality of the features, and good features are expensive to design, domain specific and often brittle.

Deep learning removes the boundary between the two stages. A deep network receives (almost) raw data—pixels, characters, audio samples—and learns the features as part of the same optimisation that fits the final predictor. Mathematically, a network with L layers computes the composite function in Equation (2.1), where each $f_l$ is a simple

parameterised map such as an affine transformation followed by an element-wise non-linearity. The contrast between the two pipelines is illustrated in Figure 2.1.

$$f(\mathbf{x};\theta) = f_L(f_{L-1}(\cdots f_2(f_1(\mathbf{x};\theta_1);\theta_2)\cdots;\theta_{L-1});\theta_L) \tag{2.1}$$

where $\mathbf{x}$ is the input, $f_l$ is the transformation computed by layer $l$ with parameters $\theta_l$, and $\theta = (\theta_1, \ldots, \theta_L)$ collects all trainable parameters.

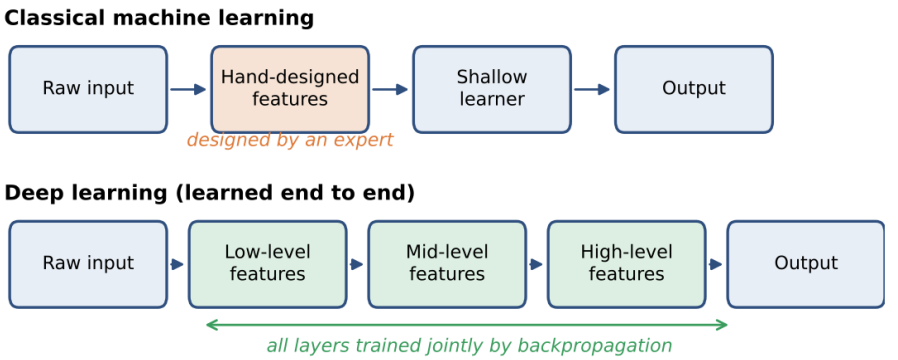



**Figure 2.1** Classical machine learning relies on hand-engineered features fed to a shallow learner, whereas deep learning learns a hierarchy of features and the predictor jointly, end to end.

In a convolutional network trained on photographs, for instance, the first layers come to respond to oriented edges and colour blobs, intermediate layers to textures and object parts, and the deepest layers to whole objects. Nobody programs this hierarchy; it emerges because it is a useful way of reducing the loss. This ability to learn **distributed, hierarchical representations** is the central idea of deep learning and the main reason it transfers so well between tasks.

### 2.1.2 Why Deep Learning Works Now

Neural networks have existed since the 1940s, yet their dominance dates only from around 2012. Three factors converged. First, **data**: the web, smartphones and digitised records produced labelled datasets of unprecedented size, such as ImageNet with over a million labelled images. Deep networks have high capacity and keep improving as data grows, whereas many classical methods saturate. Second, **compute**: graphics processing units (GPUs), originally built for games, turned out to be ideal for the dense matrix multiplications at the heart of neural networks, giving speed-ups of one to two orders of magnitude over CPUs; specialised accelerators followed. Third, **algorithms and software**: rectified linear activations, careful initialisation, dropout, batch normalisation, residual connections and adaptive optimisers made very deep networks train-

able, while open-source frameworks with automatic differentiation (Theano, TensorFlow, PyTorch, JAX) made experimentation cheap.

**Table 2.2** Classical machine learning compared with deep learning.

| Aspect | Classical machine learning | Deep learning |
|---|---|---|
| Features | Hand-engineered by domain experts | Learned automatically from raw data |
| Data requirement | Works well with small to medium data | Benefits from large datasets; can overfit small ones |
| Compute | CPU usually sufficient | GPUs/accelerators usually required |
| Best data types | Tabular, structured features | Images, audio, text, video, graphs |
| Interpretability | Often high (linear models, trees) | Generally low; needs explanation methods |
| Optimisation | Often convex with unique optimum | Non-convex; relies on SGD heuristics |
| Transfer | Limited | Pre-trained models fine-tuned for new tasks |

Table 2.2 summarises the practical differences. It should not be read as saying that deep learning is always better. On small tabular datasets, gradient-boosted trees (Chapter 1) frequently match or beat neural networks at a fraction of the cost, and they are easier to interpret. Deep learning shines when the input is high-dimensional and has structure—spatial, temporal or relational—that layered architectures can exploit, and when enough data or a suitable pre-trained model is available.

### 2.1.3 Strengths and Limitations

The strengths of deep learning follow from its flexibility. A single framework covers regression, classification, sequence generation, segmentation and density estimation simply by changing the output layer and the loss. Models scale gracefully with data and compute, and representations learned on one task can be reused on another through transfer learning. Multimodal models can combine text, images and audio in one network.

The limitations are equally important. Deep networks are data-hungry and computationally expensive; training a large language model consumes substantial energy. They are opaque, which complicates their use in regulated domains such as medicine and finance. They can be brittle, failing under distribution shift or small adversarial perturbations, and they can absorb and amplify biases in their training data. Many of these issues are addressed in the sections on trustworthy and efficient deep learning later in this chapter.

**Key Point | Key Takeaways**

- Deep learning learns a hierarchy of representations end to end instead of relying on hand-crafted features.
- Its success since 2012 rests on large datasets, GPU computing and algorithmic advances that made deep networks trainable.
- It excels on unstructured, high-dimensional data; classical methods remain strong baselines for small tabular problems.

## 2.2 History and Evolution of Deep Learning

The history of deep learning is a story of bold ideas, disappointments and revivals. Understanding it helps to explain why the field uses the vocabulary it does, why certain architectures look the way they do, and why apparently old ideas—backpropagation, convolution, recurrence—returned with such force once data and hardware caught up. Figure 2.2 places the main milestones on a timeline, and Table 2.3 lists them with their significance.

### 2.2.1 The Cybernetic Era (1943–1969)

In 1943 the neurophysiologist Warren McCulloch and the logician Walter Pitts proposed a mathematical model of a neuron as a binary threshold unit: it fires if the weighted sum of its binary inputs reaches a threshold, as in Equation (2.2). They showed that networks of such units can compute any Boolean function, but the weights were fixed by hand. In 1949 Donald Hebb suggested that synapses strengthen when the neurons they connect are active together, the first learning principle for neural networks.

$$y = \operatorname{step}\left(\sum_{i=1}^{n} w_i x_i - \tau\right), \quad \operatorname{step}(z) = 1 \text{ if } z \geq 0,\ 0 \text{ otherwise} \tag{2.2}$$

where $x_i \in \{0, 1\}$ are inputs, $w_i$ fixed weights and $\tau$ the firing threshold.

Frank Rosenblatt's **perceptron** (1958) added a learning rule: the weights are adjusted whenever the unit misclassifies a training example. Implemented in custom hardware as the Mark I Perceptron, it attracted great excitement. Bernard Widrow and Ted Hoff's ADALINE (1960) used a continuous error and the least-mean-squares rule, an early form of gradient descent. In 1969, however, Marvin Minsky and Seymour Papert's book *Perceptrons* proved rigorously that a single-layer perceptron cannot represent functions that are not linearly separable, such as XOR. Although they acknowledged

that multilayer networks might overcome this, no practical training method for them was known, and funding for neural network research fell sharply.

### 2.2.2 Connectionism and Backpropagation (1970–1995)

The key missing ingredient—an efficient way to compute gradients in multilayer networks—was developed in stages. Seppo Linnainmaa described reverse-mode automatic differentiation in 1970, and Paul Werbos proposed applying it to neural networks in his 1974 thesis. The idea became widely known through the 1986 *Nature* paper of David Rumelhart, Geoffrey Hinton and Ronald Williams, "Learning representations by back-propagating errors", and the *Parallel Distributed Processing* volumes. Backpropagation showed that hidden units could learn useful internal representations, reviving the field under the name **connectionism**.

Important architectures appeared in this period. Kunihiko Fukushima's Neocognitron (1980) introduced a hierarchy of local feature detectors inspired by the visual cortex. Yann LeCun and colleagues trained convolutional networks with backpropagation to read handwritten zip codes (1989), culminating in LeNet-5 (LeCun, Bottou, Bengio and Haffner, 1998), which was deployed to read cheques. Hornik, Stinchcombe and White (1989) and Cybenko (1989) proved universal approximation theorems. Sepp Hochreiter's 1991 analysis identified the vanishing gradient problem, and Hochreiter and Jürgen Schmidhuber's **long short-term memory** (LSTM, 1997) addressed it for recurrent networks.

> **Note | The AI Winters**
>
> The term *AI winter* describes periods of reduced funding and interest after inflated expectations were not met. Neural networks suffered such winters after 1969 and again in the late 1990s, when support vector machines and kernel methods offered convex optimisation, strong theory and better results on the small datasets of the time. Many researchers avoided the term *neural network* in paper titles during this second winter.
>
> The lesson for practitioners is that a method's apparent failure may reflect the data and hardware of its era rather than a fundamental flaw.

### 2.2.3 The Deep Learning Revival (2006–2016)

In 2006 Hinton, Simon Osindero and Yee-Whye Teh showed that **deep belief networks** could be trained by greedy layer-wise unsupervised pre-training with restricted Boltzmann machines, followed by fine-tuning. Bengio and colleagues obtained similar results with stacked autoencoders. These papers popularised the term *deep learning* and demonstrated that deep architectures were trainable. Soon after, researchers found that with enough labelled data, GPUs, rectified linear units (Nair and Hinton, 2010;

Glorot, Bordes and Bengio, 2011) and good initialisation (Glorot and Bengio, 2010), pre-training was often unnecessary.

The decisive moment came in 2012, when **AlexNet** (Krizhevsky, Sutskever and Hinton) won the ImageNet Large Scale Visual Recognition Challenge with a top-5 error of about 15.3%, compared with about 26.2% for the runner-up. The same year, deep networks substantially reduced error rates in speech recognition (Hinton et al., 2012). Progress then accelerated: word2vec embeddings (Mikolov et al., 2013); dropout (Srivastava et al., 2014); sequence-to-sequence learning (Sutskever, Vinyals and Le, 2014) and attention for translation (Bahdanau, Cho and Bengio, 2014); **generative adversarial networks** (Goodfellow et al., 2014); the Adam optimiser (Kingma and Ba, 2014); batch normalisation (Ioffe and Szegedy, 2015); and **residual networks** (He, Zhang, Ren and Sun, 2015), which trained networks with over 150 layers and surpassed estimated human-level error on ImageNet classification. In 2016 DeepMind's AlphaGo, combining deep networks with tree search and reinforcement learning, defeated the Go champion Lee Sedol.

> **Case Study 2.1 | ImageNet and AlexNet**
>
> The ImageNet dataset, led by Fei-Fei Li, contains millions of images labelled with WordNet categories; its annual challenge (ILSVRC, 2010–2017) used 1.2 million training images in 1,000 classes. AlexNet combined five convolutional and three fully connected layers (about 60 million parameters), ReLU activations, dropout, data augmentation and training on two GPUs.
>
> Its margin of victory in 2012 convinced the computer vision community almost overnight. Within three years nearly every competitive entry was a deep convolutional network, and top-5 error had fallen below 4%.

### 2.2.4 The Transformer and Foundation-Model Era (2017–present)

In 2017 Vaswani et al. introduced the **transformer** in "Attention Is All You Need", replacing recurrence entirely with self-attention and enabling massively parallel training on sequences. In 2018 BERT (Devlin et al.) and the first GPT model (Radford et al.) showed that transformers pre-trained on large unlabelled text corpora could be adapted to many language tasks. Scaling up produced GPT-2 (2019) and GPT-3 (2020, 175 billion parameters), which exhibited in-context learning from a handful of examples in the prompt. In 2020 **denoising diffusion probabilistic models** (Ho, Jain and Abbeel) achieved high-quality image synthesis, leading to text-to-image systems such as DALL·E 2 and Stable Diffusion (2022). The same year, the Vision Transformer (Dosovitskiy et al.) brought transformers to images, and in 2021 AlphaFold 2 (Jumper et al.) predicted protein structures with near-experimental accuracy. The public re-

lease of ChatGPT in November 2022 brought large language models to a mass audience, and subsequent models have become multimodal, accepting and producing text, images and audio.

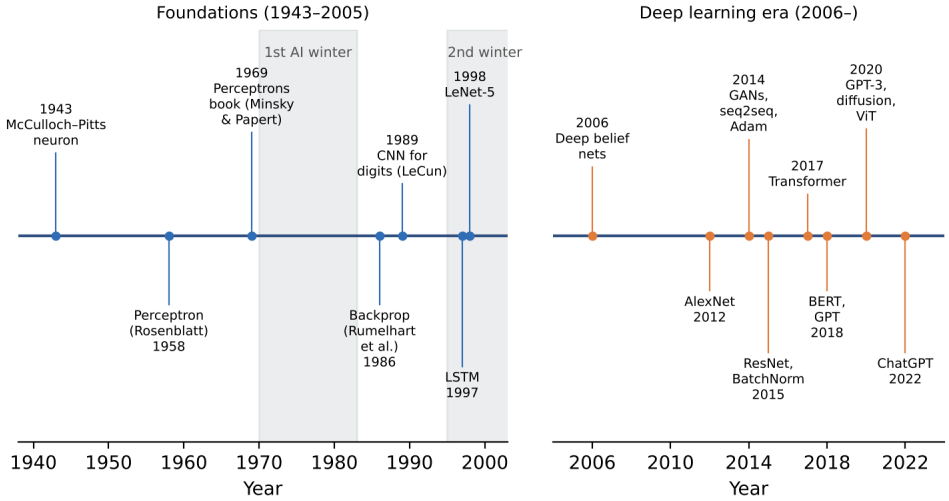



**Figure 2.2** Timeline of major milestones in neural network and deep learning research; the left panel covers 1940–2005 (shaded bands mark the two AI winters) and the right panel expands the dense period from 2006 onwards.

Two recurring themes stand out from Table 2.3. First, many breakthroughs were rediscoveries or re-combinations of older ideas made feasible by new data and hardware: convolution dates from the 1980s and backpropagation from the 1970s. Second, each era's leading architecture encodes an assumption about the data—local spatial structure for CNNs, sequential order for RNNs, and pairwise interactions for transformers—and progress has often come from finding weaker, more general assumptions that scale better with data.

## 2.3 Relationship Between Artificial Intelligence, Machine Learning, and Deep Learning

The terms artificial intelligence, machine learning and deep learning are often used interchangeably in the media, but they denote nested fields with different scopes, as depicted in Figure 2.3. Making the distinction precise helps in choosing the right tool for a problem and in reading the literature critically.

**Table 2.3** Selected milestones in the history of deep learning.

| Year | Milestone | Key contributors | Significance |
|---|---|---|---|
| 1943 | Threshold logic neuron | McCulloch & Pitts | First mathematical model of a neuron |
| 1958 | Perceptron | Rosenblatt | First neuron model that learns from data |
| 1969 | Perceptrons (book) | Minsky & Papert | Proved limits of single-layer networks (XOR) |
| 1986 | Backpropagation popularised | Rumelhart, Hinton & Williams | Training of multilayer networks |
| 1989 / 1998 | Convolutional networks, LeNet-5 | LeCun et al. | Weight sharing for images; cheque reading |
| 1997 | LSTM | Hochreiter & Schmidhuber | Gated memory for long sequences |
| 2006 | Deep belief networks | Hinton, Osindero & Teh | Layer-wise pre-training; deep learning revival |
| 2012 | AlexNet | Krizhevsky, Sutskever & Hinton | GPU-trained CNN wins ImageNet by wide margin |
| 2014 | Generative adversarial networks | Goodfellow et al. | Adversarial training of generators |
| 2015 | ResNet | He, Zhang, Ren & Sun | Residual connections enable 100+ layers |
| 2017 | Transformer | Vaswani et al. | Self-attention replaces recurrence |
| 2018 | BERT, GPT | Devlin et al.; Radford et al. | Large-scale pre-training for language |
| 2020 | GPT-3; DDPM diffusion | Brown et al.; Ho, Jain & Abbeel | In-context learning; diffusion image synthesis |
| 2021 | AlphaFold 2 | Jumper et al. | Protein structure prediction |
| 2022 | ChatGPT | OpenAI | Instruction-tuned LLM reaches mass audience |

### 2.3.1 Three Nested Fields

**D Definition 2.2 | AI, Machine Learning and Deep Learning**

- **Artificial intelligence (AI)**: the broad field concerned with building systems that perform tasks requiring intelligence when done by humans—reasoning, planning, perception, language and learning. It includes symbolic methods such as search, logic, expert systems and planning.
- **Machine learning (ML)**: the subfield of AI in which systems improve their performance on a task through experience (data), rather than being explicitly programmed with rules.
- **Deep learning (DL)**: the subfield of ML that uses multilayer neural networks to learn hierarchical representations directly from data.

Artificial intelligence was named at the 1956 Dartmouth workshop organised by John McCarthy, Marvin Minsky, Nathaniel Rochester and Claude Shannon. Early AI was dominated by **symbolic** approaches: knowledge was encoded as rules and facts, and intelligence was achieved by search and logical inference. Expert systems of the 1970s

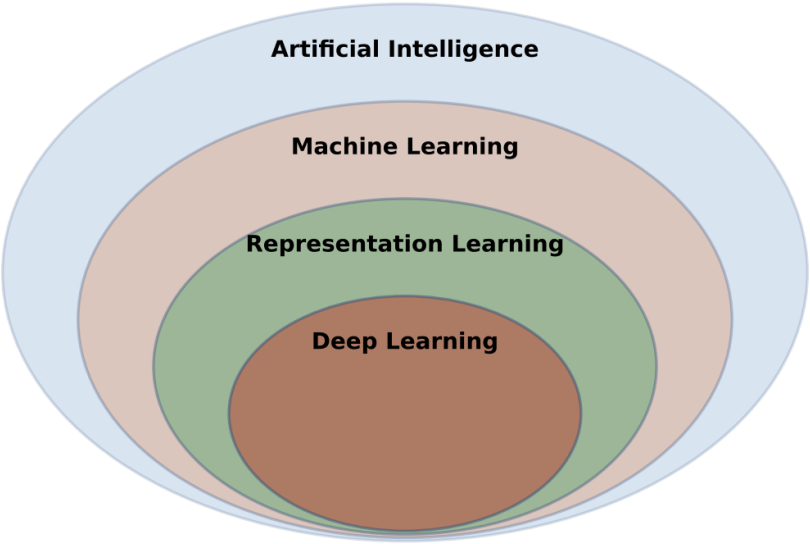



**Figure 2.3** Deep learning is a subfield of machine learning, which is itself one approach to artificial intelligence; each inner field inherits the goals of the outer one but restricts the methods.

and 1980s, such as MYCIN for diagnosing bacterial infections, encoded hundreds of hand-written if–then rules. Such systems are transparent but difficult to scale, because the rules must be elicited from experts and they break on situations the experts did not anticipate.

Machine learning inverts this approach. Following Tom Mitchell's classic definition, a program learns from experience E with respect to a task T and performance measure P if its performance on T, as measured by P, improves with E. Instead of writing rules, we provide examples and a learning algorithm that induces the rules. Chapter 1 covered the main ML families: linear models, decision trees and ensembles, support vector machines, probabilistic models and clustering.

Between ML and DL sits **representation learning**, the set of methods that learn the features as well as the predictor; principal component analysis and dictionary learning are shallow examples, and deep learning is the most successful, multilayer case. Deep learning is therefore not a separate field but a particular, highly expressive choice of hypothesis class and training procedure within machine learning. Everything learned in Chapter 1 about loss functions, generalisation, regularisation and evaluation carries over directly.

**Table 2.4** Characteristics of artificial intelligence, machine learning and deep learning.

| Characteristic | Artificial intelligence | Machine learning | Deep learning |
|---|---|---|---|
| Scope | Any intelligent behaviour | Learning from data | Learning with deep neural networks |
| Knowledge source | Rules, logic, search, or data | Data plus engineered features | Data (raw or minimally processed) |
| Typical methods | Search, planning, expert systems, ML | Regression, trees, SVMs, ensembles | MLPs, CNNs, RNNs, transformers |
| Data needs | May need none | Moderate | Large, or a pre-trained model |
| Interpretability | High for symbolic systems | Moderate | Low without explanation tools |
| Example | Chess engine using search | Credit scoring with gradient boosting | Speech recognition, image captioning |

### 2.3.2 Comparing the Three Levels

Table 2.4 highlights that the three levels differ mainly in where knowledge comes from. Modern systems frequently combine them. AlphaGo used deep networks to evaluate positions and propose moves, but classical Monte Carlo tree search to plan; robotic systems combine learned perception with model-based control; and large language model agents use learned language understanding with symbolic tools such as calculators, code interpreters and databases. This **neuro-symbolic** integration is an active research area.

**E Worked Example 2.1 | Classifying a Spam E-mail at Three Levels**

- **Rule-based AI**: flag an e-mail if it contains phrases like "free money" and comes from an unknown sender. Simple, transparent, easily evaded.
- **Classical ML**: compute word-frequency features and train a naïve Bayes or logistic regression classifier on thousands of labelled e-mails.
- **Deep learning**: feed the raw token sequence to a transformer pre-trained on large text corpora and fine-tune it; the model learns that paraphrases of "free money" are also suspicious, without any feature design.

**K Key Point | Key Takeaways**

- AI ⊃ ML ⊃ representation learning ⊃ DL: every deep learning system is a machine learning system, but not vice versa.
- The fields differ in how knowledge is acquired: hand-coded rules, learned predictors on engineered features, or learned features and predictors together.
- Practical systems increasingly combine learned neural components with symbolic reasoning and search.

# 2.4 Fundamentals of Neural Networks

An **artificial neural network** is a directed graph of simple computing units, called neurons or nodes, organised in layers. Each neuron receives numerical inputs, combines them linearly using learnable weights, adds a bias, and passes the result through a non-linear **activation function**. This section introduces the vocabulary and notation used throughout the chapter; subsequent sections examine each component in depth.

## 2.4.1 The Artificial Neuron

A single neuron with inputs $x_1, \ldots, x_d$ computes a **pre-activation** $z$ (also called the logit or net input) and an **activation** $a$, as in Equation (2.3). The weights determine how strongly each input influences the neuron, the bias shifts the threshold at which it becomes active, and the activation function $\varphi$ introduces non-linearity.

$$z = \sum_{j=1}^{d} w_j x_j + b = \mathbf{w}^{\top}\mathbf{x} + b \qquad a = \phi(z) \tag{2.3}$$

where $\mathbf{x} \in \mathbb{R}^d$ is the input vector, $\mathbf{w} \in \mathbb{R}^d$ the weight vector, $b \in \mathbb{R}$ the bias, and $\varphi$ the activation function (for example the sigmoid or ReLU).

With $\varphi$ equal to the identity, the neuron is linear regression; with $\varphi$ the logistic sigmoid and a cross-entropy loss, it is logistic regression. A neural network is thus a generalisation of models already met in Chapter 1, obtained by connecting many such units.

## 2.4.2 Layers and Networks

Neurons are grouped into **layers**. The **input layer** simply holds the feature vector; it performs no computation. One or more **hidden layers** compute intermediate representations, and the **output layer** produces the prediction. In a **fully connected** (dense) layer every neuron receives every output of the previous layer. Stacking the neurons of layer $l$ into a vector, the whole layer is computed at once by the matrix expression in Equation (2.4). Figure 2.4 shows a network with three inputs, two hidden layers of four neurons and two outputs.

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \qquad \mathbf{a}^{(l)} = \phi^{(l)}(\mathbf{z}^{(l)}), \qquad \mathbf{a}^{(0)} = \mathbf{x} \tag{2.4}$$

where $W^{(l)} \in \mathbb{R}^{(n_l \times n_{l-1})}$ is the weight matrix of layer $l$, $b^{(l)} \in \mathbb{R}^{(n_l)}$ its bias vector, $n_l$ the number of neurons in layer $l$, and $\varphi^{(l)}$ is applied element-wise.

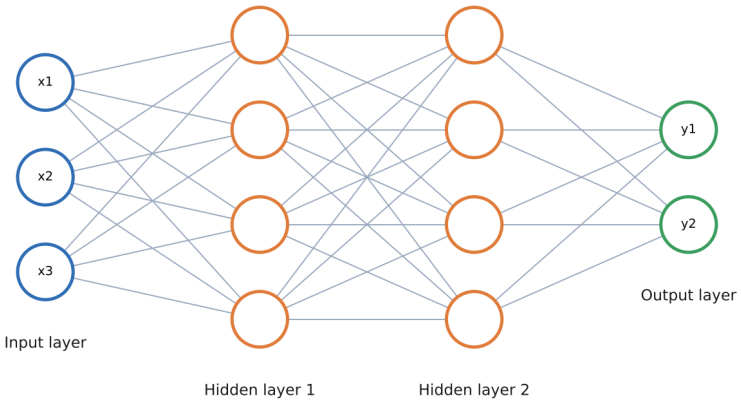



**Figure 2.4** A fully connected feed-forward network with three inputs, two hidden layers of four neurons each, and two outputs; every line represents one learnable weight.

Counting parameters is a useful sanity check. A dense layer mapping $n_{l-1}$ inputs to $n_l$ outputs has $n_l \cdot n_{l-1}$ weights and $n_l$ biases. For the network in Figure 2.4 this gives $(4 \cdot 3 + 4) + (4 \cdot 4 + 4) + (2 \cdot 4 + 2) = 16 + 20 + 10 = 46$ parameters. The general count is given by Equation (2.5).

$$|\theta| = \sum_{l=1}^{L} (n_l \, n_{l-1} + n_l) \tag{2.5}$$

where $L$ is the number of layers with parameters and $n_0 = d$ is the input dimension.

**Table 2.5** Notation for fully connected networks used throughout this chapter.

| Symbol | Meaning | Shape |
|---|---|---|
| $x$, $a^{(0)}$ | Input vector (one example) | $d \times 1$ |
| $X$ | Mini-batch of inputs (rows are examples) | $m \times d$ |
| $W^{(l)}$ | Weight matrix of layer $l$ | $n_l \times n_{l-1}$ |
| $b^{(l)}$ | Bias vector of layer $l$ | $n_l \times 1$ |
| $z^{(l)}$ | Pre-activations of layer $l$ | $n_l \times 1$ |
| $a^{(l)}$ | Activations (outputs) of layer $l$ | $n_l \times 1$ |
| $\varphi^{(l)}$ | Activation function of layer $l$ | element-wise |
| $\hat{y} = a^{(L)}$ | Network output | $n_L \times 1$ |
| $\mathcal{L}$, $J(\theta)$ | Per-example loss; average cost | scalar |
| $\eta$ | Learning rate | scalar |

### 2.4.3 Why Non-linearity and Depth Matter

If every activation were the identity, the composition of layers would collapse: $W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = W'x + b'$ is again a single affine map. Non-linear activations are therefore essential; they allow each layer to bend and fold the input space so that later layers see a representation in which the task is easier, often linearly separable.

How expressive are such networks? The **universal approximation theorem** (Cybenko, 1989; Hornik, Stinchcombe and White, 1989; extended by Leshno et al., 1993 to any non-polynomial activation) states that a network with a single hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy, provided it has enough hidden units. Formally, for any continuous $g$ and $\varepsilon > 0$ there exist $N$, weights and biases such that Equation (2.6) holds.

$$\sup_{\mathbf{x}\in K} \left| g(\mathbf{x}) - \sum_{k=1}^{N} v_k \, \phi\!\left(\mathbf{w}_k^{\top}\mathbf{x} + b_k\right) \right| < \varepsilon \tag{2.6}$$

where $K \subset \mathbb{R}^d$ is compact, $N$ is the number of hidden units, and $v_k$, $w_k$, $b_k$ are output weights, input weights and biases.

The theorem guarantees existence but says nothing about how many units are needed, nor whether gradient descent will find the weights, nor whether the result will generalise. In practice, **depth** is far more efficient than width: there are functions that a deep network represents with a polynomial number of units but a shallow network needs exponentially many (Telgarsky, 2016; Montúfar et al., 2014). Deep networks reuse intermediate features—an edge detector feeds many part detectors—giving a compositional, parameter-efficient representation.

### 2.4.4 A First Implementation

Listing 2.1 implements a dense layer and a two-layer forward pass directly in NumPy, following Equation (2.4). Processing a mini-batch of $m$ examples at once is done by storing examples as rows of a matrix $X$, so the layer computes $XW^{\mathsf{T}} + b^{\mathsf{T}}$; this row-major convention is the one used by PyTorch and Keras.

**Listing 2.1** A dense layer and a two-layer network forward pass in NumPy.

```
import numpy as np

rng = np.random.default_rng(42)

def dense(X, W, b, act):
    """X: (m, n_in), W: (n_out, n_in), b: (n_out,). Returns (m, n_out)."""
```

```
    return act(X @ W.T + b)

relu = lambda z: np.maximum(0.0, z)
identity = lambda z: z

# a 3-4-2 network
W1, b1 = rng.normal(0, 0.5, (4, 3)), np.zeros(4)
W2, b2 = rng.normal(0, 0.5, (2, 4)), np.zeros(2)

X = rng.normal(size=(5, 3))          # mini-batch of 5 examples
H = dense(X, W1, b1, relu)           # hidden representation (5, 4)
Y_hat = dense(H, W2, b2, identity)   # outputs (5, 2)
n_params = W1.size + b1.size + W2.size + b2.size
print(Y_hat.shape, n_params)         # (5, 2) 26
```

**Practice Tip** | **Practical Tip: Check Shapes First**

Most bugs in hand-written networks are shape mismatches. Before training, pass a small batch through the model and print the shape of every intermediate tensor, and verify the parameter count against Equation (2.5). Frameworks provide summaries (`model.summary()` in Keras, `torchinfo.summary` in PyTorch) for exactly this purpose.

## 2.5 Biological Neurons and Artificial Neurons

Artificial neural networks were originally inspired by the brain, and much of their vocabulary—neurons, synapses, firing, activation—comes from neuroscience. This section describes the biological neuron in enough detail to appreciate the analogy, and then explains where the analogy holds and where it breaks down. Modern deep learning is best regarded as an engineering discipline loosely inspired by neuroscience, not as a model of how the brain works.

### 2.5.1 The Biological Neuron

The human brain contains roughly 86 billion neurons, each connected to thousands of others through synapses, giving on the order of $10^{14}$ connections. A typical neuron, sketched in Figure 2.5 (left), has three main parts. The **dendrites** are branching fibres that receive chemical signals from other neurons. The **cell body** (soma) integrates these signals; incoming signals change the electrical potential across the cell membrane, some making it more positive (excitatory) and some more negative (inhibitory). When the membrane potential at the axon hillock exceeds a threshold, the neuron fires an **action potential**, an all-or-nothing electrical spike that travels along

the **axon** to the axon terminals. There, neurotransmitters are released across **synapses** to the dendrites of downstream neurons.

Information is carried both by whether a neuron fires and by its **firing rate**: a strongly stimulated neuron emits spikes more frequently, up to a few hundred per second. Learning in the brain is believed to occur largely by changing synaptic strengths—a process called synaptic plasticity. Hebb's postulate (1949), often summarised as "cells that fire together wire together", describes one such mechanism, and its mathematical form is shown in Equation (2.7).

$$\Delta w_{ij} = \eta \, x_i \, y_j \tag{2.7}$$

where $\Delta w_{ij}$ is the change in the strength of the connection from neuron i to neuron j, $x_i$ and $y_j$ are their activities, and $\eta > 0$ is a learning rate.

## 2.5.2 From Biology to Mathematics

The artificial neuron of Equation (2.3) abstracts this picture. Inputs $x_j$ play the role of signals arriving on dendrites; weights $w_j$ correspond to synaptic strengths, with positive weights excitatory and negative weights inhibitory; the weighted sum z models integration in the soma; the bias b sets the firing threshold; and the activation function $\varphi$ converts the integrated input into an output, which can be read as a firing rate. The correspondence is shown side by side in Figure 2.5.

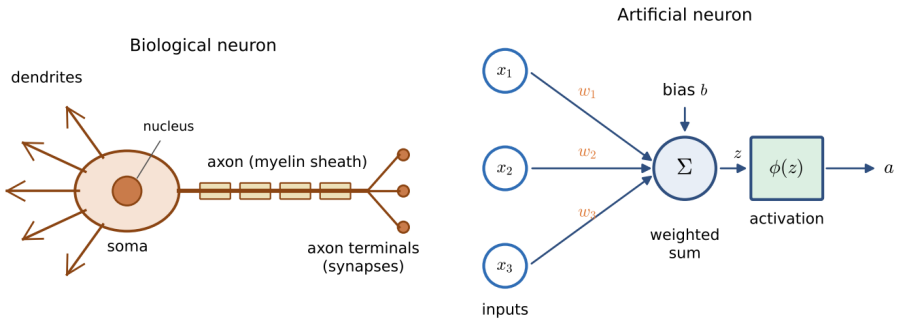



**Figure 2.5** Left: schematic biological neuron with dendrites, soma, axon and synaptic terminals. Right: artificial neuron computing a weighted sum of inputs plus bias followed by an activation function.

## 2.5.3 Where the Analogy Breaks Down

Table 2.6 should not be over-interpreted. Biological neurons communicate with discrete spikes whose precise timing carries information; artificial neurons exchange real

**Table 2.6** Correspondence between biological and artificial neurons.

| Biological neuron | Artificial neuron | Comment |
|---|---|---|
| Dendrites | Input connections $x_j$ | Receive signals from other units |
| Synaptic strength | Weight $w_j$ | Learned; sign gives excitation or inhibition |
| Soma integration | Weighted sum $z = w^T x + b$ | Linear in artificial case; non-linear in biology |
| Firing threshold | Bias b | Shifts the activation point |
| Firing rate | Activation $a = \varphi(z)$ | Continuous value instead of spike train |
| Axon and terminals | Output connections | Fan-out to the next layer |
| Synaptic plasticity | Gradient-based weight update | Biology does not appear to use exact backprop |

numbers synchronously. Dendrites perform substantial non-linear computation themselves, so a single biological neuron may be closer to a small network than to a single unit. The brain contains dozens of neuron types, extensive feedback connections, neuromodulators and glial cells with no counterpart in standard networks. Most importantly, backpropagation requires each synapse to know an error signal computed far downstream and transmitted backwards with the same weights (the *weight transport problem*), which is not biologically plausible in its standard form. Research on spiking neural networks, neuromorphic hardware and biologically plausible learning rules continues to explore closer analogues, but mainstream deep learning has advanced mostly by mathematical and engineering insight.

> **Note | Brains and Networks: Orders of Magnitude**
>
> The brain runs on about 20 watts, whereas training a large neural network may require megawatt-hours of energy. Conversely, a GPU performs floating-point multiplications billions of times faster than a neuron fires. Comparisons of "number of neurons" or "number of parameters" between brains and models are therefore of limited meaning: the units, the communication and the learning rules are fundamentally different.

## 2.6 Perceptrons and Multilayer Perceptrons (MLPs)

The perceptron is the simplest neural network that learns, and the multilayer perceptron is the simplest deep network. Studying them reveals both the power of learning from mistakes and the fundamental limitation that motivated hidden layers. This section covers the perceptron model and learning rule, its convergence and its failure on XOR, and how adding a hidden layer overcomes it.

### 2.6.1 The Perceptron Model

Rosenblatt's perceptron is a binary classifier that outputs $+1$ or $-1$ according to which side of a hyperplane the input lies, as in Equation (2.8). Geometrically, $\mathbf{w}$ is the normal vector of the decision boundary $\mathrm{w}^{\mathrm{T}}\mathrm{x} + \mathrm{b} = 0$ and $-\mathrm{b}/\|\mathrm{w}\|$ is its signed distance from the origin.

$$\hat{y} = \operatorname{sign}(\mathbf{w}^{\top}\mathbf{x} + b), \qquad \operatorname{sign}(z) = +1 \text{ if } z \geq 0, \;\; -1 \text{ otherwise} \tag{2.8}$$

where $\mathrm{y} \in \{-1, +1\}$ is the class label and $\hat{\mathrm{y}}$ the prediction.

### 2.6.2 The Perceptron Learning Rule

The perceptron learns by correcting its mistakes. It cycles through the training data; whenever an example is misclassified, meaning $\mathrm{y}(\mathrm{w}^{\mathrm{T}}\mathrm{x} + \mathrm{b}) \leq 0$, it moves the weight vector towards the example if the true label is $+1$ and away from it if the label is $-1$, as in Equation (2.9). Correctly classified examples leave the weights unchanged. The complete procedure is given in Algorithm 2.1.

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} + \eta\, y_i\, \mathbf{x}_i \\ b &\leftarrow b + \eta\, y_i \qquad \text{if } \; y_i(\mathbf{w}^{\top}\mathbf{x}_i + b) \leq 0 \end{aligned} \tag{2.9}$$

where $(\mathrm{x}_i, \mathrm{y}_i)$ is the current training example and $\eta > 0$ is the learning rate (for the perceptron, $\eta = 1$ is common because scaling $\mathrm{w}$ does not change the decision boundary).

**Algorithm 2.1** Perceptron Learning Algorithm

**Input:** Training set $\{(\mathrm{x}_i, \mathrm{y}_i)\}_{i=1}^{n}$ with $\mathrm{x}_i \in \mathbb{R}^d$, $\mathrm{y}_i \in \{-1, +1\}$; learning rate $\eta$; maximum epochs $\mathrm{T}$

**Output:** Weight vector $\mathrm{w}$ and bias $\mathrm{b}$ defining the separating hyperplane

**initialise** $\mathrm{w} \leftarrow 0$, $\mathrm{b} \leftarrow 0$
**for** epoch $= 1$ **to** $\mathrm{T}$ **do**
  errors $\leftarrow 0$
  **for** $\mathrm{i} = 1$ **to** $\mathrm{n}$ (**in** random order) **do**
    **if** $\mathrm{y}_i\,(\mathrm{w}^{\mathrm{T}}\mathrm{x}_i + \mathrm{b}) \leq 0$ **then**
      $\mathrm{w} \leftarrow \mathrm{w} + \eta\, \mathrm{y}_i\, \mathrm{x}_i$
      $\mathrm{b} \leftarrow \mathrm{b} + \eta\, \mathrm{y}_i$
      errors $\leftarrow$ errors $+ 1$
  **if** errors $= 0$ **then**
    **break** ▷ *all examples correctly classified*
**return** $\mathrm{w}$, $\mathrm{b}$

The rule can be interpreted as stochastic gradient descent on the **perceptron criterion** $J(w, b) = \Sigma \max(0, -y_i(w^T x_i + b))$, which is zero for correctly classified points and grows linearly with the margin violation. The **perceptron convergence theorem** (Novikoff, 1962) guarantees that if the data are linearly separable with margin $\gamma$ and all inputs satisfy $\|x_i\| \le R$, the algorithm makes at most $(R/\gamma)^2$ mistakes, regardless of the number of examples or their dimension (Equation (2.10)).

$$\text{number of updates} \le \left(\frac{R}{\gamma}\right)^2 \tag{2.10}$$

where $R = \max_i \|x_i\|$ and $\gamma$ is the largest margin achievable by any separating hyperplane with unit-norm weight vector.

**E Worked Example 2.2 | Learning the AND Function**

Use inputs $x \in \{0,1\}^2$, labels $y = +1$ only for $(1,1)$, $\eta = 1$, and start from $w = (0, 0)$, $b = 0$. Present examples in the order $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$.

- Epoch 1: $(0,0)$, $y = -1$: $z = 0$, so $y \cdot z = 0 \le 0 \to b = -1$. $(0,1)$: $z = -1$ ✓. $(1,0)$: $z = -1$ ✓. $(1,1)$, $y = +1$: $z = -1 \to w = (1,1)$, $b = 0$.
- Epoch 2: $(0,0)$: $z = 0 \to b = -1$. $(0,1)$: $z = 0 \to w = (1,0)$, $b = -2$. $(1,0)$: $z = -1$ ✓. $(1,1)$: $z = -1 \to w = (2,1)$, $b = -1$.
- Epoch 3: $(0,0)$ ✓; $(0,1)$: $z = 0 \to w = (2,0)$, $b = -2$; $(1,0)$: $z = 0 \to w = (1,0)$, $b = -3$; $(1,1)$: $z = -2 \to w = (2,1)$, $b = -2$.
- Epochs 4–7 continue in the same way; every update is triggered either by a negative example with $z = 0$ (a point lying exactly on the boundary) or by $(1,1)$ with $z \le 0$.
- Epoch 8: $(0,0)$ ✓; $(0,1)$: $z = 0 \to w = (3,2)$, $b = -4$; $(1,0)$: $z = -1$ ✓; $(1,1)$: $z = 1$ ✓.

Epoch 9 makes no mistakes: $z = -4, -2, -1, +1$ for the four inputs. The line $3x_1 + 2x_2 = 4$ separates $(1,1)$ from the other points. Treating $z = 0$ as an error (the condition $y \cdot z \le 0$) forces a strict margin, which is why several epochs are needed; convergence is guaranteed because AND is linearly separable.

### 2.6.3 The XOR Problem

The exclusive-or function outputs 1 when exactly one input is 1. Its four points cannot be separated by any straight line, as Figure 2.6 (left) shows: the positive points $(0,1)$ and $(1,0)$ lie on one diagonal and the negative points $(0,0)$ and $(1,1)$ on the other. Suppose a perceptron did solve XOR; then we would need $b < 0$ (for $(0,0)$), $w_1 + b \ge 0$ and $w_2 + b \ge 0$ (for the positives), and $w_1 + w_2 + b < 0$ (for $(1,1)$). Adding the middle two inequalities gives $w_1 + w_2 + 2b \ge 0$, so $w_1 + w_2 + b \ge -b > 0$, a contradiction. This simple argument is at the heart of Minsky and Papert's critique.

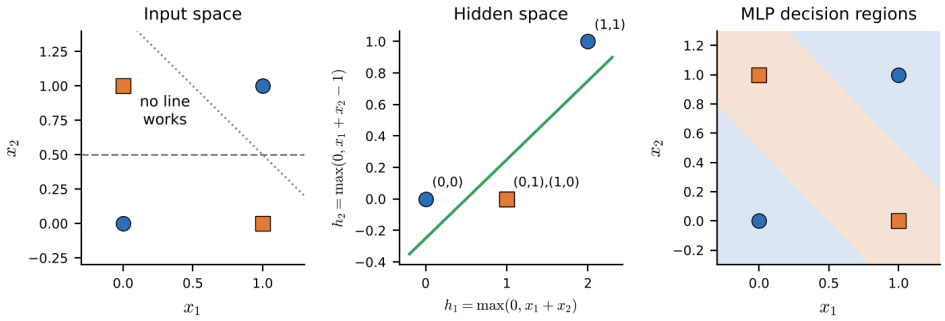



**Figure 2.6** Left: no single line separates the XOR classes. Middle: a hidden layer of two ReLU units maps the four inputs to a new space where they are linearly separable. Right: the resulting non-linear decision regions of the MLP in the original input space.

### 2.6.4 Multilayer Perceptrons

Adding a hidden layer of non-linear units solves XOR. With ReLU activations, the network in Equation (2.11) computes XOR exactly. The first hidden unit counts how many inputs are on; the second fires only when both are on; the output subtracts twice the second from the first. The middle panel of Figure 2.6 shows that in the hidden space the three distinct points (0,0), (1,0) and (2,1) are linearly separable, and the right panel shows the resulting decision regions in the input space.

$$h_1 = \max(0,\ x_1 + x_2), \qquad h_2 = \max(0,\ x_1 + x_2 - 1)$$
$$\hat{y} = h_1 - 2h_2 \tag{2.11}$$

giving $\hat{y} = 0, 1, 1, 0$ for inputs (0,0), (0,1), (1,0), (1,1).

A **multilayer perceptron** (MLP) is a feed-forward network of one or more hidden layers of such units with differentiable activations. The name is historical: modern MLPs use smooth or piecewise-linear activations rather than Rosenblatt's step function, because the step function has zero gradient almost everywhere and cannot be trained by gradient descent. With differentiable activations, the weights of all layers can be learned jointly by backpropagation (see the Section on Backpropagation). Listing 2.2 implements the perceptron of Algorithm 2.1 and shows that it fails on XOR, while Listing 2.3 trains a small MLP in PyTorch that succeeds.

**Listing 2.2** Perceptron learning from scratch in NumPy, applied to AND and XOR.

```
import numpy as np

def perceptron(X, y, eta=1.0, epochs=50, seed=0):
    rng = np.random.default_rng(seed)
    w, b = np.zeros(X.shape[1]), 0.0
```

```
    for _ in range(epochs):
        errors = 0
        for i in rng.permutation(len(y)):
            if y[i] * (X[i] @ w + b) <= 0:      # misclassified
                w += eta * y[i] * X[i]
                b += eta * y[i]
                errors += 1
        if errors == 0:
            break
    return w, b

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
for name, y in {"AND": [-1, -1, -1, 1], "XOR": [-1, 1, 1, -1]}.items():
    y = np.array(y)
    w, b = perceptron(X, y)
    acc = np.mean(np.where(X @ w + b >= 0, 1, -1) == y)
    print(f"{name}: w={w}, b={b}, accuracy={acc:.2f}")
# AND reaches accuracy 1.00; XOR never exceeds 0.75
```

**Listing 2.3** A two-layer MLP learning XOR in PyTorch.

```
import torch
from torch import nn

torch.manual_seed(0)
X = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [0.]])

model = nn.Sequential(nn.Linear(2, 4), nn.Tanh(), nn.Linear(4, 1))
loss_fn = nn.BCEWithLogitsLoss()            # sigmoid + binary cross-entropy
opt = torch.optim.Adam(model.parameters(), lr=0.05)

for epoch in range(1000):
    opt.zero_grad()
    loss = loss_fn(model(X), y)
    loss.backward()
    opt.step()

print(f"final loss {loss.item():.4f}")
print(torch.sigmoid(model(X)).round().squeeze())   # tensor([0., 1., 1., 0.])
```

**Key Point | Key Takeaways**

- The perceptron learns a linear decision boundary by correcting mistakes and converges whenever the data are linearly separable.
- A single linear threshold unit cannot represent XOR or any non-linearly-separable function.
- A hidden layer with non-linear activations re-represents the input so that the classes become linearly separable; this is the essence of deep learning.

# 2.7 Neural Network Architecture

The **architecture** of a neural network is its blueprint: the types of layers, how many there are, how wide each one is, how they are connected, and which activation functions and output layer they use. Architecture encodes prior knowledge, or **inductive bias**, about the problem. Choosing it is one of the most consequential decisions in a deep learning project, alongside the loss and the data.

## 2.7.1 Depth, Width and Connectivity

The **depth** of a network is the number of layers with trainable parameters; its **width** is the number of units in a layer. Deeper networks can represent more complex, compositional functions with fewer parameters, but are harder to optimise; wider networks are easier to optimise but less parameter-efficient. In fully connected networks, a common pattern is a **funnel**: the hidden widths shrink towards the output (for example 512 → 256 → 128), progressively compressing information. Very deep modern architectures add **skip (residual) connections**, in which a layer's input is added to its output, $a^{(l+1)} = a^{(l)} + F(a^{(l)})$, as in Equation (2.12); these create short paths for gradients and are discussed with ResNets in the Section on Convolutional Neural Networks.

$$\mathbf{a}^{(l+1)} = \mathbf{a}^{(l)} + F\left(\mathbf{a}^{(l)};\, \theta^{(l)}\right) \tag{2.12}$$

where F is a small sub-network (for example two dense layers) whose output is added to its input.

Connectivity patterns define the main architectural families, summarised in Table 2.7 and illustrated schematically in Figure 2.7. Each family is treated in detail in a later section of this chapter; here we only note how its connectivity matches a type of data.

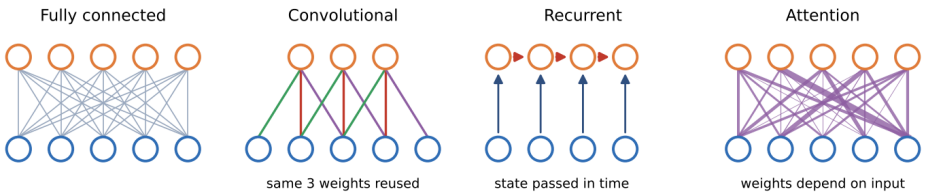



**Figure 2.7** Connectivity patterns of four architectural families: fully connected (every unit to every unit), convolutional (local, shared weights), recurrent (feedback through time), and attention-based (input-dependent all-to-all weights).

**Table 2.7** Major neural network architecture families and their inductive biases.

| Family | Connectivity | Inductive bias | Typical data |
|---|---|---|---|
| MLP (feed-forward) | Dense, layer to layer | None beyond smoothness | Tabular, fixed-size vectors |
| Convolutional (CNN) | Local, weight-shared filters | Locality, translation equivariance | Images, audio spectrograms |
| Recurrent (RNN, LSTM, GRU) | Feedback over time steps | Sequential order, shared dynamics | Time series, text, speech |
| Transformer | Input-dependent attention | Pairwise interactions; permutation equivariance | Text, images, multimodal |
| Graph neural network | Along graph edges | Permutation invariance over nodes | Molecules, social networks |
| Autoencoder / generative | Encoder–decoder | Low-dimensional latent structure | Compression, synthesis |

### 2.7.2 Designing the Output Layer

Whatever the hidden architecture, the output layer must match the task. Its activation determines the range and interpretation of the outputs, and it should be paired with a compatible loss so that training maximises the likelihood of the data under a sensible probabilistic model. Table 2.8 lists the standard combinations, which will be justified in the Sections on Activation Functions and Loss Functions.

**Table 2.8** Output layer design for common tasks.

| Task | Output units | Output activation | Loss function |
|---|---|---|---|
| Regression | 1 per target | Identity (linear) | MSE, MAE or Huber |
| Binary classification | 1 | Sigmoid | Binary cross-entropy |
| Multi-class (one label) | K (classes) | Softmax | Categorical cross-entropy |
| Multi-label | K | Sigmoid per unit | Sum of binary cross-entropies |
| Positive quantity / counts | 1 | Softplus or exp | Poisson or Gamma negative log-likelihood |
| Bounded output in $[-1, 1]$ | 1 per target | Tanh | MSE |

### 2.7.3 Choosing an Architecture in Practice

For a new problem, a few principles help. Start from an architecture known to work on similar data—a pre-trained CNN or vision transformer for images, a pre-trained transformer for text, an MLP or gradient-boosted trees for tabular data. For MLPs, two to four hidden layers of 64–512 units with ReLU activations are a reasonable default; increase capacity until the training loss is low, then add regularisation until the validation loss is acceptable. Prefer depth with residual connections and normalisation over extreme width. Finally, treat depth and width as hyperparameters to be tuned (see the

Section on Hyperparameter Tuning). Listing 2.4 shows how such an MLP is specified in Keras.

**Listing 2.4** Specifying a funnel-shaped MLP for 10-class classification in Keras.

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    keras.Input(shape=(784,)),                # e.g. flattened 28x28 image
    layers.Dense(512, activation="relu"),
    layers.Dense(256, activation="relu"),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax"),   # one probability per class
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.summary()   # 784*512+512 + 512*256+256 + 256*128+128 + 128*10+10 = 567,434
↪ params
```

> **Practice Tip** | **Practical Tip: Inductive Bias Beats Raw Capacity**
>
> An MLP can in principle learn anything a CNN can, but it must learn translation invariance from data, which requires far more examples. Choosing an architecture whose connectivity matches the structure of the data is usually the single most effective way to improve sample efficiency.

## 2.8 Activation Functions

Activation functions supply the non-linearity that gives neural networks their expressive power. Their choice also has a strong effect on optimisation, because the derivative $\varphi'(z)$ multiplies every gradient that flows backwards through a unit. A good hidden-layer activation should be non-linear, cheap to compute, have derivatives that neither vanish nor explode over the typical range of inputs, and ideally produce outputs centred near zero. This section reviews the most important activations; their shapes and derivatives are plotted in Figure 2.8 and compared in Table 2.9.

### 2.8.1 Sigmoid and Hyperbolic Tangent

The **logistic sigmoid** $\sigma$ squashes any real number into $(0, 1)$, which makes it a natural model of a firing rate or a probability. Its derivative has the convenient form in Equation (2.13). The **hyperbolic tangent** is a rescaled sigmoid with range $(-1, 1)$, $\tanh(z) = 2\sigma(2z) - 1$, whose outputs are zero-centred (Equation (2.14)).

$$\sigma(z) = \frac{1}{1+e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{2.13}$$

$$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$
$$\tanh'(z) = 1-\tanh^{2}(z) \tag{2.14}$$

Both functions **saturate**: for large |z| they flatten and their derivatives approach zero. The sigmoid's derivative is at most 0.25 (at z = 0), and tanh's at most 1. In a deep network, the backpropagated gradient is multiplied by such a factor at every layer, so gradients shrink geometrically and early layers learn very slowly—the vanishing gradient problem discussed later. The sigmoid's outputs are also always positive, which makes gradients of the next layer's weights all share a sign and causes inefficient zigzag updates. For these reasons sigmoid and tanh are now rarely used in hidden layers of feed-forward networks, although the sigmoid remains the standard output for binary probabilities and both are used as gates inside LSTMs and GRUs.

### 2.8.2 Rectified Linear Units and Variants

The **rectified linear unit** (ReLU) passes positive inputs unchanged and sets negative inputs to zero (Equation (2.15)). Its derivative is exactly 1 for positive inputs, so gradients pass through active units without shrinking, and it is extremely cheap to compute. Networks with ReLUs produce sparse activations and compute piecewise-linear functions. ReLU's adoption (Nair and Hinton, 2010; Glorot, Bordes and Bengio, 2011; Krizhevsky et al., 2012) was a key enabler of deep learning.

$$\mathrm{ReLU}(z) = \max(0, z)$$
$$\mathrm{ReLU}'(z) = 1 \text{ if } z > 0,\ \ 0 \text{ if } z < 0 \tag{2.15}$$

(the derivative at z = 0 is undefined; implementations conventionally use 0).

ReLU's weakness is the **dying ReLU** problem: if a unit's pre-activation becomes negative for every training input—for example after a large update or because of a large negative bias—its gradient is zero and it never recovers. **Leaky ReLU** (Maas, Hannun and Ng, 2013) keeps a small slope $\alpha$ (typically 0.01) for negative inputs; **Parametric ReLU** (PReLU; He et al., 2015) learns $\alpha$; and the **exponential linear unit** (ELU; Clevert, Unterthiner and Hochreiter, 2016) uses a smooth exponential for negative inputs, pushing mean activations towards zero (Equation (2.16)).

$$\begin{aligned} \mathrm{LeakyReLU}(z) &= \max(\alpha z,\ z), \qquad 0 < \alpha < 1 \\ \mathrm{ELU}(z) &= z \text{ if } z > 0, \quad \alpha(e^{z} - 1) \text{ if } z \le 0 \end{aligned} \tag{2.16}$$

### 2.8.3 Smooth Modern Activations: GELU and Swish

Transformers and many recent architectures use smooth, non-monotonic activations. The **Gaussian error linear unit** (GELU; Hendrycks and Gimpel, 2016) weights its input by the probability that a standard normal variable is below it, and is the default in BERT and GPT models. **Swish** or SiLU (Ramachandran, Zoph and Le, 2017) multiplies the input by its sigmoid. Both behave like ReLU for large |z| but are smooth near zero and allow small negative outputs (Equation (2.17)).

$$\begin{aligned} \mathrm{GELU}(z) &= z\,\Phi(z) \approx 0.5\,z\Big(1 + \tanh\Big[\sqrt{2/\pi}\,(z + 0.044715\,z^{3})\Big]\Big) \\ \mathrm{Swish}(z) &= z\,\sigma(\beta z) \end{aligned} \tag{2.17}$$

where $\Phi$ is the standard normal cumulative distribution function and $\beta$ is a constant or learnable parameter ($\beta = 1$ gives SiLU).

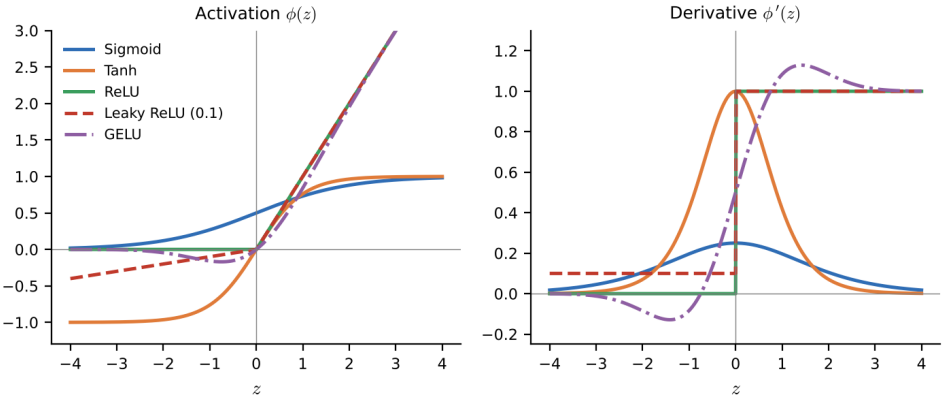



**Figure 2.8** Common activation functions (left) and their derivatives (right). Sigmoid and tanh saturate for large |z|, so their derivatives vanish; ReLU-family functions keep a unit derivative for positive inputs.

### 2.8.4 Softmax for Multi-class Outputs

For a classification problem with K mutually exclusive classes, the output layer produces K logits $z_1$, ..., $z_K$ and the **softmax** function converts them into a probability distribution (Equation (2.18)). Softmax is a smooth approximation of the arg max: it

emphasises the largest logit but remains differentiable. It is invariant to adding a constant to all logits, a property used for numerical stability: implementations subtract $\max_k z_k$ before exponentiating to avoid overflow. A temperature parameter T, $\mathrm{softmax}(z/T)$, sharpens ($T<1$) or flattens ($T>1$) the distribution; it is used in knowledge distillation and in sampling from language models.

$$\mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K$$
$$\frac{\partial \,\mathrm{softmax}(\mathbf{z})_k}{\partial z_j} = s_k(\delta_{kj} - s_j) \tag{2.18}$$

where $s_k = \mathrm{softmax}(z)_k$ and $\delta_{kj}$ is the Kronecker delta (1 if $k = j$, 0 otherwise).

**E Worked Example 2.3 | Softmax**

Logits $z = (2.0, 1.0, 0.1)$. Exponentials: $e^2 = 7.389$, $e^1 = 2.718$, $e^{0.1} = 1.105$; their sum is 11.212.

- Probabilities: $7.389/11.212 = 0.659$, $2.718/11.212 = 0.242$, $1.105/11.212 = 0.099$ (sum = 1).
- Subtracting the maximum, $z - 2 = (0, -1, -1.9)$, gives exponentials 1, 0.368, 0.150 with sum 1.518 and the same probabilities 0.659, 0.242, 0.099—confirming shift invariance.
- With temperature $T = 0.5$ the logits become $(4, 2, 0.2)$ and the probabilities $(0.864, 0.117, 0.019)$: sharper.

**Table 2.9** Comparison of activation functions.

| Function | Range | Max derivative | Zero-centred | Main use / issue |
|---|---|---|---|---|
| Sigmoid | $(0, 1)$ | 0.25 | No | Binary output; saturates, vanishing gradients |
| Tanh | $(-1, 1)$ | 1 | Yes | RNN/LSTM state; saturates |
| ReLU | $[0, \infty)$ | 1 | No | Default for hidden layers; dying units |
| Leaky ReLU / PReLU | $(-\infty, \infty)$ | 1 | Approx. | Avoids dying ReLU |
| ELU | $(-\alpha, \infty)$ | 1 | Approx. | Smooth; slightly costlier |
| GELU / Swish | $\approx(-0.17, \infty)$ / $\approx(-0.28, \infty)$ | $\approx$1.13 / $\approx$1.10 | Approx. | Transformers, modern CNNs |
| Softmax | $(0, 1)$, sums to 1 | — | — | Multi-class output layer |

**T Practice Tip | Practical Tip: Choosing an Activation**

- Hidden layers: start with ReLU; switch to GELU or Swish for transformers and very deep models, or to Leaky ReLU if many units die.
- Output layer: identity for regression, sigmoid for binary or multi-label, softmax for multi-class (see Table 2.8).

- In code, prefer losses that take raw logits (e.g. `BCEWithLogitsLoss`, `CrossEntropyLoss`) and apply the sigmoid or softmax internally; this is numerically more stable than applying the activation yourself.

# 2.9 Forward Propagation

**Forward propagation** (the forward pass) is the computation of a network's output from its input: information flows from the input layer through each hidden layer to the output, and finally into the loss. It is used both at inference time, to make predictions, and during training, where it also stores the intermediate values that backpropagation will need. This section formalises the forward pass for a single example and for a mini-batch, and works through a complete numerical example.

## 2.9.1 Layer-by-Layer Computation

For a network with L layers, forward propagation applies Equation (2.4) repeatedly, starting from $a^{(0)} = x$. Each layer performs an affine transformation followed by an element-wise non-linearity, and the final activation is the prediction $\hat{y} = a^{(L)}$. The loss $\mathcal{L}(\hat{y}, y)$ compares this prediction with the target. Algorithm 2.2 states the procedure; note that it caches every $z^{(l)}$ and $a^{(l)}$.

**Algorithm 2.2** Forward Propagation for a Fully Connected Network

**Input:** Input $x$; parameters $\{W^{(l)}, b^{(l)}\}_{l=1}^{L}$; activations $\{\varphi^{(l)}\}$; target $y$; loss $\mathcal{L}$
**Output:** Prediction $\hat{y}$, loss value, and cache of $\{z^{(l)}, a^{(l)}\}$

$a^{(0)} \leftarrow x$
**for** $l = 1$ **to** $L$ **do**
  $z^{(l)} \leftarrow W^{(l)}\, a^{(l-1)} + b^{(l)}$ ▷ *affine transformation*
  $a^{(l)} \leftarrow \varphi^{(l)}(z^{(l)})$ ▷ *element-wise activation*
  cache $z^{(l)}, a^{(l)}$
$\hat{y} \leftarrow a^{(L)}$
loss $\leftarrow \mathcal{L}(\hat{y}, y)$
**return** $\hat{y}$, loss, cache

## 2.9.2 Vectorised Mini-Batch Form

In practice, examples are processed in mini-batches to exploit fast matrix multiplication on GPUs. Stacking m examples as the rows of $X \in \mathbb{R}^{\wedge}(m \times d)$, the forward pass becomes Equation (2.19). The bias row vector is **broadcast**, that is, added to every row.

The cost of a dense layer is dominated by the matrix product, which requires about $2 \cdot m \cdot n_l \cdot n_{l-1}$ floating-point operations.

$$Z^{(l)} = A^{(l-1)} W^{(l)\top} + \mathbf{1}_m \mathbf{b}^{(l)\top}$$
$$A^{(l)} = \phi^{(l)}(Z^{(l)}), \qquad A^{(0)} = X \tag{2.19}$$

where $A^{(l)} \in \mathbb{R}^{(m \times n_l)}$ holds the activations of all $m$ examples (one per row) and $\mathbf{1}_m$ is a column vector of $m$ ones.

For a whole mini-batch the scalar cost is the average of the per-example losses, as in Equation (2.20). Averaging rather than summing makes the gradient scale independent of the batch size, so the learning rate need not be retuned when $m$ changes.

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{\mathbf{y}}_i, \mathbf{y}_i) \tag{2.20}$$

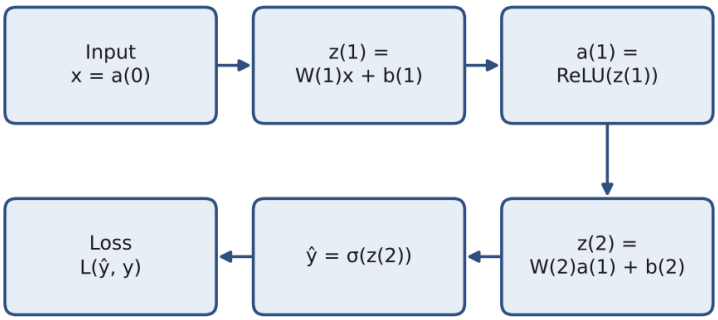



**Figure 2.9** Data flow of the forward pass in a two-layer network: each layer applies an affine map followed by an activation, and the output enters the loss together with the target.

## 2.9.3 Worked Numerical Example

Consider the 2–2–1 network in Figure 2.9 with ReLU hidden units and a sigmoid output, used for binary classification with a binary cross-entropy loss. The arithmetic is shown in the example box below; the same network and values will be reused in the Section on Backpropagation to compute gradients.

**E Worked Example 2.4 | A Forward Pass**

Input $x = (1.0, 2.0)^T$, target $y = 1$. Parameters: $W^{(1)} = [[0.5, -0.2], [0.3, 0.4]]$, $b^{(1)} = (0.1, -0.1)^T$; $W^{(2)} = [0.7, -0.5]$, $b^{(2)} = 0.2$.

- Hidden pre-activations: $z_1^{(1)} = 0.5 \cdot 1 - 0.2 \cdot 2 + 0.1 = 0.2$; $z_2^{(1)} = 0.3 \cdot 1 + 0.4 \cdot 2 - 0.1 = 1.0$.
- Hidden activations (ReLU): $a^{(1)} = (0.2, 1.0)^T$.
- Output pre-activation: $z^{(2)} = 0.7 \cdot 0.2 - 0.5 \cdot 1.0 + 0.2 = -0.16$.
- Output: $\hat{y} = \sigma(-0.16) = 1/(1 + e^{\wedge}0.16) = 1/(1 + 1.1735) = 0.4601$.
- Loss: $\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] = -\log 0.4601 = 0.7763$.

The network currently assigns probability 0.46 to the correct class; training should increase it.

Listing 2.5 reproduces this computation in NumPy and prints the same numbers; in PyTorch the identical forward pass would be expressed with `nn.Linear` layers, and the framework would record the operations for automatic differentiation.

**Listing 2.5** The forward pass of the worked example in NumPy.

```
import numpy as np

x  = np.array([1.0, 2.0]); y = 1.0
W1 = np.array([[0.5, -0.2], [0.3, 0.4]]); b1 = np.array([0.1, -0.1])
W2 = np.array([[0.7, -0.5]]);             b2 = np.array([0.2])

sigmoid = lambda z: 1.0 / (1.0 + np.exp(-z))

z1 = W1 @ x + b1            # [0.2, 1.0]
a1 = np.maximum(0, z1)      # ReLU -> [0.2, 1.0]
z2 = W2 @ a1 + b2           # [-0.16]
y_hat = sigmoid(z2)         # [0.4601]
loss = -(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))
print(z1, a1, z2, y_hat, loss)   # loss = [0.7763]
```

**N Note | Memory Cost of the Forward Pass**

During training, every intermediate activation must be stored until the backward pass uses it. Activation memory grows with batch size × depth × width and often exceeds the memory needed for the parameters themselves. Techniques such as gradient checkpointing (recomputing some activations during the backward pass) trade extra computation for memory.

## 2.10 Loss Functions and Cost Functions

A **loss function** $\mathcal{L}(\hat{y}, y)$ measures the discrepancy between a prediction and the target for a single example; the **cost function** (or objective) $J(\theta)$ aggregates it over the train-

ing set, usually as an average, possibly plus a regularisation term. The choice of loss defines what the network is optimised to do, and it interacts with the output activation and with the gradients seen by the optimiser. Most standard losses arise as negative log-likelihoods of a probabilistic model of the targets, which gives them a principled justification.

**D Definition 2.3 | Loss, Cost and Objective**

- **Loss** $\mathcal{L}(\hat{y}_i, y_i)$: error on one example.
- **Cost** $J(\theta) = (1/n)\,\Sigma_i\,\mathcal{L}(\hat{y}_i, y_i)$: average loss over the dataset (the empirical risk).
- **Objective**: the quantity actually minimised, e.g. $J(\theta) + \lambda\,R(\theta)$ with a regulariser R. In practice the three terms are often used interchangeably.

## 2.10.1 Losses for Regression

The **mean squared error** (MSE) is the negative log-likelihood of a Gaussian noise model with fixed variance; minimising it estimates the conditional mean of y given x. The **mean absolute error** (MAE) corresponds to Laplacian noise and estimates the conditional median, making it more robust to outliers, but its gradient has constant magnitude, which can make convergence near the optimum slow. The **Huber loss** (Huber, 1964) is quadratic for small errors and linear for large ones, combining the smooth optimum of MSE with the robustness of MAE (Equation (2.21)).

$$\begin{aligned} \mathcal{L}_{\mathrm{MSE}} &= (y-\hat{y})^2, \qquad \mathcal{L}_{\mathrm{MAE}} = |\,y-\hat{y}\,| \\ \mathcal{L}_{\delta}^{\mathrm{Huber}} &= \tfrac{1}{2}(y-\hat{y})^2 \;\text{ if }\; |y-\hat{y}| \le \delta, \qquad \delta\big(|y-\hat{y}| - \tfrac{\delta}{2}\big) \;\text{ otherwise} \end{aligned} \tag{2.21}$$

where $\delta > 0$ is the threshold at which the Huber loss switches from quadratic to linear.

Figure 2.10 (left) plots the three regression losses against the residual $r = y - \hat{y}$. The MSE penalises a residual of 4 sixteen times more than a residual of 1, so a few outliers can dominate training; the Huber loss limits their influence.

## 2.10.2 Losses for Classification

For binary classification with a sigmoid output $\hat{y} = \sigma(z)$ interpreted as $P(y = 1 \mid x)$, the Bernoulli negative log-likelihood gives the **binary cross-entropy** (BCE) loss. For K classes with softmax outputs and a one-hot target vector, the categorical negative log-likelihood gives the **categorical cross-entropy** (CCE), which reduces to the negative log-probability of the correct class c (Equation (2.22)).

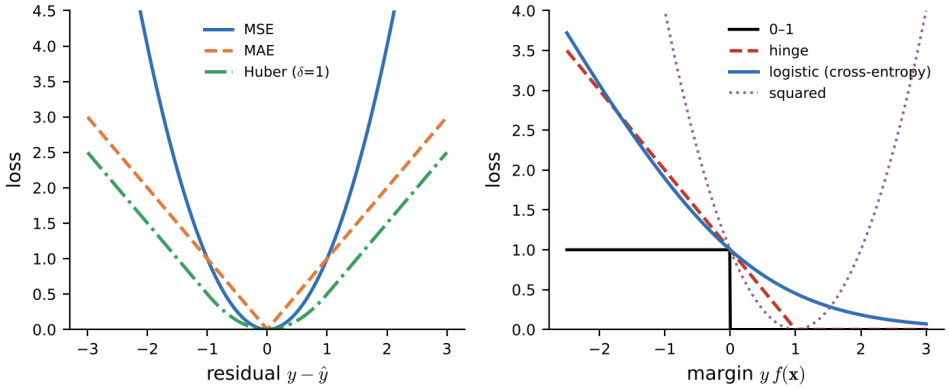



**Figure 2.10** Left: regression losses as a function of the residual. Right: classification losses as a function of the margin $y \cdot f(x)$ for labels in $\{-1, +1\}$; the 0–1 loss is what we care about, the others are differentiable surrogates.

$$\mathcal{L}_{\text{BCE}} = -\big[\, y \log \hat{y} + (1-y)\log(1-\hat{y}) \,\big]$$
$$\mathcal{L}_{\text{CCE}} = -\sum_{k=1}^{K} y_k \log \hat{y}_k = -\log \hat{y}_c \tag{2.22}$$

where $y \in \{0, 1\}$ for BCE, $y$ is a one-hot vector for CCE with correct class $c$, and $\hat{y}_k = \text{softmax}(z)_k$.

Cross-entropy pairs beautifully with its output activation. Differentiating BCE composed with the sigmoid, or CCE composed with the softmax, gives the remarkably simple gradient with respect to the logits shown in Equation (2.23): prediction minus target. The gradient is large when the network is confidently wrong, unlike MSE with a sigmoid output, whose gradient contains a factor $\sigma'(z)$ that vanishes exactly when the prediction is saturated at the wrong value. This is why cross-entropy, not MSE, is the standard classification loss.

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y, \qquad \frac{\partial \mathcal{L}_{\text{CCE}}}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y} \tag{2.23}$$

The **hinge loss** used by support vector machines, $\mathcal{L} = \max(0, 1 - y \cdot f(x))$ with $y \in \{-1, +1\}$, penalises predictions that are correct but within a margin of 1 and ignores those beyond it. It yields sparse gradients and no probability estimates, and is occasionally used for the output layer of neural classifiers. The right panel of Figure 2.10 compares classification losses as functions of the margin; all are convex upper bounds (surrogates) of the non-differentiable 0–1 loss. Two useful refinements are **label smoothing**, discussed under regularisation, and the **focal loss** (Lin et al., 2017), which down-weights easy examples for heavily imbalanced problems.

**E Worked Example 2.5 | Cross-Entropy versus MSE for a Confident Mistake**

Let $y = 1$ and the logit $z = -4$, so $\hat{y} = \sigma(-4) = 0.018$.

- BCE: $\mathcal{L} = -\log 0.018 = 4.02$; gradient $\partial\mathcal{L}/\partial z = \hat{y} - y = -0.982$ (strong correction).
- MSE: $\mathcal{L} = (1 - 0.018)^2 = 0.964$; gradient $\partial\mathcal{L}/\partial z = -2(1 - \hat{y})\sigma'(z) = -2 \times 0.982 \times 0.0177 = -0.035$, almost 30 times weaker.

Cross-entropy therefore escapes confidently wrong predictions far faster.

### 2.10.3 Divergence-Based Losses

The **Kullback–Leibler (KL) divergence** measures how a distribution q differs from a reference distribution p (Equation (2.24)). It is non-negative, zero only when $p = q$, and asymmetric. Cross-entropy decomposes as $H(p, q) = H(p) + D_{KL}(p \| q)$; because the entropy $H(p)$ of the data does not depend on the model, minimising cross-entropy is equivalent to minimising KL divergence from the data distribution. KL losses appear explicitly in knowledge distillation (matching a student's softmax to a teacher's), in variational autoencoders (regularising the latent distribution towards a prior), and in reinforcement learning from human feedback (keeping a fine-tuned policy close to a reference model).

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_k p_k \log \frac{p_k}{q_k}$$
$$H(p, q) = -\sum_k p_k \log q_k = H(p) + D_{\mathrm{KL}}(p \,\|\, q) \tag{2.24}$$

where p is the target distribution, q the model distribution, and $H(p) = -\Sigma_k\, p_k \log p_k$ the entropy of p.

**Table 2.10** Summary of common loss functions.

| Loss | Task | Probabilistic model | Properties |
|---|---|---|---|
| MSE (L2) | Regression | Gaussian noise | Smooth; sensitive to outliers |
| MAE (L1) | Regression | Laplace noise | Robust; non-smooth at 0 |
| Huber | Regression | Gaussian core, Laplace tails | Robust and smooth; needs $\delta$ |
| Binary cross-entropy | Binary / multi-label | Bernoulli | Gradient $\hat{y} - y$ with sigmoid |
| Categorical cross-entropy | Multi-class | Categorical | Gradient $\hat{y} - y$ with softmax |
| Hinge | Binary (margin) | None (max-margin) | Sparse gradients; no probabilities |
| KL divergence | Distribution matching | Any distributions | Asymmetric; used in VAEs, distillation |

**T Practice Tip | Practical Tip: Losses in Frameworks**

- PyTorch: `nn.MSELoss`, `nn.L1Loss`, `nn.HuberLoss`, `nn.BCEWithLogitsLoss`, `nn.CrossEntropyLoss` (takes logits and integer labels), `nn.KLDivLoss` (expects log-probabilities as input).
- Keras: `MeanSquaredError`, `Huber`, `BinaryCrossentropy(from_logits=True)`, `SparseCategoricalCrossentropy(from_logits=True)`, `KLDivergence`.
- Always check whether a loss expects logits or probabilities; passing softmax outputs into a loss that applies softmax again is a common silent bug.

# 2.11 Backpropagation

Training a network by gradient descent requires the gradient of the cost with respect to every weight and bias—often millions or billions of numbers. **Backpropagation** (backward propagation of errors) computes all of them exactly in time proportional to a single forward pass. It is nothing more than the chain rule of calculus, applied systematically from the output back to the input and organised so that intermediate results are reused. It is a special case of **reverse-mode automatic differentiation**, the technique on which every modern deep learning framework is built.

## 2.11.1 Computational Graphs

Any computation can be represented as a **computational graph**: a directed acyclic graph whose nodes are elementary operations (add, multiply, exp, matrix product) and whose edges carry intermediate values. Figure 2.11 shows the graph of a single sigmoid neuron with a squared-error loss. In the forward pass values flow left to right. In the backward pass we start with $\partial \mathcal{L}/\partial \mathcal{L} = 1$ at the output and move right to left; at each node we multiply the incoming **upstream gradient** by the node's **local gradient** and send the result to its inputs. If a value feeds several nodes, the gradients arriving from each are summed.

**E Worked Example 2.6 | Backpropagation Through a Single Neuron**

Using the values of Figure 2.11: $z = 0.5 \cdot 2 + (-1) \cdot 1 + 0.5 = 0.5$, $\hat{y} = \sigma(0.5) = 0.6225$, $\mathcal{L} = (0.6225 - 1)^2 = 0.1425$.

- $\partial \mathcal{L}/\partial \hat{y} = 2(\hat{y} - y) = -0.755$.
- $\partial \mathcal{L}/\partial z = \partial \mathcal{L}/\partial \hat{y} \cdot \sigma'(z)$, with $\sigma'(0.5) = \hat{y}(1 - \hat{y}) = 0.2350$, gives $\partial \mathcal{L}/\partial z = -0.755 \times 0.2350 = -0.1774$.
- The $+$ node passes the gradient unchanged to each input: $\partial \mathcal{L}/\partial b = -0.1774$, and the gradient into each $\times$ node is $-0.1774$.

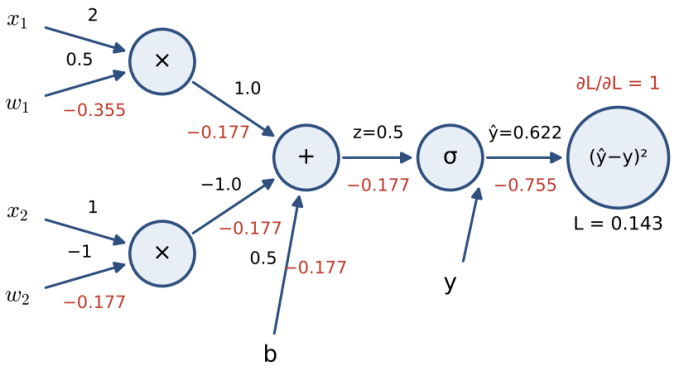



**Figure 2.11** Computational graph of a single sigmoid neuron with squared-error loss, for w = (0.5, −1), x = (2, 1), b = 0.5, y = 1. Forward values are shown above each edge (black), backward gradients below (red).

- A × node multiplies the upstream gradient by the *other* input: $\partial\mathcal{L}/\partial w_1 = -0.1774 \times x_1 = -0.355$ and $\partial\mathcal{L}/\partial w_2 = -0.1774 \times x_2 = -0.177$.

All gradients are negative, so gradient descent increases $w_1$, $w_2$ and b, raising z and moving $\hat{y}$ towards the target 1.

## 2.11.2 Derivation for Fully Connected Networks

Consider a fully connected network computed by Equation (2.4) and a per-example loss $\mathcal{L}$. Define the **error signal** of layer l as the gradient of the loss with respect to that layer's pre-activations, $\delta^{(l)} = \partial\mathcal{L}/\partial z^{(l)}$. Four equations, sometimes called the fundamental equations of backpropagation, follow from the chain rule (Equation (2.25)).

$$\begin{aligned} \boldsymbol{\delta}^{(L)} &= \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot \phi'^{(L)}(\mathbf{z}^{(L)}) \\ \boldsymbol{\delta}^{(l)} &= (W^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}) \odot \phi'^{(l)}(\mathbf{z}^{(l)}), \qquad l = L-1, \ldots, 1 \\ \frac{\partial\mathcal{L}}{\partial W^{(l)}} &= \boldsymbol{\delta}^{(l)}\,\mathbf{a}^{(l-1)\top} \\ \frac{\partial\mathcal{L}}{\partial \mathbf{b}^{(l)}} &= \boldsymbol{\delta}^{(l)} \end{aligned} \tag{2.25}$$

where $\odot$ denotes element-wise multiplication and $\varphi'^{(l)}$ is the derivative of the activation of layer l, applied element-wise.

Each line follows from the chain rule. For the output layer, the loss depends on $z_j^{(L)}$ only through $a_j^{(L)} = \varphi(z_j^{(L)})$, so $\delta_j^{(L)} = (\partial\mathcal{L}/\partial a_j^{(L)})\cdot\varphi'(z_j^{(L)})$. For a hidden layer, $z_j^{(l)}$ influences the loss through every unit k of the next layer, via $z_k^{(l+1)} = \Sigma_j\, W_{kj}^{(l+1)}\, \varphi(z_j^{(l)})$

+ $b_k^{(l+1)}$; summing over these paths gives the scalar form in Equation (2.26), whose vector form is the second line of Equation (2.25). Finally, since $z_k^{(l)} = \Sigma_j\, W_{kj}^{(l)}\, a_j^{(l-1)} + b_k^{(l)}$, we have $\partial z_k^{(l)} / \partial W_{kj}^{(l)} = a_j^{(l-1)}$, which gives the weight gradient as an outer product.

$$\delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} = \sum_k \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \phi'\left(z_j^{(l)}\right) \sum_k W_{kj}^{(l+1)}\, \delta_k^{(l+1)} \tag{2.26}$$

The structure is now clear: errors propagate backwards through the transposed weight matrices, being scaled at each layer by the local activation derivative. When the output uses a sigmoid or softmax with the matching cross-entropy loss, the first line simplifies to $\delta^{(L)} = \hat{y} - y$ (Equation (2.23)). For a mini-batch, the gradients are averaged over examples; in matrix form with $\Delta^{(l)} \in \mathbb{R}^{\wedge}(m \times n_l)$ holding the error signals as rows, $\partial J / \partial W^{(l)} = (1/m)\, \Delta^{(l)T}\, A^{(l-1)}$.

**Algorithm 2.3** Backpropagation for a Fully Connected Network (one example)

**Input:** Cache $\{z^{(l)}, a^{(l)}\}$ from Algorithm 2.2; target y; weights $\{W^{(l)}\}$
**Output:** Gradients $\{\partial \mathcal{L}/\partial W^{(l)}, \partial \mathcal{L}/\partial b^{(l)}\}$ for $l = 1, \ldots, L$

$\delta^{(L)} \leftarrow \nabla_a \mathcal{L}(a^{(L)}, y) \odot \varphi'^{(L)}(z^{(L)})$ ▷ *equals $a^{(L)} - y$ for softmax/sigmoid + cross-entropy*
**for** $l = L$ down **to** 1 **do**
  $\partial \mathcal{L}/\partial W^{(l)} \leftarrow \delta^{(l)}\, a^{(l-1)T}$
  $\partial \mathcal{L}/\partial b^{(l)} \leftarrow \delta^{(l)}$
  **if** $l > 1$ **then**
    $\delta^{(l-1)} \leftarrow (W^{(l)T}\, \delta^{(l)}) \odot \varphi'^{(l-1)}(z^{(l-1)})$
**return** $\{\partial \mathcal{L}/\partial W^{(l)}, \partial \mathcal{L}/\partial b^{(l)}\}$

### 2.11.3 Worked Example: Two-Layer Network

We now complete the example of the Section on Forward Propagation by computing every gradient with Algorithm 2.3 and taking one gradient-descent step. The output uses a sigmoid with binary cross-entropy, so $\delta^{(2)} = \hat{y} - y$.

**Worked Example 2.7 | Backward Pass and Update**

From the forward pass: $a^{(1)} = (0.2, 1.0)^T$, $z^{(1)} = (0.2, 1.0)^T$, $\hat{y} = 0.4601$, $y = 1$, $\mathcal{L} = 0.7763$.

- Output error: $\delta^{(2)} = \hat{y} - y = -0.5399$.
- Output gradients: $\partial \mathcal{L}/\partial W^{(2)} = \delta^{(2)}\, a^{(1)T} = (-0.1080, -0.5399)$; $\partial \mathcal{L}/\partial b^{(2)} = -0.5399$.
- Back to the hidden layer: $W^{(2)T}\delta^{(2)} = (0.7, -0.5)^T \times (-0.5399) = (-0.3779, 0.2700)^T$. Both hidden pre-activations are positive, so ReLU′ = (1, 1) and $\delta^{(1)} = (-0.3779, 0.2700)^T$.
- Hidden gradients: $\partial \mathcal{L}/\partial W^{(1)} = \delta^{(1)} x^T = [[-0.3779, -0.7559], [0.2700, 0.5399]]$; $\partial \mathcal{L}/\partial b^{(1)} = (-0.3779, 0.2700)^T$.

- Update with $\eta = 0.5$: $W^{(2)} \leftarrow (0.7540, -0.2300)$, $b^{(2)} \leftarrow 0.4700$; $W^{(1)} \leftarrow [[0.6890, 0.1779], [0.1650, 0.1300]]$, $b^{(1)} \leftarrow (0.2890, -0.2350)$.
- New forward pass: $z^{(1)} = (1.3338, 0.1901)$, $a^{(1)} = (1.3338, 0.1901)$, $z^{(2)} = 0.7540 \times 1.3338 - 0.2300 \times 0.1901 + 0.4700 = 1.4319$, $\hat{y} = 0.8072$, $\mathcal{L} = 0.2142$.

A single step reduced the loss from 0.776 to 0.214, raising the probability of the correct class from 0.46 to 0.81.

### 2.11.4 Gradient Checking and Automatic Differentiation

Hand-derived gradients are error-prone. **Gradient checking** compares an analytic gradient with the centred finite-difference approximation in Equation (2.27), which has error $O(\varepsilon^2)$. A relative difference below about $10^{-7}$ (in double precision) indicates a correct implementation. Because it requires two forward passes per parameter, gradient checking is used only for debugging on small models.

$$\frac{\partial J}{\partial \theta_i} \approx \frac{J(\theta + \varepsilon\, \mathbf{e}_i) - J(\theta - \varepsilon\, \mathbf{e}_i)}{2\varepsilon} \tag{2.27}$$

where $\mathbf{e}_i$ is the i-th unit vector and $\varepsilon$ is small, e.g. $10^{-5}$.

Modern frameworks implement backpropagation generically through **automatic differentiation**. During the forward pass they record every operation on a tape (PyTorch, TensorFlow's GradientTape) or trace the program into a graph (JAX); each primitive operation knows its local derivative (a vector–Jacobian product). Calling `loss.backward()` traverses the recorded graph in reverse and accumulates gradients into each parameter's `.grad` field. Reverse mode is efficient for deep learning because there are many inputs (parameters) and a single scalar output (the loss): one backward pass costs only a small constant factor more than the forward pass, whereas forward-mode differentiation or finite differences would need one pass per parameter. Listing 2.6 implements Algorithm 2.3 in NumPy and checks it against PyTorch's autograd.

**Listing 2.6** Manual backpropagation in NumPy verified against PyTorch autograd.

```
import numpy as np, torch

x = np.array([1.0, 2.0]); y = 1.0
W1 = np.array([[0.5, -0.2], [0.3, 0.4]]); b1 = np.array([0.1, -0.1])
W2 = np.array([[0.7, -0.5]]);             b2 = np.array([0.2])
sig = lambda z: 1 / (1 + np.exp(-z))

# forward
z1 = W1 @ x + b1; a1 = np.maximum(0, z1)
z2 = W2 @ a1 + b2; y_hat = sig(z2)
# backward (sigmoid + BCE => delta2 = y_hat - y)
```

```
d2  = y_hat - y
gW2 = np.outer(d2, a1); gb2 = d2
d1  = (W2.T @ d2) * (z1 > 0)
gW1 = np.outer(d1, x);  gb1 = d1

# same computation with autograd
T = lambda a: torch.tensor(a, requires_grad=True)
tW1, tb1, tW2, tb2 = T(W1), T(b1), T(W2), T(b2)
out = torch.sigmoid(tW2 @ torch.relu(tW1 @ torch.tensor(x) + tb1) + tb2)
loss = torch.nn.functional.binary_cross_entropy(out, torch.tensor([y]))
loss.backward()
print(np.allclose(gW1, tW1.grad.numpy()), np.allclose(gW2, tW2.grad.numpy()))  # True
↪ True
```

**Key Point | Key Takeaways**

- Backpropagation is the chain rule applied in reverse topological order over the computational graph, reusing intermediate results.
- Error signals propagate backwards through transposed weight matrices and are scaled by activation derivatives at each layer.
- Its cost is a small constant multiple of a forward pass, independent of the number of parameters; frameworks implement it as reverse-mode automatic differentiation.

# 2.12 Gradient Descent and Optimization in Deep Learning

Training a neural network means solving the optimisation problem $\theta^* = \arg\min_\theta J(\theta)$, where $J$ is the average training loss plus any regulariser. Chapter 1 introduced gradient descent for convex problems such as linear and logistic regression, where any local minimum is global. Neural network objectives are highly **non-convex**, have millions of dimensions, and are evaluated on noisy mini-batches. Remarkably, simple first-order methods still work very well. This section explains the basic algorithm and the geometry of the loss surfaces it must navigate; the following sections describe its stochastic variants and adaptive refinements.

## 2.12.1 The Gradient Descent Update

Gradient descent repeatedly moves the parameters a small step in the direction of steepest descent, the negative gradient (Equation (2.28)). The justification is a first-order Taylor expansion: for a small step $\Delta\theta$, $J(\theta + \Delta\theta) \approx J(\theta) + \nabla J(\theta)^{\mathrm{T}}\Delta\theta$, which decreases most rapidly, for a fixed step length, when $\Delta\theta$ points along $-\nabla J(\theta)$.

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t) \tag{2.28}$$

where $\theta_t$ are the parameters at iteration t and $\eta > 0$ is the learning rate (step size).

The learning rate is the single most important hyperparameter. If it is too small, progress is painfully slow; if it is too large, the iterates overshoot the minimum, oscillate, or diverge. For a quadratic objective with Hessian H, gradient descent converges if and only if $\eta < 2/\lambda_max(H)$, where $\lambda_max$ is the largest eigenvalue (the sharpest curvature), as stated in Equation (2.29). The number of iterations then scales with the **condition number** $\kappa = \lambda_max/\lambda_min$: long, narrow valleys (large $\kappa$) force a small learning rate dictated by the steep direction, so progress along the shallow direction is slow. Figure 2.12 illustrates these regimes on a one-dimensional quadratic.

$$J(\theta) \approx J(\theta^*) + \tfrac{1}{2}(\theta - \theta^*)^\top H(\theta - \theta^*)$$
$$\text{convergence requires} \quad 0 < \eta < \frac{2}{\lambda_{\max}(H)} \tag{2.29}$$

where $H = \nabla^2 J(\theta)$ *is the Hessian matrix of second derivatives at the minimum* $\theta^*$.

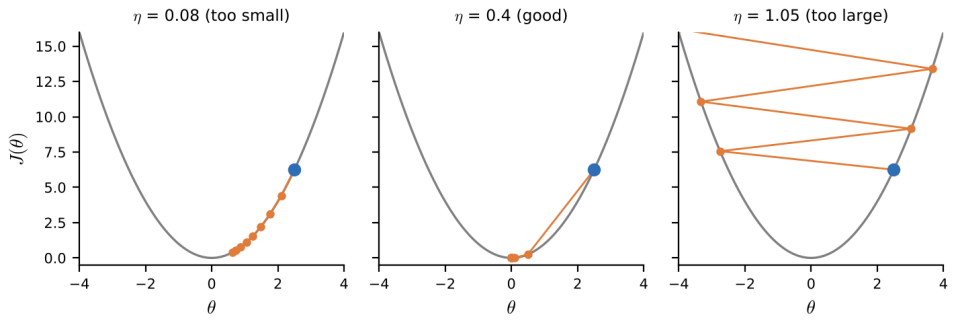



**Figure 2.12** Gradient descent on $J(\theta) = \theta^2$ (curvature 2, so the stability limit is $\eta < 1$) from $\theta_0 = 2.5$ with three learning rates: too small converges slowly, a good value converges quickly, and too large diverges.

## 2.12.2 The Geometry of Loss Landscapes

The loss surface of a deep network is a function over millions of dimensions, but several of its features are well understood. A **critical point** is any $\theta$ with $\nabla J(\theta) = 0$. Its nature is determined by the eigenvalues of the Hessian: all positive at a local minimum, all negative at a local maximum, and of mixed sign at a **saddle point**. Early worries focused on poor local minima, but theory and experiment (Dauphin et al., 2014; Choromanska et al., 2015) suggest that in high dimensions most critical points with high loss are saddle points: for a point to be a minimum, the curvature must be positive in *every*

one of millions of directions, which is exponentially unlikely unless the loss is already low. Local minima that do exist in large networks tend to have loss values close to the global minimum.

Saddle points and **plateaus**—large flat regions with tiny gradients—are therefore the main obstacles. Plain gradient descent slows down near a saddle because the gradient is small there, although it is unstable along negative-curvature directions and will eventually escape. The noise in stochastic gradients and the momentum of the optimisers described next help to escape much faster. Figure 2.13 contrasts a convex bowl, a saddle and a non-convex surface with several minima.

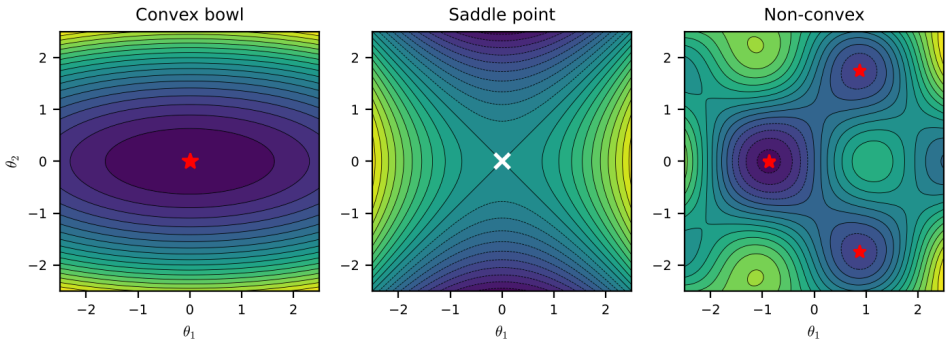



**Figure 2.13** Contour plots of three two-dimensional loss surfaces: a convex, ill-conditioned bowl (single global minimum), a saddle point (minimum along one axis, maximum along the other), and a non-convex surface with several local minima.

Other features matter too. **Sharp minima**, where the loss rises steeply in some direction, are thought to generalise worse than **flat minima**, where the loss stays low in a wide neighbourhood (Hochreiter and Schmidhuber, 1997; Keskar et al., 2017), because small shifts between the training and test distributions change the loss less in a flat basin. This is one explanation for the observation that small-batch SGD, whose noise prevents it from settling into narrow minima, often generalises better than large-batch training. Neural loss landscapes also have many symmetries—permuting hidden units, or rescaling weights across ReLU layers, gives the same function—so there are always many equivalent minima, often connected by low-loss paths (Garipov et al., 2018; Draxler et al., 2018).

### 2.12.3 Practical Optimisation Challenges

Beyond the landscape geometry, practical training must cope with **ill-conditioning** (curvature differs by orders of magnitude across directions, so no single learning rate suits all parameters), **noisy gradients** (estimated on mini-batches), **cliffs** (regions

**Table 2.11** Types of critical points and their effect on gradient descent.

| Critical point | Hessian eigenvalues | Behaviour of gradient descent | Prevalence in deep nets |
|---|---|---|---|
| Local / global minimum | All $\geq 0$ | Converges and stops | Many, mostly with near-optimal loss |
| Local maximum | All $\leq 0$ | Unstable; easily escaped | Rare |
| Saddle point | Mixed signs | Slows down; eventually escapes | Dominant at high loss |
| Plateau | Near zero | Very slow progress | Common, e.g. with saturated units |
| Sharp minimum | Some very large | Requires small $\eta$; may generalise poorly | Favoured by large batches |

where the loss changes abruptly, common in recurrent networks, which produce exploding gradients) and **poor initialisation** (which can place the network in a saturated or dead region). Each of the next sections addresses one or more of these: stochastic and mini-batch gradients for scale, momentum and adaptive methods for ill-conditioning, initialisation and normalisation for well-behaved signals, learning-rate schedules for convergence, and gradient clipping for cliffs.

> **Note** | **Why Not Second-Order Methods?**
>
> Newton's method, $\theta \leftarrow \theta - H^{-1}\nabla J$, rescales each direction by its curvature and solves ill-conditioning directly. But for d parameters the Hessian has $d^2$ entries—$10^{18}$ for a model with a billion parameters—and inverting it costs $O(d^3)$. Moreover, Newton's method is attracted to saddle points. Approximations such as L-BFGS, K-FAC and Shampoo exist, but first-order methods with adaptive per-parameter scaling (Adam and relatives) remain the default in deep learning.

## 2.13 Stochastic Gradient Descent, Mini-Batch Gradient Descent, and Batch Gradient Descent

The cost $J(\theta)$ is an average over all n training examples, so its exact gradient requires a pass over the whole dataset. With millions of examples this is wasteful: a step is taken only after every example has been seen, and nearby examples contribute almost identical gradient information. The three variants of gradient descent differ in how many examples are used to estimate the gradient for each update, and they trade the accuracy of each step against the number of steps per unit of computation.

### 2.13.1 Three Ways to Estimate the Gradient

**Batch gradient descent** (full-batch GD) uses all n examples per update, giving the exact gradient. **Stochastic gradient descent** (SGD) in its original sense uses a single randomly chosen example. **Mini-batch gradient descent** uses a random subset $\mathcal{B}$ of m examples, typically 32 to a few thousand. The three updates are written together in Equation (2.30). In modern usage, "SGD" almost always means the mini-batch version.

$$\begin{aligned} \text{Batch}: &\quad \theta \leftarrow \theta - \eta \tfrac{1}{n} \sum_{i=1}^{n} \nabla_\theta \mathcal{L}_i(\theta) \\ \text{Stochastic}: &\quad \theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_i(\theta), \qquad i \sim \text{Uniform}\{1, \dots, n\} \\ \text{Mini-batch}: &\quad \theta \leftarrow \theta - \eta \tfrac{1}{m} \sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta), \qquad |\mathcal{B}| = m \end{aligned} \tag{2.30}$$

where $\mathcal{L}_i$ is the loss on example i and $\mathcal{B}$ is a mini-batch sampled without replacement.

The mini-batch gradient is an **unbiased estimator** of the full gradient: its expectation over the random choice of $\mathcal{B}$ equals $\nabla J(\theta)$. Its variance, however, decreases only as $1/m$ (Equation (2.31)), so the standard error falls as $1/\sqrt{m}$. Using 100 times more examples per step reduces the noise only 10-fold while costing 100 times more computation. This diminishing return is the fundamental reason why small batches, which allow many more updates per epoch, are so efficient.

$$\begin{aligned} \mathbb{E}_{\mathcal{B}}[\mathbf{g}_{\mathcal{B}}] &= \nabla_\theta J(\theta) \\ \text{Var}[\mathbf{g}_{\mathcal{B}}] &\approx \tfrac{1}{m} \Sigma(\theta) \end{aligned} \tag{2.31}$$

where g_$\mathcal{B}$ is the mini-batch gradient and $\Sigma(\theta)$ the covariance of the per-example gradients.

**Algorithm 2.4** Mini-Batch Stochastic Gradient Descent

**Input:** Training set D of size n; batch size m; learning rate $\eta$ (or schedule $\eta_t$); number of epochs E; initial parameters $\theta_0$

**Output:** Trained parameters $\theta$

```
θ ← θ₀; t ← 0
for epoch = 1 to E do
    shuffle D randomly
    partition D into ⌈n/m⌉ mini-batches B_1, B_2, ...
    for each mini-batch B do
        g ← (1/|B|) Σ_{i∈B} ∇_θ L(f(x_i; θ), y_i)        ▷ forward + backward pass
        θ ← θ − η_t g
        t ← t + 1
```

9 **return** $\theta$

Algorithm 2.4 shows the standard implementation. One full pass over the shuffled training set is an **epoch**; one parameter update is an **iteration** or **step**. With $n = 50{,}000$ and $m = 100$ an epoch contains 500 iterations. Reshuffling every epoch prevents the model from seeing batches in a fixed order, which could bias the updates.

### 2.13.2 Comparing the Variants

Figure 2.14 shows the trajectories of the three methods on a two-parameter linear regression problem. Batch GD follows a smooth path along the steepest descent direction. SGD with single examples moves erratically, because each example's gradient points in a different direction, but it makes n updates per epoch and quickly reaches the vicinity of the minimum, where it keeps fluctuating unless the learning rate is reduced. Mini-batch GD is in between. Table 2.12 summarises the trade-offs.

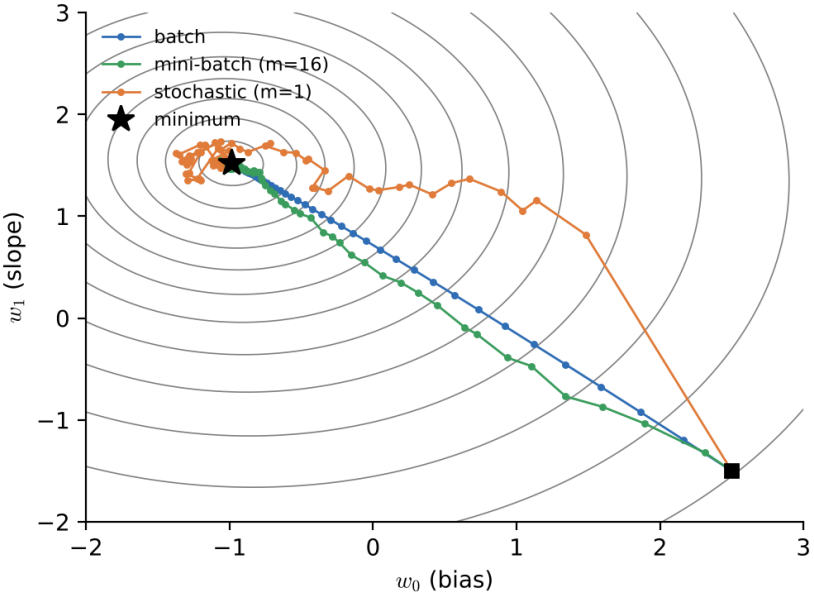



**Figure 2.14** Parameter trajectories of batch, mini-batch ($m = 16$) and stochastic ($m = 1$) gradient descent on a least-squares problem, each run for 60 updates from the same start; contours show the full training loss.

### 2.13.3 Choosing the Batch Size

The batch size interacts with hardware, learning rate and generalisation. Larger batches use the parallelism of GPUs more fully, up to the limit of device memory,

**Table 2.12** Comparison of batch, stochastic and mini-batch gradient descent.

| Property | Batch GD | Stochastic GD (m = 1) | Mini-batch GD |
|---|---|---|---|
| Examples per update | All n | 1 | m (e.g. 32–1024) |
| Updates per epoch | 1 | n | n/m |
| Gradient noise | None | Very high | Moderate, controlled by m |
| Hardware efficiency | Good, but memory-bound for large n | Poor (no parallelism) | Excellent on GPUs |
| Convergence | Smooth, deterministic | Noisy; needs decaying $\eta$ | Fast and reasonably stable |
| Escaping saddles / sharp minima | Weak | Strong | Good |
| Typical use | Small datasets, convex problems | Online / streaming learning | Default for deep learning |

and reduce wall-clock time per epoch. But beyond a **critical batch size** the reduction in gradient noise no longer yields proportionally fewer steps, so total computation increases. Larger batches also tend to converge to sharper minima that can generalise slightly worse (Keskar et al., 2017), although careful tuning largely closes this gap. A widely used heuristic is the **linear scaling rule** (Goyal et al., 2017): when the batch size is multiplied by k, multiply the learning rate by k as well, combined with a warm-up period (see the Section on Learning Rate Scheduling). Powers of two (32, 64, 128, 256) are conventional because they align with memory layouts, though the benefit is modest.

> **T Practice Tip | Practical Tip: Batch Size Defaults**
>
> - Start with m = 32–256 for most problems; use the largest size that fits in memory if training time is critical, and scale $\eta$ accordingly.
> - If the batch size is limited by memory, **gradient accumulation** sums gradients over several small batches before each update, emulating a larger batch.
> - Batch normalisation behaves poorly with very small batches ($m < 8$); use group or layer normalisation instead (see the Section on Dropout and Batch Normalization).

> **N Note | Data Loading Matters**
>
> Mini-batch training is only fast if the next batch is ready when the GPU needs it. Frameworks provide asynchronous loaders (`torch.utils.data.DataLoader` with `num_workers > 0`, `tf.data` with `prefetch`) that read, decode and augment data in parallel worker processes. An under-utilised GPU is very often caused by a slow input pipeline rather than by the model.

## 2.14 Optimizers: SGD, Momentum, RMSprop, and Adam

Plain (mini-batch) SGD uses the same learning rate for every parameter and takes no account of past gradients. On ill-conditioned loss surfaces—long narrow valleys with steep walls—it oscillates across the valley while creeping slowly along its floor. Modern **optimisers** modify the update in two complementary ways: **momentum** accumulates a running average of gradients to damp oscillations and accelerate along consistent directions, and **adaptive learning rates** rescale each parameter's step by an estimate of its gradient magnitude. Adam combines both. In what follows, $g_t = \nabla_\theta J(\theta_t)$ denotes the mini-batch gradient at step t, and all operations on vectors are element-wise.

### 2.14.1 SGD with Momentum and Nesterov Acceleration

**Momentum** (Polyak, 1964) maintains a velocity vector v that is an exponentially decaying sum of past gradients and moves the parameters along it (Equation (2.32)). A useful physical analogy is a heavy ball rolling down the loss surface: components of the gradient that keep the same sign accumulate, while components that flip sign from step to step (oscillations across a valley) cancel. With a momentum coefficient $\beta$, a constant gradient g produces a terminal velocity of $g/(1-\beta)$; for $\beta = 0.9$ the effective step is ten times larger along consistent directions.

$$\begin{aligned} \mathbf{v}_t &= \beta\,\mathbf{v}_{t-1} + \mathbf{g}_t \\ \theta_{t+1} &= \theta_t - \eta\,\mathbf{v}_t \end{aligned} \tag{2.32}$$

where $v_t$ is the velocity ($v_0 = 0$) and $\beta \in [0, 1)$ is the momentum coefficient, typically 0.9.

**Nesterov accelerated gradient** (NAG; Nesterov, 1983, introduced to deep learning by Sutskever et al., 2013) evaluates the gradient not at the current position but at the look-ahead point the momentum is about to carry us to, $\theta_t - \eta\beta v_{t-1}$ (Equation (2.33)). This anticipatory correction reduces overshooting and often converges slightly faster than classical momentum.

$$\begin{aligned} \mathbf{v}_t &= \beta\,\mathbf{v}_{t-1} + \nabla_\theta J(\theta_t - \eta\beta\,\mathbf{v}_{t-1}) \\ \theta_{t+1} &= \theta_t - \eta\,\mathbf{v}_t \end{aligned} \tag{2.33}$$

### 2.14.2 Adaptive Learning Rates: AdaGrad and RMSprop

**AdaGrad** (Duchi, Hazan and Singer, 2011) gives each parameter its own effective learning rate, inversely proportional to the square root of the sum of its past squared gradients (Equation (2.34)). Parameters with large or frequent gradients take small

steps; rarely updated parameters, such as embeddings of rare words, take large steps. Its weakness is that the accumulator only grows, so the effective learning rate decays monotonically towards zero and training can stall.

$$\begin{aligned} \mathbf{r}_t &= \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\mathbf{r}_t + \epsilon}} \odot \mathbf{g}_t \end{aligned} \tag{2.34}$$

where $\mathbf{r}_t$ accumulates squared gradients and $\varepsilon \approx 10^{-8}$ prevents division by zero.

**RMSprop** (proposed by Geoffrey Hinton in his 2012 Coursera lectures) fixes this by replacing the sum with an exponential moving average, so that the scaling reflects only recent gradient magnitudes (Equation (2.35)). Dividing by the root-mean-square gradient equalises step sizes across directions of very different curvature, which counteracts ill-conditioning.

$$\begin{aligned} \mathbf{s}_t &= \rho\, \mathbf{s}_{t-1} + (1-\rho)\, \mathbf{g}_t \odot \mathbf{g}_t \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t \end{aligned} \tag{2.35}$$

where $\mathbf{s}_t$ is the moving average of squared gradients and $\rho \approx 0.9$ is its decay rate.

### 2.14.3 Adam and AdamW

**Adam** (adaptive moment estimation; Kingma and Ba, 2015) combines momentum with RMSprop-style scaling. It keeps exponential moving averages of the gradient (the first moment, $\mathbf{m}$) and of the squared gradient (the second moment, $\mathbf{v}$). Because both averages start at zero, they are biased towards zero in early iterations; Adam divides by $(1-\beta^t)$ to correct this bias. The full update is Equation (2.36) and the algorithm is given in Algorithm 2.5. The default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ and $\eta = 10^{-3}$ work well across a wide range of problems, which is a major reason for Adam's popularity.

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\, \mathbf{g}_t, \qquad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\, \mathbf{g}_t \odot \mathbf{g}_t \\ \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \\ \theta_{t+1} &= \theta_t - \eta\, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}} \end{aligned} \tag{2.36}$$

where $\mathbf{m}_t$ and $\mathbf{v}_t$ are the first- and second-moment estimates, $\beta_1$ and $\beta_2$ their decay rates, and $\hat{\mathbf{m}}_t$, $\hat{\mathbf{v}}_t$ the bias-corrected estimates.

**Algorithm 2.5** Adam Optimiser

**Input:** Objective $J(\theta)$; learning rate $\eta$ (default $10^{-3}$); decay rates $\beta_1 = 0.9$, $\beta_2 = 0.999$; $\varepsilon = 10^{-8}$; weight decay $\lambda$ ($0$ for Adam, $> 0$ for AdamW); initial $\theta_0$

**Output:** Optimised parameters $\theta$

$m_0 \leftarrow 0$, $v_0 \leftarrow 0$, $t \leftarrow 0$
**while not** converged **do**
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla_\theta J(\theta_{t-1})$ on a mini-batch
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$ ▷ *first moment (momentum)*
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)\, g_t \odot g_t$ ▷ *second moment*
  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$; $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ ▷ *bias correction*
  $\theta_t \leftarrow \theta_{t-1} - \eta \left( \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda\, \theta_{t-1} \right)$ ▷ *$\lambda$ term only **in** AdamW*
**return** $\theta_t$

A subtle issue concerns weight decay (L2 regularisation, see the Section on Regularization). In plain SGD, adding $\lambda\|\theta\|^2/2$ to the loss is equivalent to shrinking the weights by a factor $(1 - \eta\lambda)$ each step. In Adam, however, the L2 gradient $\lambda\theta$ is divided by $\sqrt{\hat{v}_t}$ along with the data gradient, so parameters with large gradients are regularised less. **AdamW** (Loshchilov and Hutter, 2019) **decouples** weight decay from the adaptive scaling by applying it directly to the weights, as in the last line of Algorithm 2.5. AdamW generalises better and is the standard optimiser for transformers and large language models.

**E Worked Example 2.8 | The First Adam Step**

Take a single parameter with gradient $g_1 = 0.2$ at $t = 1$, $\eta = 0.001$ and default $\beta$'s.

- $m_1 = 0.1 \times 0.2 = 0.02$; $v_1 = 0.001 \times 0.04 = 0.00004$.
- Bias correction: $\hat{m}_1 = 0.02/(1 - 0.9) = 0.2$; $\hat{v}_1 = 0.00004/(1 - 0.999) = 0.04$.
- Step: $\Delta\theta = -0.001 \times 0.2/(\sqrt{0.04} + 10^{-8}) = -0.001 \times 0.2/0.2 = -0.001$.

The first step has magnitude $\approx \eta$ regardless of the gradient's scale: Adam's step size is approximately invariant to rescaling of the gradient. Without bias correction the step would be $0.001 \times 0.02/0.0063 = 0.0032$, distorted by the zero initialisation.

Figure 2.15 compares the trajectories of SGD, momentum, RMSprop and Adam on an ill-conditioned quadratic. Plain SGD zig-zags across the narrow valley; momentum overshoots but accelerates along the floor; RMSprop and Adam rescale the axes and head almost directly towards the minimum.

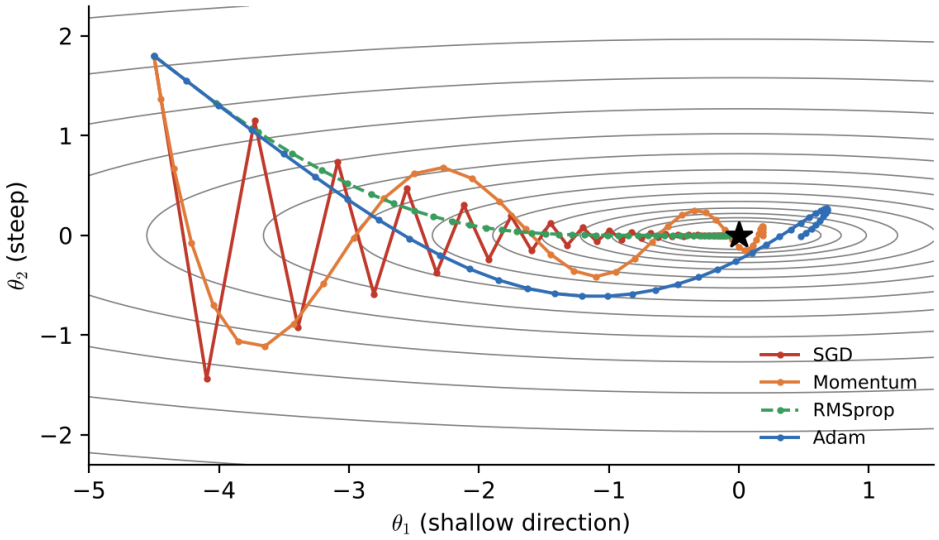



**Figure 2.15** Trajectories of four optimisers for 40 steps on the ill-conditioned quadratic $J = 0.5(0.1\theta_1^2 + 2\theta_2^2)$ from $\theta_0 = (-4.5, 1.8)$; SGD oscillates along the steep axis, momentum accelerates along the shallow axis, and the adaptive methods rescale both.

**Table 2.13** Comparison of gradient-based optimisers.

| Optimiser | Key idea | Typical hyperparameters | Strengths | Weaknesses |
|---|---|---|---|---|
| SGD | Step along $-g$ | $\eta$ | Simple; good generalisation | Slow on ill-conditioned problems; $\eta$ sensitive |
| SGD + momentum / Nesterov | Velocity accumulates gradients | $\eta \approx 0.01$–$0.1$, $\beta = 0.9$ | Fast; strong results in vision | Needs $\eta$ schedule and tuning |
| AdaGrad | Per-parameter $\eta \propto 1/\sqrt{\Sigma g^2}$ | $\eta \approx 0.01$ | Good for sparse features | Learning rate decays to zero |
| RMSprop | $\eta \propto 1/\text{RMS}$ of recent g | $\eta \approx 10^{-3}$, $\rho = 0.9$ | Handles non-stationarity; RNNs | No momentum or bias correction |
| Adam | Momentum + RMSprop + bias correction | $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ | Robust defaults; fast progress | Coupled L2; sometimes generalises worse |
| AdamW | Adam with decoupled weight decay | As Adam, $\lambda \approx 0.01$–$0.1$ | Standard for transformers | Extra hyperparameter $\lambda$ |

### 2.14.4 Choosing an Optimiser

Table 2.13 summarises the main options. In practice, Adam or AdamW with default settings is the fastest way to obtain a working model and is the standard for transformers. For convolutional networks on image classification, well-tuned SGD with momentum and a learning-rate schedule often achieves slightly better final accuracy (Wilson et al., 2017), so it remains common in that setting. Whatever the optimiser, the learning rate and its schedule matter more than the choice between adaptive methods. Listing 2.7 implements Adam from scratch, and Listing 2.8 shows the corresponding PyTorch and Keras calls.

**Listing 2.7** The Adam update implemented from scratch in NumPy.

```python
import numpy as np

class Adam:
    def __init__(self, params, lr=1e-3, b1=0.9, b2=0.999, eps=1e-8, wd=0.0):
        self.p, self.lr, self.b1, self.b2, self.eps, self.wd = params, lr, b1, b2, eps, wd
        self.m = [np.zeros_like(w) for w in params]
        self.v = [np.zeros_like(w) for w in params]
        self.t = 0

    def step(self, grads):
        self.t += 1
        for w, g, m, v in zip(self.p, grads, self.m, self.v):
            m[:] = self.b1 * m + (1 - self.b1) * g
            v[:] = self.b2 * v + (1 - self.b2) * g * g
            m_hat = m / (1 - self.b1 ** self.t)
            v_hat = v / (1 - self.b2 ** self.t)
            w -= self.lr * (m_hat / (np.sqrt(v_hat) + self.eps) + self.wd * w)  # AdamW if wd > 0

# minimise f(w) = (w0 - 3)^2 + 10 (w1 + 1)^2
w = np.array([0.0, 0.0]); opt = Adam([w], lr=0.1)
for _ in range(300):
    opt.step([np.array([2 * (w[0] - 3), 20 * (w[1] + 1)])])
print(w.round(3))   # approximately [ 3. -1.]
```

**Listing 2.8** Creating optimisers in PyTorch and Keras.

```python
import torch
from torch import nn
model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 1))

sgd     = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9, nesterov=True)
rmsprop = torch.optim.RMSprop(model.parameters(), lr=1e-3, alpha=0.9)
adam    = torch.optim.Adam(model.parameters(), lr=1e-3, betas=(0.9, 0.999))
adamw   = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)

# the canonical update step (identical for every optimiser)
```

```
x, y = torch.randn(32, 20), torch.randn(32, 1)
adamw.zero_grad()
loss = nn.functional.mse_loss(model(x), y)
loss.backward()
adamw.step()

# Keras equivalents
from tensorflow import keras
keras.optimizers.SGD(learning_rate=0.05, momentum=0.9, nesterov=True)
keras.optimizers.AdamW(learning_rate=1e-3, weight_decay=0.01)
```

**Key Point | Key Takeaways**

- Momentum smooths noisy gradients and accelerates progress along consistent directions.
- Adaptive methods (AdaGrad, RMSprop, Adam) scale each parameter's step by its recent gradient magnitude, counteracting ill-conditioning.
- Adam combines both with bias correction; AdamW decouples weight decay and is the default for transformers, while SGD with momentum remains strong for CNNs.

# 2.15 Weight Initialization Techniques

Before training begins, every weight must be given a value. This choice is surprisingly consequential. Because deep networks multiply signals by many weight matrices in succession, a poor initial scale makes activations and gradients either shrink to zero or blow up exponentially with depth, and training never gets started. Good initialisation keeps the variance of signals roughly constant from layer to layer in both the forward and backward passes.

## 2.15.1 Why Not Zeros? Symmetry Breaking

Initialising all weights to zero (or to any identical constant) is fatal for hidden layers. Every unit in a layer then computes the same output, receives the same gradient, and receives the same update, so the units remain identical forever: the layer effectively has one unit. Random initialisation **breaks the symmetry** so that different units can learn different features. Biases, by contrast, can safely be initialised to zero, because the random weights already break symmetry. (A small positive bias such as 0.01 is sometimes used with ReLU to make units initially active.)

Random values must still have the right scale. Drawing weights from $\mathrm{N}(0, 1)$ in a layer with 1,000 inputs gives pre-activations with standard deviation about $\sqrt{1000} \approx 32$, which saturates sigmoids and tanh units immediately; drawing them from $\mathrm{N}(0,$

$0.01^2$) makes activations shrink by a factor of about 0.3 per layer, vanishing after a few layers. Figure 2.16 demonstrates both failure modes.

### 2.15.2 Variance-Preserving Initialisation

Consider a unit $z_j = \Sigma_i W_{ji} a_i$ with $n_{in}$ inputs, where weights and inputs are independent with zero mean. Then $\mathrm{Var}(z_j) = n_{in} \cdot \mathrm{Var}(W) \cdot \mathrm{E}[a_i^2]$. To keep the variance unchanged from layer to layer we need $n_{in} \cdot \mathrm{Var}(W) \approx 1$ for the forward pass; the analogous argument for backpropagated gradients requires $n_{out} \cdot \mathrm{Var}(W) \approx 1$. **Xavier (Glorot) initialisation** (Glorot and Bengio, 2010) compromises between the two by using the average fan, as in Equation (2.37). It was derived for symmetric activations that are approximately linear near zero, such as tanh.

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$
$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \quad \text{or} \quad W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}},\ \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right) \tag{2.37}$$

where $n_{in}$ and $n_{out}$ are the numbers of inputs (fan-in) and outputs (fan-out) of the layer.

ReLU sets roughly half of its inputs to zero, halving the second moment of the activations. **He (Kaiming) initialisation** (He, Zhang, Ren and Sun, 2015) compensates by doubling the variance (Equation (2.38)). With it, He et al. trained 30-layer plain ReLU networks from scratch that failed to converge with Xavier initialisation.

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}}}, \qquad W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}}\right) \tag{2.38}$$

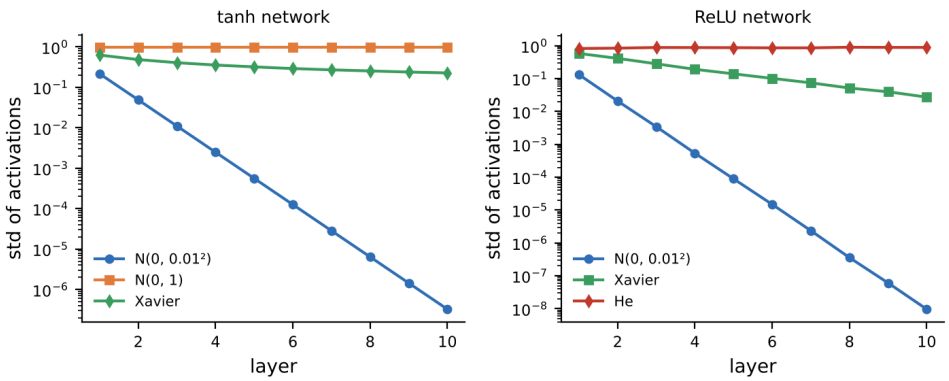



**Figure 2.16** Standard deviation of activations across ten 512-unit layers for different initialisations: with tanh, a too-small scale vanishes and N(0,1) saturates while Xavier stays stable; with ReLU, Xavier slowly decays while He initialisation preserves the scale.

**Table 2.14** Weight initialisation schemes and when to use them.

| Scheme | Distribution | Variance | Suited to |
|---|---|---|---|
| Zeros / constant | $W = c$ | 0 | Biases only (never weights) |
| Small random | $N(0, 0.01^2)$ | $10^{-4}$ | Shallow nets only; vanishes in deep nets |
| Xavier / Glorot | Normal or uniform | $2/(n_{in} + n_{out})$ | tanh, sigmoid, linear, softmax layers |
| He / Kaiming | Normal or uniform | $2/n_{in}$ | ReLU, Leaky ReLU (with adjusted gain) |
| LeCun | Normal | $1/n_{in}$ | SELU, self-normalising networks |
| Orthogonal | Random orthogonal matrix × gain | Preserves norms exactly | RNNs, very deep linear-like nets |

## 2.15.3 Practice and Beyond

Frameworks apply sensible defaults: Keras `Dense` layers use Glorot uniform, and PyTorch `nn.Linear` uses a Kaiming-uniform variant. When using ReLU networks without normalisation, explicitly selecting He initialisation is good practice. Normalisation layers (see the Section on Dropout and Batch Normalization) and residual connections reduce, but do not remove, sensitivity to initialisation. Some very deep residual networks scale the last layer of each residual branch towards zero at initialisation, so that every block starts as the identity (Goyal et al., 2017; Zhang, Dauphin and Ma, 2019). Finally, the most powerful initialisation of all is a **pre-trained model**: starting from weights learned on a large related dataset, discussed under transfer learning later in this chapter.

**Listing 2.9** Applying explicit initialisation schemes in PyTorch.

```
import torch
from torch import nn

def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity="relu")   # He initialisation
        nn.init.zeros_(m.bias)

model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(),
                      nn.Linear(256, 256), nn.ReLU(),
                      nn.Linear(256, 10))
model.apply(init_weights)

# for a tanh layer use Xavier instead:
layer = nn.Linear(256, 128)
nn.init.xavier_uniform_(layer.weight, gain=nn.init.calculate_gain("tanh"))
print(model[0].weight.std().item(), (2 / 784) ** 0.5)   # both about 0.0505
```

**Practice Tip | Practical Tip: Diagnosing Initialisation Problems**

Before training, push one mini-batch through the network and print the mean and standard deviation of activations in each layer, and of gradients after one backward pass. Standard deviations should be of similar order across layers. Values collapsing towards 0 or growing by orders of magnitude indicate a scale problem. For classification, the initial loss should be close to log K (e.g. 2.30 for 10 classes), corresponding to uniform predictions.

# 2.16 Learning Rate Scheduling

A constant learning rate is rarely optimal. Early in training, large steps make rapid progress and help explore the loss landscape; later, the noise in mini-batch gradients causes the parameters to bounce around the minimum, and smaller steps are needed to settle into it. Theory for SGD on convex problems requires the learning rate to decay such that $\Sigma_t\ \eta_t = \infty$ and $\Sigma_t\ \eta_t^2 < \infty$ (the Robbins–Monro conditions). A **learning-rate schedule** specifies $\eta_t$ as a function of the iteration or epoch. Schedules typically improve final accuracy by a noticeable margin at no extra cost, and are used with every optimiser, including Adam.

## 2.16.1 Decay Schedules

The classical schedules, written as functions of the step t with initial rate $\eta_0$, are collected in Equation (2.39). **Step decay** divides the learning rate by a constant factor (often 10) at fixed milestones, for example at epochs 30, 60 and 90 when training ResNets on ImageNet. **Exponential decay** multiplies it by $\gamma < 1$ every step or epoch. **Cosine annealing** (Loshchilov and Hutter, 2017) follows half a cosine wave from $\eta_{\max}$ to $\eta_{\min}$, decaying slowly at first, fastest in the middle and slowly again at the end; it has few hyperparameters and is a strong default.

$$\begin{aligned} \text{Step}: &\quad \eta_t = \eta_0\,\gamma^{\lfloor t/s \rfloor} \\ \text{Exponential}: &\quad \eta_t = \eta_0\,e^{-kt} \\ \text{Cosine}: &\quad \eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\tfrac{\pi t}{T}\right) \end{aligned} \tag{2.39}$$

where $\gamma$ is the decay factor, s the step interval, k the decay rate, T the total number of steps, and $\eta_{\max}$, $\eta_{\min}$ the maximum and minimum rates.

**Reduce-on-plateau** is an adaptive alternative: the learning rate is reduced by a factor whenever the validation loss has not improved for a given number of epochs (the patience). It requires no advance knowledge of training length, but makes the schedule depend on noisy validation measurements.

### 2.16.2 Warm-up and the One-Cycle Policy

At the start of training the weights are random, gradients can be large and erratic, and adaptive optimisers' second-moment estimates are based on very few samples. Using the full learning rate immediately can destabilise training, especially with large batches or transformers. **Warm-up** increases the learning rate linearly from near zero to $\eta_{\max}$ over the first $T_w$ steps (typically a few hundred to a few thousand), as in Equation (2.40), after which a decay schedule takes over. Linear warm-up followed by cosine or linear decay is the standard schedule for training transformers.

$$\eta_t = \eta_{\max} \, \frac{t}{T_w} \quad \text{for} \;\; t \le T_w, \qquad \text{then decay (e.g. cosine) for} \;\; t > T_w \tag{2.40}$$

where $T_w$ is the number of warm-up steps.

The **one-cycle policy** (Smith, 2018; Smith and Topin, 2019) raises the learning rate from a low value to a high maximum over roughly the first 30% of training and then anneals it to a value far below the starting point, while moving momentum in the opposite direction. The high-learning-rate phase acts as a regulariser and enables so-called super-convergence: reaching good accuracy in far fewer epochs. The maximum is usually chosen with a **learning-rate range test**, which trains briefly while increasing $\eta$ exponentially and picks a value somewhat below the point where the loss starts to diverge. Figure 2.17 plots these schedules and Algorithm 2.6 gives the warm-up plus cosine schedule.

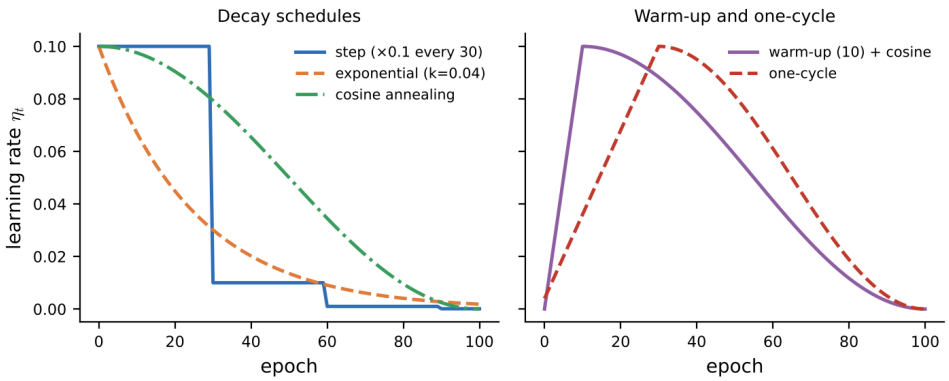



**Figure 2.17** Common learning-rate schedules over 100 epochs with maximum rate 0.1: step decay, exponential decay, cosine annealing, linear warm-up followed by cosine decay, and the one-cycle policy.

**Algorithm 2.6** Linear Warm-up with Cosine Decay

**Input:** Current step $t$; total steps $T$; warm-up steps $T_w$; peak rate $\eta_{max}$; final rate $\eta_{min}$
**Output:** Learning rate $\eta_t$ for step $t$

**if** $t < T_w$ **then**
  **return** $\eta_{max} \cdot (t + 1) \,/\, T_w$
progress $\leftarrow (t - T_w) \,/\, (T - T_w)$ ▷ *fraction of decay phase completed,* ***in*** $[0, 1]$
**return** $\eta_{min} + \tfrac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi \cdot \text{progress}))$

## 2.16.3 Schedules in Practice

**Table 2.15** Learning-rate schedules and their typical uses.

| Schedule | Hyperparameters | Advantages | Typical use |
|---|---|---|---|
| Constant | $\eta$ | Simplest | Quick experiments, fine-tuning small heads |
| Step decay | $\eta_0$, $\gamma$, milestones | Simple; well-studied | CNNs with SGD (e.g. ResNet) |
| Exponential | $\eta_0$, $k$ | Smooth decay | Older CNN/RNN recipes |
| Cosine annealing | $\eta_{max}$, $\eta_{min}$, $T$ | Few settings; strong default | Vision and general use |
| Warm-up + cosine/-linear | $T_w$, $\eta_{max}$, $T$ | Stabilises early training | Transformers, large batches |
| One-cycle | $\eta_{max}$, cycle length | Fast convergence; regularising | Fixed-budget training |
| Reduce on plateau | factor, patience | No need to know $T$ | Unknown training length |

Table 2.15 summarises the options. Listing 2.10 shows how schedulers are attached to an optimiser in PyTorch; note that some schedulers are stepped once per epoch and others once per mini-batch, a frequent source of bugs.

**Listing 2.10** Learning-rate schedulers in PyTorch.

```python
import math, torch
from torch import nn
model = nn.Linear(10, 1)
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)

# (a) cosine annealing over 100 epochs  -> call sched.step() once per epoch
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=100, eta_min=1e-4)

# (b) one-cycle over all mini-batches  -> call sched.step() after every batch
# sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=0.1, total_steps=100 * 390)

# (c) linear warm-up + cosine decay with LambdaLR (per batch)
T, Tw = 10_000, 500
lam = lambda t: (t + 1) / Tw if t < Tw else 0.5 * (1 + math.cos(math.pi * (t - Tw) / (T - Tw)))
# sched = torch.optim.lr_scheduler.LambdaLR(opt, lr_lambda=lam)
```

```

for epoch in range(100):
    # ... loop over mini-batches: forward, backward, opt.step() ...
    sched.step()
print(opt.param_groups[0]["lr"])   # 1e-4 at the end of cosine annealing
```

**Practice Tip** | **Practical Tip: Finding the Peak Learning Rate**

Run a learning-rate range test for a few hundred iterations, increasing $\eta$ exponentially from $10^{-7}$ to 10. Plot the (smoothed) loss against $\eta$ on a log axis. Choose $\eta_{\max}$ about one order of magnitude below the value at which the loss is lowest, or just before it starts to rise steeply. Then use warm-up plus cosine decay, or one-cycle, with that peak.

## 2.17 Vanishing and Exploding Gradients

Backpropagation multiplies the error signal by a weight matrix and an activation derivative at every layer it passes through. In a deep network, or in a recurrent network unrolled over many time steps, these repeated products can shrink the gradient exponentially—the **vanishing gradient** problem—or grow it exponentially—the **exploding gradient** problem. Identified by Hochreiter (1991) and Bengio, Simard and Frasconi (1994), these problems were among the main reasons deep networks were considered untrainable before 2006, and most of the techniques in this part of the chapter exist partly to address them.

### 2.17.1 Mathematical Origin

Unrolling the backpropagation recursion of Equation (2.25) from the output down to layer $l$ gives Equation (2.41). The gradient reaching layer $l$ is the output error multiplied by a product of $L-l$ Jacobian matrices, each of the form $\operatorname{diag}(\varphi'(z^{(k)}))\,W^{(k+1)\mathrm{T}}$.

$$\begin{aligned} \boldsymbol{\delta}^{(l)} &= \left[\prod_{k=l}^{L-1} \operatorname{diag}\big(\phi'(\mathbf{z}^{(k)})\big) W^{(k+1)\,\top}\right] \boldsymbol{\delta}^{(L)} \\ \|\boldsymbol{\delta}^{(l)}\| &\le \prod_{k=l}^{L-1} \big(\max|\phi'| \cdot \|W^{(k+1)}\|\big)\, \|\boldsymbol{\delta}^{(L)}\| \end{aligned} \tag{2.41}$$

where $\|W\|$ is the spectral norm (largest singular value) of the weight matrix.

If each factor $\max|\varphi'|\cdot\|W\|$ is below 1, the bound decays geometrically with depth; if the factors are consistently above 1, the gradient can grow geometrically. For a sig-

moid network, $|\sigma'| \le 0.25$, so even with $\|W\| = 2$ the factor is 0.5 and after 20 layers the gradient is reduced by roughly $0.5^{20} \approx 10^{-6}$. The early layers then learn a million times more slowly than the last layers. Recurrent networks are even more exposed, because the *same* matrix is multiplied at every time step: its largest eigenvalue being slightly below or above 1 decides between vanishing and exploding over long sequences. Figure 2.18 shows measured gradient norms per layer in a 20-layer network.

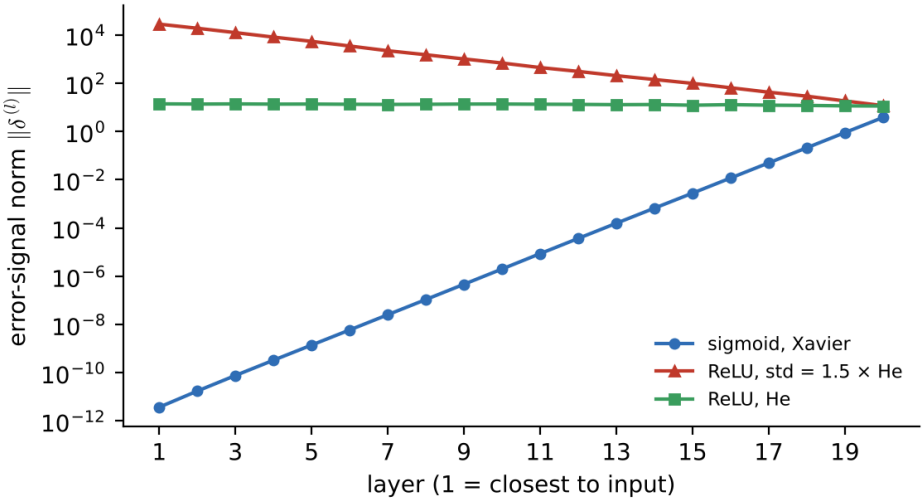



**Figure 2.18** Norm of the backpropagated error signal $\delta^{(1)}$ at each layer of a 20-layer, 256-unit MLP after one backward pass: with sigmoid and Xavier initialisation the gradient vanishes towards the input; with ReLU and an oversized weight scale it explodes; with ReLU and He initialisation it remains roughly constant.

### 2.17.2 Symptoms and Diagnosis

Vanishing gradients manifest as a training loss that plateaus early at a high value, early layers whose weights barely change, and saturated activations (sigmoid outputs stuck near 0 or 1, or a large fraction of ReLUs that are always zero). Exploding gradients manifest as sudden spikes in the loss, weights or loss becoming `NaN` or `inf`, and wildly fluctuating updates. Both are easy to detect by logging the gradient norm of each layer during training, for instance with TensorBoard or Weights & Biases histograms.

### 2.17.3 Remedies

No single fix exists; instead, modern architectures combine several measures, summarised in Table 2.16. Non-saturating activations (ReLU, GELU) keep $\varphi'$ near 1 for active units. Variance-preserving initialisation (Xavier, He) sets $\|W\|$ so that each factor

in Equation (2.41) is close to 1 on average. Normalisation layers re-standardise activations at every layer. **Residual connections** change the Jacobian of each block from $\partial F/\partial a$ to $I + \partial F/\partial a$, so the identity term provides a direct path along which gradients flow undiminished—the key to training networks with hundreds of layers. In recurrent networks, the gated cells of LSTMs and GRUs serve a similar role (see the Section on Recurrent Neural Networks).

For exploding gradients the standard remedy is **gradient clipping** (Pascanu, Mikolov and Bengio, 2013). Clipping by global norm rescales the full gradient vector whenever its norm exceeds a threshold c, preserving its direction while bounding the step (Equation (2.42)). It is routinely used when training RNNs and transformers, with c typically between 0.5 and 5.

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1,\ \frac{c}{\|\mathbf{g}\|}\right) \tag{2.42}$$

where g is the concatenation of all parameter gradients and $c > 0$ the clipping threshold.

**Table 2.16** Remedies for vanishing and exploding gradients.

| Technique | Addresses | Mechanism |
|---|---|---|
| ReLU / GELU activations | Vanishing | Derivative $\approx 1$ for active units, no saturation |
| Xavier / He initialisation | Both | Keeps per-layer gain near 1 at initialisation |
| Batch / layer normalisation | Both | Re-standardises activations at every layer |
| Residual (skip) connections | Vanishing | Identity path for gradients through each block |
| LSTM / GRU gating | Vanishing (RNNs) | Additive cell-state updates controlled by gates |
| Gradient clipping | Exploding | Caps the norm of the update |
| Smaller learning rate / warm-up | Exploding | Limits the size of early updates |
| Weight decay | Exploding | Keeps weight norms bounded |

**Listing 2.11** Monitoring gradient norms and applying gradient clipping in PyTorch.

```
import torch
from torch import nn

model = nn.LSTM(input_size=32, hidden_size=128, num_layers=2, batch_first=True)
head = nn.Linear(128, 1)
params = list(model.parameters()) + list(head.parameters())
opt = torch.optim.Adam(params, lr=1e-3)

x, y = torch.randn(16, 200, 32), torch.randn(16, 1)    # long sequences
out, _ = model(x)
loss = nn.functional.mse_loss(head(out[:, -1]), y)
opt.zero_grad(); loss.backward()
```

```

for name, p in model.named_parameters():                # per-layer diagnostics
    print(f"{name:20s} grad norm = {p.grad.norm():.2e}")

total = torch.nn.utils.clip_grad_norm_(params, max_norm=1.0)   # returns norm before clipping
print("global norm before clipping:", float(total))
opt.step()
```

**Case Study 2.2 | Residual Networks**

Before 2015, adding layers to plain CNNs beyond about 20 made even the *training* error worse—not overfitting, but an optimisation failure partly attributable to poor gradient flow. He et al. (2016) reformulated each block to learn a residual F(a) added to its input. A 152-layer ResNet won ILSVRC 2015 with 3.57% top-5 error, and 1,000-layer variants were trained on CIFAR-10. Residual connections are now present in virtually every deep architecture, including transformers.

## 2.18 Regularization in Deep Learning

Deep networks frequently have more parameters than training examples, so they can memorise the training set—Zhang et al. (2017) showed that standard image classifiers can fit even randomly assigned labels perfectly. **Regularisation** is any modification of the learning procedure intended to reduce generalisation error without reducing training error correspondingly. Chapter 1 introduced regularisation for linear models; this section adapts those ideas to deep networks and adds methods specific to them. Dropout and normalisation layers, which also have a regularising effect, are treated in the next section.

### 2.18.1 Parameter Norm Penalties and Weight Decay

The most common explicit regulariser adds a penalty on the size of the weights to the cost (Equation (2.43)). **L2 regularisation** penalises the squared Euclidean norm, encouraging many small weights and smoother functions; it corresponds to a Gaussian prior on the weights in a Bayesian interpretation. **L1 regularisation** penalises absolute values and drives many weights exactly to zero, producing sparse networks; it corresponds to a Laplace prior. Biases and normalisation parameters are usually excluded from the penalty.

$$\tilde{J}(\theta) = J(\theta) + \tfrac{\lambda}{2}\sum_{l}\|W^{(l)}\|_F^2 \qquad \text{(L2)}$$
$$\tilde{J}(\theta) = J(\theta) + \lambda\sum_{l}\sum_{i,j}\left|W_{ij}^{(l)}\right| \qquad \text{(L1)} \tag{2.43}$$

where $\lambda \geq 0$ controls the strength of regularisation and $\|\cdot\|_F$ is the Frobenius norm (square root of the sum of squared entries).

Differentiating the L2 penalty adds $\lambda W$ to the gradient, so the SGD update becomes Equation (2.44): before the usual gradient step, every weight is multiplied by $(1 - \eta\lambda)$, shrinking or *decaying* towards zero. This is why L2 regularisation is called **weight decay**. As explained in the Section on Optimizers, the two are equivalent for SGD but not for adaptive methods, where decoupled weight decay (AdamW) should be used. Typical values are $\lambda = 10^{-5}$ to $10^{-3}$ for L2 with SGD and 0.01 to 0.1 for AdamW's decoupled decay.

$$W \leftarrow W - \eta(\nabla_W J + \lambda W) = (1 - \eta\lambda)\, W - \eta\, \nabla_W J \tag{2.44}$$

### 2.18.2 Early Stopping

As training proceeds, the training loss keeps decreasing, but the validation loss typically decreases and then starts to rise as the network begins to overfit, as shown in Figure 2.19. **Early stopping** monitors the validation loss after every epoch, saves the parameters whenever it reaches a new minimum, and halts training when no improvement has occurred for a specified number of epochs (the **patience**). The saved best parameters are returned (Algorithm 2.7). Early stopping is cheap, effective and nearly universal. For a quadratic loss and gradient descent, Bishop (1995) and Goodfellow, Bengio and Courville (2016) show that it is approximately equivalent to L2 regularisation, with the number of steps playing the role of $1/\lambda$: the parameters can only travel a limited distance from their initialisation.

**Algorithm 2.7** Training with Early Stopping

**Input:** Training and validation sets; model with parameters θ; maximum epochs E; patience p; minimum improvement δ_min
**Output:** Parameters θ* with the lowest validation loss

```
best ← +∞; θ* ← θ; wait ← 0
for epoch = 1 to E do
    train θ for one epoch on the training set
    v ← validation loss of θ
    if v < best − δ_min then
        best ← v; θ* ← copy of θ; wait ← 0
```

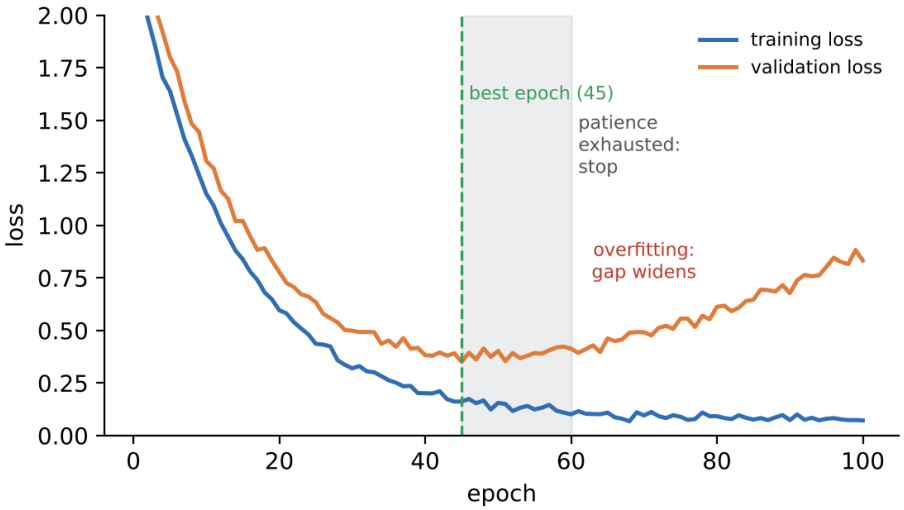



**Figure 2.19** Typical training and validation loss curves: the validation loss reaches its minimum and then rises as the model overfits; early stopping restores the parameters from the best epoch once the patience is exhausted.

```
    else
        wait ← wait + 1
        if wait ≥ p then
            break
return θ*
```

## 2.18.3 Data Augmentation

The best way to improve generalisation is more data. When collecting it is impossible, **data augmentation** creates additional training examples by applying label-preserving transformations to existing ones. For images these include random crops, horizontal flips, small rotations and scalings, colour jitter and blurring; for audio, time shifts, speed changes, added noise and SpecAugment-style masking of frequency bands; for text, synonym replacement, back-translation and random token masking. Augmentation encodes prior knowledge of **invariances**: a cat flipped horizontally is still a cat. It must respect the task, however—flipping a handwritten "6" vertically turns it into a "9". Mixing methods such as **mixup** (Zhang et al., 2018), which trains on convex combinations of pairs of examples and their labels, and **CutMix** (Yun et al., 2019), which pastes patches between images, provide further regularisation. Listing 2.12 shows an augmentation pipeline; augmentation is applied only to the training set, never to validation or test data.

**Listing 2.12** An image data-augmentation pipeline with torchvision and its Keras equivalent.

```python
from torchvision import transforms

train_tf = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),   # random crop + rescale
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
eval_tf = transforms.Compose([            # deterministic: no augmentation
    transforms.Resize(256), transforms.CenterCrop(224), transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Keras: augmentation as layers at the start of the model (active only in training)
from tensorflow import keras
from tensorflow.keras import layers
augment = keras.Sequential([layers.RandomFlip("horizontal"),
                            layers.RandomRotation(0.03), layers.RandomZoom(0.1)])
```

### 2.18.4 Label Smoothing and Other Techniques

With one-hot targets, cross-entropy is minimised only when the correct-class probability reaches 1, which requires the correct logit to grow without bound. The network becomes over-confident and its probability estimates poorly calibrated. **Label smoothing** (Szegedy et al., 2016) replaces the one-hot target with a mixture of the one-hot vector and the uniform distribution (Equation (2.45)). With $\varepsilon = 0.1$ and $K = 10$ classes the target becomes 0.91 for the correct class and 0.01 for each of the others. Label smoothing improves generalisation and calibration of large image classifiers and transformers, although it can hurt knowledge distillation because it erases information in the relative logits.

$$y_k^{\mathrm{LS}} = (1 - \varepsilon)\, y_k + \frac{\varepsilon}{K} \tag{2.45}$$

where $y_k$ is the original one-hot target, $K$ the number of classes and $\varepsilon$ the smoothing parameter (typically 0.1).

Several other techniques act as regularisers. **Noise injection** adds Gaussian noise to inputs, weights or hidden units; adding noise to inputs is, for small noise, equivalent to a penalty on the gradient of the output with respect to the input (Bishop, 1995). **Ensembling** several independently trained networks and averaging their predictions reliably reduces variance, at multiplied cost; **stochastic weight averaging** (Izmailov et al., 2018) approximates an ensemble by averaging weights along the end of the train-

ing trajectory. **Parameter sharing**, as in convolutional networks, is a form of architectural regularisation. Finally, **multi-task learning** and **transfer learning** regularise by forcing representations to be useful for several tasks. Table 2.17 summarises the main techniques.

**Table 2.17** Summary of regularisation techniques for deep networks.

| Technique | How it works | Key hyperparameter | Typical setting |
|---|---|---|---|
| L2 / weight decay | Shrinks weights towards zero | $\lambda$ | $10^{-4}$ (SGD); 0.01–0.1 (AdamW) |
| L1 | Drives weights to exactly zero | $\lambda$ | $10^{-6}$–$10^{-4}$; used for sparsity |
| Early stopping | Halts at best validation loss | Patience | 5–20 epochs |
| Data augmentation | Label-preserving transformations | Transform strengths | Task-dependent |
| Label smoothing | Softens one-hot targets | $\varepsilon$ | 0.1 |
| Mixup / CutMix | Interpolates examples and labels | Mixing parameter $\alpha$ | 0.2–1.0 |
| Dropout | Randomly zeros units (next section) | Drop probability p | 0.1–0.5 |
| Ensembling / SWA | Averages models or weights | Number of members | 3–10 models |

**Key Point | Key Takeaways**

- Regularisation trades a little training performance for better generalisation; deep networks almost always need some.
- Weight decay, early stopping and data augmentation are the three most widely used techniques; augmentation is often the most powerful.
- Label smoothing prevents over-confident predictions and improves calibration.

## 2.19 Dropout and Batch Normalization

Dropout and batch normalisation are two of the most influential techniques introduced during the deep learning revival. Both are implemented as layers that behave differently during training and inference, both inject noise that has a regularising effect, and both made it practical to train deeper and larger networks. Dropout primarily combats overfitting; batch normalisation primarily accelerates and stabilises optimisation.

### 2.19.1 Dropout

**Dropout** (Hinton et al., 2012; Srivastava et al., 2014) randomly sets each hidden unit's output to zero with probability p during each training forward pass, independently for every example and every iteration. A network with N droppable units thus trains a different random sub-network, one of $2^N$ possible "thinned" networks, on each mini-batch, as illustrated in Figure 2.20. Because any unit may disappear, units cannot rely on specific other units being present, which discourages fragile **co-adaptations** and pushes each unit to learn features that are useful on their own.

At test time all units are used. To keep expected activations consistent, modern implementations use **inverted dropout**: during training the surviving activations are scaled by $1/(1-\mathrm{p})$, so nothing needs to change at inference (Equation (2.46)). Using the full network at test time can be viewed as an efficient approximation to averaging the predictions of the exponentially many thinned networks—a cheap form of ensembling. Gal and Ghahramani (2016) also showed that keeping dropout active at test time and averaging several stochastic passes (**Monte Carlo dropout**) yields an approximate Bayesian estimate of predictive uncertainty.

$$r_j \sim \mathrm{Bernoulli}(1-p), \qquad \tilde{a}_j = \frac{r_j a_j}{1-p} \qquad \text{(training)}$$
$$\tilde{a}_j = a_j \qquad \text{(inference)}, \qquad \mathbb{E}[\tilde{a}_j] = a_j \tag{2.46}$$

where p is the drop probability, $r_j$ a random keep mask and $\tilde{a}_j$ the output of the dropout layer.

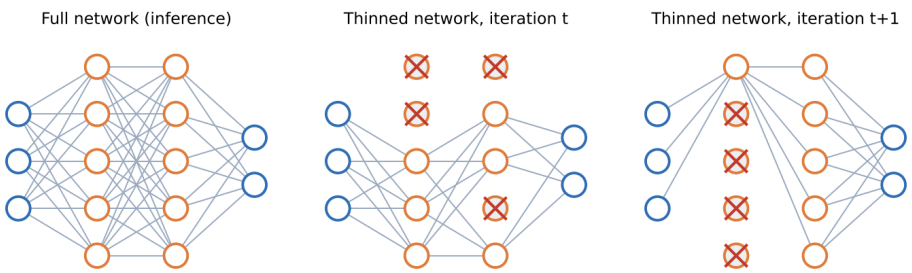



**Figure 2.20** Dropout: the full network (left) and two random thinned networks sampled on different training iterations with p = 0.5 (centre, right); crossed-out units and their connections are removed for that iteration.

Typical drop probabilities are p = 0.5 for large fully connected layers, 0.1–0.3 for smaller layers and transformers (applied to attention weights and residual branches), and little or none in convolutional layers, where spatially correlated activations make standard dropout less effective and variants such as spatial dropout (dropping entire feature maps) or DropBlock are preferred. Dropout slows convergence, often requiring more

epochs, and its benefit decreases as the training set grows; very large modern language models are often pre-trained with little or no dropout.

### 2.19.2 Batch Normalisation

**Batch normalisation** (BN; Ioffe and Szegedy, 2015) standardises each feature of a layer's pre-activations using the mean and variance computed over the current mini-batch, and then applies a learnable scale $\gamma$ and shift $\beta$ so that the layer can still represent any desired distribution, including the identity (Equation (2.47)). Ioffe and Szegedy motivated BN as a remedy for *internal covariate shift*—the changing distribution of a layer's inputs as earlier layers are updated. Later work (Santurkar et al., 2018) argued that its main benefit is rather to make the loss landscape smoother, so that gradients are more predictive and larger learning rates can be used.

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} z_i, \qquad \sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m} (z_i - \mu_{\mathcal{B}})^2$$
$$\hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma\,\hat{z}_i + \beta \tag{2.47}$$

where $z_i$ is one feature of example $i$ in a mini-batch of size $m$, $\epsilon$ a small constant for numerical stability, and $\gamma$, $\beta$ learnable per-feature parameters.

During inference, statistics of a single example or a small batch would be unreliable and would make predictions depend on other examples. BN therefore maintains **running averages** of the mean and variance during training (an exponential moving average with momentum around 0.1) and uses them in place of the batch statistics at test time. Algorithm 2.8 gives both modes. In a convolutional layer, statistics are computed per channel over the batch and both spatial dimensions.

**Algorithm 2.8** Batch Normalisation Layer (training and inference)

**Input:** Mini-batch of pre-activations $\{z_1, \dots, z_m\}$ for one feature; parameters $\gamma$, $\beta$; running statistics $\mu_run$, $\sigma^2_run$; momentum $\alpha$ (e.g. 0.1); $\epsilon$

**Output:** Normalised outputs $\{y_1, \dots, y_m\}$; updated running statistics

**if** training **then**
  $\mu \leftarrow (1/m)\,\Sigma_i\, z_i$
  $\sigma^2 \leftarrow (1/m)\,\Sigma_i\,(z_i - \mu)^2$
  $\mu_run \leftarrow (1-\alpha)\,\mu_run + \alpha\,\mu$
  $\sigma^2_run \leftarrow (1-\alpha)\,\sigma^2_run + \alpha\,\sigma^2$
**else** ▷ *inference*
  $\mu \leftarrow \mu_run$; $\sigma^2 \leftarrow \sigma^2_run$
**for** $i = 1$ **to** $m$ **do**

$\hat{z}_i \leftarrow (z_i - \mu) \,/\, \sqrt{(\sigma^2 + \varepsilon)}$
$y_i \leftarrow \gamma\, \hat{z}_i + \beta$
**return** $\{y_i\}$

The practical benefits of BN are substantial: much higher learning rates can be used, training is far less sensitive to initialisation, and networks converge in many fewer epochs—Ioffe and Szegedy matched the accuracy of an Inception network with 14 times fewer training steps. The noise from batch statistics also regularises, often reducing the need for dropout. BN is usually placed between the linear (or convolutional) operation and the activation, and the preceding layer's bias becomes redundant because $\beta$ plays its role. Its drawbacks are a dependence on batch size (statistics are noisy for m below about 8), a discrepancy between training and inference behaviour, and awkwardness in recurrent networks and distributed training.

**E Worked Example 2.9 | Batch-Normalising One Feature**

A mini-batch of four pre-activations $z = (2, 4, 6, 8)$ with $\gamma = 1.5$, $\beta = 0.5$ and $\varepsilon \approx 0$.

- Mean $\mu = 20/4 = 5$. Variance $\sigma^2 = (9 + 1 + 1 + 9)/4 = 5$, so $\sigma = 2.236$.
- Normalised: $\hat{z} = (-3, -1, 1, 3)/2.236 = (-1.342, -0.447, 0.447, 1.342)$, with mean 0 and variance 1.
- Scaled and shifted: $y = 1.5\hat{z} + 0.5 = (-1.512, -0.171, 1.171, 2.512)$.
- With running-statistics momentum $\alpha = 0.1$ and initial $\mu_run = 0$, $\sigma^2_run = 1$: $\mu_run \leftarrow 0.5$, $\sigma^2_run \leftarrow 0.9 + 0.5 = 1.4$.

## 2.19.3 Layer Normalisation and Other Variants

**Layer normalisation** (LN; Ba, Kiros and Hinton, 2016) computes the mean and variance over the features of each individual example instead of over the batch. It therefore behaves identically in training and inference, works with any batch size including 1, and handles variable-length sequences naturally. It is the standard normalisation in transformers and recurrent networks. **RMSNorm** (Zhang and Sennrich, 2019) simplifies LN by dropping the mean subtraction and is used in many recent large language models. **Group normalisation** (Wu and He, 2018) normalises over groups of channels and is preferred for vision tasks with small batches, such as detection and segmentation; **instance normalisation** normalises each channel of each image separately and is used in style transfer. Figure 2.21 shows over which dimensions each method computes its statistics, and Table 2.18 compares them.

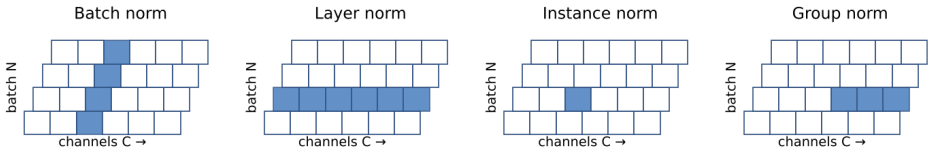



**Figure 2.21** Normalisation methods differ in the set of values (shaded) over which the mean and variance are computed; each cell stands for all spatial (H·W) or sequence positions of one channel of one example in a batch of N examples with C channels.

**Table 2.18** Comparison of normalisation layers.

| Method | Statistics over | Batch-size dependent | Typical use |
|---|---|---|---|
| Batch norm | Batch (and spatial) per channel | Yes | CNNs with batches $\geq 16$ |
| Layer norm | All features of one example | No | Transformers, RNNs |
| RMSNorm | All features (no centring) | No | Large language models |
| Instance norm | Spatial positions, one channel, one example | No | Style transfer, generation |
| Group norm | Groups of channels, one example | No | Detection, segmentation, small batches |

**Listing 2.13** Inverted dropout and batch normalisation from scratch in NumPy, and the equivalent PyTorch block.

```python
import numpy as np

def dropout(a, p, training, rng=np.random.default_rng()):
    if not training or p == 0:
        return a
    mask = rng.random(a.shape) >= p          # keep with probability 1 - p
    return a * mask / (1 - p)                # inverted scaling

def batchnorm_train(Z, gamma, beta, eps=1e-5):
    mu, var = Z.mean(axis=0), Z.var(axis=0)  # statistics per feature over the batch
    Z_hat = (Z - mu) / np.sqrt(var + eps)
    return gamma * Z_hat + beta, mu, var

Z = np.random.default_rng(0).normal(3.0, 2.0, size=(64, 4))
Y, mu, var = batchnorm_train(Z, gamma=np.ones(4), beta=np.zeros(4))
print(Y.mean(0).round(3), Y.std(0).round(3))      # ~0 and ~1 per feature
print(dropout(np.ones((2, 6)), p=0.5, training=True))

import torch.nn as nn
block = nn.Sequential(nn.Linear(256, 128, bias=False),  # bias redundant before BN
                      nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(p=0.3))
# block.train() uses batch statistics and dropout; block.eval() uses running stats, no dropout
```

> **Note | Always Switch Modes**
>
> Dropout and batch normalisation behave differently in training and inference. Forgetting to call `model.eval()` in PyTorch before validation or deployment (or calling a Keras model with `training=True`) leaves dropout active and uses batch statistics, giving noisy and degraded predictions. Conversely, forgetting `model.train()` afterwards stops running statistics from being updated.

# 2.20 Training, Validation, and Testing Deep Neural Networks

A trained network is only as trustworthy as the protocol used to develop and evaluate it. Chapter 1 introduced data splitting and cross-validation; deep learning keeps the same principles but adapts them to large datasets, long training runs and many hyperparameters. This section describes how to split data, how a training loop is organised, how to read learning curves, and how to debug a network that does not learn.

## 2.20.1 Data Splits

The available labelled data are divided into three disjoint subsets. The **training set** is used to fit the parameters by gradient descent. The **validation set** (or development set) is used to make every other decision: early stopping, choice of architecture, learning rate and regularisation strength. The **test set** is used once, at the very end, to estimate the performance of the final model on unseen data. Because validation performance is used for model selection, it becomes an optimistically biased estimate; only an untouched test set provides an unbiased one.

With small datasets the classical 60/20/20 or 70/15/15 split is common, and k-fold cross-validation may be used if training is cheap. With millions of examples, validation and test sets of a few thousand to tens of thousands of examples are usually sufficient for precise estimates, so splits such as 98/1/1 are reasonable. Cross-validation is rarely used for large deep networks because each fold requires a full training run. Splits must respect the structure of the data: use **stratified** splits for imbalanced classes, **group** splits so that all data from one patient, speaker or document fall in the same subset, and **temporal** splits (train on the past, test on the future) for time-dependent data. Validation and test sets should come from the distribution on which the model will be deployed.

> **Note | Data Leakage**
>
> Leakage occurs when information from the validation or test set influences training. Common causes include computing normalisation statistics on the full dataset before splitting, duplicated or near-duplicated examples across splits (frequent in web-scraped image and text corpora), multiple images of the same patient in different splits, and features that are proxies for the label. Leakage produces excellent offline results that collapse in deployment. Always fit preprocessing on the training split only and deduplicate across splits.

### 2.20.2 The Training Loop

Every deep learning framework ultimately executes the loop formalised in Algorithm 2.9. Each epoch iterates over shuffled mini-batches of the training set, performing a forward pass, loss computation, backward pass and optimiser step; the learning-rate scheduler is stepped; and then the model is evaluated on the validation set in inference mode, metrics are logged, and the best checkpoint is saved. High-level APIs such as Keras `model.fit`, PyTorch Lightning or Hugging Face `Trainer` wrap this loop, but understanding it is essential for debugging. Listing 2.14 is a complete PyTorch version.

**Algorithm 2.9** Complete Training Loop with Validation and Checkpointing

**Input:** Training loader, validation loader; model $f_\theta$; loss $\mathcal{L}$; optimiser; scheduler; epochs E; patience p
**Output:** Best model checkpoint $\theta^*$; history of training and validation metrics

```
best ← +∞; wait ← 0
for epoch = 1 to E do
    set model to training mode
    for each mini-batch (X, Y) in training loader do
        Ŷ ← f_θ(X)                                   ▷ forward pass
        loss ← ℒ(Ŷ, Y)
        zero gradients; backpropagate loss           ▷ backward pass
        clip gradients (optional); optimiser step
        scheduler step (if per-batch)
    scheduler step (if per-epoch)
    set model to evaluation mode; disable gradient tracking
    compute validation loss and metrics over validation loader
    log training and validation metrics
    if validation loss < best then
        best ← validation loss; save checkpoint θ*; wait ← 0
    else
        wait ← wait + 1; if wait ≥ p then break
load θ*; evaluate once on the test set
return θ*, history
```

**Listing 2.14** A complete training loop with validation, early stopping and checkpointing in PyTorch.

```python
import copy, torch
from torch import nn

def run_epoch(model, loader, loss_fn, opt=None, device="cpu"):
    train = opt is not None
    model.train(train)
    total, correct, n = 0.0, 0, 0
    with torch.set_grad_enabled(train):
        for X, y in loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            loss = loss_fn(logits, y)
            if train:
                opt.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                opt.step()
            total += loss.item() * len(y)
            correct += (logits.argmax(1) == y).sum().item(); n += len(y)
    return total / n, correct / n

def fit(model, train_dl, val_dl, epochs=50, patience=8, lr=1e-3):
    loss_fn = nn.CrossEntropyLoss()
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    best, wait, best_state = float("inf"), 0, None
    for epoch in range(epochs):
        tr_loss, tr_acc = run_epoch(model, train_dl, loss_fn, opt)
        va_loss, va_acc = run_epoch(model, val_dl, loss_fn)
        sched.step()
        print(f"{epoch:3d} train {tr_loss:.3f}/{tr_acc:.3f}  val
        ↪  {va_loss:.3f}/{va_acc:.3f}")
        if va_loss < best:
            best, wait, best_state = va_loss, 0, copy.deepcopy(model.state_dict())
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    return model
```

### 2.20.3 Reading Learning Curves

**Learning curves**—training and validation loss (and metrics) plotted against epochs—are the primary diagnostic tool. Figure 2.22 shows three characteristic patterns. **Underfitting** (high bias): both losses are high and close together; the model lacks capacity or has not trained long enough, so increase model size, train longer, raise the learning rate or reduce regularisation. **Overfitting** (high variance): the training loss keeps falling while the validation loss rises, and a large gap opens; add data or augmenta-

tion, increase regularisation, reduce capacity or stop earlier. A **good fit** shows both losses decreasing and levelling off with a small gap. Other patterns are informative too: a loss that explodes or becomes NaN signals too high a learning rate or numerical problems; a loss that does not move at all suggests a bug, a dead network or too small a learning rate; and a validation loss lower than the training loss is often explained by regularisation (dropout, augmentation) being active only during training.

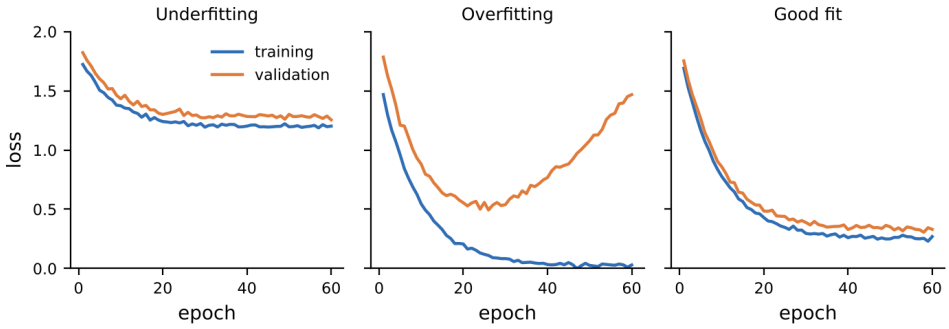



**Figure 2.22** Diagnostic learning-curve patterns: underfitting (both losses high), overfitting (validation loss diverges from training loss), and a good fit (both low with a small gap).

**Table 2.19** Diagnosing training problems from learning curves.

| Symptom | Likely cause | Remedies |
|---|---|---|
| Training and validation loss both high | Underfitting; too little capacity or training | Bigger model, train longer, tune $\eta$, less regularisation |
| Training loss low, validation loss rising | Overfitting | More data/augmentation, weight decay, dropout, early stopping |
| Loss NaN or exploding | $\eta$ too high; numerical instability | Lower $\eta$, warm-up, gradient clipping, check log/divide operations |
| Loss flat from the start | Bug, dead units, $\eta$ far too small | Overfit one batch; check labels, gradients and initialisation |
| Validation loss noisy | Small validation set or batch | Larger validation set; smooth curves; average over seeds |
| Good validation, poor test | Selection overfitting or leakage | Fresh test set; check splits for duplicates |

## 2.20.4 Debugging and Reproducibility

A powerful sanity check before full training is to **overfit a single small batch**: a correctly implemented network with sufficient capacity should drive the training loss on a handful of examples to almost zero within a few hundred steps. If it cannot, there is a bug in the data pipeline, the loss, or the model. Other checks are to verify that the

initial loss matches its theoretical value (log K for K balanced classes), to visualise a few input batches after augmentation, and to compare against a trivial baseline such as predicting the majority class.

Deep learning results vary with the random seed, through initialisation, data order, dropout masks and non-deterministic GPU kernels. For reproducibility, set seeds for Python, NumPy and the framework, record library versions and hyperparameters, and enable deterministic algorithms where available. For reliable conclusions, report the mean and standard deviation over several seeds rather than a single run, and track experiments with tools such as TensorBoard, MLflow or Weights & Biases.

> **Practice Tip** | **Practical Tip: A Debugging Checklist**
>
> - Check shapes, dtypes and value ranges of a batch of inputs and labels.
> - Confirm the initial loss ≈ log K for classification.
> - Overfit one batch; then overfit a small subset (e.g. 1% of data).
> - Plot learning curves from the first run; log gradient norms per layer.
> - Change one thing at a time and keep a log of every experiment.

## 2.21 Hyperparameter Tuning in Deep Learning

**Hyperparameters** are settings chosen before training rather than learned by gradient descent: the learning rate and its schedule, batch size, optimiser parameters, number and width of layers, activation functions, dropout rate, weight decay, augmentation strength and number of epochs. They can change performance dramatically, and good values depend on the dataset and architecture. **Hyperparameter tuning** (or optimisation, HPO) searches for the configuration $\lambda^*$ that minimises the validation loss of the model trained with it (Equation (2.48)). This is a bilevel optimisation problem: each evaluation of the outer objective requires a complete inner training run, which makes tuning expensive and motivates efficient search strategies.

$$\lambda^* = \arg\min_{\lambda \in \Lambda} \; J_{\text{val}}(\theta^*(\lambda))$$
$$\theta^*(\lambda) = \arg\min_{\theta} \; J_{\text{train}}(\theta;\, \lambda) \tag{2.48}$$

where $\Lambda$ is the search space of hyperparameter configurations, $J_{val}$ the validation loss and J_train the training objective.

### 2.21.1 What to Tune First

Not all hyperparameters are equally important. The learning rate is almost always the most influential, followed by its schedule, weight decay, batch size and model size; momentum or Adam's $\beta$ parameters rarely need changing from their defaults. Many hyperparameters span orders of magnitude and should be searched on a **logarithmic scale**: for example, sampling the learning rate as $\eta = 10^{u}$ with $u$ uniform on $[-5, -1]$ gives equal attention to $10^{-5}$–$10^{-4}$ and $10^{-2}$–$10^{-1}$. Table 2.20 lists typical ranges.

**Table 2.20** Important hyperparameters, their typical ranges and search scales.

| Hyperparameter | Typical range | Scale | Importance |
|---|---|---|---|
| Learning rate $\eta$ | $10^{-5}$ – $10^{-1}$ | Log | Very high |
| LR schedule / warm-up | Cosine, step, one-cycle; 0–10% warm-up | Categorical | High |
| Weight decay $\lambda$ | $10^{-6}$ – $10^{-1}$ | Log | High |
| Batch size $m$ | 16 – 4096 | Log (powers of 2) | Medium (couple with $\eta$) |
| Number of layers | 2 – 50+ | Integer | Medium |
| Units per layer | 32 – 4096 | Log | Medium |
| Dropout $p$ | 0 – 0.5 | Linear | Medium |
| Adam $\beta_1$, $\beta_2$ | 0.9, 0.999 (rarely changed) | $1 -$ log | Low |
| Epochs | Set by early stopping | — | Controlled by patience |

### 2.21.2 Search Strategies

**Manual search**, guided by learning curves and experience, remains surprisingly common and is effective for the first few coarse decisions. **Grid search** evaluates every combination of a few values per hyperparameter; its cost grows exponentially with the number of hyperparameters, and it wastes effort because it tests only a few distinct values of each one. **Random search** samples each configuration independently from the search distribution. Bergstra and Bengio (2012) showed that it is markedly more efficient than grid search when only a few hyperparameters really matter—the usual case—because every trial tries a new value of each important hyperparameter, as illustrated in Figure 2.23.

**Bayesian optimisation** goes further by building a probabilistic **surrogate model** of the validation loss as a function of the hyperparameters—a Gaussian process, a random forest, or a tree-structured Parzen estimator (TPE; Bergstra et al., 2011)—and choosing each new trial by maximising an **acquisition function** that balances exploring uncertain regions against exploiting promising ones (Snoek, Larochelle and Adams, 2012). A common acquisition function is expected improvement (Equation (2.49)). Bayesian

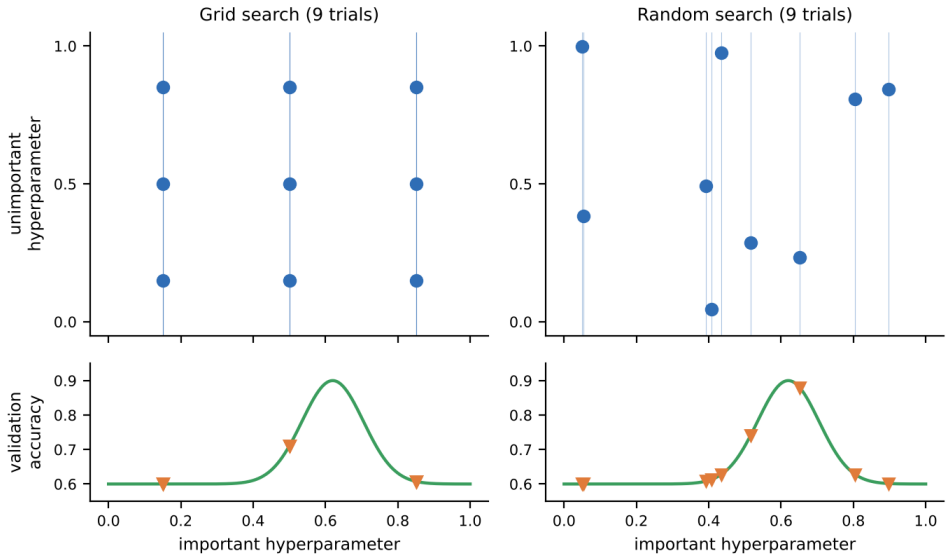



**Figure 2.23** Grid search versus random search with nine trials when one hyperparameter is important and one is not. Top: trial locations; bottom: validation accuracy as a function of the important hyperparameter, with triangles marking the values tested. The grid tests only three distinct values and misses the peak, whereas random search tests nine (after Bergstra and Bengio, 2012).

optimisation typically finds good configurations in fewer trials than random search, but the trials are sequential and harder to parallelise.

$$\mathrm{EI}(\lambda) = \mathbb{E}\left[\max\left(0,\; J_{\text{best}} - J(\lambda)\right)\right] \tag{2.49}$$

where $J_{\text{best}}$ is the lowest validation loss observed so far and the expectation is taken under the surrogate model's predictive distribution for $J(\lambda)$.

Because full training runs are expensive, **multi-fidelity** methods evaluate many configurations cheaply—with few epochs or a data subset—and allocate more resources only to the most promising. **Successive halving** trains $n$ configurations for a small budget, keeps the best half (or third), doubles their budget, and repeats; **Hyperband** (Li et al., 2018) runs successive halving with several trade-offs between $n$ and the starting budget, as described in Algorithm 2.10. **BOHB** combines Hyperband with Bayesian optimisation. **Population-based training** (Jaderberg et al., 2017) trains a population of models in parallel and periodically replaces poor members with perturbed copies of good ones, adapting hyperparameters such as the learning rate during training.

**Algorithm 2.10** Successive Halving for Hyperparameter Search

**Input:** Search space $\Lambda$; number of initial configurations n; minimum budget r (e.g. epochs); reduction factor $\eta_r$ (e.g. 3); maximum budget R
**Output:** Best configuration found

sample n configurations $\lambda_1, \ldots, \lambda_n$ from $\Lambda$
$S \leftarrow \{\lambda_1, \ldots, \lambda_n\}$; $b \leftarrow r$
**while** $|S| > 1$ **and** $b \leq R$ **do**
  **for each** $\lambda$ **in** S **do**
    train model with $\lambda$ **for** budget b (resuming **if** possible)
    record validation loss $J(\lambda)$
  keep the $\lceil |S| / \eta_r \rceil$ configurations **in** S with the lowest J
  $b \leftarrow b \cdot \eta_r$
**return** configuration **in** S with the lowest J

**Table 2.21** Hyperparameter search strategies compared.

| Strategy | How it chooses trials | Parallelism | Best for |
|---|---|---|---|
| Manual | Expert judgement from learning curves | Low | Early exploration, few runs |
| Grid search | All combinations of fixed values | High | 1–2 hyperparameters |
| Random search | Independent random samples | High | Strong, simple baseline |
| Bayesian optimisation | Surrogate model + acquisition function | Low–medium | Expensive runs, $< 20$ hyperparameters |
| Hyperband / ASHA | Early-stops poor configurations | High | Large search spaces, cheap partial runs |
| Population-based training | Exploit and explore during training | High | Schedules, RL, large compute |

### 2.21.3 Tools and Best Practices

Libraries such as Optuna, Ray Tune, KerasTuner, Hyperopt and Weights & Biases Sweeps implement these strategies with parallel execution, pruning of unpromising trials and result visualisation. Listing 2.15 uses Optuna's TPE sampler with median pruning to tune a small PyTorch MLP. Whatever the tool, a few practices make tuning more reliable: start with a coarse search over wide ranges, then refine around the best region; tune on a validation set (never the test set); use log scales for scale-type hyperparameters; tune the learning rate again whenever the batch size or architecture changes; and verify the final choice with several random seeds, because differences smaller than the seed-to-seed variance are not meaningful.

**Listing 2.15** Hyperparameter tuning with Optuna (TPE sampler and median pruning).

```python
import optuna, torch
from torch import nn
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=4000, n_features=20, n_informative=10,
    random_state=0)
Xtr, Xva, ytr, yva = map(torch.tensor, train_test_split(X, y, test_size=0.25,
    random_state=0))
Xtr, Xva = Xtr.float(), Xva.float()

def objective(trial):
    lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
    wd = trial.suggest_float("weight_decay", 1e-6, 1e-1, log=True)
    width = trial.suggest_int("width", 32, 512, log=True)
    p = trial.suggest_float("dropout", 0.0, 0.5)
    model = nn.Sequential(nn.Linear(20, width), nn.ReLU(), nn.Dropout(p),
        nn.Linear(width, 2))
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=wd)
    for epoch in range(30):
        model.train(); opt.zero_grad()
        nn.functional.cross_entropy(model(Xtr), ytr).backward(); opt.step()
        model.eval()
        with torch.no_grad():
            val_loss = nn.functional.cross_entropy(model(Xva), yva).item()
        trial.report(val_loss, epoch)
        if trial.should_prune():                # stop unpromising trials early
            raise optuna.TrialPruned()
    return val_loss

study = optuna.create_study(direction="minimize",
    sampler=optuna.samplers.TPESampler(seed=0),
                            pruner=optuna.pruners.MedianPruner(n_warmup_steps=5))
study.optimize(objective, n_trials=50)
print(study.best_params, round(study.best_value, 4))
```

**Case Study 2.3 | The Cost of Tuning**

Neural architecture search (NAS) extends hyperparameter tuning to the architecture itself. Zoph and Le (2017) used reinforcement learning to design CNN architectures, consuming hundreds of GPUs for weeks; later methods such as weight sharing (ENAS) and differentiable search (DARTS; Liu, Simonyan and Yang, 2019) cut the cost by orders of magnitude. For very large language models, a single training run is too expensive to tune directly, so practitioners tune on small proxy models and extrapolate using scaling laws or parameterisations designed to transfer hyperparameters across model sizes (such as µP; Yang et al., 2021).

**Key Point | Key Takeaways**

- The learning rate is the most important hyperparameter; search it on a log scale and retune

it when batch size or architecture changes.

- Random search beats grid search; Bayesian optimisation and Hyperband-style early stopping reduce cost further.
- Tune on validation data only, refine coarse-to-fine, and confirm conclusions over several random seeds.

## 2.22 Deep Feedforward Neural Networks

A **deep feedforward neural network**, also called a deep multilayer perceptron (MLP), is the simplest deep architecture and the reference point against which every specialised model in the rest of this chapter is measured. Information flows in one direction only: from the input, through a sequence of hidden layers, to the output. There are no loops, no shared weights across positions and no memory of previous inputs. The earlier sections of this chapter introduced the artificial neuron, activation functions, backpropagation, optimisers and regularisation; here we put those ingredients together, ask what *depth* buys us, and set out the design decisions a practitioner faces when building a fully connected network for tabular or vector data.

### 2.22.1 Layered Composition of Functions

A feedforward network with L layers computes a composition of simple parametric functions. Each layer applies an affine map followed by an element-wise non-linearity, as written in Equation (2.50). The output layer replaces the hidden activation with a task-specific function: the identity for regression, the logistic sigmoid for binary classification and the softmax for multi-class classification. Stacking layers yields the overall mapping in Equation (2.51).

$$\begin{aligned} \mathbf{z}^{(l)} &= W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{h}^{(l)} &= \phi(\mathbf{z}^{(l)}), \quad \mathbf{h}^{(0)} = \mathbf{x} \end{aligned} \tag{2.50}$$

where $W^{(l)} \in \mathbb{R}^{\wedge}(n_l \times n_{l-1})$ is the weight matrix of layer $l$, $\mathbf{b}^{(l)}$ its bias vector, $\mathbf{z}^{(l)}$ the pre-activation, $\varphi$ an element-wise activation such as ReLU, and $\mathbf{h}^{(l)}$ the layer's output (activation).

$$f(\mathbf{x};\theta) = f^{(L)}\big(f^{(L-1)}\big(\cdots f^{(1)}(\mathbf{x})\big)\big) \tag{2.51}$$

where $\theta = \{W^{(l)}, \mathbf{b}^{(l)} : l = 1,\ldots,L\}$ collects all trainable parameters.

Figure 2.24 shows a network with four inputs, two hidden layers and three outputs. Every unit in one layer connects to every unit in the next, which is why such layers are

called *dense* or *fully connected*. The number of parameters in a dense layer is $n_l(n_{l-1} + 1)$, so a single layer mapping a 1,000-dimensional input to 1,000 hidden units already holds about one million weights. This quadratic growth is the main reason fully connected networks are poorly suited to raw images or long sequences, and it motivates the weight sharing of convolutional and recurrent networks discussed later in this part.

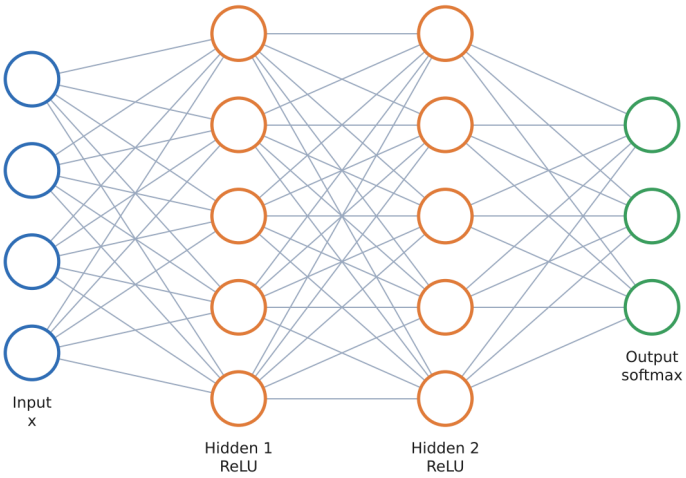



**Figure 2.24** A deep feedforward network with four inputs, two hidden layers of five units and three softmax outputs; every unit is connected to every unit in the next layer.

## 2.22.2 Why Depth Helps

The universal approximation theorem (Cybenko 1989; Hornik, Stinchcombe & White 1989) states that a network with a single hidden layer and a suitable non-linearity can approximate any continuous function on a compact set to arbitrary accuracy, given enough hidden units. The theorem says nothing, however, about *how many* units are needed. For many functions of practical interest, a shallow network requires a number of units exponential in the input dimension, whereas a deep network can represent the same function with polynomially many units. Intuitively, each layer can reuse the features computed by the layer below: a ReLU network partitions the input space into linear regions, and the number of such regions can grow exponentially with depth but only polynomially with width (Montúfar et al. 2014).

Depth also supports **hierarchical representation learning**. Early layers learn simple features; later layers combine them into increasingly abstract concepts. In a credit-scoring network, for example, the first layer may learn ratios such as debt-to-income, while deeper layers combine these into latent notions of financial stress. This compositional view underlies all of the architectures in this part: convolutional networks build

edges into textures into object parts, and transformers build tokens into phrases into meanings.

**Definition 2.4 | Deep Feedforward Network**

A deep feedforward network is a directed acyclic composition of two or more parametrised layers, each an affine transformation followed by a fixed non-linearity, trained end-to-end by gradient descent on a differentiable loss. "Deep" conventionally means more than one hidden layer.

### 2.22.3 Design Choices

Building an MLP involves a modest set of decisions summarised in Table 2.22. Most have sensible defaults: ReLU or GELU activations in hidden layers, He initialisation, the Adam optimiser, batch or layer normalisation and dropout for regularisation (all covered in the earlier sections on training deep networks). The loss function is dictated by the output: mean squared error for regression and cross-entropy for classification, as written in Equation (2.52).

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log \hat{y}_{ik}, \qquad \hat{y}_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^{K}\exp(z_{ij})} \tag{2.52}$$

where $y_{ik}$ is 1 if example $i$ belongs to class $k$ and 0 otherwise, $z_{ik}$ the output logit and $\hat{y}_{ik}$ the softmax probability.

Listing 2.16 gives a compact PyTorch implementation. Note how the architecture is defined declaratively with `nn.Sequential`, and how the training loop follows the generic pattern of forward pass, loss, backward pass and parameter update.

**Listing 2.16** A deep feedforward classifier and training loop in PyTorch.

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, n_classes, p_drop=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, d_hidden), nn.BatchNorm1d(d_hidden), nn.ReLU(),
            nn.Dropout(p_drop),
            nn.Linear(d_hidden, d_hidden), nn.BatchNorm1d(d_hidden), nn.ReLU(),
            nn.Dropout(p_drop),
            nn.Linear(d_hidden, n_classes))          # logits; softmax is inside the loss
    def forward(self, x):
        return self.net(x)
```

**Table 2.22** Key design decisions for a deep feedforward network and common defaults.

| Decision | Common default | Notes |
|---|---|---|
| Depth | 2–4 hidden layers | Deeper MLPs on tabular data rarely help without residual connections |
| Width | 64–1024 units | Funnel (decreasing) or constant widths both work |
| Hidden activation | ReLU / GELU | Avoid sigmoid/tanh in deep stacks (vanishing gradients) |
| Output activation | Identity / sigmoid / softmax | Matches regression / binary / multi-class |
| Initialisation | He (ReLU), Glorot (tanh) | Keeps activation variance stable across layers |
| Optimiser | Adam or AdamW, lr ≈ 1e-3 | Combine with a learning-rate schedule |
| Regularisation | Dropout 0.1–0.5, weight decay | Early stopping on validation loss |
| Normalisation | Batch norm or layer norm | Also standardise input features |

```
13
14 model = MLP(d_in=20, d_hidden=128, n_classes=3)
15 opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
16 loss_fn = nn.CrossEntropyLoss()
17
18 for epoch in range(50):
19     model.train()
20     for xb, yb in train_loader:                    # mini-batches from a DataLoader
21         opt.zero_grad()
22         loss = loss_fn(model(xb), yb)
23         loss.backward()                            # backpropagation
24         opt.step()
```

## 2.22.4 Strengths and Limitations

Feedforward networks are flexible, easy to implement and fast at inference. They are the standard head placed on top of every other architecture: the classifier at the end of a CNN, the position-wise feed-forward sublayer of a transformer and the value head of a reinforcement learning agent are all MLPs. On their own, however, they ignore any structure in the input. Permuting the pixels of every training image identically leaves an MLP's accuracy unchanged, which shows that it does not exploit spatial locality. On tabular data, carefully tuned gradient-boosted trees frequently match or beat MLPs (Grinsztajn, Oyallon & Varoquaux 2022), so deep networks are most compelling when the input has exploitable structure or when representations must be learned jointly

with other modalities.

**Key Point | Takeaways**

- A deep feedforward network is a composition of affine maps and non-linearities trained by backpropagation.
- Depth gives exponential gains in representational efficiency and supports hierarchical features.
- Dense layers do not share weights, so parameter counts explode on high-dimensional structured inputs; CNNs, RNNs and transformers introduce *inductive biases* to fix this.

# 2.23 Convolutional Neural Networks (CNNs)

Convolutional neural networks (CNNs) are deep networks specialised for data arranged on a regular grid: one-dimensional signals such as audio, two-dimensional images and three-dimensional volumes such as MRI scans or video. They were popularised for handwritten digit recognition by LeCun et al. (1989, 1998) and became the dominant approach to computer vision after AlexNet won the 2012 ImageNet challenge. CNNs rest on three ideas that encode prior knowledge about images: **local connectivity** (a unit looks only at a small neighbourhood), **weight sharing** (the same filter is applied at every location) and **pooling** (summaries that make the representation robust to small translations).

## 2.23.1 The Convolution Operation

In deep learning, a convolutional layer slides a small learnable filter, or **kernel**, over the input and computes a weighted sum at each position. Strictly, most libraries implement *cross-correlation* (the kernel is not flipped), but because the kernel is learned the distinction is immaterial and the operation is universally called convolution. For a single-channel input I and a k × k kernel K, the output **feature map** S is given by Equation (2.53). With C input channels the kernel becomes a k × k × C tensor and the sum runs over channels as well, as in Equation (2.54); a layer with C′ such kernels produces C′ output channels.

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k-1}\sum_{n=0}^{k-1} I(i+m,\; j+n)\, K(m,n) + b \tag{2.53}$$

where (i, j) indexes the output position, (m, n) the kernel position and b is a scalar bias.

$$S_{c'}(i,j) = b_{c'} + \sum_{c=1}^{C} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I_c(s\,i+m,\; s\,j+n)\, K_{c',c}(m,n) \tag{2.54}$$

where $c'$ indexes the output channel, c the input channel and s the stride.

Figure 2.25 illustrates the operation: the highlighted 3 × 3 window of the input is multiplied element-wise with the kernel and summed to give one output value; sliding the window produces the full feature map. Because the same nine weights are reused at every location, the layer is **translation equivariant**: shifting the input shifts the feature map by the same amount. Weight sharing also slashes the parameter count. A 3 × 3 convolution from 64 to 64 channels has 3·3·64·64 + 64 ≈ 37 thousand parameters regardless of the image size, whereas a dense layer between two 56 × 56 × 64 tensors would need about $4 \times 10^{10}$.

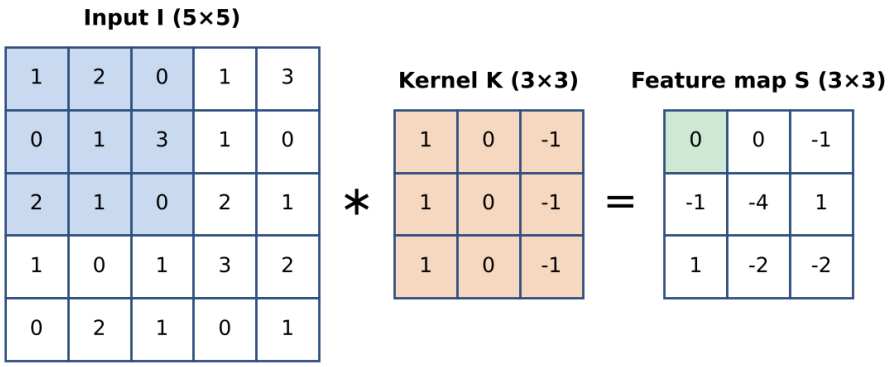



**Figure 2.25** A 3 × 3 kernel slides over a 5 × 5 input (stride 1, no padding). The shaded window is multiplied element-wise with the kernel and summed to produce the shaded output entry of the 3 × 3 feature map.

## 2.23.2 Padding, Stride and Output Size

Two hyperparameters control the spatial size of the output. **Padding** p adds a border of zeros (or reflected values) around the input; without it each convolution shrinks the map by k − 1 pixels and information at the border is under-used. *Same* padding, p = (k − 1)/2 for odd k, keeps the spatial size unchanged at stride 1. **Stride** s is the step between successive filter positions; s = 2 halves the resolution and is a common alternative to pooling. **Dilation** d inserts gaps of d − 1 pixels between kernel elements, enlarging the effective kernel size to d(k − 1) + 1 without adding parameters. The output size along one spatial dimension is given by Equation (2.55).

$$O = \left\lfloor \frac{W - d\,(k-1) - 1 + 2p}{s} \right\rfloor + 1 \tag{2.55}$$

where W is the input width (or height), k the kernel size, p the padding, s the stride and d the dilation ($d = 1$ for ordinary convolution, giving $O = \lfloor (W - k + 2p)/s \rfloor + 1$).

**E Worked Example 2.10 | Output Sizes**

A $224 \times 224 \times 3$ image passes through a convolution with 64 kernels of size $7 \times 7$, stride 2 and padding 3.

- Output width: $O = \lfloor (224 - 7 + 2 \cdot 3)/2 \rfloor + 1 = \lfloor 223/2 \rfloor + 1 = 111 + 1 = 112$. The output is $112 \times 112 \times 64$.
- Parameters: $7 \cdot 7 \cdot 3 \cdot 64 + 64 = 9{,}472$.

A following $3 \times 3$ max-pool with stride 2 and padding 1 gives $\lfloor (112 - 3 + 2)/2 \rfloor + 1 = 55 + 1 = 56$, so the map becomes $56 \times 56 \times 64$. These are exactly the first two layers of ResNet.

## 2.23.3 Worked Example of a 2D Convolution

It is worth computing a convolution by hand once. Figure 2.25 uses a vertical-edge kernel whose left column is $+1$ and right column $-1$; it responds strongly where intensity decreases from left to right. The calculation is set out in the example box below and verified with NumPy in Listing 2.17.

**E Worked Example 2.11 | Convolving a 5 × 5 Input**

Input rows: (1,2,0,1,3), (0,1,3,1,0), (2,1,0,2,1), (1,0,1,3,2), (0,2,1,0,1). Kernel rows: (1,0,−1) repeated three times. Stride 1, no padding, bias 0.

- $S(0,0)$: window columns 0–2 of rows 0–2. Left column sum = 1+0+2 = 3; right column sum = 0+3+0 = 3. $S = 3 - 3 = 0$.
- $S(0,1)$: left column (2,1,1) sums to 4; right column (1,1,2) sums to 4. $S = 0$.
- $S(0,2)$: left column (0,3,0) = 3; right column (3,0,1) = 4. $S = -1$.
- $S(1,0)$: left (0,2,1) = 3; right (3,0,1) = 4. $S = -1$. $S(1,1)$: left (1,1,0) = 2; right (1,2,3) = 6. $S = -4$.
- $S(1,2)$: left (3,0,1) = 4; right (0,1,2) = 3. $S = 1$. $S(2,0)$: left (2,1,0) = 3; right (0,1,1) = 2. $S = 1$.
- $S(2,1)$: left (1,0,2) = 3; right (2,3,0) = 5. $S = -2$. $S(2,2)$: left (0,1,1) = 2; right (1,2,1) = 4. $S = -2$.

The $3 \times 3$ feature map is therefore $((0,0,-1), (-1,-4,1), (1,-2,-2))$. Output size agrees with Equation (2.55): $\lfloor (5 - 3 + 0)/1 \rfloor + 1 = 3$.

**Listing 2.17** A direct NumPy implementation of 2D convolution (cross-correlation) with stride and padding.

```
import numpy as np

def conv2d(I, K, stride=1, pad=0, bias=0.0):
    I = np.pad(I, pad)                                  # zero padding on all sides
```

```
    k = K.shape[0]
    H = (I.shape[0] - k) // stride + 1              # output size, Eq. for O
    W = (I.shape[1] - k) // stride + 1
    S = np.empty((H, W))
    for i in range(H):
        for j in range(W):
            win = I[i*stride:i*stride+k, j*stride:j*stride+k]
            S[i, j] = np.sum(win * K) + bias
    return S

I = np.array([[1,2,0,1,3],[0,1,3,1,0],[2,1,0,2,1],[1,0,1,3,2],[0,2,1,0,1]])
K = np.array([[1,0,-1]] * 3)
print(conv2d(I, K))            # [[ 0  0 -1] [-1 -4  1] [ 1 -2 -2]]
print(conv2d(I, K, pad=1).shape)   # (5, 5): 'same' padding
```

### 2.23.4 Pooling

A **pooling** layer replaces each local window of a feature map with a summary statistic, independently per channel. **Max pooling** keeps the largest value, as in Equation (2.56), and **average pooling** keeps the mean. A 2 × 2 pool with stride 2 halves each spatial dimension and discards 75% of the activations, reducing computation in later layers and providing a small degree of invariance to local translations. **Global average pooling** averages each channel over the entire spatial extent, converting a H × W × C tensor into a C-dimensional vector; it replaced large fully connected heads in modern architectures and greatly reduces overfitting. Pooling has no learnable parameters.

$$P_c(i,j) = \max_{0 \le m,\, n < k} F_c(s\,i + m,\; s\,j + n) \tag{2.56}$$

where $F_c$ is channel c of the input feature map, k the pooling window size and s the stride.

### 2.23.5 Receptive Field

The **receptive field** of a unit is the region of the input image that can influence its value. A unit in the first 3 × 3 convolution sees a 3 × 3 patch; a unit in the second sees 5 × 5; and the field keeps growing with depth. For a stack of layers with kernel sizes $k_l$ and strides $s_l$, the receptive field $r_L$ follows the recursion in Equation (2.57). Strided layers and pooling multiply the growth rate, which is why CNNs downsample early. Two stacked 3 × 3 convolutions have the same 5 × 5 receptive field as one 5 × 5 convolution but use $18C^2$ rather than $25C^2$ weights and include an extra non-linearity, an observation central to VGG (next section). In practice the *effective* receptive field, where gradients are substantial, is considerably smaller than the theoretical one and roughly Gaussian in shape (Luo et al. 2016).

$$r_l = r_{l-1} + (k_l - 1)\prod_{i=1}^{l-1} s_i, \qquad r_0 = 1 \tag{2.57}$$

where $k_l$ and $s_l$ are the kernel size and stride of layer $l$.

### 2.23.6 The CNN Pipeline

A typical image-classification CNN alternates convolution, non-linearity and down-sampling, progressively trading spatial resolution for channel depth, and ends with a classifier head. Figure 2.26 shows this canonical pipeline: early layers detect edges and colour blobs, middle layers textures and motifs, and late layers object parts and whole objects (Zeiler & Fergus 2014). Table 2.23 summarises the building blocks.

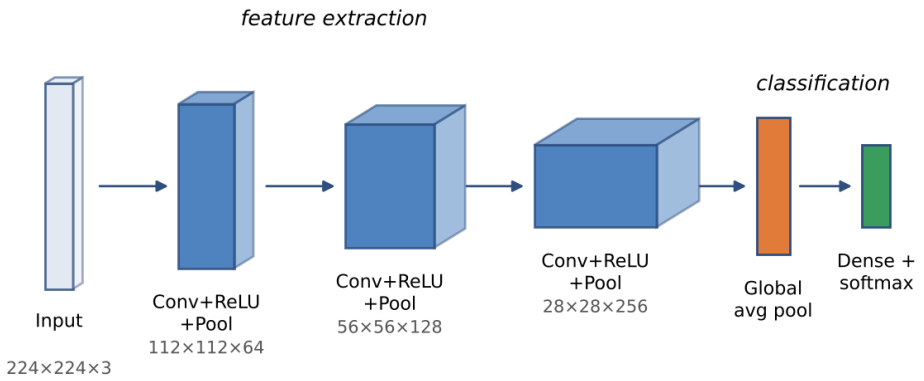



**Figure 2.26** The canonical CNN pipeline: stacked convolution–ReLU–pooling stages reduce spatial size while increasing channel depth, followed by global pooling and a dense classifier.

**Table 2.23** Building blocks of a convolutional neural network.

| Layer | Purpose | Learnable parameters | Effect on shape |
|---|---|---|---|
| Convolution | Local feature detection with shared weights | $k{\cdot}k{\cdot}C{\cdot}C' + C'$ | H,W by Equation (2.55); $C \to C'$ |
| Activation (ReLU) | Non-linearity | None | Unchanged |
| Batch normalisation | Stabilise and speed up training | $2C$ ($\gamma$, $\beta$) | Unchanged |
| Max / average pooling | Downsample; local invariance | None | H,W divided by stride |
| Global average pooling | Collapse spatial dimensions | None | $H{\times}W{\times}C \to C$ |
| Dense (fully connected) | Classification head | $C{\cdot}K + K$ | $C \to K$ classes |

In Keras, the whole pipeline takes only a few lines, as shown in Listing 2.18.

**Listing 2.18** A small CNN for 32 × 32 colour images (e.g. CIFAR-10) in Keras.

```
from tensorflow import keras
from tensorflow.keras import layers

def conv_block(x, filters):
    x = layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    return layers.MaxPooling2D(2)(x)                # halve H and W

inputs = keras.Input(shape=(32, 32, 3))
x = conv_block(inputs, 32)                          # 16x16x32
x = conv_block(x, 64)                               # 8x8x64
x = conv_block(x, 128)                              # 4x4x128
x = layers.GlobalAveragePooling2D()(x)              # 128
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(10, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
model.summary()
```

**Practice Tip | Practical Tips for CNNs**

- Prefer stacks of 3 × 3 convolutions; use 1 × 1 convolutions to change channel counts cheaply.
- Double the number of channels whenever you halve the spatial resolution, to keep computation per layer roughly constant.
- Use data augmentation (random crops, flips, colour jitter) — it is often worth more than architectural tweaks.
- Start from a pretrained backbone whenever your dataset is small (see the section on transfer learning later in this chapter).

## 2.24 CNN Architectures: LeNet, AlexNet, VGG, Inception, and ResNet

The history of CNN architectures between 1998 and 2016 is a story of going deeper while keeping training stable and computation affordable. Much of this progress was measured on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), a 1,000-class classification benchmark with about 1.2 million training images, where the standard metric is *top-5 error*: the fraction of test images whose true label is not among

the model's five highest-scoring classes. Table 2.24 summarises the five landmark architectures discussed in this section.

**Table 2.24** Landmark CNN architectures. Top-5 error is on ImageNet (ILSVRC) for the competition or paper's best reported single-model or ensemble result; LeNet-5 was evaluated on MNIST, not ImageNet.

| Model | Year | Depth (weight layers) | Parameters | Top-5 error | Key idea |
|---|---|---|---|---|---|
| LeNet-5 | 1998 | 5 (7 incl. pooling) | ≈ 60 K | n/a (MNIST ≈ 0.95% error) | Conv + subsampling + dense; gradient-trained end-to-end |
| AlexNet | 2012 | 8 | ≈ 60 M | 15.3% | ReLU, dropout, GPU training, data augmentation |
| VGG-16 | 2014 | 16 | ≈ 138 M | 7.3% | Uniform stacks of 3 × 3 convolutions |
| GoogLeNet (Inception v1) | 2014 | 22 | ≈ 6.8 M | 6.7% | Inception modules with 1 × 1 bottlenecks; global average pooling |
| ResNet-152 | 2015 | 152 | ≈ 60 M | 3.57% | Residual (skip) connections enable very deep nets |

### 2.24.1 LeNet-5 (1998)

LeCun, Bottou, Bengio and Haffner's LeNet-5 (1998) read handwritten digits on bank cheques and established the template still used today. A 32 × 32 grey-scale input passes through a 5 × 5 convolution with six channels, a 2 × 2 subsampling (average-pooling) layer, a second 5 × 5 convolution with sixteen channels, another subsampling layer, and three fully connected layers of 120, 84 and 10 units. With about 60 thousand parameters and tanh-like activations, it was trained end-to-end with backpropagation, demonstrating that learned features could outperform hand-crafted ones.

### 2.24.2 AlexNet (2012)

Krizhevsky, Sutskever and Hinton's AlexNet won ILSVRC 2012 with a top-5 error of 15.3%, against 26.2% for the runner-up, and is widely credited with launching the deep learning era in computer vision. It had five convolutional and three fully connected layers (about 60 million parameters) and introduced several now-standard practices: **ReLU** activations, which trained several times faster than tanh; **dropout** in the fully connected layers; aggressive **data augmentation**; and training split across two **GPUs**. Its first layer used large 11 × 11 kernels with stride 4.

### 2.24.3 VGG (2014)

Simonyan and Zisserman's VGG networks (2014) asked a simple question: what happens if we use only $3 \times 3$ convolutions and simply add more of them? VGG-16 and VGG-19 consist of five blocks of two to four $3 \times 3$ convolutions, each block followed by $2 \times 2$ max pooling, with channels doubling from 64 to 512, and three large dense layers at the end. As noted in the previous section, stacked small kernels achieve large receptive fields with fewer parameters and more non-linearities. VGG's uniformity made it a favourite feature extractor, but its 138 million parameters (most in the first dense layer) and roughly 15.5 GFLOPs per image make it expensive.

### 2.24.4 Inception / GoogLeNet (2014)

Szegedy et al.'s GoogLeNet (2014), winner of ILSVRC 2014, took a different route to depth: making each layer *wider* and smarter rather than bigger. Its building block, the **Inception module** shown in Figure 2.27, applies $1 \times 1$, $3 \times 3$ and $5 \times 5$ convolutions and a $3 \times 3$ max pool in parallel and concatenates their outputs along the channel axis, letting the network choose the appropriate scale at each location. The key trick is the **$1 \times 1$ bottleneck convolution** placed before the expensive $3 \times 3$ and $5 \times 5$ branches to reduce the number of channels. A $1 \times 1$ convolution is a per-pixel dense layer across channels, so it mixes channel information at negligible spatial cost. With global average pooling instead of large dense layers, the 22-layer GoogLeNet uses only about 6.8 million parameters, roughly twenty times fewer than VGG-16. Later versions (Inception v2–v4) added batch normalisation and factorised convolutions, e.g. a $5 \times 5$ replaced by two $3 \times 3$s and $n \times n$ by $1 \times n$ followed by $n \times 1$.

**E Worked Example 2.12 | Saving Computation with a 1 × 1 Bottleneck**

Consider a $28 \times 28 \times 192$ input to a $5 \times 5$ convolution producing 32 channels (same padding).

- Direct: multiplications $= 28 \cdot 28 \cdot 32 \times (5 \cdot 5 \cdot 192) = 25{,}088 \times 4{,}800 \approx 120.4$ million.
- With a $1 \times 1$ bottleneck to 16 channels first: $28 \cdot 28 \cdot 16 \times 192 \approx 2.41$ M, plus $28 \cdot 28 \cdot 32 \times (5 \cdot 5 \cdot 16) \approx 10.04$ M; total $\approx 12.4$ M.

The bottleneck cuts the cost by almost a factor of ten while preserving the output shape.

### 2.24.5 ResNet and Residual Learning (2015)

By 2015 it was clear that simply stacking more layers did not keep improving accuracy. He, Zhang, Ren and Sun (2016) observed a **degradation problem**: a 56-layer plain network had *higher training error* than a 20-layer one. This is not overfitting, since training error rose, but an optimisation difficulty, because a deeper network could in principle

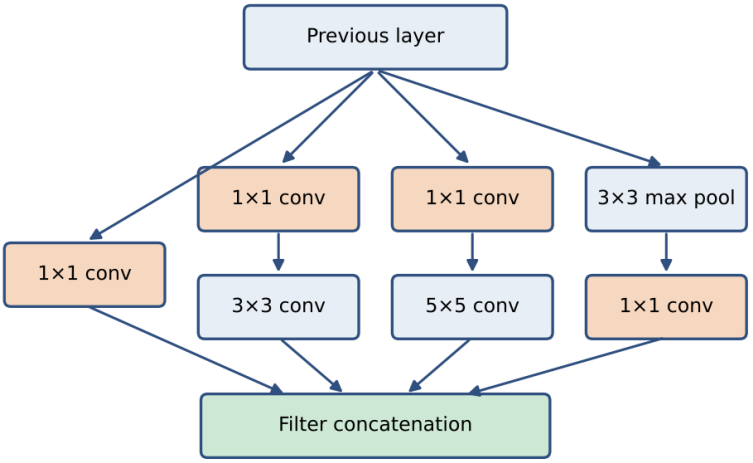



**Figure 2.27** The Inception module (GoogLeNet). Four parallel branches with different receptive fields are concatenated along the channel axis; $1 \times 1$ convolutions (shaded) reduce channel depth before the costly $3 \times 3$ and $5 \times 5$ convolutions.

copy the shallower one and set the extra layers to the identity. Their solution, the **residual block** in Figure 2.28, makes the identity easy to learn. Instead of asking a stack of layers to learn a desired mapping $\mathrm{H}(\mathbf{x})$ directly, the block learns the *residual* $\mathrm{F}(\mathbf{x}) = \mathrm{H}(\mathbf{x}) - \mathbf{x}$ and adds the input back through a **skip (shortcut) connection**, as written in Equation (2.58).

$$\begin{aligned} \mathbf{y} &= \mathcal{F}(\mathbf{x}; \{W_i\}) + \mathbf{x} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left( \frac{\partial \mathcal{F}}{\partial \mathbf{x}} + I \right) \end{aligned} \tag{2.58}$$

where $\mathcal{F}$ is the residual function computed by two or three convolution–batch-norm–ReLU layers with weights $\{W_i\}$. When channel counts or resolution differ, the shortcut becomes a $1 \times 1$ convolution $W_s\mathbf{x}$ (a *projection shortcut*).

The second line of Equation (2.58) reveals why this works so well: the identity term $I$ gives the gradient a direct path backwards through every block, so gradients cannot vanish simply because $\mathcal{F}$'s Jacobian is small. If the optimal mapping is close to the identity, the block only has to push $\mathcal{F}$ towards zero, which weight decay does naturally. ResNets with 50, 101 and 152 layers use a *bottleneck* block ($1 \times 1$ reduce, $3 \times 3$, $1 \times 1$ expand). An ensemble of ResNets won ILSVRC 2015 with 3.57% top-5 error, below the often-quoted human estimate of about 5%. Residual connections are now ubiquitous: every transformer layer in this part uses them.

Listing 2.19 implements the basic residual block in PyTorch, including the projection shortcut needed when a block changes resolution or channel count.

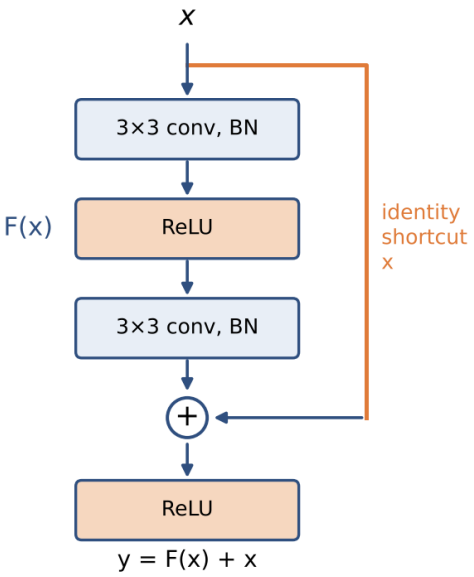



**Figure 2.28** A residual block. The input x bypasses two weight layers through an identity shortcut and is added to their output, so the layers need only learn the residual $F(x) = H(x) - x$.

**Listing 2.19** A ResNet basic block with optional projection shortcut (PyTorch).

```python
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.shortcut = nn.Identity()
        if stride != 1 or c_in != c_out:             # projection shortcut W_s x
            self.shortcut = nn.Sequential(
                nn.Conv2d(c_in, c_out, 1, stride, bias=False), nn.BatchNorm2d(c_out))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))              # residual F(x)
        return F.relu(out + self.shortcut(x))        # y = F(x) + x
```

## 2.24.6 Beyond ResNet

Subsequent architectures refined these ideas. **DenseNet** (Huang et al. 2017) concatenates each layer's output to all later layers. **MobileNet** (Howard et al. 2017) factorises

convolutions into depthwise and pointwise ($1 \times 1$) steps for mobile devices. **EfficientNet** (Tan & Le 2019) scales depth, width and resolution together using a compound coefficient. **ConvNeXt** (Liu et al. 2022) modernised the ResNet design with lessons from vision transformers and showed that pure CNNs remain competitive. Vision transformers themselves are discussed in the section on BERT, GPT and other transformer models.

> **Key Point** | **Lessons from a Decade of CNN Architectures**
>
> - Small kernels stacked deeply beat large kernels (VGG).
> - $1 \times 1$ convolutions are cheap channel mixers and bottlenecks (Inception, ResNet).
> - Skip connections are the key to training very deep networks (ResNet).
> - Global average pooling replaces heavy dense heads and reduces overfitting.

# 2.25 Image Classification and Object Recognition

**Image classification** assigns a single label, or a probability distribution over labels, to an entire image: "this is a photo of a golden retriever". **Object recognition** is the broader goal of identifying which objects are present in a scene; in the deep learning literature the term is often used interchangeably with classification, while *localising* objects is treated as detection (next section). Classification is the foundational task of computer vision: backbones trained on it serve as feature extractors for detection, segmentation, retrieval and captioning.

## 2.25.1 Problem Formulation and Pipeline

Given an image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$, a classifier outputs a vector of class probabilities $p(y \mid \mathbf{x})$ through a CNN or vision-transformer backbone followed by a softmax head, and is trained with the cross-entropy loss of Equation (2.52). Variants include **multi-label classification**, where several labels may apply and each output uses an independent sigmoid with binary cross-entropy, and **fine-grained classification** such as distinguishing 200 bird species, where differences between classes are subtle. Figure 2.29 shows the end-to-end workflow, from data preparation and augmentation through training to evaluation.

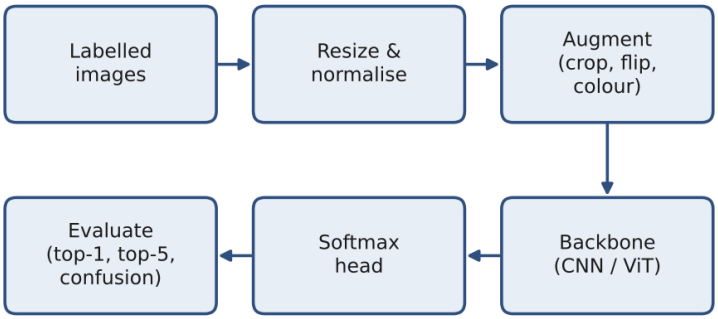



**Figure 2.29** End-to-end image classification workflow, from labelled images through augmentation, a pretrained backbone and fine-tuning to evaluation and deployment.

## 2.25.2 Training Recipes that Matter

Modern accuracy owes as much to training procedure as to architecture. **Data augmentation** generates label-preserving variants of each image: random resized crops, horizontal flips, colour jitter and, more aggressively, *Mixup* (convex combinations of two images and their labels), *CutMix* (pasting a patch of one image onto another) and automated policies such as RandAugment. **Label smoothing** replaces one-hot targets with $(1-\varepsilon)$ on the true class and $\varepsilon/(K-1)$ elsewhere, discouraging over-confident logits. Cosine learning-rate schedules with warm-up, weight decay and exponential moving averages of weights are further standard ingredients. Wightman, Touvron and Jégou (2021) showed that a plain ResNet-50 trained with such a recipe reaches about 80% top-1 accuracy on ImageNet, versus about 76% with the original 2015 procedure.

For small datasets, **transfer learning** is the default: take a backbone pretrained on ImageNet (or a larger dataset), replace the head and fine-tune, as in Listing 2.20. Transfer learning is treated in depth in a later section of this chapter; here it suffices to note that fine-tuning typically needs only hundreds to a few thousand labelled images per class.

**Listing 2.20** Fine-tuning a pretrained ResNet-50 for a new 5-class problem with torchvision.

```
import torch, torch.nn as nn
from torchvision import models, transforms

weights = models.ResNet50_Weights.IMAGENET1K_V2
model = models.resnet50(weights=weights)
for p in model.parameters():                        # freeze the backbone first
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 5)       # new trainable head
```

```

train_tf = transforms.Compose([
    transforms.RandomResizedCrop(224), transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(0.2, 0.2, 0.2), transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])

opt = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss(label_smoothing=0.1)
# ... train the head for a few epochs, then unfreeze layer4 and
# continue with a smaller learning rate (e.g. 1e-4) for the whole network
```

### 2.25.3 Evaluation

Classification performance is reported as **top-1 accuracy** (the highest-scoring class is correct) and, for many-class problems, **top-k accuracy**. Per-class precision, recall and F1 and the **confusion matrix** reveal which classes are confused; macro-averaged metrics should be used under class imbalance. **Calibration**, the agreement between predicted confidence and empirical accuracy, matters in safety-critical settings and can be improved post hoc with temperature scaling. Table 2.25 lists benchmark datasets that have driven progress.

**Table 2.25** Widely used image classification benchmarks.

| Dataset | Year | Classes | Images | Typical use |
|---|---|---|---|---|
| MNIST | 1998 | 10 | 70 K (28×28 grey) | Teaching, sanity checks |
| CIFAR-10 / 100 | 2009 | 10 / 100 | 60 K (32×32 colour) | Rapid architecture experiments |
| ImageNet-1k (ILSVRC) | 2012 | 1,000 | ≈ 1.28 M train | Standard large-scale benchmark, pretraining |
| Places365 | 2017 | 365 scenes | ≈ 1.8 M train | Scene recognition |
| iNaturalist | 2018 | 8,000+ species | ≈ 450 K+ | Fine-grained, long-tailed classification |

### 2.25.4 Interpreting Classifiers

Because CNNs are used in medicine, autonomous driving and security, it is important to understand *why* they predict a label. **Saliency maps** show the gradient of the class score with respect to the input pixels. **Grad-CAM** (Selvaraju et al. 2017) weights the final convolutional feature maps by the spatially averaged gradients of the class score and produces a coarse heatmap of the regions supporting the decision. Such tools often reveal *shortcut learning*: a pneumonia classifier that attends to hospital-specific text tokens on the X-ray rather than the lungs, or a "wolf" detector that relies on snowy backgrounds. Explainable AI is treated more fully elsewhere in the book.

**Case Study 2.4 | Dermatologist-Level Skin Cancer Classification**

Esteva et al. (2017, *Nature*) fine-tuned an Inception v3 network, pretrained on ImageNet, on about 129,000 clinical images covering over 2,000 skin diseases. On biopsy-proven test sets the network matched the performance of 21 board-certified dermatologists at distinguishing malignant from benign lesions.

The study illustrates both the power of transfer learning and its caveats: performance depends on the distribution of skin tones, cameras and lesion types in the training data, so external validation on diverse populations is essential before clinical deployment.

**Note | Robustness Caveats**

High benchmark accuracy does not guarantee robustness. CNNs can be fooled by *adversarial examples*—imperceptible perturbations crafted to change the prediction (Szegedy et al. 2014; Goodfellow, Shlens & Szegedy 2015)—and degrade under distribution shift such as new cameras, weather or image corruptions (Hendrycks & Dietterich 2019). Always evaluate on data that reflects deployment conditions.

## 2.26 Image Segmentation and Object Detection

Classification answers *what* is in an image; **object detection** and **image segmentation** also answer *where*. Detection localises each object instance with a bounding box and a class label. Segmentation assigns a label to every pixel: **semantic segmentation** labels pixels by class (all cars share one label), **instance segmentation** separates individual objects (car 1, car 2), and **panoptic segmentation** unifies both, labelling countable "things" by instance and amorphous "stuff" such as sky and road by class. These dense prediction tasks power autonomous driving, medical image analysis, robotics and satellite mapping.

### 2.26.1 Intersection over Union

Detection and segmentation quality hinge on how well a predicted region overlaps the ground truth. The standard measure is **intersection over union** (IoU), also called the Jaccard index, defined in Equation (2.59) and illustrated in Figure 2.30. IoU ranges from 0 (no overlap) to 1 (perfect match). A detection is usually counted as a true positive if its IoU with an unmatched ground-truth box of the same class exceeds a threshold, conventionally 0.5. For segmentation, the same formula applied to pixel sets gives per-class IoU, averaged over classes to obtain **mean IoU (mIoU)**; the closely related **Dice coefficient** $2|A \cap B|/(|A| + |B|)$ is popular in medical imaging.

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \tag{2.59}$$

where A and B are the predicted and ground-truth regions (boxes or pixel sets) and $|\cdot|$ denotes area.

**E Worked Example 2.13 | Computing IoU**

Ground truth box A has corners $(x_1, y_1, x_2, y_2) = (2, 2, 6, 6)$; prediction B = (4, 3, 8, 7).

- Intersection: x from max(2,4)=4 to min(6,8)=6, width 2; y from max(2,3)=3 to min(6,7)=6, height 3. Area = 6.
- Areas: $|A| = 4 \times 4 = 16$, $|B| = 4 \times 4 = 16$. Union = $16 + 16 - 6 = 26$.
- IoU = $6/26 \approx 0.23$, below the usual 0.5 threshold, so B would be a false positive.

## 2.26.2 Non-Maximum Suppression

Detectors typically produce many overlapping candidate boxes around each object. **Non-maximum suppression** (NMS), given in Algorithm 2.11, removes duplicates greedily: keep the highest-scoring box, discard all remaining boxes that overlap it by more than a threshold $\tau$ (typically 0.45–0.7), and repeat. NMS is applied per class. Its main weakness is crowded scenes, where genuinely distinct but overlapping objects suppress each other; *Soft-NMS* decays scores instead of deleting boxes, and transformer-based detectors such as DETR avoid NMS altogether by predicting a set of boxes directly.

**Algorithm 2.11** Greedy Non-Maximum Suppression (per class)

**Input:** Candidate boxes $B = \{b_1, \ldots, b_n\}$ with confidence scores $s_1, \ldots, s_n$; IoU threshold $\tau$; score threshold $\sigma$
**Output:** Kept boxes K

```
remove every box with s_i < σ from B
sort B by score in descending order
K ← ∅
while B is not empty do
    b* ← first (highest-scoring) box in B
    K ← K ∪ {b*}; remove b* from B
    for each box b in B do
        if IoU(b, b*) > τ then
            remove b from B
return K
```

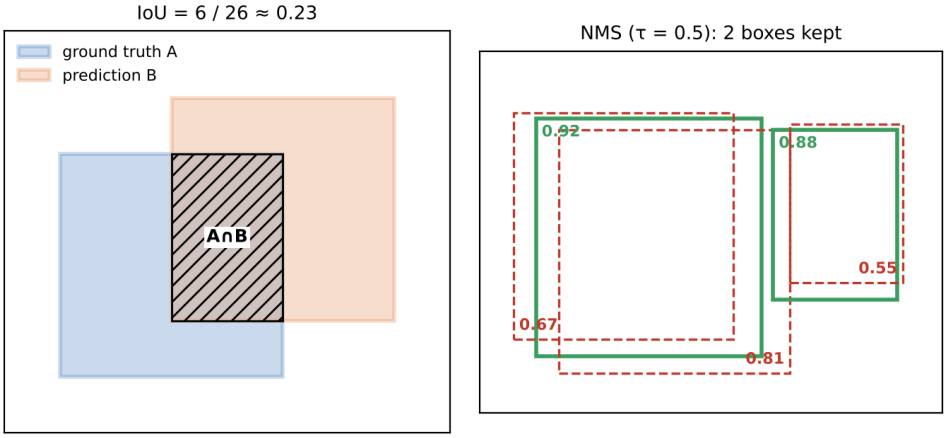



**Figure 2.30** Left: IoU is the area of overlap divided by the area of union of the predicted and ground-truth boxes. Right: NMS keeps the highest-scoring box (solid) and suppresses overlapping lower-scoring duplicates (dashed).

## 2.26.3 Evaluating Detectors: Mean Average Precision

Detectors are evaluated by **mean average precision** (mAP). For each class, detections across the test set are sorted by confidence; walking down the list and marking each as a true or false positive (using an IoU threshold) traces a precision–recall curve. **Average precision** (AP) is the area under this curve, computed with interpolated precision as in Equation (2.60), and mAP is the mean of AP over classes. The PASCAL VOC benchmark uses a single IoU threshold of 0.5 (mAP@0.5), whereas COCO averages mAP over ten thresholds from 0.5 to 0.95 in steps of 0.05 (written mAP@[.5:.95]), rewarding precise localisation.

$$\mathrm{AP} = \int_0^1 p_{\mathrm{interp}}(r)\,dr, \quad p_{\mathrm{interp}}(r) = \max_{\tilde{r} \geq r} p(\tilde{r})$$
$$\mathrm{mAP} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{AP}_k \tag{2.60}$$

where $p(r)$ is precision at recall $r$ and $K$ the number of classes; in practice the integral is a sum over recall points.

## 2.26.4 Two-Stage Detectors: the R-CNN Family

**R-CNN** (Girshick et al. 2014) brought CNNs to detection in a two-stage design: (1) generate about 2,000 class-agnostic *region proposals* with selective search, (2) warp each to a fixed size, extract CNN features and classify them with SVMs, refining boxes by

regression. It was accurate but very slow because the CNN ran on every proposal. **Fast R-CNN** (Girshick 2015) ran the CNN once per image and used **RoI pooling** to crop fixed-size features for each proposal from the shared feature map, training classification and box regression jointly. **Faster R-CNN** (Ren et al. 2015) replaced selective search with a learned **Region Proposal Network** (RPN) that slides over the feature map and predicts objectness scores and box offsets relative to a set of reference **anchor boxes** of multiple scales and aspect ratios. With a **Feature Pyramid Network** (FPN; Lin et al. 2017) that fuses features from several resolutions, Faster R-CNN remains a strong, accurate baseline.

Box regression in this family predicts offsets relative to an anchor or proposal $(x_a, y_a, w_a, h_a)$ rather than absolute coordinates, as in Equation (2.61); the parametrisation makes targets roughly scale-invariant and well conditioned for learning.

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a} \tag{2.61}$$

where $(x, y, w, h)$ are the centre, width and height of the target box and $t$ the regression targets, typically trained with the smooth-L1 loss.

## 2.26.5 One-Stage Detectors: YOLO and SSD

One-stage detectors skip the proposal step and predict boxes and classes densely in a single network pass, trading a little accuracy for large gains in speed. **YOLO** ("You Only Look Once"; Redmon et al. 2016) divides the image into an $S \times S$ grid; each cell predicts $B$ boxes with confidence scores and class probabilities, so detection becomes a single regression problem running at 45 frames per second on 2016 hardware. Successive versions added anchors, multi-scale prediction, better backbones and, more recently, anchor-free heads; YOLO variants are the default choice for real-time applications. **SSD** (Single Shot MultiBox Detector; Liu et al. 2016) predicts offsets for default boxes at several feature maps of different resolutions, so small objects are detected from high-resolution maps and large ones from coarse maps. **RetinaNet** (Lin et al. 2017) identified the extreme foreground–background imbalance of dense detectors as the reason they lagged two-stage ones and introduced the **focal loss** of Equation (2.62), which down-weights easy, well-classified examples.

$$\mathrm{FL}(p_t) = -\alpha_t \, (1 - p_t)^{\gamma} \log(p_t) \tag{2.62}$$

where $p_t$ is the predicted probability of the true class, $\gamma \geq 0$ a focusing parameter ($\gamma = 2$ typical) and $\alpha_t$ a class-balancing weight; $\gamma = 0$ recovers weighted cross-entropy.

## 2.26.6 Semantic Segmentation: FCN and U-Net

**Fully Convolutional Networks** (FCN; Long, Shelhamer & Darrell 2015) showed that replacing the dense layers of a classifier with convolutions and upsampling the coarse output with learned **transposed convolutions** yields pixel-wise predictions for images of any size. Fine detail is recovered by fusing upsampled predictions with earlier, higher-resolution features. **U-Net** (Ronneberger, Fischer & Brox 2015), designed for biomedical images with few training examples, pushed this idea to a symmetric **encoder–decoder** shown in Figure 2.31. The contracting path captures context; the expanding path restores resolution; and **skip connections** concatenate encoder features with decoder features at each scale so that precise boundaries are preserved. U-Net and its descendants dominate medical segmentation, and, as we shall see, the U-Net is also the denoising network inside most diffusion models. **DeepLab** (Chen et al. 2017–2018) instead uses *atrous* (dilated) convolutions and atrous spatial pyramid pooling to enlarge receptive fields without losing resolution.

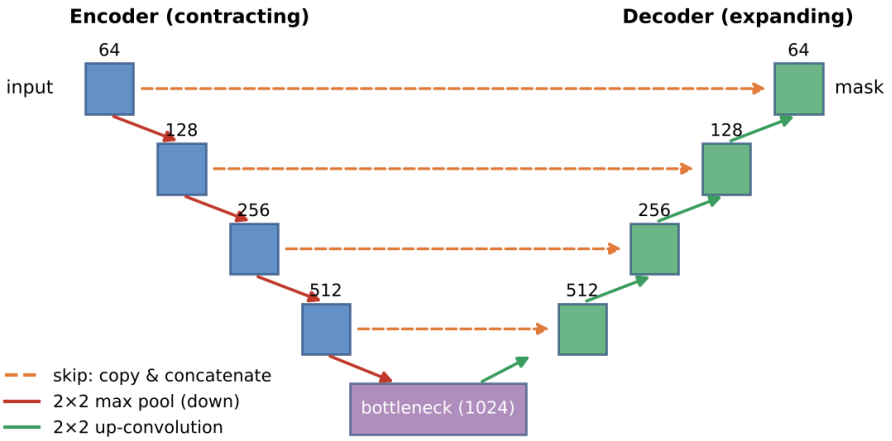



**Figure 2.31** U-Net encoder–decoder: the contracting path (left) halves resolution and doubles channels; the expanding path (right) upsamples, and skip connections concatenate encoder features at each scale.

## 2.26.7 Instance Segmentation: Mask R-CNN

**Mask R-CNN** (He, Gkioxari, Dollár & Girshick 2017) extends Faster R-CNN with a third, parallel head that predicts a small binary mask (e.g. $28 \times 28$) for each detected region, giving instance segmentation with little extra cost. A key technical contribution is **RoIAlign**, which replaces the coarse quantisation of RoI pooling with bilinear interpolation so that features align precisely with the input pixels, a detail that substantially improves mask accuracy. Mask predictions are decoupled from classifica-

tion: one mask per class is predicted with a per-pixel sigmoid, and the mask of the predicted class is selected. More recently, promptable foundation models such as the Segment Anything Model (Kirillov et al. 2023) segment arbitrary objects from point or box prompts. Table 2.26 summarises the main methods.

**Table 2.26** Summary of major detection and segmentation methods.

| Method | Year | Task | Type | Key idea |
|---|---|---|---|---|
| R-CNN | 2014 | Detection | Two-stage | CNN features on selective-search proposals |
| Fast R-CNN | 2015 | Detection | Two-stage | Shared feature map + RoI pooling |
| Faster R-CNN | 2015 | Detection | Two-stage | Learned Region Proposal Network with anchors |
| YOLO | 2016 | Detection | One-stage | Grid-based single regression; real-time |
| SSD | 2016 | Detection | One-stage | Multi-scale default boxes |
| RetinaNet | 2017 | Detection | One-stage | Focal loss for class imbalance |
| DETR | 2020 | Detection | Set prediction | Transformer + Hungarian matching; no NMS |
| FCN | 2015 | Semantic seg. | Encoder–decoder | Fully convolutional, learned upsampling |
| U-Net | 2015 | Semantic seg. | Encoder–decoder | Symmetric skips; few training images |
| DeepLab v3+ | 2018 | Semantic seg. | Encoder–decoder | Atrous convolution, ASPP |
| Mask R-CNN | 2017 | Instance seg. | Two-stage | Mask head + RoIAlign |

**Listing 2.21** Running a pretrained Faster R-CNN detector with torchvision.

```python
import torch
from torchvision.models.detection import fasterrcnn_resnet50_fpn,
↪  FasterRCNN_ResNet50_FPN_Weights
from torchvision.io import read_image

weights = FasterRCNN_ResNet50_FPN_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn(weights=weights, box_score_thresh=0.5).eval()
preprocess = weights.transforms()

img = read_image("street.jpg")                      # uint8 tensor C x H x W
with torch.no_grad():
    out = model([preprocess(img)])[0]               # NMS is applied internally

labels = [weights.meta["categories"][i] for i in out["labels"]]
for box, lab, sc in zip(out["boxes"], labels, out["scores"]):
    print(f"{lab:>12s}  {sc:.2f}  {box.int().tolist()}")
```

Listing 2.21 shows how little code is needed to run a modern detector; the real effort in practice goes into annotation, which is far more expensive for boxes and masks than for image labels.

**Case Study 2.5 | Perception in Autonomous Driving**

A self-driving perception stack runs object detection (vehicles, pedestrians, cyclists, signs) and semantic segmentation (drivable road, lane markings) on several cameras simultaneously, often fused with lidar. Latency budgets of tens of milliseconds favour one-stage detectors, while safety requirements demand high recall for vulnerable road users even at the cost of false alarms. Benchmarks such as KITTI, Cityscapes and nuScenes reflect these needs.

# 2.27 Recurrent Neural Networks (RNNs)

Many data are naturally sequential: words in a sentence, frames in a video, readings from a sensor, prices over time. A feedforward network expects inputs of fixed size and has no notion of order. **Recurrent neural networks** (RNNs) process a sequence one element at a time while maintaining a **hidden state** that summarises everything seen so far. The same weights are applied at every time step, so an RNN can handle sequences of any length, and parameter sharing across time plays the same role that sharing across space plays in CNNs. The modern form was popularised by Elman (1990), building on earlier work by Jordan (1986) and others.

## 2.27.1 The Recurrence

At time step t the RNN combines the current input $\mathbf{x}_t$ with the previous hidden state $\mathbf{h}_{t-1}$ to produce a new state and, optionally, an output, as in Equation (2.63). Because $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, which depends on $\mathbf{h}_{t-2}$, and so on, the state is in principle a function of the entire history $\mathbf{x}_1, \ldots, \mathbf{x}_t$.

$$\begin{aligned} \mathbf{h}_t &= \tanh\left(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b}_h\right) \\ \hat{\mathbf{y}}_t &= \operatorname{softmax}(W_{hy}\mathbf{h}_t + \mathbf{b}_y) \end{aligned} \tag{2.63}$$

where $W_{xh}$, $W_{hh}$ and $W_{hy}$ are the input-to-hidden, hidden-to-hidden (recurrent) and hidden-to-output weight matrices, shared across all time steps, and $\mathbf{h}_0$ is usually a zero vector.

The compact loop diagram on the left of Figure 2.32 is equivalent to the **unrolled** computational graph on the right: one copy of the cell per time step, all sharing parameters. Unrolling turns the RNN into a deep feedforward network whose depth equals the sequence length, which is precisely how it is trained.

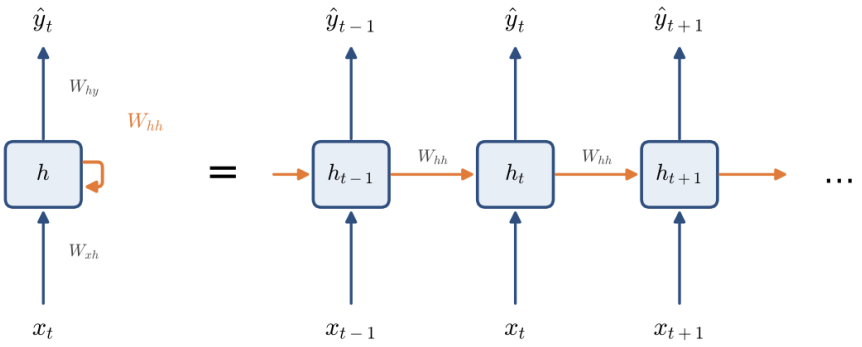



**Figure 2.32** An RNN shown as a compact loop (left) and unrolled through time (right). The same weights $W_{xh}$, $W_{hh}$ and $W_{hy}$ are reused at every time step.

## 2.27.2 Input–Output Configurations

RNNs support several mappings between input and output sequences, listed in Table 2.27. In *many-to-one* tasks only the final hidden state feeds a classifier; in *many-to-many* tasks of equal length an output is emitted at every step; and in *sequence-to-sequence* tasks of differing lengths an encoder RNN summarises the input and a decoder RNN generates the output, an architecture examined in the section on sequence modelling.

**Table 2.27** Input–output configurations of recurrent networks.

| Configuration | Input | Output | Example |
|---|---|---|---|
| One-to-many | Single vector | Sequence | Image captioning, music generation |
| Many-to-one | Sequence | Single label/value | Sentiment classification, activity recognition |
| Many-to-many (aligned) | Sequence | Sequence of same length | Part-of-speech tagging, frame labelling |
| Many-to-many (seq2seq) | Sequence | Sequence of different length | Machine translation, summarisation |

Two common extensions are **bidirectional RNNs** (Schuster & Paliwal 1997), which run one RNN forward and another backward over the sequence and concatenate their states so that each position sees both past and future context (useful for tagging, not for real-time prediction), and **stacked (deep) RNNs**, where the hidden-state sequence of one layer is the input sequence of the next.

## 2.27.3 Backpropagation Through Time

RNNs are trained with **backpropagation through time** (BPTT): unroll the network over the sequence, compute the total loss $\mathcal{L} = \Sigma_t \mathcal{L}_t$, and apply ordinary backpropaga-

tion to the unrolled graph, summing the gradient contributions to each shared weight across time steps (Algorithm 2.12). The gradient of a loss at time t with respect to an earlier hidden state $\mathbf{h}_k$ involves a product of Jacobians, as shown in Equation (2.64).

$$\frac{\partial \mathcal{L}_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left( \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial^{+} \mathbf{h}_k}{\partial W_{hh}}$$
$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\!\left(1 - \mathbf{h}_j^2\right) W_{hh} \tag{2.64}$$

where $\partial^{+}\mathbf{h}_k / \partial W_{hh}$ denotes the immediate partial derivative treating $\mathbf{h}_{k-1}$ as constant; the second line uses $\tanh'(z) = 1 - \tanh^2(z)$.

**Algorithm 2.12** Truncated Backpropagation Through Time

**Input:** Sequence $(x_1, \dots, x_T)$ with targets $(y_1, \dots, y_T)$; RNN parameters $\theta = \{W_{xh}, W_{hh}, W_{hy}, b\}$; truncation length $k$; learning rate $\eta$

**Output:** Updated parameters $\theta$

$h \leftarrow 0$
**for each** chunk $[t_0, t_0 + k - 1]$ of the sequence **do**
  detach $h$ from the previous graph (stop gradients flowing further back)
  **for** $t = t_0$ **to** $t_0 + k - 1$ **do** ▷ *forward pass*
    $h_t \leftarrow \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h)$; $\hat{y}_t \leftarrow \operatorname{softmax}(W_{hy}\, h_t + b_y)$
    $\mathcal{L} \leftarrow \mathcal{L} + \operatorname{loss}(\hat{y}_t, y_t)$
  compute $\nabla_\theta \mathcal{L}$ by backpropagating from $t_0 + k - 1$ down **to** $t_0$, summing over time steps
  clip gradient: **if** $\|\nabla_\theta \mathcal{L}\| > c$ **then** $\nabla_\theta \mathcal{L} \leftarrow c\, \nabla_\theta \mathcal{L} \,/\, \|\nabla_\theta \mathcal{L}\|$
  $\theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}$; $\mathcal{L} \leftarrow 0$
**return** $\theta$

## 2.27.4 Vanishing and Exploding Gradients

The product of $t - k$ Jacobians in Equation (2.64) is the root of the RNN's main weakness. Roughly, if the largest singular value of $W_{hh}$ (scaled by the tanh derivative, which is at most 1) is below one, the product shrinks exponentially with the time gap and gradients **vanish**; if it exceeds one, they can **explode** (Bengio, Simard & Frasconi 1994; Pascanu, Mikolov & Bengio 2013). Exploding gradients are easily tamed by **gradient clipping**, which rescales the gradient whenever its norm exceeds a threshold, as in the penultimate step of Algorithm 2.12. Vanishing gradients are more insidious: the network simply cannot learn dependencies spanning more than ten or twenty steps, because the error signal from distant time steps is negligible. Truncated BPTT, which limits backpropagation to $k$ steps for efficiency, also caps the dependencies that can be learned. Figure 2.33 visualises the effect. The cure was architectural: gated units, introduced next.

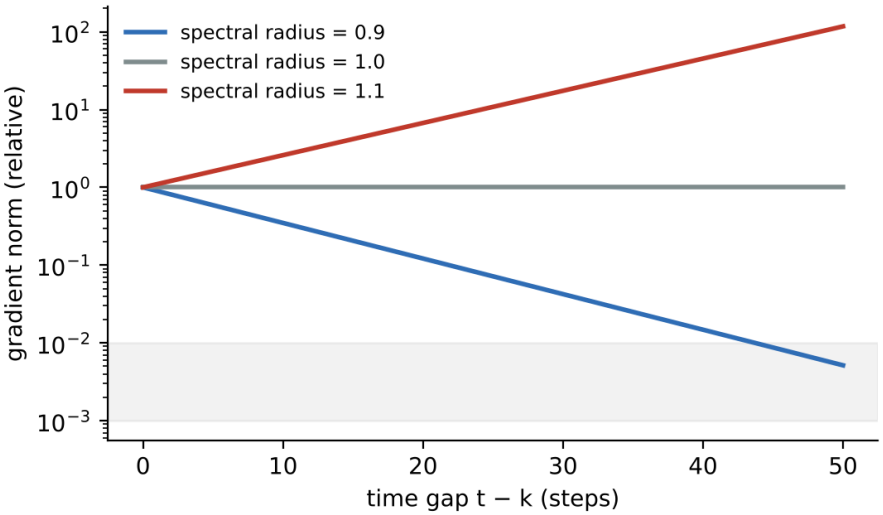



**Figure 2.33** Norm of the back-propagated gradient $\|\partial h_t / \partial h_k\|$ versus time gap $t - k$ for a simple RNN whose recurrent matrix has spectral radius 0.9, 1.0 or 1.1 (log scale): gradients vanish or explode exponentially.

**Listing 2.22** A many-to-one RNN text classifier in PyTorch, with gradient clipping.

```python
import torch, torch.nn as nn

class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, emb_dim=128, hidden=256, n_classes=2):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, emb_dim, padding_idx=0)
        self.rnn = nn.RNN(emb_dim, hidden, batch_first=True, nonlinearity="tanh")
        self.fc = nn.Linear(hidden, n_classes)
    def forward(self, tokens):                      # tokens: (batch, seq_len)
        _, h_T = self.rnn(self.emb(tokens))         # h_T: (1, batch, hidden)
        return self.fc(h_T[-1])                     # classify from final state

model = RNNClassifier(vocab_size=20_000)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for tokens, labels in train_loader:
    opt.zero_grad()
    loss = nn.functional.cross_entropy(model(tokens), labels)
    loss.backward()                                 # BPTT through the unrolled graph
    nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    opt.step()
```

Listing 2.22 shows a simple RNN classifier; swapping `nn.RNN` for `nn.LSTM` or `nn.GRU` changes only one line, which is how most practitioners use the gated variants below.

**Key Point | RNN Essentials**

- An RNN applies the same cell at every time step, carrying a hidden state that summarises the past.
- Training uses backpropagation through time on the unrolled graph.
- Products of Jacobians make gradients vanish or explode; clipping fixes explosion, gating fixes vanishing.

# 2.28 Long Short-Term Memory Networks (LSTMs)

The **Long Short-Term Memory** network, introduced by Hochreiter and Schmidhuber (1997) and extended with the forget gate by Gers, Schmidhuber and Cummins (2000), was designed specifically to overcome the vanishing gradient problem. For nearly two decades LSTMs were the workhorse of sequence modelling, powering speech recognition, handwriting recognition, machine translation (including Google's 2016 neural translation system) and language modelling, until transformers took over many of these roles. They remain widely used for time series and on-device applications.

## 2.28.1 The Cell State and Gating Idea

The LSTM's key innovation is a separate **cell state** $\mathbf{c}_t$, a memory vector that runs through the sequence and is modified only by element-wise multiplication and addition. Three **gates**, each a sigmoid layer producing values between 0 and 1, control the flow of information: the **forget gate** decides what to erase from the cell, the **input gate** decides what new information to write, and the **output gate** decides what part of the cell to expose as the hidden state. Because the gates are themselves computed from the input and the previous hidden state, the network *learns* when to remember and when to forget. Figure 2.34 depicts the cell.

## 2.28.2 Gate Equations

The full LSTM update at time step t is given by Equation (2.65) (gates and candidate) and Equation (2.66) (state updates).

$$\begin{aligned}\mathbf{f}_t &= \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f)\\ \mathbf{i}_t &= \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i)\\ \mathbf{o}_t &= \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o)\\ \tilde{\mathbf{c}}_t &= \tanh\left(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c\right)\end{aligned} \tag{2.65}$$

where $\mathbf{f}_t$, $\mathbf{i}_t$ and $\mathbf{o}_t$ are the forget, input and output gates, $\tilde{\mathbf{c}}_t$ the candidate cell content, $\sigma$ the logistic sigmoid, W_• input weight matrices, U_• recurrent weight matrices and $\mathbf{b}$_• biases.

$$\begin{aligned}\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t\\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh\left(\mathbf{c}_t\right)\end{aligned} \tag{2.66}$$

where $\odot$ denotes element-wise (Hadamard) multiplication.

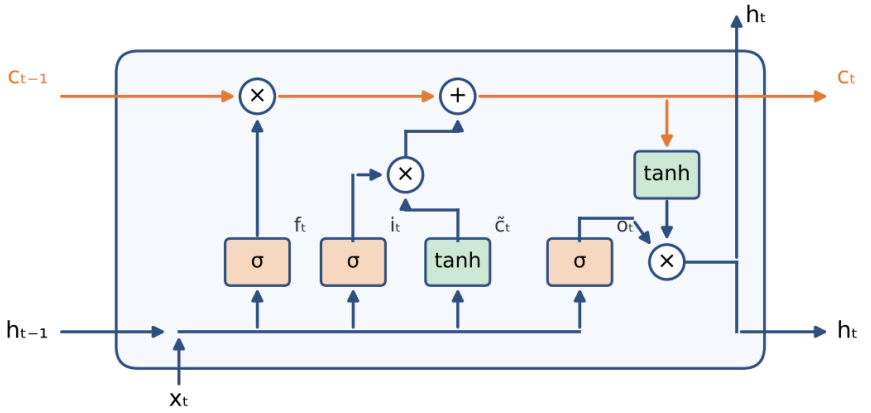



**Figure 2.34** The LSTM cell. The cell state $c_t$ (top line) is updated additively: the forget gate $f_t$ scales the old memory, the input gate $i_t$ admits the candidate $\tilde{c}_t$, and the output gate $o_t$ filters $\tanh(c_t)$ to form the hidden state $h_t$.

## 2.28.3 Why the LSTM Preserves Gradients

Consider the gradient of the cell state with respect to its predecessor. From the first line of Equation (2.66), the direct path gives $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f}_t)$ (ignoring the indirect dependence through the gates). When the forget gate is close to one the cell state is copied almost unchanged and the gradient flows back through many steps without shrinking—the **constant error carousel** of the original paper. Unlike the simple RNN, where the Jacobian involves repeated multiplication by the same matrix $W_{hh}$, here the multiplication is element-wise by gate values the network controls. This is the same idea as the residual connection in ResNet: an additive path that lets gradients bypass

the non-linear transformations. A common practical trick is to initialise the forget-gate bias $\mathbf{b}_f$ to 1 or 2 so that the network remembers by default early in training (Jozefowicz, Zaremba & Sutskever 2015).

**E Worked Example 2.14 | One LSTM Step with Scalar State**

Let the hidden size be 1 and suppose, after computing the pre-activations, $f = \sigma(2.0) \approx 0.881$, $i = \sigma(0.0) = 0.5$, $o = \sigma(1.0) \approx 0.731$ and $\tilde{c} = \tanh(0.5) \approx 0.462$. The previous cell state is $c_{-1} = 0.8$.

- New cell state: $c = 0.881 \times 0.8 + 0.5 \times 0.462 = 0.705 + 0.231 = 0.936$.
- Hidden state: $h = 0.731 \times \tanh(0.936) = 0.731 \times 0.733 \approx 0.536$.

Most of the old memory (0.705 of 0.8) survived; half of the candidate was written in; and the output gate exposed about 73% of the squashed cell.

## 2.28.4 Parameters, Variants and Practice

An LSTM layer with input dimension $d$ and hidden size $n$ has four sets of weights, so its parameter count is $4(nd + n^2 + n)$, four times that of a simple RNN of the same size. Popular variants include **peephole connections** (Gers & Schmidhuber 2000), which let gates look at the cell state, and the **coupled input–forget gate**, which sets $\mathbf{i}_t = 1 - \mathbf{f}_t$. A large empirical study by Greff et al. (2017) found that none of eight variants significantly outperformed the standard LSTM, and that the forget gate and output activation are its most critical components. Listing 2.23 builds a stacked bidirectional LSTM classifier in Keras.

**Listing 2.23** A stacked bidirectional LSTM sentiment classifier in Keras.

```
from tensorflow import keras
from tensorflow.keras import layers

vocab_size, max_len = 20_000, 200
model = keras.Sequential([
    keras.Input(shape=(max_len,), dtype="int32"),
    layers.Embedding(vocab_size, 128, mask_zero=True),     # mask padding tokens
    layers.Bidirectional(layers.LSTM(64, return_sequences=True)),  # full sequence
    layers.Bidirectional(layers.LSTM(32)),                  # final state only
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer=keras.optimizers.Adam(1e-3, clipnorm=1.0),
              loss="binary_crossentropy", metrics=["accuracy"])
# model.fit(x_train, y_train, validation_split=0.1, epochs=5, batch_size=64)
```

> **Note | Limitations of LSTMs**
>
> LSTMs mitigate but do not eliminate the difficulty of very long-range dependencies (hundreds or thousands of steps), and their strictly sequential computation prevents parallelisation across time during training. Both limitations motivated attention mechanisms and the transformer, covered later in this part.

## 2.29 Gated Recurrent Units (GRUs)

The **Gated Recurrent Unit** (GRU), proposed by Cho et al. (2014) in the paper that also introduced the RNN encoder–decoder for machine translation, is a streamlined gated cell. It merges the LSTM's cell and hidden states into a single state vector and uses two gates instead of three, giving fewer parameters and slightly faster computation while retaining the ability to model long-range dependencies.

### 2.29.1 GRU Equations

The GRU uses an **update gate** $\mathbf{z}_t$, which plays the combined role of the LSTM's forget and input gates, and a **reset gate** $\mathbf{r}_t$, which controls how much of the previous state is used when computing the candidate. The update is given in Equation (2.67).

$$\begin{aligned} \mathbf{z}_t &= \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z) \\ \mathbf{r}_t &= \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r) \\ \tilde{\mathbf{h}}_t &= \tanh\left(W_h \mathbf{x}_t + U_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h\right) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \end{aligned} \tag{2.67}$$

where $\mathbf{z}_t$ is the update gate, $\mathbf{r}_t$ the reset gate and $\tilde{\mathbf{h}}_t$ the candidate state. (Some references, including PyTorch's documentation, swap the roles of $\mathbf{z}_t$ and $1 - \mathbf{z}_t$ in the last line; the two conventions are equivalent.)

The last line of Equation (2.67) is an interpolation between the old state and the candidate. When $\mathbf{z}_t \approx 0$, the state is copied forward unchanged, providing the same additive gradient highway as the LSTM's cell state. When the reset gate $\mathbf{r}_t \approx 0$, the candidate ignores the previous state and is computed from the current input alone, letting the unit "start afresh" at boundaries such as the end of a sentence. Units that learn to capture short-term dependencies tend to have active reset gates, while those capturing long-term dependencies have update gates that stay mostly closed. Figure 2.35 shows the data flow.

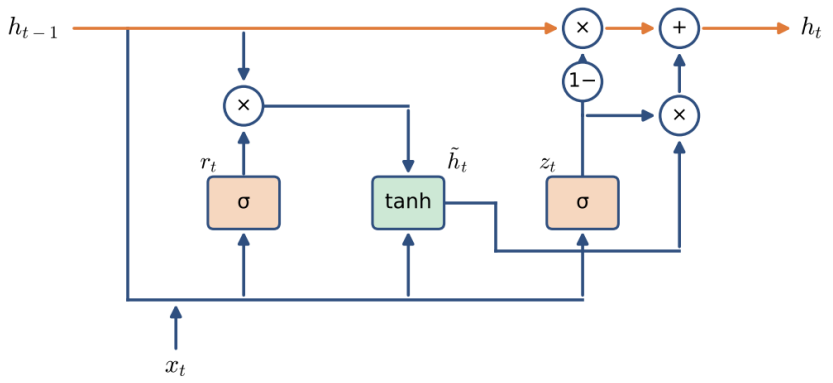



**Figure 2.35** The GRU cell. The reset gate $r_t$ modulates the previous state when forming the candidate $\tilde{h}_t$; the update gate $z_t$ interpolates between $h_{t-1}$ and $\tilde{h}_t$.

## 2.29.2 LSTM versus GRU

Empirical comparisons (Chung et al. 2014; Jozefowicz, Zaremba & Sutskever 2015) found that GRUs and LSTMs perform comparably on most tasks, with neither consistently superior; the best choice is task-dependent and is usually settled by validation. GRUs train somewhat faster and can generalise better on small datasets thanks to fewer parameters, whereas LSTMs, with their separate memory and output gating, sometimes have the edge on tasks requiring precise counting or very long memory. Table 2.28 summarises the differences.

**Table 2.28** Comparison of LSTM and GRU cells (input size d, hidden size n).

| Aspect | LSTM | GRU |
|---|---|---|
| Year / authors | 1997, Hochreiter & Schmidhuber (forget gate 2000) | 2014, Cho et al. |
| Gates | 3: forget, input, output | 2: update, reset |
| State vectors | Cell state c and hidden state h | Single hidden state h |
| Memory update | $c = f \odot c_{-1} + i \odot \tilde{c}$ (independent gates) | $h = (1-z) \odot h_{-1} + z \odot \tilde{h}$ (coupled) |
| Output exposure | Gated by o, through tanh(c) | Entire state exposed |
| Parameters per layer | $4(nd + n^2 + n)$ | $3(nd + n^2 + n)$ |
| Speed | Slower | $\approx$ 25% fewer operations per step |
| Typical strengths | Very long dependencies, large datasets | Smaller datasets, faster training |

**E Worked Example 2.15 | Parameter Counts**

For input size d = 100 and hidden size n = 128:

- Simple RNN: $nd + n^2 + n = 12{,}800 + 16{,}384 + 128 = 29{,}312$.
- GRU: $3 \times 29{,}312 = 87{,}936$. LSTM: $4 \times 29{,}312 = 117{,}248$.

(Frameworks such as PyTorch and cuDNN use two bias vectors per gate, adding 3n or 4n parameters.)

**T Practice Tip | Choosing a Recurrent Cell**

- Default to a GRU for speed or an LSTM for very long memory; try both if time allows.
- Use 1–3 layers, dropout between layers, gradient clipping and layer normalisation for stability.
- For long texts or when parallel training matters, consider transformers or temporal convolutional networks instead.

# 2.30 Sequence Modeling and Time-Series Forecasting

**Sequence modelling** is the task of learning a probability distribution over, or a mapping between, ordered sequences. Language models predict the next word; speech recognisers map audio frames to characters; translators map a sentence in one language to a sentence in another; and forecasters map the history of a time series to its future values. The recurrent architectures of the previous three sections, and the attention-based models that follow, are general tools for all of these problems. This section covers two important frameworks: the encoder–decoder (sequence-to-sequence) model and deep learning for time-series forecasting.

## 2.30.1 Autoregressive Factorisation

Most generative sequence models rely on the chain rule of probability to factorise the joint distribution of a sequence into a product of next-step conditionals, as in Equation (2.68). A model trained to predict each element from its predecessors can then generate new sequences one step at a time by sampling and feeding each output back as the next input. Training maximises the log-likelihood, i.e. minimises the sum of per-step cross-entropies.

$$p(y_1, \dots, y_T) = \prod_{t=1}^{T} p(y_t \mid y_1, \dots, y_{t-1}), \qquad \mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}) \tag{2.68}$$

where y_<t denotes all elements before position t and θ the model parameters.

## 2.30.2 Sequence-to-Sequence Models

Sutskever, Vinyals and Le (2014) and Cho et al. (2014) introduced the **encoder–decoder** or **sequence-to-sequence (seq2seq)** architecture for problems where input and output have different lengths, such as translation. An **encoder** RNN reads the source sequence $\mathbf{x}_1$,…,**x**_S and compresses it into a fixed-length **context vector** $\mathbf{c}$, typically its final hidden state. A **decoder** RNN, initialised with $\mathbf{c}$, generates the target sequence token by token, modelling the conditional in Equation (2.69). Generation starts from a special <bos> (beginning-of-sequence) token and ends when the decoder emits <eos>. Sutskever et al. found that reversing the order of the source sentence helped, because it shortened the distance between the first source words and the first target words.

$$\begin{aligned} \mathbf{c} &= \mathrm{Encoder}(\mathbf{x}_1, \dots, \mathbf{x}_S), \qquad \mathbf{s}_t = \mathrm{RNN}(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c}) \\ p(\mathbf{y}_1, \dots, \mathbf{y}_T \mid \mathbf{x}) &= \prod_{t=1}^{T} p(\mathbf{y}_t \mid \mathbf{y}_{<t}, \mathbf{c}) = \prod_{t=1}^{T} \mathrm{softmax}(W_o \mathbf{s}_t)_{y_t} \end{aligned} \tag{2.69}$$

where $\mathbf{s}_t$ is the decoder hidden state and $W_o$ projects it onto the output vocabulary.

During training the decoder is fed the *ground-truth* previous token rather than its own prediction, a technique called **teacher forcing** that makes training stable and parallelisable over time steps in the loss computation. At inference time the model must consume its own outputs, and the mismatch between the two regimes (**exposure bias**) can cause errors to compound; *scheduled sampling* (Bengio et al. 2015) gradually replaces ground-truth inputs with model predictions during training. The single fixed-length context vector is the model's bottleneck: performance degrades for long sentences because all information must squeeze through $\mathbf{c}$. Attention, introduced in a later section, removes this bottleneck. Figure 2.36 illustrates the architecture.

## 2.30.3 Time-Series Forecasting with Deep Learning

A **time series** is a sequence of observations $y_1, y_2, \dots$ indexed by time, possibly accompanied by covariates (exogenous features) such as calendar effects, prices or weather. **Forecasting** predicts future values $y_{T+1}$,…,$y_{T+H}$ over a **horizon** H. Classical methods such as ARIMA and exponential smoothing remain strong baselines for single, short

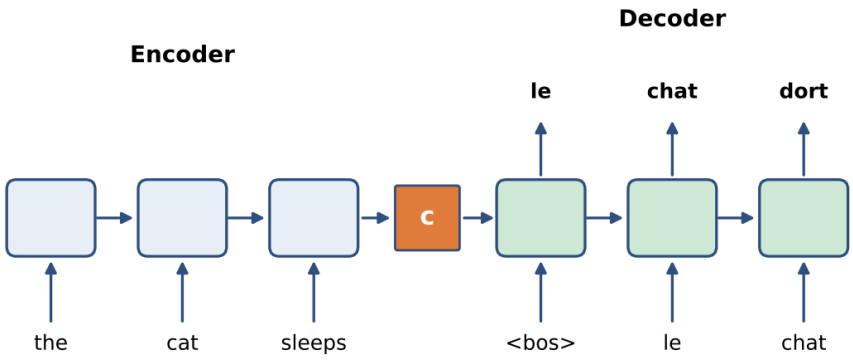



**Figure 2.36** Encoder–decoder (seq2seq) model translating a three-word sentence. The encoder's final state c is the only channel through which source information reaches the decoder.

series; deep learning shines when there are many related series (thousands of products in a retailer, sensors in a grid), long histories, rich covariates and non-linear dynamics, because a single **global model** can be trained across all series and learn shared patterns.

### Framing forecasting as supervised learning: windowing

Neural forecasters are trained on fixed-length **sliding windows**. A window of the past L observations (the *lookback* or *context*) is the input and the next H observations the target, as in Equation (2.70); sliding the window by one step across the series yields many training pairs. Inputs are usually normalised per series (e.g. by dividing by the window mean) so that one model can serve series of very different scales. Data must be split **chronologically**—train on the past, validate and test on later periods—never randomly, otherwise future information leaks into training.

$$\mathbf{X}^{(t)} = (y_{t-L+1}, \dots, y_t), \quad \mathbf{Y}^{(t)} = (y_{t+1}, \dots, y_{t+H})$$
$$\hat{\mathbf{Y}}^{(t)} = f_\theta\big(\mathbf{X}^{(t)}, \mathbf{u}^{(t)}\big), \quad t = L, \dots, T-H \tag{2.70}$$

where L is the lookback length, H the forecast horizon and $\mathbf{u}^{(t)}$ optional covariates known at time t.

Multi-step forecasts can be produced **recursively** (predict one step, append it to the input, repeat), which can accumulate error, or **directly** (output all H values at once from a vector head or a seq2seq decoder), which is generally more robust. Probabilistic forecasters output a distribution or quantiles rather than a point; the **quantile (pinball) loss** of Equation (2.71) trains a network to predict the q-th quantile, giving prediction intervals.

$$\ell_q(y, \hat{y}) = \max\left(q\,(y - \hat{y}),\ (q-1)(y - \hat{y})\right) \tag{2.71}$$

where $q \in (0, 1)$ is the target quantile; $q = 0.5$ gives half the absolute error.

Figure 2.37 shows an LSTM-style forecast of a noisy seasonal series with an 80% prediction interval obtained from the 0.1 and 0.9 quantiles. Listing 2.24 shows how to build windows and train an LSTM forecaster, and Table 2.29 lists widely used deep forecasting architectures.

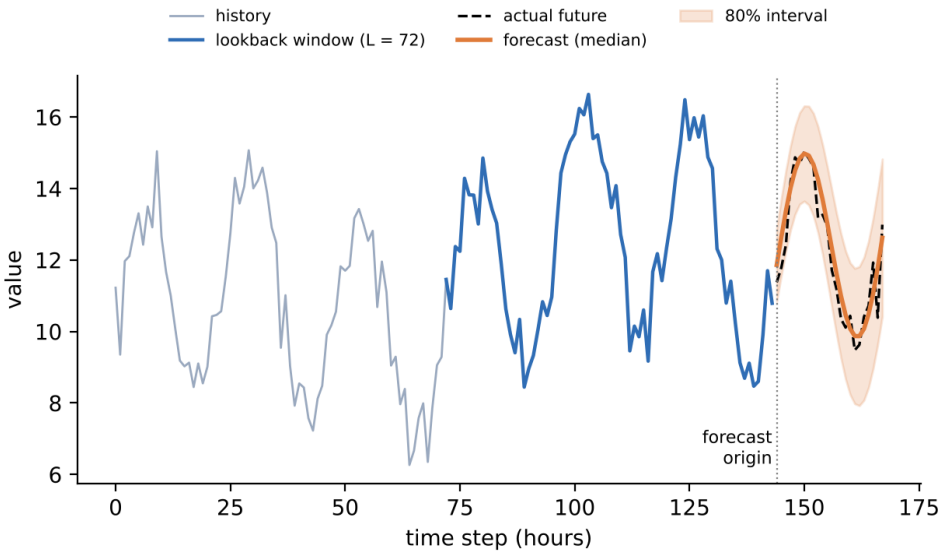



**Figure 2.37** Forecasting a seasonal series: the model sees a lookback window of history (blue) and predicts the next 24 steps (orange) with an 80% prediction interval from quantile outputs; the dashed line is the actual future.

**Listing 2.24** Sliding-window dataset and a direct multi-step LSTM forecaster in PyTorch.

```
import numpy as np, torch, torch.nn as nn

def make_windows(series, L=72, H=24):
    X, Y = [], []
    for t in range(L, len(series) - H + 1):
        X.append(series[t-L:t]); Y.append(series[t:t+H])
    return torch.tensor(np.array(X), dtype=torch.float32).unsqueeze(-1), \
           torch.tensor(np.array(Y), dtype=torch.float32)

class LSTMForecaster(nn.Module):
    def __init__(self, hidden=64, H=24):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden, num_layers=2, batch_first=True, dropout=0.1)
        self.head = nn.Linear(hidden, H)              # direct strategy: all H steps at
        ↪ once
    def forward(self, x):                             # x: (batch, L, 1)
```

```
        out, _ = self.lstm(x)
        return self.head(out[:, -1])                    # (batch, H)

series = (series - series[:split].mean()) / series[:split].std()   # scale with TRAIN
↪ stats only
X_tr, Y_tr = make_windows(series[:split])
model, loss_fn = LSTMForecaster(), nn.L1Loss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(30):
    opt.zero_grad(); loss = loss_fn(model(X_tr), Y_tr); loss.backward(); opt.step()
```

**Table 2.29** Deep learning architectures for time-series forecasting.

| Model | Year | Core idea | Notes |
|---|---|---|---|
| LSTM / GRU seq2seq | 2014+ | Recurrent encoder–decoder | Simple, strong baseline |
| Temporal Convolutional Network (TCN) | 2018 | Causal, dilated 1D convolutions | Parallel training; long receptive field |
| DeepAR | 2017–2020 | Autoregressive RNN with likelihood outputs | Probabilistic; global model over many series |
| N-BEATS | 2020 | Stacked MLP blocks with backcast/forecast residuals | Pure deep model; interpretable variant |
| Temporal Fusion Transformer | 2021 | Attention + gating + variable selection | Multi-horizon, interpretable |
| PatchTST | 2023 | Transformer on patches of the series | Channel-independent; long horizons |

> **Note | Evaluating Forecasts**
>
> Use rolling-origin (walk-forward) evaluation: repeatedly train on data up to time t and test on the next H steps. Report scale-free metrics such as MAPE (beware division by values near zero), sMAPE or MASE, which compares the error against a seasonal-naïve forecast. Always include naïve and classical baselines: Zeng et al. (2023) showed that a single linear layer can rival several transformer forecasters on popular long-horizon benchmarks.

## 2.31 Natural Language Processing (NLP) with Deep Learning

**Natural language processing** (NLP) builds systems that read, understand and generate human language. Until the early 2010s NLP relied on hand-engineered features—n-grams, part-of-speech tags, parse trees—fed to linear models such as logistic regression and conditional random fields. Deep learning transformed the field in three waves: dense **word embeddings** (2013), **recurrent and attention-based seq2seq** models (2014–2016), and **pretrained transformers** (2018 onwards), which now underpin

virtually every NLP application. This section sets out the pipeline and the core tasks; the following sections develop each component.

### 2.31.1 From Text to Tensors: Tokenisation

Neural networks operate on numbers, so text must first be split into **tokens** and mapped to integer ids from a fixed **vocabulary**. The choice of token unit involves a trade-off. **Word-level** tokenisation is intuitive but produces huge vocabularies and cannot handle unseen words, which are mapped to an `<unk>` token. **Character-level** tokenisation has a tiny vocabulary and no unknown words, but sequences become very long and individual characters carry little meaning. Modern systems use **sub-word tokenisation**, which keeps frequent words whole and splits rare words into meaningful pieces: "unhappiness" might become ["un", "happi", "ness"]. The main algorithms are **Byte-Pair Encoding** (BPE; Sennrich, Haddow & Birch 2016), used by GPT models; **WordPiece** (used by BERT); and **SentencePiece/Unigram** (Kudo 2018), which treats text as a raw stream including spaces and is popular for multilingual models. Byte-level BPE operates on UTF-8 bytes, guaranteeing that any string can be encoded. Algorithm 2.13 gives the BPE training procedure.

**Algorithm 2.13** Byte-Pair Encoding (BPE) Vocabulary Learning

**Input:** Training corpus; target number of merges M
**Output:** Ordered list of merge rules and the resulting vocabulary V

split every word into characters, appending an **end**-of-word marker
V ← set of all characters
**for** m = 1 **to** M **do**
    count the frequency of every adjacent symbol pair (a, b) **in** the corpus
    (a*, b*) ← most frequent pair
    replace every occurrence of a* b* by the new symbol a*b*
    V ← V ∪ {a*b*}; record merge rule (a*, b*)
**return** merge rules **and** V
▷ *to tokenise new text, apply the merge rules in the order learned*

**E Worked Example 2.16 | BPE Merges**

Corpus word counts: low ×5, lower ×2, newest ×6, widest ×3. Start from characters.

- Most frequent pair: (e, s) appears in newest (6) and widest (3) = 9 times → merge to "es".
- Next: (es, t) appears 9 times → "est". Next: (l, o) appears 7 times → "lo"; then (lo, w) → "low".

After four merges "lowest", never seen in training, tokenises as ["low", "est"].

### 2.31.2 The Deep NLP Pipeline

Figure 2.38 shows the standard modern pipeline. Raw text is normalised and tokenised; token ids are mapped to dense vectors by an **embedding layer**; a **contextual encoder**—historically a BiLSTM, now almost always a transformer—turns these into context-dependent representations; and a lightweight **task head** maps them to outputs. A sequence-level task such as sentiment analysis pools the representations (or uses a special `[CLS]` token), a token-level task such as named-entity recognition classifies each position, and a generation task uses a decoder to emit text. Because the encoder is pre-trained on large unlabelled corpora and only lightly fine-tuned, a single architecture serves dozens of tasks.

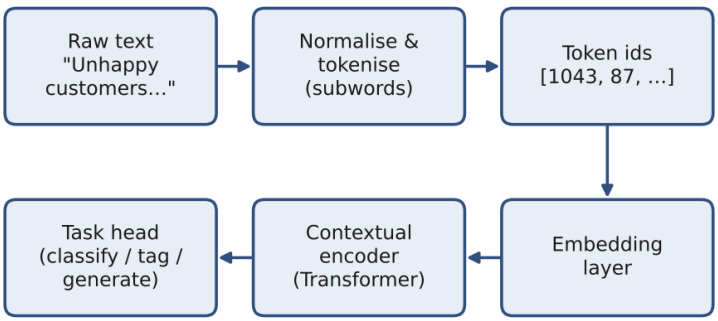



**Figure 2.38** The deep learning NLP pipeline: text is tokenised into subword ids, embedded, contextualised by an encoder and mapped to task outputs by a small head.

### 2.31.3 Core NLP Tasks

NLP tasks can be grouped by the shape of their output, as summarised in Table 2.30. Classification tasks output a label per text or text pair; sequence-labelling tasks output a label per token, typically with BIO (begin–inside–outside) tags for spans; span-extraction tasks such as extractive question answering predict start and end positions; and generation tasks output new text. Benchmarks such as GLUE and SuperGLUE (Wang et al. 2018, 2019) bundle many of these tasks to measure general language understanding.

Generation quality is notoriously hard to measure automatically. **BLEU** (Papineni et al. 2002) counts n-gram overlap with reference translations, **ROUGE** (Lin 2004) measures recall of reference n-grams for summaries, and **perplexity**, defined in Equation (2.72),

**Table 2.30** Core NLP tasks, their output structure, typical metrics and benchmark datasets.

| Task | Output | Metric | Example dataset |
|---|---|---|---|
| Sentiment analysis | Label per text | Accuracy, F1 | SST-2, IMDb |
| Natural language inference | Label per sentence pair | Accuracy | MNLI, SNLI |
| Named-entity recognition | BIO tag per token | Entity-level F1 | CoNLL-2003 |
| Part-of-speech tagging | Tag per token | Accuracy | Penn Treebank |
| Extractive question answering | Answer span (start, end) | Exact match, F1 | SQuAD |
| Machine translation | Text in another language | BLEU, chrF, COMET | WMT |
| Summarisation | Shorter text | ROUGE | CNN/DailyMail, XSum |
| Language modelling | Next-token distribution | Perplexity | WikiText-103 |

measures how well a language model predicts held-out text—lower is better, and a perplexity of k means the model is on average as uncertain as if choosing uniformly among k tokens.

$$\mathrm{PPL} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p_\theta(w_t \mid w_{<t})\right) \tag{2.72}$$

where $w_1,\ldots,w_T$ are the tokens of a held-out corpus.

**Listing 2.25** Tokenisation and inference with pretrained models using Hugging Face Transformers.

```python
from transformers import AutoTokenizer, pipeline

tok = AutoTokenizer.from_pretrained("bert-base-uncased")
enc = tok("Unhappiness is unforgettable.", return_tensors="pt")
print(tok.convert_ids_to_tokens(enc["input_ids"][0]))
# ['[CLS]', 'un', '##ha', '##pp', '##iness', 'is', 'un', '##forget', '##table', '.',
↪  '[SEP]']

sentiment = pipeline("sentiment-analysis")            # default fine-tuned model
print(sentiment("The battery life is disappointing."))
ner = pipeline("ner", aggregation_strategy="simple")
print(ner("Ada Lovelace worked with Charles Babbage in London."))
```

Listing 2.25 shows how libraries hide this pipeline behind a few calls (the exact subword split printed depends on the tokenizer's vocabulary).

**Case Study 2.6 | Neural Machine Translation in Production**

In 2016 Google replaced its phrase-based statistical translation system with GNMT (Wu et al. 2016), an 8-layer LSTM encoder–decoder with attention and WordPiece tokenisation, reporting that human-rated translation errors fell by about 60% on several language pairs. Within

two years transformer-based systems had superseded it. The episode illustrates how quickly deep models moved from research to products used by hundreds of millions of people.

# 2.32 Word Embeddings and Representation Learning

How should a word be represented as a vector? The answer shapes everything downstream. **Representation learning** is the idea that good features should be *learned* from data rather than engineered, and word embeddings were its first great success in NLP. They rest on the **distributional hypothesis**: words that occur in similar contexts tend to have similar meanings (Harris 1954; "you shall know a word by the company it keeps", Firth 1957).

## 2.32.1 One-Hot versus Dense Representations

The simplest representation is the **one-hot vector**: with a vocabulary of size V, word i is the vector $\mathbf{e}_i \in \{0,1\}^V$ with a single 1 in position i. One-hot vectors are high-dimensional (V is often 50,000 or more), sparse and, crucially, mutually orthogonal: the cosine similarity between "cat" and "kitten" is the same as between "cat" and "carburettor", namely zero. A **dense embedding** maps each word to a learned vector $\mathbf{v}_w \in \mathbb{R}^d$ with d typically 50–1,000, in which geometric closeness reflects semantic similarity. An embedding layer is simply a lookup table $E \in \mathbb{R}^{V \times d}$; multiplying a one-hot vector by E selects one row, as noted in Equation (2.73). Table 2.31 contrasts the two.

$$\mathbf{v}_w = E^{\top}\mathbf{e}_w \in \mathbb{R}^d \qquad \cos(\mathbf{u},\mathbf{v}) = \frac{\mathbf{u}^{\top}\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|} \tag{2.73}$$

where E is the embedding matrix (one row per word) and the cosine similarity measures the angle between two embeddings.

## 2.32.2 Word2Vec: CBOW and Skip-Gram

Mikolov et al. (2013) introduced **Word2Vec**, two shallow neural models that learn embeddings by predicting words from their contexts on billions of tokens. **Continuous bag-of-words (CBOW)** predicts a centre word from the average of its surrounding

**Table 2.31** One-hot versus dense word representations.

| Property | One-hot | Dense embedding |
|---|---|---|
| Dimension | Vocabulary size V ($10^4$–$10^6$) | d ≈ 50–1,000 |
| Sparsity | Exactly one non-zero | All entries non-zero |
| Similarity structure | All words equidistant | Similar words are close |
| Learned? | No (fixed) | Yes (from co-occurrence or a task) |
| Generalisation | None across words | Shares statistical strength across similar words |
| Out-of-vocabulary words | Unrepresentable | Unrepresentable (except subword models such as fastText) |

context words. **Skip-gram** does the reverse, predicting each context word within a window of size m from the centre word. Skip-gram works better for rare words; CBOW is faster. Each word has two vectors: an input (centre) vector **v**_w and an output (context) vector **u**_w. The skip-gram objective maximises the average log-probability of context words, with the softmax in Equation (2.74).

$$J(\theta) = \frac{1}{T}\sum_{t=1}^{T}\sum_{-m \le j \le m,\ j \neq 0} \log p(w_{t+j} \mid w_t)$$
$$p(o \mid c) = \frac{\exp\left(\mathbf{u}_o^{\top}\mathbf{v}_c\right)}{\sum_{w=1}^{V}\exp\left(\mathbf{u}_w^{\top}\mathbf{v}_c\right)} \tag{2.74}$$

where c is the centre word, o an observed context word, m the window size and V the vocabulary size.

The denominator sums over the entire vocabulary for every training pair, which is prohibitively expensive. **Negative sampling** replaces the multi-class softmax with a set of binary logistic regressions: distinguish the true context word from k "noise" words drawn from a noise distribution $P_n(w)$, usually the unigram distribution raised to the power 3/4, which up-weights rare words. The per-pair loss is given in Equation (2.75); with k = 5–20 for small corpora and 2–5 for large ones, training cost is independent of V. A further trick, **subsampling** of very frequent words such as "the", speeds training and improves rare-word vectors. Algorithm 2.14 summarises skip-gram with negative sampling (SGNS).

$$\mathcal{L}(c, o) = -\log\sigma\left(\mathbf{u}_o^{\top}\mathbf{v}_c\right) - \sum_{i=1}^{k}\log\sigma\left(-\mathbf{u}_{n_i}^{\top}\mathbf{v}_c\right), \quad n_i \sim P_n(w) \propto f(w)^{3/4} \tag{2.75}$$

where $\sigma$ is the logistic sigmoid, $n_1,\ldots,n_k$ are negative samples and $f(w)$ is the corpus frequency of word w.

**Algorithm 2.14** Skip-Gram with Negative Sampling (SGNS)

**Input:** Tokenised corpus $w_1,\dots,w_T$; window size m; embedding dimension d; negatives k; learning rate $\eta$; epochs E
**Output:** Word embeddings $v_w$ (rows of the input matrix)

**initialise input** vectors $v_w \sim$ small random, **output** vectors $u_w \leftarrow 0$
**for** epoch $= 1$ **to** E **do**
  **for** $t = 1$ **to** T **do**
    (optionally) skip $w_t$ with a probability that grows with its frequency
    $c \leftarrow w_t$; sample an effective window size $b \in \{1,\dots,m\}$
    **for each** context position j with $1 \le |j| \le b$ **do**
      $o \leftarrow w_{t+j}$; draw negatives $n_1,\dots,n_k \sim P_n$
      $g_o \leftarrow \sigma(u_o{}^{\top} v_c) - 1$; $g_i \leftarrow \sigma(u_{n_i}{}^{\top} v_c)$ **for** $i = 1,\dots,k$
      $\Delta v \leftarrow g_o\, u_o + \Sigma_i\, g_i\, u_{n_i}$
      $u_o \leftarrow u_o - \eta\, g_o\, v_c$; $u_{n_i} \leftarrow u_{n_i} - \eta\, g_i\, v_c$ **for each** i
      $v_c \leftarrow v_c - \eta\, \Delta v$
**return** $\{v_w\}$

## 2.32.3 GloVe and fastText

**GloVe** (Global Vectors; Pennington, Socher & Manning 2014) starts from the global word–word co-occurrence matrix X, where $X_{ij}$ counts how often word j appears in the context of word i, and fits embeddings so that their dot products approximate log co-occurrence counts, using the weighted least-squares objective in Equation (2.76). The weighting function f caps the influence of very frequent pairs and ignores zero counts. GloVe thus combines the global statistics of count-based methods such as latent semantic analysis with the efficiency of predictive methods; Levy and Goldberg (2014) showed that SGNS itself implicitly factorises a shifted pointwise mutual information matrix, so the two families are closely related.

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left( \mathbf{w}_i^{\top} \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \qquad f(x) = \min\left(1, (x/x_{\max})^{\alpha}\right) \tag{2.76}$$

where $\mathbf{w}_i$ and $\tilde{w}_j$ are word and context vectors, $b_i$ and $\tilde{b}_j$ biases, and typically $x_{\max} = 100$ and $\alpha = 3/4$.

**fastText** (Bojanowski et al. 2017) represents each word as the sum of vectors for its character n-grams (n = 3–6, plus the whole word), e.g. "where" includes ⟨wh, whe, her, ere, re⟩. It can build vectors for unseen and misspelled words and handles morphologically rich languages such as Finnish or Tamil far better than word-level models.

### 2.32.4 Properties, Analogies and Limitations

Well-trained embeddings exhibit striking linear structure: semantic and syntactic relations correspond approximately to constant vector offsets. The famous example is $\mathbf{v}(\text{king}) - \mathbf{v}(\text{man}) + \mathbf{v}(\text{woman}) \approx \mathbf{v}(\text{queen})$, visualised in Figure 2.39; similar offsets encode country–capital, verb tense and comparative–superlative relations. Analogies are solved by finding the word whose vector has the highest cosine similarity to the offset target, as in Equation (2.77) (excluding the three query words).

$$d^{*} = \arg\max_{d \in V \setminus \{a,b,c\}} \cos\left(\mathbf{v}_d,\ \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c\right) \tag{2.77}$$

for the analogy "a is to b as c is to d".

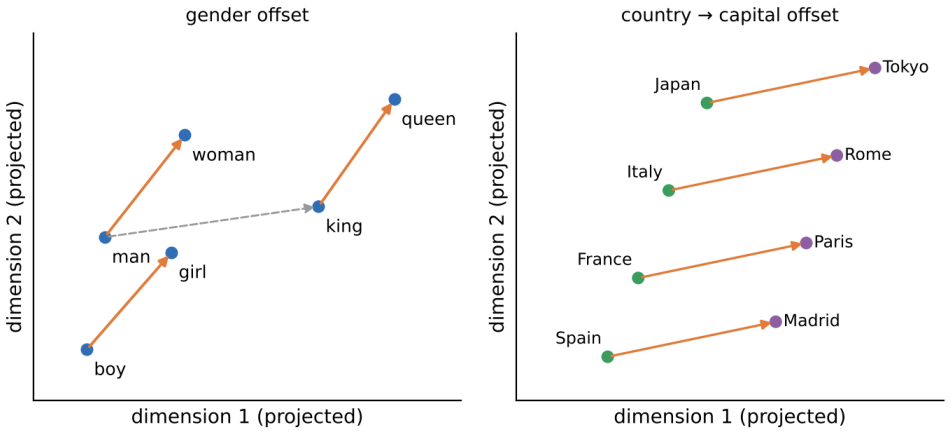



**Figure 2.39** Two-dimensional projection of word embeddings: the gender offset (man→woman) and the capital-city offset (country→capital) are approximately constant vectors, enabling analogy arithmetic.

Static embeddings have important limitations. Each word receives *one* vector regardless of context, so "bank" in "river bank" and "bank account" share a representation. Embeddings also absorb social biases present in the training text: Bolukbasi et al. (2016) showed that analogies such as man : computer programmer :: woman : homemaker emerge from news corpora. **Contextual embeddings** from ELMo (Peters et al. 2018), which used bidirectional LSTMs, and then from transformers such as BERT solve the polysemy problem by computing a different vector for each occurrence of a word; they are discussed in the section on BERT, GPT and other transformer models. Static embeddings remain useful as lightweight features and for analysing corpora. Listing 2.26 trains Word2Vec with gensim.

**Listing 2.26** Training skip-gram Word2Vec with negative sampling using gensim.

```
from gensim.models import Word2Vec

# sentences: an iterable of token lists, e.g. [["the", "cat", "sat"], ...]
model = Word2Vec(sentences,
                 vector_size=200,     # embedding dimension d
                 window=5,            # context window m
                 sg=1,                # 1 = skip-gram, 0 = CBOW
                 negative=10,         # k negative samples
                 sample=1e-4,         # subsampling of frequent words
                 min_count=5, workers=4, epochs=5)

print(model.wv.most_similar("excellent", topn=5))
print(model.wv.most_similar(positive=["king", "woman"], negative=["man"], topn=1))
```

**Key Point | Representation Learning Beyond Words**

The embedding idea generalises far beyond words: products in a recommender, nodes in a graph (see the later section on graph neural networks), proteins, and users can all be embedded so that similarity in vector space reflects similarity of role. The recurring recipe is to define a *context* and train vectors to predict it.

# 2.33 Attention Mechanisms

**Attention** lets a model focus selectively on the most relevant parts of its input when producing each part of its output. It was introduced to remove the fixed-length bottleneck of seq2seq models, and it has since become the single most important building block in deep learning: the transformer architecture of the next section is built from attention alone. The intuition mirrors human translation: when writing the third word of the French sentence, a translator looks back mainly at the corresponding words of the English source rather than holding the whole sentence in memory.

## 2.33.1 Bahdanau (Additive) Attention

Bahdanau, Cho and Bengio (2015) proposed letting the decoder compute a *different* context vector $\mathbf{c}_t$ at every output step, as a weighted average of *all* encoder hidden states $\mathbf{h}_1, \dots, \mathbf{h}_S$ (from a bidirectional RNN). The weights are produced by a small feedforward **alignment model** that scores how well each source position matches the current decoder state, as in Equation (2.78). The scores are normalised with a softmax into **attention weights** $\alpha_{ti}$ that are non-negative and sum to one.

$$e_{ti} = \mathbf{v}_a^\top \tanh\left(W_a \mathbf{s}_{t-1} + U_a \mathbf{h}_i\right)$$

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^{S} \exp(e_{tj})}, \qquad \mathbf{c}_t = \sum_{i=1}^{S} \alpha_{ti}\, \mathbf{h}_i \tag{2.78}$$

where $\mathbf{s}_{t-1}$ is the previous decoder state, $\mathbf{h}_i$ the encoder state at source position $i$, $W_a$, $U_a$ and $\mathbf{v}_a$ learned parameters, $e_{ti}$ the alignment score and $\mathbf{c}_t$ the context vector fed to the decoder at step $t$.

Because every output step can look directly at every input position, the path between any source word and any target word is short, and translation quality no longer collapses for long sentences. The attention weights are also interpretable: plotting $\alpha_{ti}$ as a matrix reveals a soft **alignment** between source and target words, as shown in Figure 2.40. The near-diagonal pattern reflects similar word order, while off-diagonal blocks reveal reordering, such as adjective–noun inversion between English and French.

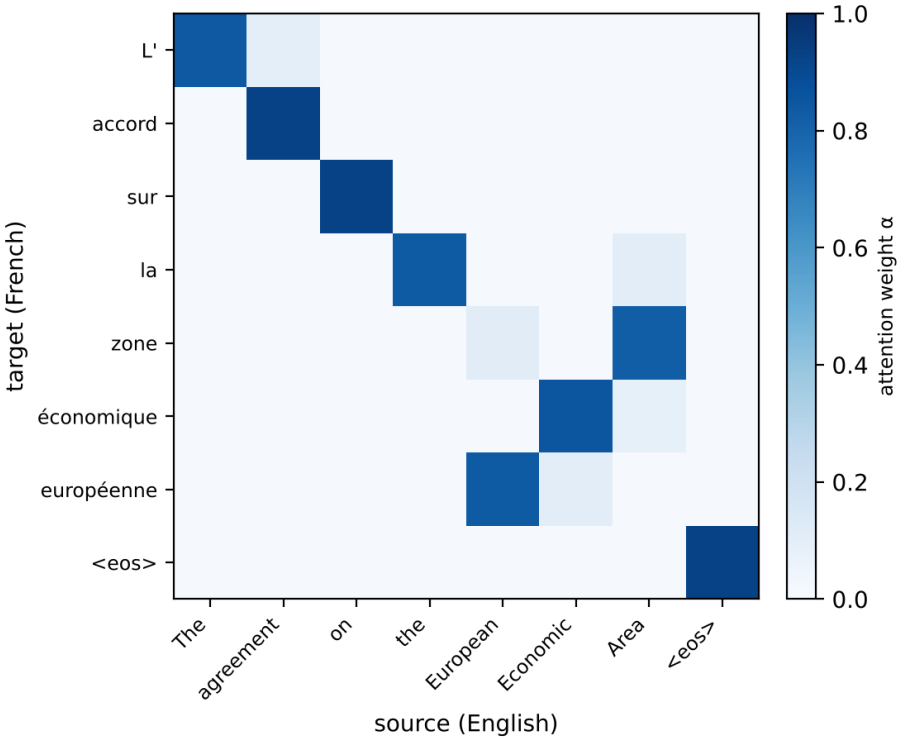



**Figure 2.40** Attention weights $\alpha_{ti}$ of an English→French translation model (illustrative). Each row is a target word and sums to one; note the reordering for "European Economic Area" → "zone économique européenne".

### 2.33.2 Luong (Multiplicative) Attention

Luong, Pham and Manning (2015) simplified and systematised attention. They compute alignment scores between the *current* decoder state $\mathbf{s}_t$ and each encoder state using one of three functions, given in Equation (2.79). The **dot** score is parameter-free and fast; the **general** (bilinear) score adds a learnable matrix; and the **concat** score is essentially Bahdanau's additive form. They also distinguished **global** attention over all source positions from **local** attention over a window around a predicted position. Multiplicative scores can be computed with highly optimised matrix multiplications, which is why the transformer adopted them.

$$\begin{aligned} \operatorname{score}(\mathbf{s}_t, \mathbf{h}_i) &= \mathbf{s}_t^{\top}\, \mathbf{h}_i \quad (\text{dot}) \\ \operatorname{score}(\mathbf{s}_t, \mathbf{h}_i) &= \mathbf{s}_t^{\top}\, W_a \mathbf{h}_i \quad (\text{general}) \\ \operatorname{score}(\mathbf{s}_t, \mathbf{h}_i) &= \mathbf{v}_a^{\top} \tanh\left(W_a [\mathbf{s}_t ; \mathbf{h}_i]\right) \quad (\text{concat}) \end{aligned} \tag{2.79}$$

where $[\cdot;\cdot]$ denotes concatenation.

### 2.33.3 Queries, Keys and Values; Scaled Dot-Product Attention

Attention can be described abstractly as a soft dictionary lookup. A **query** $\mathbf{q}$ is compared with a set of **keys** $\mathbf{k}_i$; the resulting weights are used to average the associated **values** $\mathbf{v}_i$. In the seq2seq setting, the decoder state is the query and the encoder states serve as both keys and values. Packing n queries into a matrix $Q \in \mathbb{R}^{n \times d_k}$, m keys into $K \in \mathbb{R}^{m \times d_k}$ and the values into $V \in \mathbb{R}^{m \times d_v}$, Vaswani et al. (2017) defined **scaled dot-product attention** in Equation (2.80).

$$\operatorname{Attention}(Q, K, V) = \operatorname{softmax}\!\left(\frac{Q K^{\top}}{\sqrt{d_k}}\right) V \tag{2.80}$$

where the softmax is applied row-wise, so each query's weights over the m keys sum to one, and $d_k$ is the key dimension.

The scaling by $\sqrt{d_k}$ matters. If the components of $\mathbf{q}$ and $\mathbf{k}$ are independent with zero mean and unit variance, their dot product has variance $d_k$. For large $d_k$ the logits become large, the softmax saturates towards a one-hot vector, and its gradients vanish. Dividing by $\sqrt{d_k}$ restores unit variance. When the queries, keys and values all come from the *same* sequence, the operation is called **self-attention**: every position attends to every other position of the same sequence, building context-aware representations without any recurrence. When queries come from one sequence and keys/values from

another (e.g. decoder attending to encoder), it is **cross-attention**. Listing 2.27 implements Equation (2.80) with optional masking.

**E Worked Example 2.17 | Scaled Dot-Product Attention by Hand**

Let $d_k = 2$, one query $q = (1, 0)$, and three keys $k_1 = (1, 0)$, $k_2 = (0, 1)$, $k_3 = (1, 1)$ with scalar values $v = (10, 20, 30)$.

- Dot products $q^T k_i$: 1, 0, 1. Scaled by $\sqrt{2} \approx 1.414$: 0.707, 0, 0.707.
- exp: 2.028, 1.000, 2.028; sum = 5.056. Weights $\alpha = (0.401, 0.198, 0.401)$.
- Output = $0.401 \cdot 10 + 0.198 \cdot 20 + 0.401 \cdot 30 = 4.01 + 3.96 + 12.03 \approx 20.0$.

Keys 1 and 3 align with the query and receive equal, larger weights; key 2 is orthogonal and receives less.

**Listing 2.27** Scaled dot-product attention with an optional mask (PyTorch).

```python
import math, torch

def scaled_dot_product_attention(Q, K, V, mask=None):
    """Q: (..., n, d_k), K: (..., m, d_k), V: (..., m, d_v); mask broadcastable to (..., n, m),
    with True where attention is NOT allowed."""
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))     # (..., n, m)
    if mask is not None:
        scores = scores.masked_fill(mask, float("-inf"))
    weights = torch.softmax(scores, dim=-1)                      # rows sum to 1
    return weights @ V, weights

Q = torch.tensor([[1., 0.]]); K = torch.tensor([[1., 0.], [0., 1.], [1., 1.]])
V = torch.tensor([[10.], [20.], [30.]])
out, w = scaled_dot_product_attention(Q, K, V)
print(w.round(decimals=3), out)        # [[0.401, 0.198, 0.401]]  [[20.0]]
# PyTorch >= 2.0 also provides torch.nn.functional.scaled_dot_product_attention
```

**Table 2.32** Attention scoring functions and variants.

| Variant | Score function | Parameters | Introduced in |
|---|---|---|---|
| Additive (Bahdanau) | $v^T \tanh(W s + U h)$ | W, U, v | Bahdanau et al. 2015 |
| Dot (Luong) | $s^T h$ | None | Luong et al. 2015 |
| General / bilinear (Luong) | $s^T W h$ | W | Luong et al. 2015 |
| Scaled dot-product | $q^T k / \sqrt{d_k}$ | Projections $W_Q$, $W_K$, $W_V$ | Vaswani et al. 2017 |
| Self-attention | Q, K, V from the same sequence | As above | Cheng et al. 2016; Vaswani et al. 2017 |
| Cross-attention | Q from decoder, K, V from encoder | As above | Vaswani et al. 2017 |

Table 2.32 collects the scoring functions. Attention is not limited to language: in image

captioning (Xu et al. 2015, "Show, Attend and Tell") the decoder attends over spatial CNN features, and squeeze-and-excitation networks (Hu, Shen & Sun 2018) use a form of channel attention inside CNNs.

> **Note | Is Attention Explanation?**
>
> Attention heatmaps are appealing explanations, but attention weights are not guaranteed to identify the inputs that causally drive a prediction. Jain and Wallace (2019) showed that very different attention distributions can yield the same output; Wiegreffe and Pinter (2019) argued the picture is more nuanced. Treat attention maps as a diagnostic, and confirm conclusions with attribution methods or ablations.

## 2.34 Transformer Architecture

The **transformer**, introduced by Vaswani et al. (2017) in the paper "Attention Is All You Need", dispensed with recurrence and convolution entirely and built a sequence-to-sequence model from attention and position-wise feedforward layers alone. Its motivation was practical: RNNs process tokens one after another, which prevents parallelisation across the sequence and makes long-range interactions pass through many steps. In a transformer, every token attends to every other token in a single layer, and all positions are processed in parallel on GPUs. The original model set a new state of the art in English–German and English–French translation at a fraction of the training cost of previous systems. Today the transformer is the backbone of language models, vision models, speech recognisers, protein-structure predictors and multimodal systems.

### 2.34.1 Overall Architecture

The original transformer is an encoder–decoder model, shown in Figure 2.41. The **encoder** is a stack of N identical layers (N = 6 in the base model); each layer has two sublayers: **multi-head self-attention** and a **position-wise feedforward network**. The **decoder** is also a stack of N layers with three sublayers: **masked multi-head self-attention** over the previously generated tokens, **cross-attention** whose queries come from the decoder and keys and values from the encoder output, and a feedforward network. Every sublayer is wrapped in a **residual connection** followed by **layer normalisation**. Token embeddings are summed with **positional encodings** at the input, and a final linear layer with softmax produces next-token probabilities.

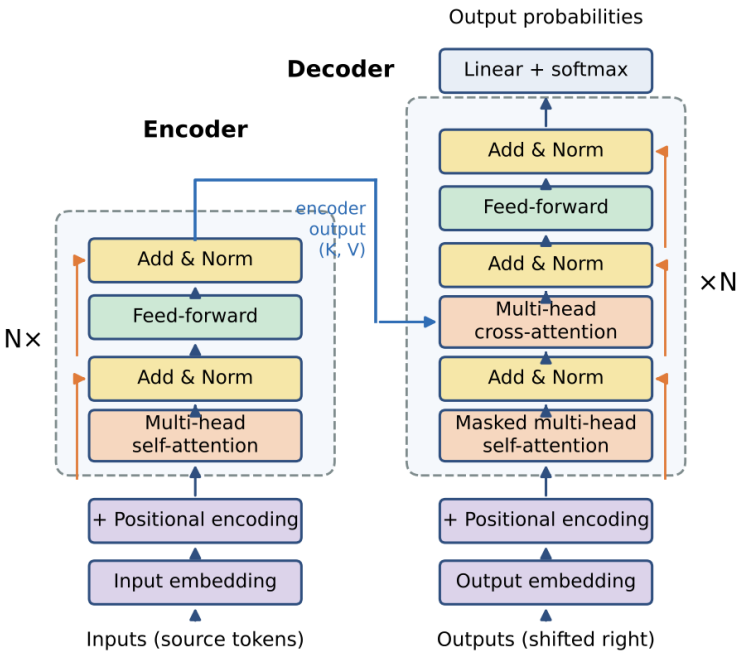



**Figure 2.41** The transformer encoder (left) and decoder (right) blocks. Each sublayer is wrapped by a residual connection and layer normalisation (Add & Norm); the decoder's cross-attention reads the encoder output. Both stacks repeat N times.

## 2.34.2 Multi-Head Attention

The transformer uses the scaled dot-product attention of Equation (2.80), but rather than performing a single attention function on d_model-dimensional representations, it runs h attention **heads** in parallel on lower-dimensional learned projections and concatenates the results, as in Equation (2.81). Each head can specialise in a different relationship—one may track syntactic dependencies such as subject–verb agreement, another coreference, another nearby positions. With $d_k = d_v = d_model/h$, the total cost is similar to that of single-head attention with full dimensionality. The base model uses d_model = 512 and h = 8 heads of dimension 64.

$$\begin{aligned} \mathrm{MultiHead}(Q,K,V) &= \mathrm{Concat}(\mathrm{head}_1,\ldots,\mathrm{head}_h)W^O \\ \mathrm{head}_i &= \mathrm{Attention}\big(QW_i^Q,\ KW_i^K,\ VW_i^V\big) \end{aligned} \tag{2.81}$$

where $W_i^Q, W_i^K \in \mathbb{R}^{d_model \times d_k}$, $W_i^V \in \mathbb{R}^{d_model \times d_v}$ are per-head projection matrices and $W^O \in \mathbb{R}^{h d_v \times d_model}$ the output projection.

In the decoder's self-attention, position t must not attend to future positions $t' > t$, otherwise training with teacher forcing would let the model simply copy the answer.

A **causal mask** sets the corresponding attention logits to $-\infty$ before the softmax, so their weights become zero. The mask allows all positions of the target sequence to be trained in parallel while preserving the autoregressive property of Equation (2.68).

### 2.34.3 Position-Wise Feedforward Network

Each layer also contains a two-layer MLP applied independently and identically at every position, given in Equation (2.82). Its inner dimension $d_{ff}$ is typically four times d_model (2,048 in the base model). Attention mixes information *across* positions; the feedforward network transforms the representation *within* each position, and it holds the majority of a transformer's parameters. Modern models often replace ReLU with GELU or with gated variants such as SwiGLU (Shazeer 2020).

$$\mathrm{FFN}(\mathbf{x}) = \max\left(0,\ \mathbf{x}W_1 + \mathbf{b}_1\right)W_2 + \mathbf{b}_2 \tag{2.82}$$

where $W_1 \in \mathbb{R}^{(d_model \times d_{ff})}$ and $W_2 \in \mathbb{R}^{(d_{ff} \times d_model)}$.

### 2.34.4 Positional Encoding

Self-attention is **permutation-equivariant**: shuffling the input tokens simply shuffles the outputs. Without additional information, a transformer would treat a sentence as a bag of words. Order is injected by adding a **positional encoding** vector to each token embedding. Vaswani et al. used fixed sinusoids of geometrically increasing wavelengths, defined in Equation (2.83) and plotted in Figure 2.42. Each dimension oscillates at a different frequency, so every position receives a unique pattern, and for any fixed offset k the encoding of position pos + k is a linear function (a rotation) of the encoding of pos, making relative positions easy to learn.

$$\begin{aligned} PE_{(pos,\ 2i)} &= \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \\ PE_{(pos,\ 2i+1)} &= \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \end{aligned} \tag{2.83}$$

where pos is the token position, $i = 0,\ldots,d_model/2 - 1$ indexes dimension pairs and the wavelengths range from $2\pi$ to $10000 \cdot 2\pi$.

Alternatives include **learned absolute** position embeddings (used by BERT and GPT-2), **relative** position representations (Shaw, Uszkoreit & Vaswani 2018), **rotary position embeddings** (RoPE; Su et al. 2021), which rotate query and key vectors by position-dependent angles and are used by most current large language models, and **ALiBi** (Press, Smith & Lewis 2022), which adds a distance-proportional penalty to attention logits and extrapolates well to longer sequences.

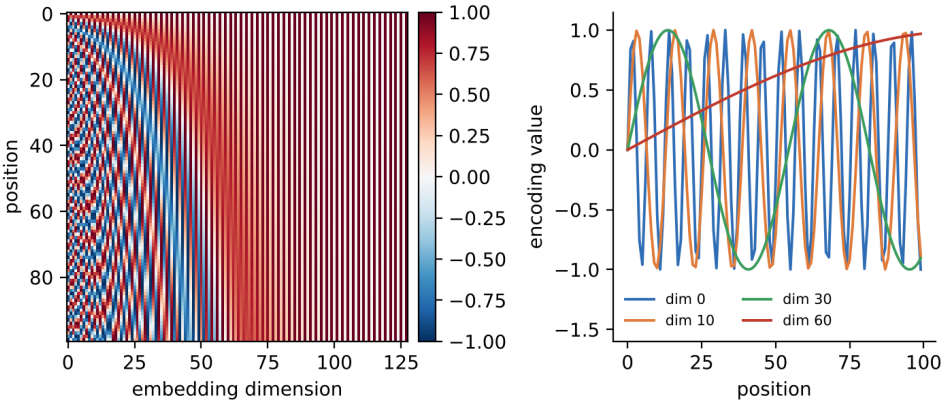



**Figure 2.42** Sinusoidal positional encodings (d_model = 128). Left: the full encoding matrix; each row is a position and each column a dimension, with low dimensions oscillating fastest. Right: four individual dimensions as a function of position.

### 2.34.5 Residual Connections and Layer Normalisation

Each sublayer is wrapped as in Equation (2.84). The residual connection, borrowed from ResNet, gives gradients a direct path through the stack. **Layer normalisation** (Ba, Kiros & Hinton 2016) normalises each token's feature vector to zero mean and unit variance, then applies a learned scale and shift; unlike batch normalisation, it is independent of the batch and of sequence length, which suits variable-length text. The original transformer applied normalisation *after* the residual addition (**Post-LN**). Most modern models use **Pre-LN**, normalising the input of each sublayer, because it trains stably without careful learning-rate warm-up (Xiong et al. 2020); many also use the cheaper **RMSNorm** (Zhang & Sennrich 2019).

$$\begin{aligned} \text{Post-LN}: &\quad \mathbf{x} \leftarrow \mathrm{LayerNorm}(\mathbf{x} + \mathrm{Sublayer}(\mathbf{x})) \\ \text{Pre-LN}: &\quad \mathbf{x} \leftarrow \mathbf{x} + \mathrm{Sublayer}(\mathrm{LayerNorm}(\mathbf{x})) \\ \mathrm{LayerNorm}(\mathbf{x}) &= \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \end{aligned} \tag{2.84}$$

where $\mu$ and $\sigma^2$ are the mean and variance of the d_model features of a single token and $\gamma$, $\beta$ are learned.

Listing 2.28 implements a Pre-LN encoder block in PyTorch, combining multi-head self-attention, the feedforward network, residual connections and layer normalisation.

**Listing 2.28** A Pre-LN transformer encoder block (PyTorch).

```python
import torch, torch.nn as nn

class TransformerBlock(nn.Module):
    def __init__(self, d_model=512, n_heads=8, d_ff=2048, dropout=0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout,
        ↪  batch_first=True)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(nn.Linear(d_model, d_ff), nn.GELU(),
                                 nn.Linear(d_ff, d_model), nn.Dropout(dropout))
        self.drop = nn.Dropout(dropout)

    def forward(self, x, attn_mask=None, pad_mask=None):   # x: (batch, seq, d_model)
        h = self.ln1(x)
        a, _ = self.attn(h, h, h, attn_mask=attn_mask, key_padding_mask=pad_mask)
        x = x + self.drop(a)                                  # residual 1
        x = x + self.ffn(self.ln2(x))                         # residual 2
        return x

x = torch.randn(2, 10, 512)
causal = torch.triu(torch.ones(10, 10, dtype=torch.bool), diagonal=1)  # True = blocked
print(TransformerBlock()(x, attn_mask=causal).shape)       # torch.Size([2, 10, 512])
```

## 2.34.6 Computational Complexity

Self-attention computes an $n \times n$ matrix of scores for a sequence of length $n$, so its time and memory scale quadratically with $n$. Table 2.33, adapted from Vaswani et al. (2017), compares layer types. Self-attention has constant path length between any two positions and is fully parallel, but becomes expensive for long sequences ($n \gg d$). This has motivated efficient attention variants: sparse patterns (Longformer, BigBird), low-rank and kernel approximations (Linformer, Performer), IO-aware exact kernels such as **FlashAttention** (Dao et al. 2022) that avoid materialising the $n \times n$ matrix in slow memory, and state-space models such as Mamba (Gu & Dao 2023) with linear-time recurrence.

**Table 2.33** Per-layer complexity, sequential operations and maximum path length for sequence length $n$, representation dimension $d$, kernel size $k$ (after Vaswani et al. 2017).

| Layer type | Complexity per layer | Sequential operations | Maximum path length |
|---|---|---|---|
| Self-attention | $O(n^2 \cdot d)$ | $O(1)$ | $O(1)$ |
| Recurrent | $O(n \cdot d^2)$ | $O(n)$ | $O(n)$ |
| Convolutional | $O(k \cdot n \cdot d^2)$ | $O(1)$ | $O(\log_k n)$ (dilated) |
| Self-attention (restricted to window $r$) | $O(r \cdot n \cdot d)$ | $O(1)$ | $O(n/r)$ |

**E Worked Example 2.18 | Parameter Count of One Transformer Layer**

Base model: d_model = 512, $d_{ff}$ = 2,048, h = 8.

- Attention projections W^Q, W^K, W^V, W^O: $4 \times (512 \times 512 + 512) = 1{,}050{,}624$.
- Feedforward: $(512 \times 2{,}048 + 2{,}048) + (2{,}048 \times 512 + 512) = 2{,}099{,}712$.
- Two layer norms: $2 \times 2 \times 512 = 2{,}048$.

Total ≈ 3.15 million per encoder layer; two-thirds sit in the feedforward network. Six encoder layers hold ≈ 18.9 M.

**K Key Point | Why Transformers Won**

- Full parallelism over sequence positions makes training on GPUs/TPUs efficient at scale.
- Constant-length paths between any two tokens ease learning long-range dependencies.
- The same block works for text, images, audio and molecules; scale (data, parameters, compute) improves it predictably.

# 2.35 BERT, GPT, and Other Transformer Models

The transformer's real impact came from **pretraining**: training a large model once on a huge unlabelled corpus with a self-supervised objective, then adapting it to many downstream tasks. Three architectural families emerged, distinguished by which part of the original transformer they keep and by their pretraining objective: **encoder-only** models (BERT and descendants) for understanding, **decoder-only** models (the GPT family) for generation, and **encoder–decoder** models (T5, BART) for text-to-text transformation. Figure 2.43 contrasts their attention patterns.

## 2.35.1 BERT and Masked Language Modelling

**BERT** (Bidirectional Encoder Representations from Transformers; Devlin et al. 2019) is a stack of transformer encoder layers—12 in BERT-Base (110 M parameters) and 24 in BERT-Large (340 M). It is pretrained with two objectives on BooksCorpus and English Wikipedia (about 3.3 billion words). In **masked language modelling (MLM)**, 15% of input tokens are selected; of these, 80% are replaced by a `[MASK]` token, 10% by a random token and 10% left unchanged; the model must predict the original tokens, with the loss in Equation (2.85). Because it sees context on *both* sides of each mask, BERT learns deeply bidirectional representations, which a left-to-right language model

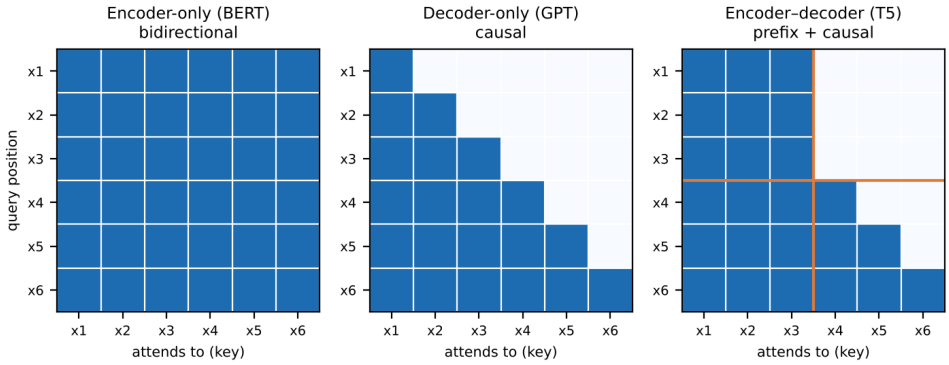



**Figure 2.43** Attention masks of the three transformer families (shaded = allowed). Encoder-only models attend bidirectionally; decoder-only models use a causal mask; encoder–decoder models combine a bidirectional encoder with a causal decoder that cross-attends to the encoder.

cannot. The second objective, **next sentence prediction (NSP)**, asks whether sentence B follows sentence A; later work found it of little value.

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log p_\theta(x_i \mid \tilde{\mathbf{x}}) \tag{2.85}$$

where $\mathcal{M}$ is the set of masked positions and $\tilde{\mathbf{x}}$ the corrupted input sequence.

For fine-tuning, a special `[CLS]` token at the start of every input provides a sequence-level representation for classification, and a `[SEP]` token separates sentence pairs. Adding a single output layer and fine-tuning all parameters for a few epochs set new records on eleven NLP tasks, including GLUE and SQuAD, when BERT was released in 2018.

## 2.35.2 Improving BERT: RoBERTa and Friends

**RoBERTa** (Liu et al. 2019) showed that BERT was significantly undertrained. Keeping the architecture, it removed NSP, used **dynamic masking** (a new mask each time a sequence is seen), larger batches, byte-level BPE and ten times more data (160 GB of text), and matched or exceeded all post-BERT models of its time. **ALBERT** (Lan et al. 2020) shared parameters across layers and factorised the embedding matrix to reduce size; **DistilBERT** (Sanh et al. 2019) used knowledge distillation to keep about 97% of BERT's performance with 40% fewer parameters; **ELECTRA** (Clark et al. 2020) replaced MLM with *replaced-token detection*, learning from every token rather than the 15% masked; and **DeBERTa** (He et al. 2021) disentangled content and position attention.

### 2.35.3 GPT and Causal Language Modelling

The **GPT** (Generative Pre-trained Transformer) family from OpenAI uses a stack of transformer *decoder* blocks without cross-attention, trained with the **causal language modelling (CLM)** objective of Equation (2.86): predict each token from the tokens to its left. GPT-1 (Radford et al. 2018, 117 M parameters) showed that generative pretraining followed by discriminative fine-tuning worked well. GPT-2 (Radford et al. 2019, 1.5 B) showed that a sufficiently large language model could perform tasks such as summarisation and translation **zero-shot**, simply by being prompted. GPT-3 (Brown et al. 2020, 175 B) demonstrated **few-shot in-context learning**: given a handful of examples in the prompt, it performed new tasks without any gradient updates. These models are the subject of the next section on large language models.

$$\mathcal{L}_{\text{CLM}} = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_1, \ldots, x_{t-1}) \tag{2.86}$$

which is Equation (2.68) applied to tokens, implemented with the causal mask.

The contrast between MLM and CLM is fundamental. MLM yields better *representations* for understanding tasks because each token sees full context, but it does not define a convenient generative model and learns only from the masked 15% of tokens. CLM learns from every token and naturally generates text, but each representation sees only left context. The empirical finding of the 2020s is that, at sufficient scale, decoder-only CLM models become excellent at understanding tasks as well, which is why nearly all frontier language models are decoder-only.

### 2.35.4 Encoder–Decoder Models: T5 and BART

**T5** (Text-to-Text Transfer Transformer; Raffel et al. 2020) cast *every* NLP problem as mapping an input string to an output string, e.g. "translate English to German: That is good." → "Das ist gut." or "sst2 sentence: a gorgeous film" → "positive". It uses the original encoder–decoder transformer pretrained with **span corruption**: contiguous spans of the input are replaced by sentinel tokens and the decoder must output the missing spans. The paper is also a landmark systematic study of pretraining objectives, architectures and datasets (the C4 corpus). **BART** (Lewis et al. 2020) is a denoising autoencoder trained to reconstruct text corrupted by masking, deletion and sentence shuffling, and excels at summarisation.

### 2.35.5 Beyond Text: Vision Transformers

The **Vision Transformer (ViT)** (Dosovitskiy et al. 2021) applies a standard transformer encoder to images with minimal changes. An image is split into fixed-size patches (e.g. 16 × 16 pixels, hence the title "An Image is Worth 16x16 Words"); each patch is flattened and linearly projected to a token embedding; a learnable `[CLS]` token and position embeddings are added; and the sequence is processed by a transformer encoder. ViT lacks the locality and translation-equivariance biases of CNNs, so it underperforms ResNets when trained on ImageNet-1k alone but surpasses them when pretrained on very large datasets (JFT-300M). DeiT (Touvron et al. 2021) closed the gap on smaller data with strong augmentation and distillation, and the **Swin Transformer** (Liu et al. 2021) reintroduced hierarchical, windowed attention for dense prediction. Multimodal models such as **CLIP** (Radford et al. 2021) pair a ViT image encoder with a text transformer trained contrastively on 400 million image–text pairs, enabling zero-shot image classification from text prompts. Table 2.34 compares the models in this section.

**Table 2.34** Representative pretrained transformer models.

| Model | Year | Architecture | Pretraining objective | Size (params) | Typical use |
|---|---|---|---|---|---|
| BERT | 2018 | Encoder | MLM + NSP | 110 M / 340 M | Classification, NER, QA |
| GPT-2 | 2019 | Decoder | Causal LM | up to 1.5 B | Text generation |
| RoBERTa | 2019 | Encoder | MLM (dynamic), no NSP | 125 M / 355 M | Understanding tasks |
| T5 | 2019 | Encoder–decoder | Span corruption | 60 M – 11 B | Text-to-text (translation, QA, summarisation) |
| BART | 2019 | Encoder–decoder | Denoising reconstruction | 140 M / 400 M | Summarisation, generation |
| ELECTRA | 2020 | Encoder | Replaced-token detection | 14 M – 335 M | Efficient understanding |
| GPT-3 | 2020 | Decoder | Causal LM | 175 B | Few-shot in-context learning |
| ViT | 2020 | Encoder | Supervised (image classes) | 86 M – 632 M | Image classification |
| CLIP | 2021 | Dual encoder | Image–text contrastive | ≈ 150 M – 430 M | Zero-shot vision, retrieval |

**Listing 2.29** Fine-tuning a pretrained BERT-style encoder for sentence classification with Hugging Face.

```
from datasets import load_dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)

```

```
ds = load_dataset("glue", "sst2")
tok = AutoTokenizer.from_pretrained("roberta-base")
ds = ds.map(lambda b: tok(b["sentence"], truncation=True, max_length=128),
↪  batched=True)

model = AutoModelForSequenceClassification.from_pretrained("roberta-base",
↪  num_labels=2)
args = TrainingArguments("out-sst2", learning_rate=2e-5,
↪  per_device_train_batch_size=32,
                         num_train_epochs=3, weight_decay=0.01, warmup_ratio=0.06,
                         eval_strategy="epoch")
trainer = Trainer(model=model, args=args, train_dataset=ds["train"],
                  eval_dataset=ds["validation"], tokenizer=tok)
trainer.train()
```

Listing 2.29 shows the standard fine-tuning recipe: small learning rate (1e-5 to 5e-5), warm-up, few epochs. Fine-tuning strategies, including parameter-efficient methods such as adapters and LoRA, are discussed in the section on transfer learning later in this chapter.

**Practice Tip | Choosing a Pretrained Transformer**

- Classification, tagging or retrieval with modest compute: an encoder (RoBERTa, DeBERTa) or a sentence-embedding model.
- Free-form generation, dialogue or tasks specified by prompts: a decoder-only LLM.
- Input-to-output transformations with paired data (summarisation, translation): an encoder–decoder (T5, BART) or a decoder-only LLM.
- Vision: a pretrained ViT, Swin or ConvNeXt backbone; for zero-shot labels, CLIP.

## 2.36 Large Language Models (LLMs)

A **large language model** (LLM) is a transformer language model, nowadays almost always decoder-only, with billions to trillions of parameters trained on trillions of tokens of text and code. Beyond a certain scale such models display capabilities that smaller models lack: following natural-language instructions, solving multi-step problems, writing programs and adapting to new tasks from a few examples in the prompt. Systems such as GPT-4, Claude, Gemini and the open-weight Llama, Mistral and Qwen families have made LLMs general-purpose tools. This section explains how they are built—pretraining, scaling laws, instruction tuning and alignment—and how they are used and decoded, along with their characteristic failure modes.

### 2.36.1 Pretraining and Scaling Laws

LLM pretraining minimises the causal language-modelling loss of Equation (2.86) over a vast corpus: filtered web crawls, books, scientific articles, code and curated multilingual text, typically de-duplicated and filtered for quality. Training a frontier model consumes on the order of $10^{24}$–$10^{26}$ floating-point operations on thousands of accelerators for months, using data, tensor and pipeline parallelism. A useful rule of thumb is that training compute is $C \approx 6ND$ FLOPs for N parameters and D training tokens.

Kaplan et al. (2020) discovered that test loss falls as a smooth **power law** in model size, data size and compute over many orders of magnitude, as in Equation (2.87). Such **scaling laws** make the performance of large training runs predictable from small ones. Hoffmann et al. (2022), in the "Chinchilla" study, refitted these laws and concluded that for a fixed compute budget, parameters and tokens should be scaled in roughly equal proportion—about 20 training tokens per parameter—so that many earlier models, including GPT-3, had been undertrained. Their 70 B-parameter Chinchilla, trained on 1.4 trillion tokens, outperformed the 280 B-parameter Gopher with the same compute. Figure 2.44 illustrates the power-law relationship. In practice, models intended for heavy deployment are trained well beyond the compute-optimal token count, because a smaller model is cheaper to serve.

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D} \qquad L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \tag{2.87}$$

where L is the test cross-entropy loss, N the number of non-embedding parameters, D the number of training tokens, and $N_c$, $D_c$, $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$ fitted constants (Kaplan et al. 2020); the second line is the Chinchilla parametric form with irreducible loss E and fitted A, B, $\alpha \approx 0.34$, $\beta \approx 0.28$ (Hoffmann et al. 2022).

### 2.36.2 In-Context Learning and Prompting

A pretrained LLM can perform a task specified entirely in its input. In **zero-shot** prompting the prompt contains only an instruction; in **few-shot** prompting it also contains k demonstrations (input–output pairs) followed by a new input. This **in-context learning** (Brown et al. 2020) involves no parameter updates: the model infers the task from the prompt and continues the pattern. Performance generally improves with model size and with the number and quality of demonstrations. **Chain-of-thought prompting** (Wei et al. 2022), which includes or elicits intermediate reasoning steps before the answer, markedly improves performance on arithmetic and multi-step

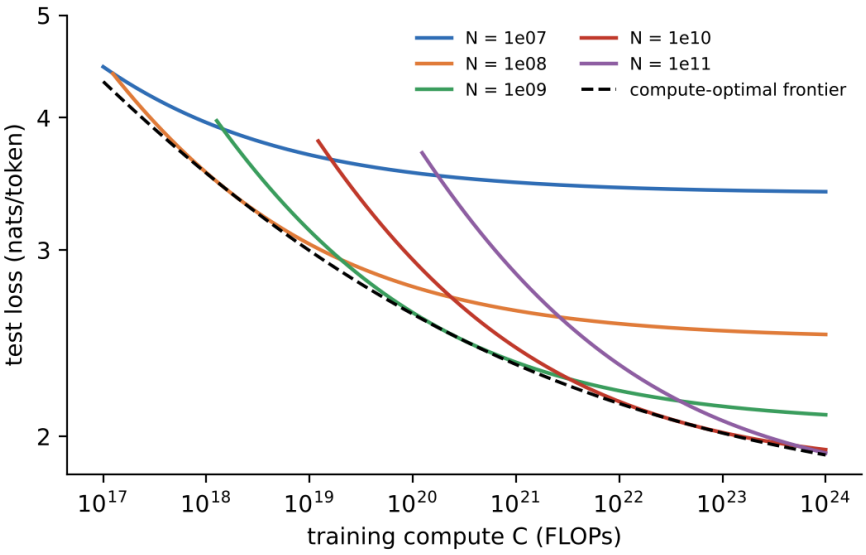



**Figure 2.44** Illustrative scaling law: test loss of language models decreases as a power law in training compute (straight lines on log–log axes); each curve is one model size, and the envelope traces the compute-optimal frontier.

reasoning in large models. Retrieval-augmented generation, tool use and agentic systems built on these abilities are discussed elsewhere in the book.

## 2.36.3 Instruction Tuning and RLHF

A base model trained only to predict internet text is not a helpful assistant: asked a question, it may continue with more questions. **Post-training** aligns it with user intent in stages, summarised in Figure 2.45 and Table 2.35. **Supervised fine-tuning (SFT)**, or **instruction tuning**, trains the model on curated (instruction, desired response) pairs with the usual next-token loss; FLAN (Wei et al. 2022) showed that instruction tuning on many tasks dramatically improves zero-shot performance on unseen tasks.

**Reinforcement learning from human feedback (RLHF)** (Christiano et al. 2017; Ouyang et al. 2022, InstructGPT) then optimises for human preferences, which are easier to give than demonstrations. Labellers compare pairs of model responses; a **reward model** $r_\varphi$ is trained on these comparisons with the Bradley–Terry loss in Equation (2.88); and the language-model policy $\pi_\theta$ is optimised with a policy-gradient method (typically PPO, covered in the section on deep reinforcement learning) to maximise reward while a KL penalty keeps it close to the SFT reference policy, as in Equation (2.89). Ouyang et al. reported that labellers preferred outputs of a 1.3 B-parameter InstructGPT model to those of the 175 B GPT-3. **Direct Preference Optimisation** (DPO; Rafailov et al. 2023) skips the explicit reward model and RL loop,

optimising the policy directly on preference pairs with a classification-style loss. Other approaches use AI feedback guided by a written set of principles (Constitutional AI; Bai et al. 2022).

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\mathbb{E}_{(x,\, y_w,\, y_l)}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{2.88}$$

where x is a prompt, $y_w$ the preferred (winning) response and $y_l$ the rejected one.

$$\max_{\theta}\ \mathbb{E}_{x \sim \mathcal{D},\ y \sim \pi_\theta(\cdot \mid x)}\left[r_\phi(x, y)\right] - \beta\, \mathrm{KL}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)) \tag{2.89}$$

where π_ref is the SFT model and β controls how far the policy may drift from it (guarding against reward hacking).

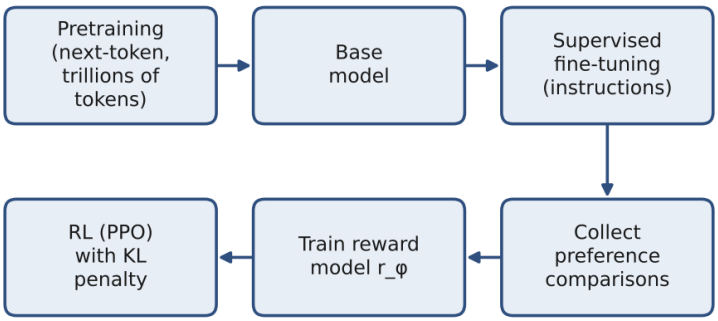



**Figure 2.45** The three-stage LLM training pipeline: self-supervised pretraining, supervised instruction tuning, and preference optimisation via a reward model and reinforcement learning (RLHF).

**Table 2.35** Stages of building an LLM assistant.

| Stage | Data | Objective | Result |
|---|---|---|---|
| Pretraining | Trillions of unlabelled tokens | Causal LM (next-token) | Base model with broad knowledge |
| Supervised fine-tuning | $10^4$–$10^6$ instruction–response pairs | Causal LM on responses | Follows instructions |
| Reward modelling | Human or AI preference comparisons | Bradley–Terry ranking loss | Scalar reward model |
| RLHF / DPO | Prompts (+ preferences for DPO) | Maximise reward − β·KL | Helpful, harmless assistant |

### 2.36.4 Decoding Strategies

An LLM outputs a probability distribution over the vocabulary for the next token; a **decoding strategy** turns these distributions into text. The choice strongly affects quality, diversity and factuality. Given logits $z_i$ for each vocabulary item, the model's distribution is the softmax with **temperature** $\tau$ in Equation (2.90). $\tau < 1$ sharpens the distribution (more deterministic), $\tau > 1$ flattens it (more random), and $\tau \to 0$ approaches greedy decoding.

$$p_\tau(w_i \mid w_{<t}) = \frac{\exp(z_i/\tau)}{\sum_{j=1}^{V} \exp(z_j/\tau)} \tag{2.90}$$

where $z_i$ is the logit of token $i$ and $\tau > 0$ the temperature.

**Greedy decoding** picks the most probable token at each step (Equation (2.91)). It is fast and deterministic but myopic, and on open-ended text it tends to produce repetitive loops. **Beam search** keeps the B highest-scoring partial sequences (beams) at each step, scoring by cumulative log-probability, usually with a length normalisation such as dividing by T^α; it is the standard for translation and summarisation where outputs are tightly constrained, as described in Algorithm 2.15. For open-ended generation, maximising likelihood produces bland text, so **sampling** methods are preferred. **Top-k sampling** (Fan, Lewis & Dauphin 2018) restricts sampling to the k most likely tokens and renormalises. **Top-p (nucleus) sampling** (Holtzman et al. 2020) instead uses the smallest set of tokens whose cumulative probability exceeds p, so the candidate set adapts to the model's confidence, as defined in Equation (2.92).

$$\begin{aligned} w_t &= \arg\max_{w \in V} p(w \mid w_{<t}) \quad \text{(greedy)} \\ \text{score}(w_{1:T}) &= \tfrac{1}{T^\alpha} \sum_{t=1}^{T} \log p(w_t \mid w_{<t}) \quad \text{(beam, length-normalised)} \end{aligned} \tag{2.91}$$

where $\alpha \in [0, 1]$ controls the length penalty.

$$\begin{aligned} V^{(k)} &= \text{top-}k \text{ tokens by } p(w \mid w_{<t}) \\ V^{(p)} &= \arg\min_{V' \subseteq V} |V'| \quad \text{s.t.} \quad \sum_{w \in V'} p(w \mid w_{<t}) \geq p \\ \tilde{p}(w) &= \frac{p(w \mid w_{<t})\, \mathbf{1}[w \in V^{(\cdot)}]}{\sum_{w' \in V^{(\cdot)}} p(w' \mid w_{<t})} \end{aligned} \tag{2.92}$$

where $V^{(k)}$ and $V^{(p)}$ are the top-k and nucleus candidate sets and $\tilde{p}$ the renormalised sampling distribution.

**Algorithm 2.15** Beam Search Decoding

**Input:** Language model $p(w \mid \text{prefix})$; beam width $B$; maximum length $T_{\max}$; length-penalty exponent $\alpha$
**Output:** Highest-scoring completed sequence

```
beams ← {(⟨bos⟩, score 0)}; finished ← ∅
for t = 1 to T_max do
    candidates ← ∅
    for each (y, s) in beams do
        for each token w in the top-B tokens of p(· | y) do
            add (y ∘ w, s + log p(w | y)) to candidates
    beams ← the B candidates with highest score
    move every beam ending in ⟨eos⟩ from beams to finished
    if beams is empty then break
return argmax over finished ∪ beams of s / |y|^α
```

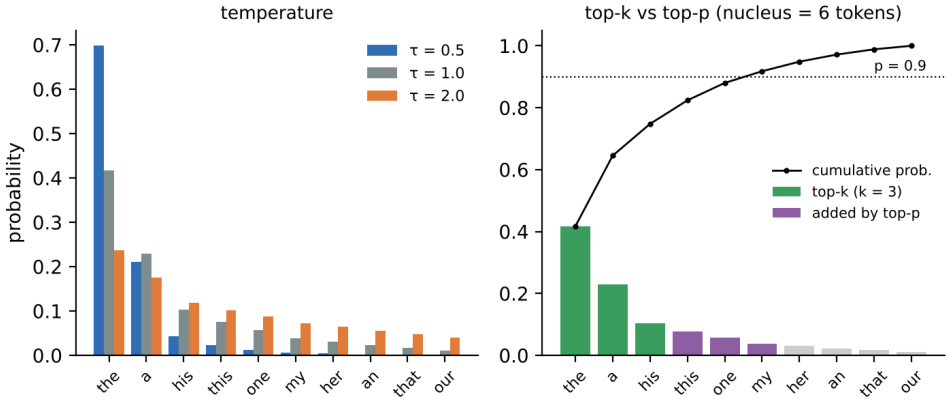



**Figure 2.46** Effect of decoding settings on a next-token distribution. Left: temperature reshapes the distribution. Right: top-k ($k = 3$) and top-p ($p = 0.9$) truncate the tail before sampling.

Figure 2.46 visualises these strategies, and Table 2.36 summarises when to use each. In practice, chat systems commonly combine a temperature of about 0.7 with top-p $\approx$ 0.9–0.95, often plus a repetition penalty; for deterministic tasks such as extraction or code, $\tau = 0$ (greedy) is common.

## 2.36.5 Hallucination and Other Limitations

LLMs can **hallucinate**: generate fluent, confident statements that are false or unsupported, such as non-existent citations. Hallucination arises because the training objective rewards plausible continuations rather than truth, because knowledge is stored imperfectly in parameters, and because decoding commits to tokens without verification.

**Table 2.36** Decoding strategies for language models.

| Strategy | Rule | Pros | Cons / typical use |
|---|---|---|---|
| Greedy | argmax token each step | Fast, deterministic | Repetitive; factual QA, code |
| Beam search (B) | Keep B best partial sequences | Higher-likelihood outputs | Bland in open text; translation, summarisation |
| Temperature $\tau$ | Softmax of $z/\tau$ | Tunes randomness smoothly | High $\tau \to$ incoherent |
| Top-k | Sample among k most likely | Removes unreliable tail | Fixed k ignores confidence |
| Top-p (nucleus) | Smallest set with mass $\geq$ p | Adapts to confidence | Default for open-ended generation |

Mitigations include **retrieval-augmented generation** (grounding answers in retrieved documents), training models to abstain or express uncertainty, tool use (calculators, code execution, search), and verification with citations. Other limitations include a fixed **context window**, a **knowledge cut-off**, sensitivity to prompt wording, biases inherited from training data, vulnerability to **prompt injection** and jailbreaks, and high inference cost. Responsible deployment requires evaluation on the target domain, human oversight for consequential decisions, and attention to privacy and safety.

**Listing 2.30** Text generation with different decoding strategies using Hugging Face Transformers.

```python
from transformers import AutoTokenizer, AutoModelForCausalLM

name = "gpt2"                                          # any causal LM checkpoint
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name)
ids = tok("Deep learning is", return_tensors="pt").input_ids

greedy = model.generate(ids, max_new_tokens=30, do_sample=False)
beam = model.generate(ids, max_new_tokens=30, num_beams=5, length_penalty=1.0,
                      no_repeat_ngram_size=3, early_stopping=True)
nucleus = model.generate(ids, max_new_tokens=30, do_sample=True,
                         temperature=0.7, top_k=50, top_p=0.9)
for out in (greedy, beam, nucleus):
    print(tok.decode(out[0], skip_special_tokens=True), "\n---")
```

Listing 2.30 compares the strategies on a small model; running it shows greedy and beam outputs repeating themselves while nucleus sampling produces more varied text.

**Case Study 2.7 | InstructGPT**

Ouyang et al. (2022) fine-tuned GPT-3 with about 13,000 demonstration prompts for SFT, 33,000 prompts for reward-model comparisons and 31,000 prompts for PPO. Despite having over 100 times fewer parameters, the 1.3 B InstructGPT was preferred by labellers over the 175

B GPT-3, and it produced fewer fabricated facts on closed-domain tasks. The recipe—pretrain, SFT, RLHF—became the template for subsequent chat assistants.

**Definition 2.5 | Emergent Ability and In-Context Learning**

**In-context learning** is the ability of a model to perform a task specified by instructions and/or examples in its input, without updating its parameters. An **emergent ability** (Wei et al. 2022) is one absent in smaller models but present in larger ones; Schaeffer, Miranda and Koyejo (2023) argued that some apparent emergence is an artefact of discontinuous evaluation metrics.

# 2.37 Generative Adversarial Networks (GANs)

The models so far have been mostly **discriminative**, learning $\mathrm{p}(\mathrm{y} \mid \mathbf{x})$. **Generative models** learn the data distribution $\mathrm{p}(\mathbf{x})$ itself, so that they can synthesise new samples—images, audio, molecules—that resemble the training data. The next four sections cover the main deep generative families: generative adversarial networks, variational autoencoders, autoencoders more broadly, and diffusion models. **Generative adversarial networks** (GANs), introduced by Goodfellow et al. (2014), frame generation as a game between two networks and produced the first strikingly realistic synthetic faces.

## 2.37.1 The Adversarial Game

A GAN consists of a **generator** G and a **discriminator** D, shown in Figure 2.47. The generator maps a random noise vector $\mathbf{z}$ drawn from a simple prior $\mathrm{p}_{\mathrm{z}}$ (e.g. a standard Gaussian) to a synthetic sample $\mathrm{G}(\mathbf{z})$. The discriminator receives either a real sample from the data or a fake one from G and outputs the probability $\mathrm{D}(\mathbf{x})$ that its input is real. The discriminator is trained to tell real from fake; the generator is trained to fool the discriminator. This competition is formalised as the two-player **minimax game** in Equation (2.93).

$$\min_{G} \max_{D} \; V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{z}}[\log (1 - D(G(\mathbf{z})))] \tag{2.93}$$

where $\mathrm{p}_{\mathrm{data}}$ is the data distribution, $\mathrm{p}_{\mathrm{z}}$ the noise prior, $\mathrm{D}(\mathbf{x}) \in (0, 1)$ the discriminator's estimate that $\mathbf{x}$ is real, and $\mathrm{G}(\mathbf{z})$ a generated sample.

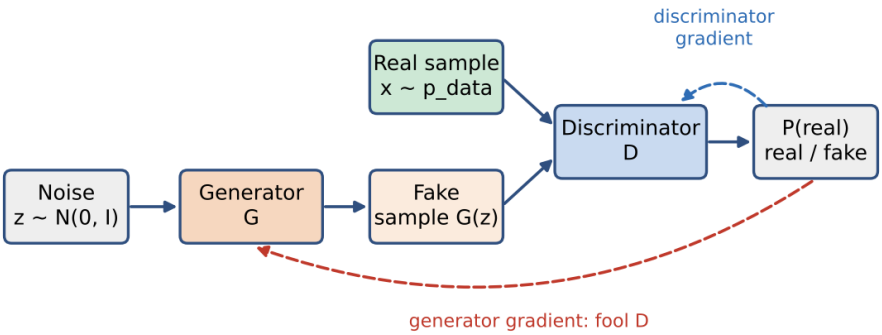



**Figure 2.47** GAN architecture. The generator turns noise z into fake samples; the discriminator classifies real versus fake. Gradients of the discriminator's output flow back to update both networks.

## 2.37.2 Theory: the Optimal Discriminator

For a fixed generator with sample distribution $p_g$, the value function is maximised by the optimal discriminator in Equation (2.94). Substituting it back shows that the generator then minimises the **Jensen–Shannon divergence** between $p_{data}$ and $p_g$, up to a constant. The global optimum is reached when $p_g = p_{data}$, at which point $D^* = ½$ everywhere and the discriminator can do no better than chance (Goodfellow et al. 2014).

$$D_G^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}$$
$$V(D_G^*, G) = -\log 4 + 2\,\mathrm{JSD}(p_{\text{data}} \| p_g) \tag{2.94}$$

where JSD is the Jensen–Shannon divergence, which is zero if and only if the two distributions are equal.

## 2.37.3 Training Algorithm

In practice, G and D are neural networks trained by alternating stochastic gradient steps, as in Algorithm 2.16. One important modification is needed. Early in training the discriminator easily rejects the poor fakes, so $\log(1 - D(G(\mathbf{z})))$ saturates and provides almost no gradient to G. Goodfellow et al. therefore proposed the **non-saturating** generator loss: maximise $\log D(G(\mathbf{z}))$ instead, which has the same fixed point but much stronger gradients when the generator is losing.

**Algorithm 2.16** GAN Training with Alternating Mini-batch SGD (non-saturating loss)

**Input:** Training data; noise prior $p_z$; generator $G_\theta$; discriminator $D_\varphi$; discriminator steps per iteration $k$; learning rates

**Output:** Trained generator $G_\theta$

**for each** training iteration **do**
  **for** step $= 1$ **to** $k$ **do** ▷ *update discriminator*
    sample real mini-batch $\{x^{(1)},\dots,x^{(m)}\}$ **and** noise $\{z^{(1)},\dots,z^{(m)}\} \sim p_z$
    $L_D \leftarrow -(1/m)\,\Sigma_i\,[\,\log D_\varphi(x^{(i)}) + \log(1 - D_\varphi(G_\theta(z^{(i)})))\,]$
    $\varphi \leftarrow \varphi - \eta_D\,\nabla_\varphi L_D$
  sample noise $\{z^{(1)},\dots,z^{(m)}\} \sim p_z$ ▷ *update generator*
  $L_G \leftarrow -(1/m)\,\Sigma_i \log D_\varphi(G_\theta(z^{(i)}))$ ▷ *non-saturating loss*
  $\theta \leftarrow \theta - \eta_G\,\nabla_\theta L_G$
**return** $G_\theta$

**Listing 2.31** One GAN training step in PyTorch using the non-saturating loss.

```python
import torch, torch.nn as nn

z_dim = 100
G = nn.Sequential(nn.Linear(z_dim, 256), nn.LeakyReLU(0.2),
                  nn.Linear(256, 784), nn.Tanh())            # 28x28 images in [-1, 1]
D = nn.Sequential(nn.Linear(784, 256), nn.LeakyReLU(0.2),
                  nn.Linear(256, 1))                          # logit of P(real)
opt_G = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_D = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
bce = nn.BCEWithLogitsLoss()

def train_step(x_real):                                       # x_real: (m, 784)
    m = x_real.size(0); ones, zeros = torch.ones(m, 1), torch.zeros(m, 1)
    # --- discriminator: real -> 1, fake -> 0
    x_fake = G(torch.randn(m, z_dim))
    loss_D = bce(D(x_real), ones) + bce(D(x_fake.detach()), zeros)
    opt_D.zero_grad(); loss_D.backward(); opt_D.step()
    # --- generator: make D output 1 on fakes (non-saturating)
    loss_G = bce(D(x_fake), ones)
    opt_G.zero_grad(); loss_G.backward(); opt_G.step()
    return loss_D.item(), loss_G.item()
```

Listing 2.31 implements the algorithm for MNIST-sized images. The Adam settings (learning rate $2 \times 10^{-4}$, $\beta_1 = 0.5$) follow the DCGAN recommendations described next.

## 2.37.4 Training Difficulties: Mode Collapse and Instability

GAN training seeks a saddle point rather than a minimum, and simultaneous gradient descent on a two-player game need not converge: the losses can oscillate indefinitely. The most notorious failure is **mode collapse**, in which the generator maps many different $\mathbf{z}$ to the same few outputs—for example producing only one or two digit classes—

because a small set of convincing samples temporarily fools the discriminator. When the discriminator adapts, the generator jumps to another mode rather than covering all of them. A second problem is **vanishing gradients** when the discriminator becomes too good and the supports of $p_{\text{data}}$ and $p_g$ barely overlap: the Jensen–Shannon divergence saturates at $\log 2$ and provides no useful signal.

Many remedies have been proposed. The **Wasserstein GAN** (WGAN; Arjovsky, Chintala & Bottou 2017) replaces the JS divergence with the Earth-Mover (Wasserstein-1) distance, estimated by a 1-Lipschitz "critic" as in Equation (2.95), giving smooth gradients even for non-overlapping distributions; WGAN-GP (Gulrajani et al. 2017) enforces the Lipschitz constraint with a gradient penalty. **Spectral normalisation** (Miyato et al. 2018) constrains each discriminator layer's spectral norm. Minibatch discrimination, feature matching, two-time-scale learning rates (TTUR) and progressive growing also help.

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \le 1} \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[f(\mathbf{x})] - \mathbb{E}_{\mathbf{z}\sim p_z}[f(G(\mathbf{z}))] \tag{2.95}$$

where the supremum is over 1-Lipschitz functions $f$, approximated by the critic network.

## 2.37.5 Notable GAN Architectures

**DCGAN** (Radford, Metz & Chintala 2016) established architectural guidelines that made convolutional GANs train reliably: replace pooling with strided convolutions (discriminator) and transposed convolutions (generator); use batch normalisation in both networks; remove fully connected hidden layers; use ReLU in the generator (tanh at the output) and LeakyReLU in the discriminator. DCGAN also showed that the latent space supports vector arithmetic—"smiling woman" − "neutral woman" + "neutral man" yields a smiling man.

**CycleGAN** (Zhu et al. 2017) performs **unpaired image-to-image translation**—horses to zebras, summer to winter, photographs to Monet paintings—without any paired examples. Two generators $G: X \to Y$ and $F: Y \to X$ are trained with adversarial losses in each domain plus a **cycle-consistency loss**, Equation (2.96), requiring that translating an image to the other domain and back recovers the original. Its paired counterpart, **pix2pix** (Isola et al. 2017), uses a conditional GAN with a U-Net generator for tasks such as turning segmentation maps into photographs.

$$\mathcal{L}_{\text{cyc}}(G, F) = \mathbb{E}_{\mathbf{x}}[\|F(G(\mathbf{x})) - \mathbf{x}\|_1] + \mathbb{E}_{\mathbf{y}}[\|G(F(\mathbf{y})) - \mathbf{y}\|_1] \tag{2.96}$$

where $\mathbf{x}$ and $\mathbf{y}$ are images from domains $X$ and $Y$.

**StyleGAN** (Karras, Laine & Aila 2019) and StyleGAN2 (2020) from NVIDIA produced photorealistic $1024 \times 1024$ faces. Building on progressive growing (Karras et al.

2018), StyleGAN maps $\mathbf{z}$ through an 8-layer MLP to an intermediate latent $\mathbf{w}$, which controls the generator at each resolution through adaptive instance normalisation ("styles"), with per-pixel noise adding stochastic detail. Coarse layers control pose and face shape, middle layers features and hairstyle, and fine layers colour and micro-texture, allowing style mixing between images. **BigGAN** (Brock, Donahue & Simonyan 2019) scaled class-conditional GANs to ImageNet. Table 2.37 summarises these models.

**Table 2.37** Influential GAN variants.

| Model | Year | Key contribution | Application |
|---|---|---|---|
| GAN | 2014 | Adversarial minimax training | Proof of concept on MNIST, CIFAR |
| Conditional GAN | 2014 | Condition G and D on labels y | Class-controlled generation |
| DCGAN | 2016 | Stable all-convolutional design | Image synthesis, latent arithmetic |
| WGAN / WGAN-GP | 2017 | Wasserstein distance; gradient penalty | Stable training, less mode collapse |
| pix2pix | 2017 | Conditional GAN + U-Net + L1 loss | Paired image-to-image translation |
| CycleGAN | 2017 | Cycle-consistency loss | Unpaired domain translation |
| BigGAN | 2019 | Large-scale class-conditional training | High-fidelity ImageNet synthesis |
| StyleGAN / 2 / 3 | 2019–2021 | Style-based generator, mapping network | Photorealistic faces, editing |

**Note | Evaluating Generative Models**

Sample quality is commonly measured with the **Fréchet Inception Distance** (FID; Heusel et al. 2017), the Fréchet distance between Gaussians fitted to Inception-network features of real and generated images (lower is better), and the older Inception Score. FID captures both fidelity and diversity but depends on the feature extractor and sample size, so it should be compared only under identical protocols. Precision and recall for distributions separate fidelity from coverage.

**Case Study 2.8 | Synthetic Data and Deepfakes**

GANs have been used to augment scarce medical-imaging datasets, to super-resolve images (SRGAN; Ledig et al. 2017) and to generate synthetic tabular data that protects privacy. The same technology underlies "deepfakes"—realistic manipulated videos of real people—raising concerns about misinformation and consent and motivating research into detection and provenance watermarking.

## 2.38 Variational Autoencoders (VAEs)

The **variational autoencoder** (VAE), introduced independently by Kingma and Welling (2014) and Rezende, Mohamed and Wierstra (2014), combines deep neural networks with probabilistic latent-variable modelling. Unlike a GAN, a VAE defines an explicit (approximate) likelihood, trains with a stable maximum-likelihood-style objective and learns an **encoder** that maps data to a structured latent space. Its samples are often blurrier than those of GANs, but VAEs are prized for representation learning, anomaly detection, controllable generation, and as the image compressor inside latent diffusion models.

## 2.38.1 Latent-Variable Generative Model

A VAE assumes each observation $\mathbf{x}$ is generated by first drawing a low-dimensional latent code $\mathbf{z}$ from a simple prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$, then drawing $\mathbf{x}$ from a **decoder** distribution $p_\theta(\mathbf{x} \mid \mathbf{z})$ whose parameters are output by a neural network. The marginal likelihood $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$ is intractable because the integral runs over all latent codes, and so is the true posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. The VAE therefore introduces an **encoder** (inference network) $q_\varphi(\mathbf{z} \mid \mathbf{x})$, a Gaussian with mean $\boldsymbol{\mu}_\varphi(\mathbf{x})$ and diagonal variance $\sigma^2_\varphi(\mathbf{x})$, that approximates the posterior. Because one network amortises inference across all data points, this is called **amortised variational inference**. Figure 2.48 shows the architecture.

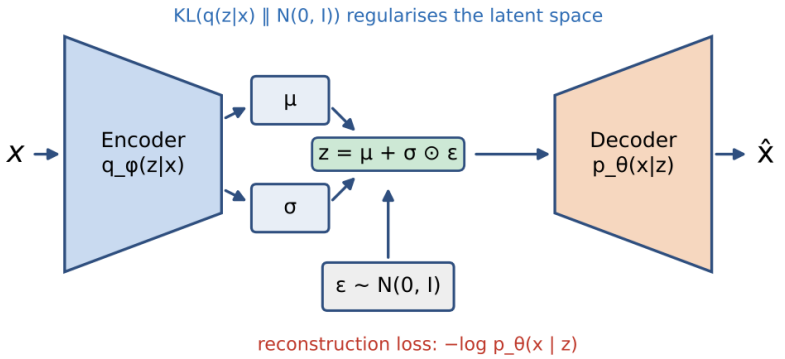



**Figure 2.48** Variational autoencoder. The encoder outputs the mean and log-variance of $q(z \mid x)$; a latent code is sampled via the reparameterisation trick $z = \mu + \sigma \odot \varepsilon$; the decoder reconstructs x from z.

## 2.38.2 Deriving the Evidence Lower Bound

The VAE is trained by maximising a lower bound on the log-likelihood, the **evidence lower bound** (ELBO). Starting from $\log p_\theta(\mathbf{x})$, we multiply and divide by $q_\varphi(\mathbf{z} \mid$

$\mathbf{x}$) inside the integral and apply **Jensen's inequality** ($\log \mathrm{E}[\cdot] \geq \mathrm{E}[\log \cdot]$, since log is concave), as in Equation (2.97).

$$\begin{aligned} \log p_\theta(\mathbf{x}) &= \log \int p_\theta(\mathbf{x}, \mathbf{z})\, d\mathbf{z} = \log \mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})}\left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}\,|\,\mathbf{x})}\right] \\ &\geq \mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})}\left[\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z}\,|\,\mathbf{x})\right] \equiv \mathcal{L}(\theta, \phi; \mathbf{x}) \end{aligned} \tag{2.97}$$

where p_θ(x, z) = p_θ(x | z) p(z) is the joint distribution and $\mathcal{L}$ is the ELBO.

An equivalent derivation reveals exactly how loose the bound is. Writing log p_θ(x) = $\mathrm{E}_q[\log$ p_θ(x)$]$ and using p_θ(x) = p_θ(x, z)/p_θ(z | x) gives the identity in Equation (2.98): the log-likelihood equals the ELBO plus the KL divergence from the approximate to the true posterior. Since KL ≥ 0, maximising the ELBO over φ pushes q towards the true posterior, while maximising over θ increases the likelihood.

$$\log p_\theta(\mathbf{x}) = \mathcal{L}(\theta, \phi; \mathbf{x}) + \mathrm{KL}(q_\phi(\mathbf{z}\,|\,\mathbf{x}) \,\|\, p_\theta(\mathbf{z}\,|\,\mathbf{x})) \tag{2.98}$$

where $\mathrm{KL}(q \,\|\, p) = \mathrm{E}_q[\log q - \log p] \geq 0$ is the Kullback–Leibler divergence.

Splitting log p_θ(x, z) = log p_θ(x | z) + log p(z) rearranges the ELBO into the intuitive form of Equation (2.99): a **reconstruction term** that rewards decoding $\mathbf{x}$ accurately from codes sampled from the encoder, minus a **regularisation term** that keeps the encoder's distribution close to the prior. For a Gaussian encoder and standard-normal prior the KL term has the closed form shown on the second line. With a Gaussian decoder of fixed variance, the reconstruction term reduces to (scaled) squared error; with a Bernoulli decoder for binary pixels it is binary cross-entropy.

$$\begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{x}) &= \mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})}\left[\log p_\theta(\mathbf{x}\,|\,\mathbf{z})\right] - \mathrm{KL}(q_\phi(\mathbf{z}\,|\,\mathbf{x}) \,\|\, p(\mathbf{z})) \\ \mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, I)\big) &= \tfrac{1}{2}\sum_{j=1}^{J}\big(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\big) \end{aligned} \tag{2.99}$$

where J is the latent dimension and $\mu_j$, $\sigma_j^2$ are the encoder outputs for dimension j.

## 2.38.3 The Reparameterisation Trick

To train the encoder by gradient descent we need ∇_φ of an expectation over q_φ. Sampling $\mathbf{z}$ ~ q_φ($\mathbf{z}$ | $\mathbf{x}$) is a stochastic operation through which gradients cannot flow directly. The **reparameterisation trick** rewrites the sample as a deterministic, differentiable function of φ and an independent noise variable, as in Equation (2.100). The randomness is moved into ε, which does not depend on φ, so the gradient can pass through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ by ordinary backpropagation. The resulting Monte Carlo gradient

estimator has much lower variance than the alternative score-function (REINFORCE) estimator discussed in the deep reinforcement learning section. In practice a single sample per data point suffices, and the encoder outputs $\log \sigma^2$ for numerical stability.

$$\mathbf{z} = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(\mathbf{0}, I)$$
$$\nabla_\phi \, \mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})}[f(\mathbf{z})] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0,\,I)}[\nabla_\phi f(\mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \odot \epsilon)] \tag{2.100}$$

where $\varepsilon$ is standard Gaussian noise independent of the parameters $\varphi$.

**Algorithm 2.17** VAE Training with the Reparameterisation Trick

**Input:** Dataset $\{x^{(i)}\}$; encoder $(\mu_\varphi, \log \sigma^2_\varphi)$; decoder $p_\theta(x \mid z)$; learning rate $\eta$
**Output:** Trained parameters $\theta, \varphi$

**initialise** $\theta, \varphi$
**repeat until** convergence
  sample a mini-batch $\{x^{(1)}, \dots, x^{(m)}\}$
  **for** $i = 1$ **to** $m$ **do**
    $(\mu_i, \log \sigma_i^2) \leftarrow \text{encoder}_\varphi(x^{(i)})$
    $\varepsilon \sim \mathcal{N}(0, I)$; $z_i \leftarrow \mu_i + \exp(\tfrac{1}{2} \log \sigma_i^2) \odot \varepsilon$
    $\text{rec}_i \leftarrow -\log p_\theta(x^{(i)} \mid z_i)$ ▷ *e.g. BCE or squared error*
    $\text{kl}_i \leftarrow \tfrac{1}{2} \sum_j (\mu_{ij}^2 + \sigma_{ij}^2 - \log \sigma_{ij}^2 - 1)$
  $L \leftarrow (1/m) \sum_i (\text{rec}_i + \beta \cdot \text{kl}_i)$ ▷ *negative ELBO ($\beta = 1$ for a standard VAE)*
  $(\theta, \varphi) \leftarrow (\theta, \varphi) - \eta \, \nabla L$
**return** $\theta, \varphi$

**Listing 2.32** A fully connected VAE for 28 × 28 images in PyTorch.

```python
import torch, torch.nn as nn, torch.nn.functional as F

class VAE(nn.Module):
    def __init__(self, d_in=784, d_h=400, d_z=20):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(d_in, d_h), nn.ReLU())
        self.mu, self.logvar = nn.Linear(d_h, d_z), nn.Linear(d_h, d_z)
        self.dec = nn.Sequential(nn.Linear(d_z, d_h), nn.ReLU(), nn.Linear(d_h, d_in))

    def forward(self, x):
        h = self.enc(x)
        mu, logvar = self.mu(h), self.logvar(h)
        z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)   # reparameterisation
        return self.dec(z), mu, logvar                            # decoder returns
        ↪ logits

def neg_elbo(x_logits, x, mu, logvar, beta=1.0):
    rec = F.binary_cross_entropy_with_logits(x_logits, x, reduction="sum")
    kl = 0.5 * torch.sum(mu.pow(2) + logvar.exp() - logvar - 1)
    return (rec + beta * kl) / x.size(0)

```

```
model = VAE(); opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for x, _ in train_loader:                                   # x in [0, 1]
    x = x.view(x.size(0), -1)
    loss = neg_elbo(*model(x), x)
    opt.zero_grad(); loss.backward(); opt.step()
samples = torch.sigmoid(model.dec(torch.randn(16, 20)))     # generate
```

Algorithm 2.17 and Listing 2.32 implement the procedure. To generate new data, sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathrm{I})$ and pass it through the decoder, as in the last line of the listing.

## 2.38.4 Latent Space, Variants and Limitations

The KL term encourages the aggregate of encoder distributions to fill the prior smoothly, so nearby latent codes decode to similar outputs and interpolating between two codes yields plausible intermediate images (Figure 2.49). **β-VAE** (Higgins et al. 2017) multiplies the KL term by $\beta > 1$, encouraging **disentangled** latent factors (e.g. one dimension for rotation, another for scale) at the cost of reconstruction quality. **Conditional VAEs** condition both encoder and decoder on a label for controlled generation, and **VQ-VAE** (van den Oord, Vinyals & Kavukcuoglu 2017) uses a discrete codebook of latent vectors, an idea used in many image and audio tokenisers.

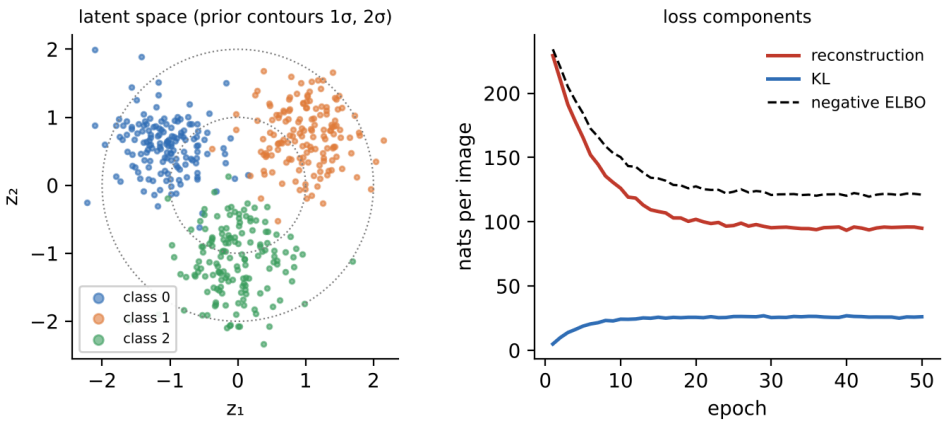



**Figure 2.49** Left: two-dimensional VAE latent space of a three-class dataset; the KL term arranges classes into a compact, continuous region around the origin of the prior. Right: KL and reconstruction terms of the loss during training.

VAEs have two well-known weaknesses. Their samples tend to be **blurry**, partly because a Gaussian decoder with pixel-wise loss averages over plausible outputs, and partly because the ELBO penalises missing modes more than generating implausible ones. With powerful autoregressive decoders, VAEs can suffer **posterior collapse**: the

decoder ignores $\mathbf{z}$, the KL term goes to zero, and the latent code carries no information. KL annealing (gradually increasing $\beta$ from 0 to 1) and free-bits constraints mitigate it.

**E Worked Example 2.19 | Evaluating the KL Term**

A 2-dimensional encoder outputs $\mu = (0.5, -1.0)$ and $\sigma^2 = (0.25, 1.0)$ for an input x.

- Dimension 1: $\mu^2 + \sigma^2 - \log \sigma^2 - 1 = 0.25 + 0.25 - (-1.386) - 1 = 0.886$.
- Dimension 2: $1.0 + 1.0 - 0 - 1 = 1.0$.
- KL $= \tfrac{1}{2}(0.886 + 1.0) = 0.943$ nats. A reparameterised sample with $\varepsilon = (0.2, -0.4)$ is $z = (0.5 + 0.5 \cdot 0.2, -1.0 + 1.0 \cdot (-0.4)) = (0.6, -1.4)$.

# 2.39 Autoencoders and Representation Learning

An **autoencoder** is a neural network trained to reproduce its input at its output. It consists of an **encoder** f that maps the input $\mathbf{x}$ to a **code** (latent representation) $\mathbf{h} = f(\mathbf{x})$ and a **decoder** g that reconstructs $\hat{\mathbf{x}} = g(\mathbf{h})$. Trained naively, an autoencoder could learn the useless identity function; the art lies in *constraining* it so that it must capture the most salient structure of the data. Autoencoders predate the deep learning revival—Hinton and Salakhutdinov (2006) used deep autoencoders, pretrained layer-wise with restricted Boltzmann machines, to obtain far better low-dimensional codes than PCA—and they remain a central tool of **unsupervised representation learning**, dimensionality reduction, denoising and anomaly detection. The VAE of the previous section is a probabilistic relative; this section covers the deterministic variants.

## 2.39.1 Undercomplete Autoencoders

The simplest constraint is a **bottleneck**: make the code dimension smaller than the input dimension, giving an **undercomplete** autoencoder. Training minimises the reconstruction loss in Equation (2.101). If the encoder and decoder are linear and the loss is squared error, the optimal autoencoder spans the same subspace as the top principal components, so an undercomplete autoencoder with non-linear layers is a **non-linear generalisation of PCA**, able to learn curved manifolds. Figure 2.50 shows the classic hourglass architecture.

$$\min_{f,\,g} \; \frac{1}{N} \sum_{i=1}^{N} \left\| \mathbf{x}^{(i)} - g\big(f(\mathbf{x}^{(i)})\big) \right\|^2 \tag{2.101}$$

where f is the encoder, g the decoder and N the number of training examples (binary cross-entropy is used for inputs in $[0, 1]$).

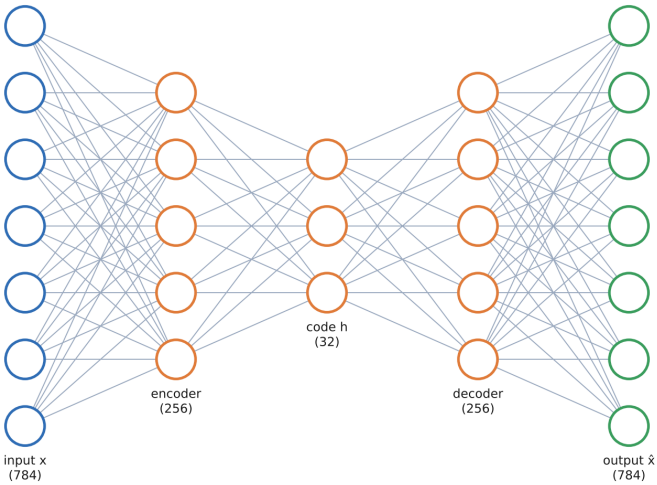



**Figure 2.50** An undercomplete autoencoder: the encoder compresses a 784-dimensional input to a 32-dimensional code, and the decoder reconstructs the input from the code.

## 2.39.2 Regularised Autoencoders

Rather than limiting capacity through the bottleneck alone, **regularised autoencoders** add a penalty or corruption that forces useful structure even when the code is as large as, or larger than, the input (**overcomplete**).

### Sparse autoencoders

A **sparse autoencoder** adds a penalty that encourages most code units to be inactive for any given input, as in Equation (2.102). The L1 form penalises absolute activations; the KL form, used in early work (Ng 2011), drives the average activation $\hat{\rho}_j$ of each hidden unit towards a small target $\rho$ such as 0.05. Each input is then explained by a small set of active features, which tends to produce interpretable, parts-based detectors. Sparse autoencoders have recently become a key tool in **mechanistic interpretability**, where they are trained on the internal activations of language models to decompose them into human-interpretable features.

$$\mathcal{L}_{\text{sparse}} = \mathcal{L}_{\text{rec}} + \lambda \sum_{j} |h_j| \quad \text{(L1)}$$
$$\mathcal{L}_{\text{sparse}} = \mathcal{L}_{\text{rec}} + \beta \sum_{j} \mathrm{KL}\big(\rho \,\|\, \hat{\rho}_j\big), \quad \hat{\rho}_j = \tfrac{1}{N}\sum_{i} h_j(\mathbf{x}^{(i)}) \tag{2.102}$$

where $h_j$ is the activation of code unit $j$, $\lambda$ and $\beta$ are penalty weights, and $\mathrm{KL}(\rho \,\|\, \hat{\rho}_j)$ is the divergence between Bernoulli distributions with means $\rho$ and $\hat{\rho}_j$.

### Denoising autoencoders

A **denoising autoencoder** (DAE; Vincent et al. 2008) corrupts the input—adding Gaussian noise, masking random pixels to zero, or applying salt-and-pepper noise—and trains the network to reconstruct the *clean* original, as in Equation (2.103). Because it cannot simply copy its input, the network must learn the structure of the data manifold to project corrupted points back onto it. Vincent et al. showed that stacking DAEs gave excellent pretrained features. The idea has had an enormous legacy: BERT's masked language modelling and the **masked autoencoders** (MAE; He et al. 2022) for vision—which mask 75% of image patches and reconstruct them—are denoising autoencoders, and diffusion models, covered next, are trained as denoisers at many noise levels.

$$\mathcal{L}_{\text{DAE}} = \mathbb{E}_{\mathbf{x}\sim\mathcal{D},\ \tilde{\mathbf{x}}\sim C(\tilde{\mathbf{x}}\,|\,\mathbf{x})} \big\| \mathbf{x} - g\big(f(\tilde{\mathbf{x}})\big) \big\|^2 \tag{2.103}$$

where $C$ is a stochastic corruption process, e.g. $\tilde{\mathbf{x}} = \mathbf{x} + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2 \mathrm{I})$.

### Contractive autoencoders

A **contractive autoencoder** (CAE; Rifai et al. 2011) adds the squared Frobenius norm of the encoder's Jacobian to the loss, Equation (2.104). The penalty makes the code insensitive to small perturbations of the input in most directions, while the reconstruction term forces sensitivity along the directions that matter—those tangent to the data manifold. The DAE and CAE are closely related: for small Gaussian noise the denoising objective approximately penalises the Jacobian of the *full* reconstruction function (Alain & Bengio 2014).

$$\mathcal{L}_{\text{CAE}} = \mathcal{L}_{\text{rec}} + \lambda \big\| J_f(\mathbf{x}) \big\|_F^2 = \mathcal{L}_{\text{rec}} + \lambda \sum_{i,j} \Big( \frac{\partial h_j(\mathbf{x})}{\partial x_i} \Big)^2 \tag{2.104}$$

where $J_f$ is the Jacobian of the encoder and $\|\cdot\|_F$ the Frobenius norm.

**Table 2.38** Types of autoencoder and how each prevents trivial identity mappings.

| Type | Constraint / regulariser | Learned representation | Typical use |
|---|---|---|---|
| Undercomplete | Code smaller than input | Compressed non-linear subspace | Dimensionality reduction, visualisation |
| Sparse | L1 or KL penalty on activations | Few active, interpretable features | Feature learning, interpretability |
| Denoising | Reconstruct clean x from corrupted $\tilde{x}$ | Robust manifold structure | Denoising, pretraining (MAE, BERT) |
| Contractive | Penalise encoder Jacobian $\|J_f\|^2$ | Locally invariant features | Robust representations |
| Variational | KL to prior; stochastic code | Smooth, generative latent space | Generation, anomaly detection |
| Convolutional | Conv encoder / transposed-conv decoder | Spatial feature maps | Images, segmentation pretraining |

## 2.39.3 Applications: Anomaly Detection and Beyond

Because an autoencoder trained only on normal data reconstructs normal patterns well and unusual patterns poorly, the **reconstruction error** is a natural anomaly score. This approach is used to detect fraudulent transactions, faults in industrial sensor data, intrusions in network traffic and defects in manufactured parts. Listing 2.33 trains a convolutional autoencoder in Keras and flags test images whose error exceeds a threshold chosen on validation data. Autoencoders are also used for denoising images and audio, for learning compact codes for retrieval, and, as in latent diffusion, for compressing images into a latent space where generation is cheaper.

**Listing 2.33** A convolutional denoising autoencoder in Keras used for anomaly detection.

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

inp = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, 3, strides=2, padding="same", activation="relu")(inp)    # 14x14
x = layers.Conv2D(64, 3, strides=2, padding="same", activation="relu")(x)      # 7x7 code
x = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(x)
out = layers.Conv2DTranspose(1, 3, strides=2, padding="same", activation="sigmoid")(x)
ae = keras.Model(inp, out)
ae.compile(optimizer="adam", loss="binary_crossentropy")

noisy = np.clip(x_normal + 0.2 * np.random.randn(*x_normal.shape), 0, 1)   # corrupt
ae.fit(noisy, x_normal, epochs=20, batch_size=128, validation_split=0.1)  # train on NORMAL data only

err = np.mean((ae.predict(x_test) - x_test) ** 2, axis=(1, 2, 3))          # reconstruction error
threshold = np.percentile(np.mean((ae.predict(x_val) - x_val) ** 2, axis=(1, 2, 3)), 99)
```

```
is_anomaly = err > threshold
```

### 2.39.4 From Autoencoders to Self-Supervised Learning

Autoencoders exemplify **self-supervised learning**: the supervisory signal is derived from the data itself. Modern self-supervised methods have broadened the toolkit beyond reconstruction. **Contrastive learning** (SimCLR, Chen et al. 2020; MoCo, He et al. 2020) trains an encoder so that two augmented views of the same image have similar embeddings while views of different images are pushed apart, using the InfoNCE loss. Non-contrastive methods such as BYOL (Grill et al. 2020) and DINO (Caron et al. 2021) avoid negative pairs altogether. Masked modelling (BERT, MAE) has become the dominant paradigm for pretraining transformers. All share the aim of this section: learning representations that make downstream tasks easy with few labels.

**Key Point** | **What Makes a Good Representation?**

- It retains the information needed for downstream tasks while discarding nuisance variation.
- It is robust: small input perturbations produce small changes in the code.
- It is simple to use: ideally linearly separable for classes of interest ("linear probe" accuracy is a common test).
- Ideally it disentangles independent factors of variation.

## 2.40 Diffusion Models and Generative AI

**Diffusion models** are the generative models behind most of today's text-to-image and video systems, including Stable Diffusion, DALL·E 2 and 3, Imagen and Midjourney-style tools, and they are increasingly used for audio, molecules and proteins. The core idea, introduced by Sohl-Dickstein et al. (2015) and made practical by **Denoising Diffusion Probabilistic Models** (DDPM; Ho, Jain & Abbeel 2020) and score-based models (Song & Ermon 2019; Song et al. 2021), is simple: gradually destroy the structure in data by adding noise over many steps, then train a neural network to reverse the process one step at a time. Generation starts from pure noise and denoises it into a sample. Diffusion models combine stable, likelihood-based training (no adversarial game) with sample quality that surpassed GANs on ImageNet (Dhariwal & Nichol 2021).

## 2.40.1 The Forward (Noising) Process

The **forward process** is a fixed Markov chain that adds a small amount of Gaussian noise at each of T steps (T = 1,000 in DDPM), according to a **variance schedule** $\beta_1 < \beta_2 < \ldots < \beta_T$, as in Equation (2.105). Because each step is Gaussian, we can jump directly from $\mathbf{x}_0$ to any $\mathbf{x}_t$ in closed form. With $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_s \alpha_s$, the noisy sample is a weighted combination of the clean data and fresh noise. As $t \to T$, $\bar{\alpha}_t \to 0$ and $\mathbf{x}_T$ is essentially pure standard Gaussian noise. Figure 2.51 illustrates the process on a two-dimensional dataset.

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t I\big)$$

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1-\bar{\alpha}_t)I\big), \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s) \tag{2.105}$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, I)$$

where $\mathbf{x}_0$ is a data sample, $\mathbf{x}_t$ its noised version at step t and $\beta_t$ the noise variance added at step t (linearly from $10^{-4}$ to 0.02 in DDPM).

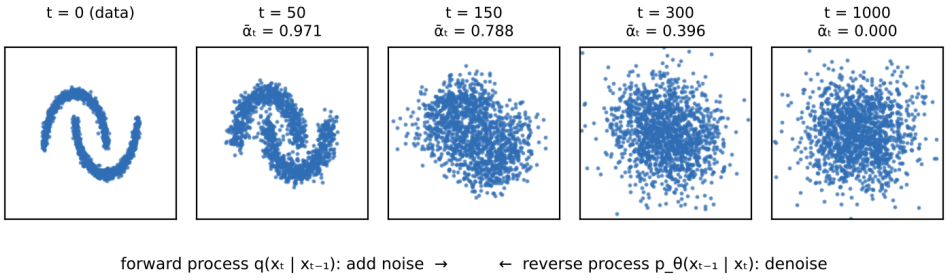



**Figure 2.51** Forward diffusion on a two-dimensional 'two moons' dataset: as t increases the data are progressively blurred into an isotropic Gaussian. The learned reverse process runs right to left, turning noise into data.

## 2.40.2 The Reverse (Denoising) Process

Generation requires the reverse conditionals $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$, which are intractable because they depend on the whole data distribution. For small $\beta_t$, however, they are approximately Gaussian, so the model learns a Gaussian reverse process $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ with a neural network predicting its mean, as in Equation (2.106). Ho et al. found that it works best to parameterise the network to predict the **noise** $\varepsilon$ that was added, $\varepsilon_\theta(\mathbf{x}_t, t)$, and to derive the mean from it. The variance is usually fixed to $\sigma_t^2 = \beta_t$ (or the posterior variance $\tilde{\beta}_t$).

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1};\ \mu_\theta(\mathbf{x}_t, t),\ \sigma_t^2 I\big)$$
$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon_\theta(\mathbf{x}_t, t)\Big) \tag{2.106}$$

where ε_θ is the noise-prediction network, typically a U-Net (or a transformer) that receives t through a sinusoidal time embedding.

## 2.40.3 Training Objective and Algorithms

A diffusion model is a deep latent-variable model with latents $\mathbf{x}_1$,...,$\mathbf{x}$_T, so it can be trained by maximising an ELBO, exactly as for the VAE but with a fixed encoder (the forward process). After simplification, Ho et al. obtained a strikingly simple **noise-prediction loss**, Equation (2.107): pick a random time step, noise a training example to that step in closed form, and regress the network's output onto the noise that was added. Training (Algorithm 2.18) is therefore ordinary supervised regression, with no adversary and no sequential simulation. Sampling (Algorithm 2.19) runs the learned reverse chain from pure noise. The noise-prediction network is equivalent, up to scaling, to estimating the **score** $\nabla_x \log p(\mathbf{x}_t)$, linking DDPMs to score-based generative modelling.

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}\{1,T\},\ \mathbf{x}_0,\ \epsilon\sim\mathcal{N}(0,I)}\Big[\big\|\epsilon - \epsilon_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon,\ t\big)\big\|^2\Big] \tag{2.107}$$

where t is drawn uniformly from {1,...,T}.

**Algorithm 2.18** DDPM Training (Ho et al. 2020)

**Input:** Training data distribution $q(x_0)$; noise schedule $\beta_1$,...,$\beta$_T with $\bar{\alpha}_t = \prod_s(1-\beta_s)$; network ε_θ; learning rate η

**Output:** Trained noise-prediction network ε_θ

**repeat**
  $x_0 \sim q(x_0)$ ▷ *a training example (mini-batch in practice)*
  $t \sim \text{Uniform}(\{1, \ldots, T\})$
  $\varepsilon \sim N(0, I)$
  $x_t \leftarrow \sqrt{\bar{\alpha}_t}\cdot x_0 + \sqrt{(1-\bar{\alpha}_t)}\cdot\varepsilon$
  take a gradient step $\theta \leftarrow \theta - \eta\,\nabla\theta\,\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$
**until** converged
**return** ε_θ

**Algorithm 2.19** DDPM Sampling (Ancestral)

**Input:** Trained $\varepsilon_\theta$; schedule $\beta_t$, $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t$; noise scales $\sigma_t$ (e.g. $\sigma_t^2 = \beta_t$)
**Output:** Generated sample $x_0$

$x_T \sim \mathcal{N}(0, I)$
**for** $t = T$ down **to** $1$ **do**
  $z \sim \mathcal{N}(0, I)$ **if** $t > 1$, **else** $z \leftarrow 0$
  $x_{t-1} \leftarrow (1/\sqrt{\alpha_t}) \cdot ( x_t - (\beta_t / \sqrt{(1 - \bar{\alpha}_t)}) \cdot \varepsilon_\theta(x_t, t) ) + \sigma_t z$
**return** $x_0$

**Listing 2.34** Core DDPM training step and sampling loop in PyTorch ($\varepsilon_\theta$ is any network taking $(x, t)$).

```python
import torch, torch.nn.functional as F

T = 1000
betas = torch.linspace(1e-4, 0.02, T)
alphas = 1 - betas
abar = torch.cumprod(alphas, dim=0)

def train_step(eps_model, x0, opt):
    t = torch.randint(0, T, (x0.size(0),))                    # random step per example
    eps = torch.randn_like(x0)
    a = abar[t].view(-1, *[1] * (x0.dim() - 1))
    xt = a.sqrt() * x0 + (1 - a).sqrt() * eps                  # closed-form forward process
    loss = F.mse_loss(eps_model(xt, t), eps)                   # L_simple
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()

@torch.no_grad()
def sample(eps_model, shape):
    x = torch.randn(shape)                                     # x_T
    for t in reversed(range(T)):
        tt = torch.full((shape[0],), t, dtype=torch.long)
        mean = (x - betas[t] / (1 - abar[t]).sqrt() * eps_model(x, tt)) / alphas[t].sqrt()
        x = mean + (betas[t].sqrt() * torch.randn_like(x) if t > 0 else 0)
    return x
```

Listing 2.34 translates both algorithms almost line for line. The main practical drawback is speed: ancestral sampling needs one network evaluation per step, i.e. 1,000 forward passes per image. **DDIM** (Song, Meng & Ermon 2021) defines a non-Markovian, deterministic sampler that uses the same trained network with 20–50 steps; higher-order ODE solvers (e.g. DPM-Solver), **progressive distillation** and **consistency models** (Song et al. 2023) reduce this further to a handful of steps or even one.

### 2.40.4 Latent Diffusion

Running diffusion directly on high-resolution pixels is expensive. **Latent diffusion models** (LDM; Rombach et al. 2022), the basis of Stable Diffusion, first train an autoencoder (a VAE with perceptual and adversarial losses) that compresses a $512 \times 512 \times 3$ image into a latent tensor of, for example, $64 \times 64 \times 4$—a 48-fold reduction—and then train the diffusion model in this latent space. The decoder maps generated latents back to pixels. Conditioning information such as text is injected through **cross-attention** layers in the denoising U-Net, with keys and values computed from the output of a text encoder (CLIP's text transformer in Stable Diffusion v1). This design separates perceptual compression (handled by the autoencoder) from semantic generation (handled by diffusion) and made high-quality text-to-image generation feasible on consumer GPUs. More recent systems replace the U-Net with a **diffusion transformer** (DiT; Peebles & Xie 2023) operating on latent patches. Figure 2.52 summarises the pipeline.

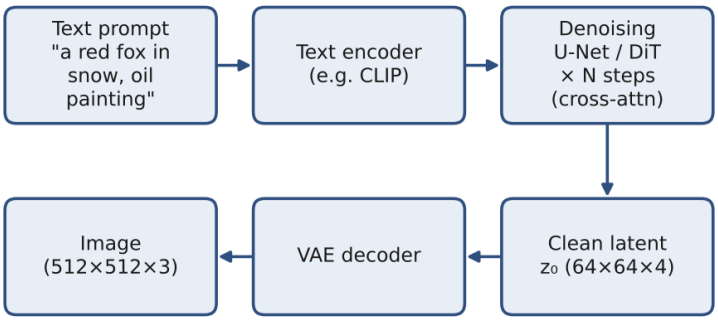



**Figure 2.52** Latent diffusion (Stable Diffusion) at inference: a text encoder conditions a denoising network via cross-attention; denoising runs in a compact latent space, and a VAE decoder maps the final latent to pixels.

### 2.40.5 Conditioning and Classifier-Free Guidance

Conditional diffusion models learn $\varepsilon_\theta(\mathbf{x}_t, t, \mathbf{c})$ for a condition $\mathbf{c}$ (class label, text embedding, image). To trade diversity for fidelity to the condition, Dhariwal and Nichol (2021) used **classifier guidance**, adding the gradient of a separately trained noisy-image classifier to the score. **Classifier-free guidance** (CFG; Ho & Salimans 2022) removes the need for a classifier. A single network is trained both conditionally and unconditionally by randomly replacing $\mathbf{c}$ with a null token $\emptyset$ (say 10–20% of the time).

At sampling time the two predictions are extrapolated as in Equation (2.108). A guidance scale $w = 1$ recovers the plain conditional model; larger values (typically 5–10 for text-to-image) yield images that match the prompt more closely, at the cost of diversity and, at extreme values, oversaturated colours.

$$\begin{aligned}\tilde{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}) &= \epsilon_\theta(\mathbf{x}_t, t, \varnothing) + w(\epsilon_\theta(\mathbf{x}_t, t, \mathbf{c}) - \epsilon_\theta(\mathbf{x}_t, t, \varnothing)) \\ \nabla_{\mathbf{x}_t} \log p_w(\mathbf{x}_t \mid \mathbf{c}) &\approx \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + w\, \nabla_{\mathbf{x}_t} \log p(\mathbf{c} \mid \mathbf{x}_t)\end{aligned} \tag{2.108}$$

where $\varnothing$ is the null condition and $w \geq 1$ the guidance scale; the second line shows the implied score, an implicit classifier raised to power $w$.

**Table 2.39** Comparison of deep generative model families.

| Family | Training | Sampling speed | Sample quality | Likelihood | Main weakness |
|---|---|---|---|---|---|
| GAN | Adversarial minimax | Fast (1 pass) | Very sharp | None | Unstable; mode collapse |
| VAE | Maximise ELBO | Fast (1 pass) | Often blurry | Lower bound | Blurriness; posterior collapse |
| Autoregressive (PixelCNN, GPT) | Exact max. likelihood | Slow (per token) | High | Exact | Sequential generation |
| Normalising flow | Exact max. likelihood | Fast | Moderate | Exact | Invertible architectures constrain design |
| Diffusion | Denoising (ELBO / score matching) | Slow (many steps) | State of the art | Lower bound | Sampling cost (mitigated by DDIM, distillation) |

Table 2.39 places diffusion models among the generative families discussed in this chapter. Generative AI—the use of such models together with LLMs to create text, images, audio, video and code—has moved from research to everyday products in a few years.

> **Note | Responsible Generative AI**
>
> Image and video generators raise questions of copyright and consent for training data, the creation of misleading or non-consensual imagery, and amplification of stereotypes present in web-scale datasets. Mitigations include dataset curation, output filtering, provenance standards and invisible watermarks, and human review for sensitive uses.

> **Case Study 2.9 | Diffusion Beyond Images**
>
> Diffusion has proved remarkably general. In structural biology, RFdiffusion (Watson et al. 2023) generates novel protein backbones that have been validated experimentally as binders, and AlphaFold 3 (Abramson et al. 2024) uses a diffusion module to produce atomic coordinates of biomolecular complexes. In audio, diffusion models synthesise speech and music; in

robotics, *diffusion policies* (Chi et al. 2023) generate action sequences for manipulation.

# 2.41 Deep Reinforcement Learning

**Reinforcement learning** (RL) studies agents that learn by trial and error to maximise cumulative reward through interaction with an environment. Chapter 1 introduced the RL framework—Markov decision processes, returns, value functions, Bellman equations, Q-learning—and value-based **deep Q-networks (DQN)** (Mnih et al. 2015), which approximate the action-value function with a neural network and learned to play Atari games from pixels; we do not repeat that material here. **Deep reinforcement learning** combines RL with deep neural networks as function approximators for policies and value functions. This section focuses on the other major family, **policy-gradient** methods, culminating in actor–critic algorithms and **Proximal Policy Optimisation (PPO)**, the algorithm most widely used in practice, including for RLHF of large language models.

## 2.41.1 Policy-Based Learning

Value-based methods like DQN learn Q(s, a) and act greedily; they handle discrete actions well but struggle with continuous or high-dimensional action spaces, where maximising over actions is itself an optimisation problem. **Policy-gradient methods** instead parameterise the policy directly as a neural network π_θ(a | s) that outputs a distribution over actions—a softmax over discrete actions, or the mean and standard deviation of a Gaussian for continuous control—and adjust θ by gradient ascent on the expected return J(θ), defined in Equation (2.109). Stochastic policies also explore naturally and can represent optimal mixed strategies.

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] \tag{2.109}$$

where $\tau = (s_0, a_0, r_0, s_1, \ldots)$ is a trajectory generated by following $\pi_\theta$, $r_t$ the reward at time t and $\gamma \in [0, 1]$ the discount factor.

## 2.41.2 The Policy Gradient Theorem and REINFORCE

The difficulty is that J depends on θ through the distribution of trajectories, which also involves unknown environment dynamics. The **log-derivative trick**, ∇_θ p_θ = p_θ ∇_θ log p_θ, resolves this. Because the dynamics do not depend on θ, they drop

out of $\nabla_\theta \log p_\theta(\tau)$, leaving only the policy terms. The result is the **policy gradient theorem** (Williams 1992; Sutton et al. 2000) in Equation (2.110), where $G_t$ is the **return-to-go** from time t. The gradient increases the log-probability of actions in proportion to how much return followed them.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\right], \quad G_t = \sum_{k=t}^{T}\gamma^{k-t} r_k$$
$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t}\nabla_\theta \log \pi_\theta(a_t \mid s_t)(G_t - b(s_t))\right] \tag{2.110}$$

where the second line subtracts a baseline $b(s_t)$, which leaves the gradient unbiased but can greatly reduce its variance.

**REINFORCE** (Williams 1992), given in Algorithm 2.20, estimates this expectation with Monte Carlo rollouts. It is simple and unbiased but has very **high variance**, because the return of a whole episode is a noisy signal for the quality of any single action. Subtracting a **baseline** $b(s_t)$ that does not depend on the action—most naturally an estimate of the state value $V(s_t)$—keeps the estimator unbiased (since $\mathbb{E}[\nabla \log \pi \cdot b] = 0$) while reducing variance considerably.

**Algorithm 2.20** REINFORCE with Baseline

**Input:** Policy network $\pi_\theta(a|s)$; value network $V_w(s)$ (baseline); discount $\gamma$; learning rates $\alpha_\theta$, $\alpha_w$
**Output:** Trained policy $\pi_\theta$

**for each** episode **do**
  generate a trajectory $s_0, a_0, r_0, \ldots, s_T, a_T, r_T$ by following $\pi_\theta$
  **for** $t = 0$ **to** $T$ **do**
    $G_t \leftarrow \Sigma_\{k=t\}^\{T\} \gamma^\{k-t\} r_k$ ▷ *return-to-go*
    $\delta_t \leftarrow G_t - V_w(s_t)$ ▷ *advantage estimate*
    $w \leftarrow w + \alpha_w\, \delta_t\, \nabla_w V_w(s_t)$ ▷ *fit baseline*
    $\theta \leftarrow \theta + \alpha_\theta\, \gamma^t\, \delta_t\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$ ▷ *policy-gradient ascent*
**return** $\pi_\theta$

## 2.41.3 Actor–Critic Methods

**Actor–critic** methods replace the Monte Carlo return with a bootstrapped estimate from a learned value function. The **actor** is the policy $\pi_\theta$; the **critic** is a value network $V_w(s)$ trained by temporal-difference learning. The policy is updated with the **advantage** $A(s,a) = Q(s,a) - V(s)$, which measures how much better an action is than the policy's average behaviour. The simplest estimate is the one-step TD error $\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$. Bootstrapping introduces some bias but greatly reduces variance

and allows updates at every step instead of the end of an episode. **Generalised Advantage Estimation** (GAE; Schulman et al. 2016) interpolates between the one-step TD error and the full Monte Carlo return with a parameter $\lambda$, Equation (2.111). Figure 2.53 shows the interaction between actor, critic and environment.

$$\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$$
$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l} \tag{2.111}$$

where $\lambda \in [0, 1]$: $\lambda = 0$ gives the low-variance, higher-bias TD advantage and $\lambda = 1$ the unbiased, high-variance Monte Carlo advantage ($\lambda \approx 0.95$ is typical).

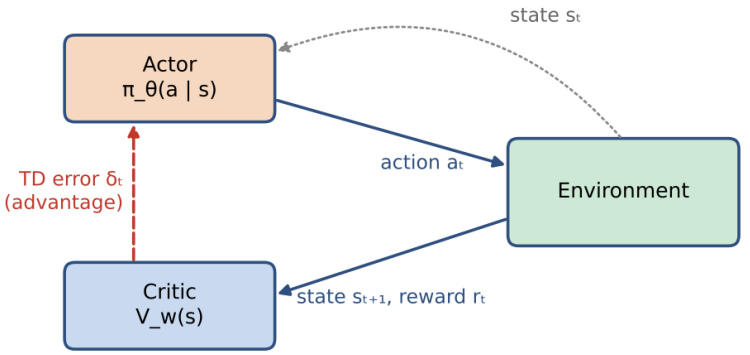



**Figure 2.53** Actor–critic architecture. The actor selects actions; the critic evaluates states and computes the TD error/advantage, which drives the updates of both networks.

Influential actor–critic algorithms include **A2C/A3C** (Mnih et al. 2016), which run many actors in parallel to decorrelate experience; **DDPG** (Lillicrap et al. 2016) and **TD3** (Fujimoto et al. 2018) for continuous control with deterministic policies; and **Soft Actor-Critic** (SAC; Haarnoja et al. 2018), which adds an entropy bonus to the objective to encourage exploration and is a strong default for robotics-style continuous control.

## 2.41.4 Proximal Policy Optimisation

Policy-gradient updates are fragile: a single step that is too large can collapse the policy, after which the agent collects poor data and may never recover. **Trust Region Policy Optimisation** (TRPO; Schulman et al. 2015) constrained each update so that the KL divergence between old and new policies stayed below a threshold, but required second-order optimisation. **Proximal Policy Optimisation** (PPO; Schulman et al. 2017) achieves similar stability with a simple first-order **clipped surrogate objective**, Equation (2.112). The probability ratio $r_t(\theta)$ measures how much more likely the new policy

is to take the logged action than the old one. Clipping the ratio to $[1-\varepsilon, 1+\varepsilon]$ ($\varepsilon \approx 0.1$–$0.2$) and taking the minimum with the unclipped term removes any incentive to move the ratio outside this range: when the advantage is positive, the objective stops rewarding increases in the action's probability beyond $1+\varepsilon$, and when it is negative, it stops rewarding decreases below $1-\varepsilon$. This makes it safe to perform several epochs of mini-batch updates on each batch of collected data, which greatly improves sample efficiency.

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{2.112}$$

$$L^{\text{PPO}}(\theta, w) = \hat{\mathbb{E}}_t\left[L_t^{\text{CLIP}}(\theta) - c_1\left(V_w(s_t) - \hat{G}_t\right)^2 + c_2\,\mathcal{H}[\pi_\theta(\cdot \mid s_t)]\right]$$

where $\hat{A}_t$ is the advantage estimate (usually GAE), $\varepsilon$ the clipping parameter, $\hat{G}_t$ the value target, $\mathcal{H}$ the policy entropy, and $c_1$, $c_2$ weighting coefficients (e.g. 0.5 and 0.01). The objective is maximised.

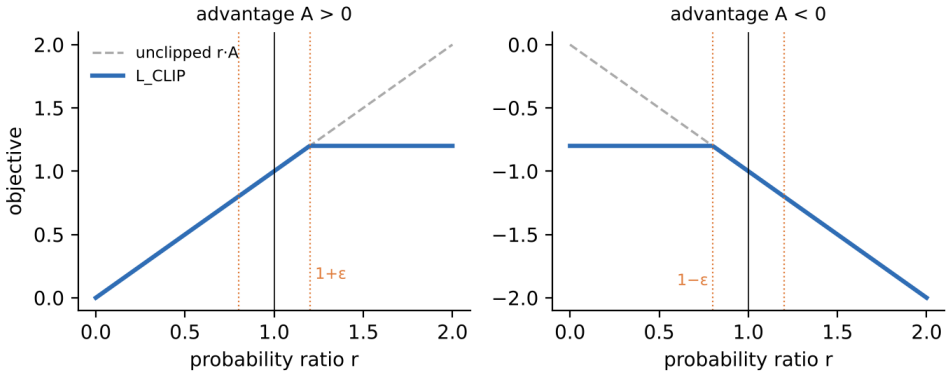



**Figure 2.54** The PPO clipped objective for a single time step as a function of the probability ratio r ($\varepsilon = 0.2$). For a positive advantage the gain is capped at $r = 1+\varepsilon$; for a negative advantage the objective is flat below $r = 1-\varepsilon$, so there is no incentive for large policy changes.

Figure 2.54 visualises the clipping, and Algorithm 2.21 gives the full algorithm. PPO's robustness to hyperparameters made it the default on-policy algorithm: OpenAI Five used it to defeat world-champion Dota 2 players, it trains simulated and real robots, and, as described in the section on large language models, it is the RL algorithm used in RLHF, where the "environment" is a prompt, the "action" is a generated response and the reward comes from a learned reward model.

**Algorithm 2.21** Proximal Policy Optimisation (PPO-Clip), actor–critic style

**Input:** Policy $\pi_\theta$; value function $V_w$; N parallel actors; horizon T; epochs K; mini-batch size M; clip $\varepsilon$; $\gamma$, $\lambda$

**Output:** Trained policy $\pi_\theta$

**for** iteration $= 1, 2, \ldots$ **do**
  **for** actor $= 1$ **to** N **do**
    run $\pi_\theta_old$ **in** the environment **for** T steps; store $(s_t, a_t, r_t, \log \pi_\theta_old(a_t|s_t))$
    compute advantages $\hat{A}_1,\ldots,\hat{A}_T$ with GAE$(\gamma, \lambda)$ using $V_w$; value targets $\hat{G}_t \leftarrow \hat{A}_t + V_w(s_t)$
  normalise advantages over the batch
  **for** epoch $= 1$ **to** K **do**
    **for each** mini-batch of size M from the N·T samples **do**
      $r_t(\theta) \leftarrow \exp(\log \pi_\theta(a_t|s_t) - \log \pi_\theta_old(a_t|s_t))$
      maximise L^PPO$(\theta, w)$ of the clipped objective by a gradient step on $\theta$ **and** w
  $\theta_old \leftarrow \theta$
**return** $\pi_\theta$

**Listing 2.35** The PPO clipped loss with value and entropy terms (PyTorch).

```python
import torch

def ppo_loss(new_logp, old_logp, adv, values, returns, entropy,
             clip_eps=0.2, c1=0.5, c2=0.01):
    """All arguments are 1-D tensors over a mini-batch of time steps."""
    adv = (adv - adv.mean()) / (adv.std() + 1e-8)            # advantage normalisation
    ratio = torch.exp(new_logp - old_logp)                    # r_t(theta)
    surr1 = ratio * adv
    surr2 = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps) * adv
    policy_loss = -torch.min(surr1, surr2).mean()             # maximise L_CLIP
    value_loss = (values - returns).pow(2).mean()
    return policy_loss + c1 * value_loss - c2 * entropy.mean()

# In practice: libraries such as Stable-Baselines3 provide tested implementations, e.g.
# from stable_baselines3 import PPO
# model = PPO("MlpPolicy", "CartPole-v1", verbose=0).learn(total_timesteps=100_000)
```

Listing 2.35 shows the loss in a few lines; in practice, well-tested libraries are strongly recommended because small implementation details—advantage normalisation, observation scaling, learning-rate annealing, value clipping—have large effects on results (Engstrom et al. 2020). Table 2.40 compares the main deep RL algorithm families.

## 2.41.5 Landmarks, Challenges and Outlook

Deep RL has produced some of the most celebrated results in AI. **AlphaGo** (Silver et al. 2016) combined policy and value networks with Monte Carlo tree search to defeat Lee Sedol at Go; **AlphaZero** (Silver et al. 2018) learned chess, shogi and Go from

**Table 2.40** Families of deep reinforcement learning algorithms.

| Algorithm | Type | Policy | Action space | Sample efficiency | Notes |
|---|---|---|---|---|---|
| DQN (Chapter 1) | Value-based, off-policy | Implicit (ε-greedy) | Discrete | Medium | Replay buffer, target network |
| REINFORCE | Policy gradient, on-policy | Stochastic | Any | Low | Unbiased, high variance |
| A2C / A3C | Actor–critic, on-policy | Stochastic | Any | Low–medium | Parallel actors |
| TRPO | Actor–critic, on-policy | Stochastic | Any | Medium | KL trust region; second-order |
| PPO | Actor–critic, on-policy | Stochastic | Any | Medium | Clipped objective; robust default; RLHF |
| DDPG / TD3 | Actor–critic, off-policy | Deterministic | Continuous | High | Replay buffer; twin critics (TD3) |
| SAC | Actor–critic, off-policy | Stochastic (max-entropy) | Continuous | High | Entropy bonus; stable |

self-play alone; and **MuZero** (Schrittwieser et al. 2020) learned its own model of the environment. Deep RL has controlled plasma in a tokamak fusion reactor (Degrave et al. 2022) and, through RLHF and related methods, shapes the behaviour of modern language models, including those trained to reason with RL from verifiable rewards such as passing unit tests or reaching correct mathematical answers.

The field still faces serious challenges: **sample inefficiency** (millions of environment steps are common), sensitivity to hyperparameters and random seeds, difficult **exploration** in sparse-reward problems, the **sim-to-real gap** when policies trained in simulation are deployed on physical robots, and **reward hacking**, where an agent exploits flaws in a misspecified reward. Offline RL, model-based RL and learning from human preferences are active responses to these problems.

**K Key Point | Deep Reinforcement Learning in Brief**

- Policy-gradient methods optimise $\pi_\theta$ directly using $\nabla J = E[\nabla \log \pi \cdot \text{advantage}]$.
- Baselines and critics reduce variance; GAE balances bias and variance.
- PPO's clipped surrogate objective keeps updates conservative and is the default on-policy algorithm, including for RLHF.
- Value-based DQN (Chapter 1) and off-policy actor–critics (SAC, TD3) are more sample-efficient alternatives for suitable problems.

**T Practice Tip | Practical Advice for Deep RL**

- Start with a proven implementation (e.g. Stable-Baselines3, CleanRL) and a benchmark environment before your own problem.

- Normalise observations and advantages; tune the learning rate and number of PPO epochs first.
- Report results over several random seeds with confidence intervals—single runs are unreliable (Henderson et al. 2018).
- Inspect agent behaviour visually: reward curves alone can hide reward hacking.

# 2.42 Graph Neural Networks (GNNs)

Many important data sets are neither grids of pixels nor ordered sequences of tokens. Molecules are atoms joined by chemical bonds, social networks are people joined by friendships, road systems are junctions joined by streets, and knowledge bases are entities joined by typed relations. In each case the natural representation is a **graph**, and the information we care about lives as much in the connections as in the individual nodes. The convolutional networks and transformers discussed earlier in this chapter assume a fixed neighbourhood structure (a 3×3 pixel window, or a position in a sequence); a graph has no such regularity. Each node may have a different number of neighbours, and there is no canonical ordering of those neighbours. **Graph neural networks (GNNs)** are deep architectures designed to learn from such irregular, relational data while respecting its symmetries.

**Definition 2.6 | Graph and Graph Neural Network**

A **graph** $G = (V, E)$ consists of a set of nodes $V$ ($|V| = n$) and a set of edges $E \subseteq V \times V$. It is usually stored as an adjacency matrix $A \in \{0,1\}^{n \times n}$ together with a node-feature matrix $X \in \mathbb{R}^{n \times d}$; edges may also carry feature vectors.

A **graph neural network** is a parameterised function that maps a graph to node-level, edge-level or graph-level representations, and whose output is unchanged (invariant) or changes consistently (equivariant) when the nodes are relabelled.

## 2.42.1 Message Passing: The Unifying Framework

Almost every modern GNN can be written in the **message-passing** form introduced by Gilmer et al. (2017). Each node $v$ holds a hidden state $h_v^{(l)}$ at layer $l$, initialised with its input features $x_v$. In each layer, every node collects *messages* from its neighbours, *aggregates* them with a permutation-invariant function (sum, mean or max), and *updates* its own state. Equation (2.113) states the general rule, and Figure 2.55 illustrates one round on a small graph.

$$\mathbf{m}_v^{(l)} = \bigoplus_{u \in \mathcal{N}(v)} \phi^{(l)}\left(\mathbf{h}_v^{(l)}, \mathbf{h}_u^{(l)}, \mathbf{e}_{uv}\right)$$
$$\mathbf{h}_v^{(l+1)} = \psi^{(l)}\left(\mathbf{h}_v^{(l)}, \mathbf{m}_v^{(l)}\right) \tag{2.113}$$

where $\mathcal{N}(v)$ is the set of neighbours of v, $e_{uv}$ is the (optional) edge feature, $\varphi$ is a learnable message function, $\oplus$ is a permutation-invariant aggregator such as sum or mean, and $\psi$ is a learnable update function (often an MLP or GRU cell).

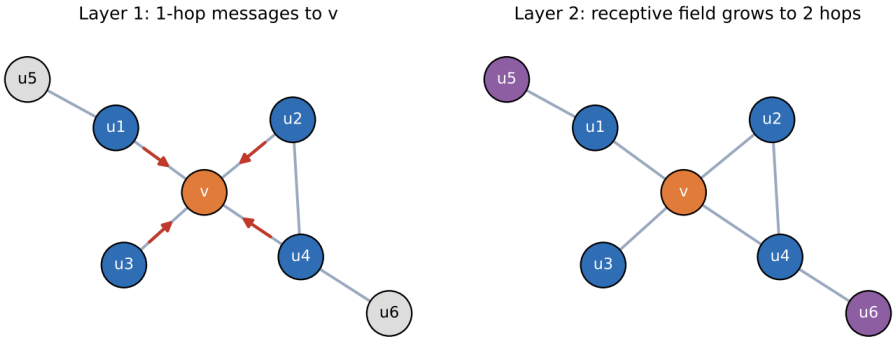



**Figure 2.55** One round of message passing: the target node v (orange) receives messages from its neighbours (blue), aggregates them and updates its state; after k rounds each node has information from its k-hop neighbourhood.

Two properties follow immediately from this design. First, because the aggregator $\oplus$ ignores the order of its arguments, the network is **permutation equivariant** at the node level: relabelling the nodes simply relabels the outputs. Second, the same $\varphi$ and $\psi$ are shared by every node, so the number of parameters is independent of graph size, exactly as weight sharing makes a CNN independent of image size. Stacking k layers gives every node a receptive field equal to its k-hop neighbourhood.

## 2.42.2 Graph Convolutional Networks (GCN)

The **graph convolutional network** of Kipf and Welling (2017) is the simplest widely used instance of message passing. It derives from a first-order approximation of spectral graph convolution, but its final layer-wise propagation rule, given in Equation (2.114), is easy to read as a normalised neighbourhood average followed by a linear map and a non-linearity.

$$\tilde{A} = A + I_n, \qquad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$$
$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}\,H^{(l)}\,W^{(l)}\right) \tag{2.114}$$

where A is the adjacency matrix, adding the identity $I_n$ inserts self-loops so that a node keeps its own features, $\tilde{D}$ is the degree matrix of $\tilde{A}$, $H^{(l)} \in \mathbb{R}^{n \times d_l}$ stacks the node states ($H^{(0)} = X$), $W^{(l)}$ is a learnable weight matrix and $\sigma$ is a non-linearity such as ReLU.

The symmetric normalisation $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ weights the message from $u$ to $v$ by $1/\sqrt{d_u d_v}$, which prevents high-degree nodes from dominating and keeps the scale of the activations stable across layers. Computationally, one GCN layer is a sparse–dense matrix product and costs $O(|E| \cdot d)$, so GCNs scale to graphs with millions of edges.

**Worked Example 2.20 | One GCN Layer on a Path Graph**

Consider the path graph $1 - 2 - 3$ with scalar features $x = (1, 2, 3)^{\top}$, weight $W = 1$ and identity activation.

- With self-loops, the degrees are $\tilde{d} = (2, 3, 2)$.
- Node 1: $h_1 = x_1/\sqrt{2 \cdot 2} + x_2/\sqrt{2 \cdot 3} = 1/2 + 2/2.449 = 0.500 + 0.816 = 1.316$.
- Node 2: $h_2 = x_1/\sqrt{3 \cdot 2} + x_2/\sqrt{3 \cdot 3} + x_3/\sqrt{3 \cdot 2} = 0.408 + 0.667 + 1.225 = 2.300$.
- Node 3: $h_3 = x_2/\sqrt{2 \cdot 3} + x_3/\sqrt{2 \cdot 2} = 0.816 + 1.500 = 2.316$.

Each node's new value is a degree-normalised blend of itself and its neighbours, a smoothing operation on the graph.

## 2.42.3 Graph Attention Networks (GAT) and GraphSAGE

A limitation of the GCN is that the weight given to each neighbour is fixed by the graph's degrees. The **graph attention network** (Veličković et al., 2018) instead *learns* how much each neighbour should contribute, using an attention mechanism similar to the one described in the earlier section on attention. The attention coefficients and update are given in Equation (2.115); in practice several attention heads are computed in parallel and their outputs concatenated, just as in multi-head self-attention.

$$e_{vu} = \mathrm{LeakyReLU}\big(\mathbf{a}^{\top}\,[W\mathbf{h}_v \,\|\, W\mathbf{h}_u]\big)$$

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in \mathcal{N}(v)\cup\{v\}} \exp(e_{vk})}, \qquad \mathbf{h}'_v = \sigma\!\left(\sum_{u \in \mathcal{N}(v)\cup\{v\}} \alpha_{vu} W \mathbf{h}_u\right) \tag{2.115}$$

where $W$ is a shared linear projection, $a$ is a learnable attention vector, $\|$ denotes concatenation, and $\alpha_{vu}$ is the normalised importance of neighbour $u$ to node $v$.

**GraphSAGE** (Hamilton, Ying and Leskovec, 2017) addresses scalability and the *inductive* setting, in which the model must produce embeddings for nodes never seen during training (for example, new users joining a platform). Rather than using the full neighbourhood, GraphSAGE *samples* a fixed number of neighbours per layer and learns an

aggregator, as set out in Algorithm 2.22. Because the learned function depends only on local features, it generalises to unseen nodes and even entirely new graphs.

**Algorithm 2.22** GraphSAGE Forward Pass with Neighbour Sampling

**Input:** Graph $G = (V, E)$; features $\{x_v\}$; depth $K$; sample sizes $S_1 \dots S_K$; weights $W^{(1)} \dots W^{(K)}$; aggregator AGG

**Output:** Node embeddings $z_v$ for all $v$ in the target batch $B$

$h_v^{(0)} \leftarrow x_v$ **for** all $v$
**for** $k = 1$ **to** $K$ **do**
  **for each** node $v$ **in** the (sampled) computation set **do**
    $N_s(v) \leftarrow$ uniformly sample $S_k$ neighbours of $v$
    $m \leftarrow \mathrm{AGG}(\{ h_u^{(k-1)} : u \in N_s(v) \})$ // mean, max-pool **or** LSTM
    $h_v^{(k)} \leftarrow \sigma( W^{(k)} \cdot [ h_v^{(k-1)} \,\|\, m ] )$
    $h_v^{(k)} \leftarrow h_v^{(k)} / \|h_v^{(k)}\|_2$
**return** $z_v \leftarrow h_v^{(K)}$ **for** $v \in B$

## 2.42.4 Tasks, Readout and Practical Issues

GNN tasks fall into three families, summarised in Table 2.41. Node-level tasks use the final node states directly; edge-level tasks combine the states of the two endpoints (for example by a dot product); graph-level tasks need a **readout** (global pooling) that maps the set of node states to a single vector, typically by summation, averaging or attention-weighted pooling.

**Table 2.41** Typical tasks solved with graph neural networks.

| Level | Task | Example application | Output head |
|---|---|---|---|
| Node | Node classification | Classifying papers in a citation network (Cora) | Softmax on $h_v$ |
| Node | Node regression | Traffic speed at each road sensor | Linear layer on $h_v$ |
| Edge | Link prediction | Friend or product recommendation | $\sigma(h_u^{\mathsf{T}} h_v)$ |
| Edge | Edge classification | Fraudulent transaction detection | MLP on $[h_u \,\|\, h_v]$ |
| Graph | Graph classification | Molecular toxicity prediction | Readout + MLP |
| Graph | Graph regression | Predicting solubility or energy of a molecule | Readout + linear |

Deep GNNs suffer from **over-smoothing**: repeated neighbourhood averaging drives all node states towards the same vector, so performance often peaks at two to four layers. Remedies include residual connections, normalisation such as PairNorm and jumping-knowledge connections that combine states from all layers. A related problem, **over-squashing**, arises when information from an exponentially growing neighbourhood must be compressed into a fixed-size vector. The expressive power

of message-passing GNNs is also bounded by the Weisfeiler–Lehman graph isomorphism test (Xu et al., 2019); the Graph Isomorphism Network (GIN) uses sum aggregation followed by an MLP to reach that bound.

**C Case Study 2.10 | GNNs in Practice**

Pinterest's PinSage, a GraphSAGE variant trained on a graph of billions of pins and boards, powers visual recommendations. DeepMind used GNNs to improve estimated arrival times in Google Maps by modelling road segments as a graph, and GNN-based models are standard in molecular property prediction and drug discovery.

Libraries such as PyTorch Geometric and the Deep Graph Library (DGL) supply efficient sparse kernels, standard benchmark data sets and implementations of GCN, GAT, GraphSAGE and GIN.

## 2.43 Self-Supervised and Contrastive Learning

Supervised deep learning is hungry for labels, and labels are expensive: annotating medical images requires specialists, and transcribing speech requires hours of human effort. Unlabelled data, by contrast, is abundant. **Self-supervised learning** (**SSL**) creates a supervisory signal from the data itself by posing a *pretext task* whose answer is known without human labelling. Language models trained to predict the next or a masked token, discussed in the sections on BERT and GPT, are the most successful example. This section focuses on SSL for vision and other continuous signals, where two families dominate: **contrastive** (and related joint-embedding) methods and **masked reconstruction** methods.

**D Definition 2.7 | Self-Supervised Learning**

A learning paradigm in which a model is trained on unlabelled data using targets derived automatically from the input (for example, a hidden part of the input, or the identity of an augmented view). The learned representation is then reused for downstream tasks via linear probing or fine-tuning.

### 2.43.1 Contrastive Learning and the InfoNCE Loss

The core idea of contrastive learning is simple: two different augmented *views* of the same image should map to nearby points in embedding space (a **positive pair**), whereas views of different images should map far apart (**negative pairs**). The model thereby learns which variations are irrelevant (crop, colour, blur) and which are semantically meaningful. This idea is formalised by the **InfoNCE** loss (van den Oord

et al., 2018), which is a softmax cross-entropy over one positive and many negatives, as shown in Equation (2.116).

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp\left(\text{sim}(\mathbf{z}, \mathbf{z}^{+})/\tau\right)}{\exp\left(\text{sim}(\mathbf{z}, \mathbf{z}^{+})/\tau\right) + \sum_{k=1}^{K} \exp\left(\text{sim}(\mathbf{z}, \mathbf{z}_{k}^{-})/\tau\right)} \tag{2.116}$$

where z is the anchor embedding, $z^{+}$ the embedding of its positive view, $z^{-}{}_{k}$ the K negative embeddings, $\text{sim}(a, b) = a^{T}b/(\|a\|\|b\|)$ is cosine similarity, and $\tau > 0$ is a temperature that controls how sharply the loss concentrates on hard negatives.

Minimising InfoNCE maximises a lower bound on the mutual information between the two views. The temperature $\tau$ is an important hyperparameter: small values ($\approx 0.1$) make the loss focus on the hardest negatives, while large values treat all negatives more uniformly.

## 2.43.2 SimCLR

**SimCLR** (Chen et al., 2020) showed that a remarkably simple recipe could rival supervised pre-training on ImageNet. Its pipeline, shown in Figure 2.56, has four components: a stochastic augmentation module (random crop with resize, colour distortion, Gaussian blur), a base encoder $f(\cdot)$ such as ResNet-50, a small MLP **projection head** $g(\cdot)$, and the contrastive loss. In a mini-batch of N images, each image yields two views, giving 2N examples; for each positive pair, the other $2(N-1)$ views act as negatives. The resulting **NT-Xent** (normalised temperature-scaled cross-entropy) loss for a positive pair $(i, j)$ is given in Equation (2.117).

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell_{2k-1,\,2k} + \ell_{2k,\,2k-1}] \tag{2.117}$$

where $\mathbb{1}[k \neq i]$ excludes the anchor itself from the denominator and the loss is averaged over both orderings of every positive pair.

Two empirical findings from SimCLR are worth remembering. Strong augmentation, especially the composition of random cropping and colour distortion, is essential; without colour distortion the network can solve the task using colour histograms alone. Second, the representation *before* the projection head (h) transfers better than the projection z, because g learns to discard information that is useful downstream but irrelevant to the contrastive task. SimCLR's weakness is its need for very large batches (4,096 or more) to supply enough negatives.

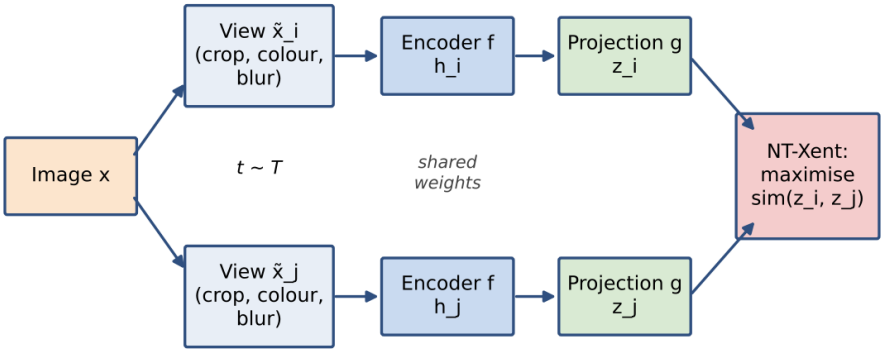



**Figure 2.56** The SimCLR framework: two augmented views of one image pass through a shared encoder f and projection head g; the NT-Xent loss pulls their projections together and pushes other images in the batch apart.

### 2.43.3 MoCo, BYOL and Masked Autoencoders

**Momentum Contrast (MoCo)** (He et al., 2020) decouples the number of negatives from the batch size. It keeps a queue of embeddings from recent batches as a dictionary of negatives, and encodes keys with a *momentum encoder* whose parameters $\theta_k$ are an exponential moving average of the query encoder's parameters θ_q, as in Equation (2.118). The slowly changing key encoder keeps the queued keys consistent with each other.

$$\theta_k \leftarrow m\,\theta_k + (1-m)\,\theta_q, \qquad m \in [0.99, 0.999] \tag{2.118}$$

where m is the momentum coefficient; only θ_q is updated by back-propagation.

**BYOL** (Bootstrap Your Own Latent; Grill et al., 2020) surprised the community by dispensing with negatives altogether. An *online* network with an extra predictor head is trained to predict the projection produced by a momentum *target* network for another view of the same image. Collapse to a constant output is avoided in practice by the asymmetry of the predictor and the stop-gradient on the target branch. SimSiam and DINO follow similar ideas, and DINO's self-distillation produces vision transformer features whose attention maps segment objects without any labels.

**Masked autoencoders (MAE)** (He et al., 2022) bring the masked-language-modelling idea of BERT to images. An image is split into patches, a large fraction (typically 75%) is randomly masked, a ViT encoder processes only the visible patches, and a lightweight decoder reconstructs the pixels of the masked patches using a mean-squared error computed on masked patches only. Because the encoder sees only a quarter of the

patches, pre-training is fast, and the high masking ratio forces the model to learn holistic, semantic structure rather than interpolating local texture. Table 2.42 compares the main methods, and Listing 2.36 implements the NT-Xent loss in a few lines of PyTorch.

**Table 2.42** Comparison of representative self-supervised methods for vision.

| Method | Year | Key idea | Needs negatives? | Main limitation |
|---|---|---|---|---|
| SimCLR | 2020 | Two views, NT-Xent, projection head | Yes (in-batch) | Very large batches |
| MoCo v2/v3 | 2020–21 | Queue of keys + momentum encoder | Yes (queue) | Extra memory and encoder |
| BYOL | 2020 | Predict target network's projection | No | Collapse avoidance is subtle |
| DINO | 2021 | Self-distillation with centring/sharpening | No | Sensitive to temperature settings |
| MAE | 2022 | Reconstruct 75% masked patches | No | Weaker linear-probe accuracy; best when fine-tuned |

**Listing 2.36** A compact PyTorch implementation of the NT-Xent loss used by SimCLR.

```python
import torch
import torch.nn.functional as F

def nt_xent(z1, z2, tau=0.5):
    """z1, z2: (N, d) projections of two views of the same N images."""
    N = z1.size(0)
    z = F.normalize(torch.cat([z1, z2], dim=0), dim=1)   # (2N, d), unit length
    sim = z @ z.t() / tau                                  # cosine similarities
    sim.fill_diagonal_(float("-inf"))                      # exclude self-pairs
    # positive of i is i+N (and vice versa)
    targets = torch.cat([torch.arange(N, 2 * N), torch.arange(0, N)]).to(z.device)
    return F.cross_entropy(sim, targets)

z1, z2 = torch.randn(8, 128), torch.randn(8, 128)
print(nt_xent(z1, z2).item())
```

**Key Point | Why Self-Supervision Matters**

- SSL turns unlimited unlabelled data into a training signal, and its representations often match or exceed supervised pre-training when transferred.
- Contrastive methods learn invariances specified by the augmentations; choosing augmentations is therefore a way of injecting domain knowledge.
- Masked modelling (BERT, MAE, wav2vec 2.0) has become the dominant pre-training recipe across text, images and speech.

# 2.44 Transfer Learning and Fine-Tuning

**Transfer learning** is the practice of reusing knowledge learned on a *source* task to improve learning on a *target* task. In deep learning this almost always means initialising a network with weights pre-trained on a large data set (ImageNet for vision, web text for language, thousands of hours of speech for audio) and adapting it to a smaller, task-specific data set. Transfer learning is the main reason deep learning is practical for organisations without web-scale data: a few hundred labelled examples can be enough to reach strong accuracy.

> **Definition 2.8 | Transfer Learning**
>
> Given a source domain $\mathcal{D}_S$ with task $\mathcal{T}_S$ and a target domain $\mathcal{D}_T$ with task $\mathcal{T}_T$, transfer learning aims to improve the target predictive function $f_T$ using knowledge gained from $\mathcal{D}_S$ and $\mathcal{T}_S$, where $\mathcal{D}_S \neq \mathcal{D}_T$ or $\mathcal{T}_S \neq \mathcal{T}_T$ (Pan and Yang, 2010).

## 2.44.1 Why Pre-Trained Features Transfer

Visualisation studies of CNNs (Zeiler and Fergus, 2014; Yosinski et al., 2014) showed that early layers learn generic detectors of edges, colour blobs and textures that are useful for almost any natural-image task, whereas later layers become increasingly specific to the source classes. The same gradient from general to specific is seen in transformers. This suggests a spectrum of strategies, from reusing the network as a frozen feature extractor to updating every weight. The right choice depends mainly on two factors: the size of the target data set and the similarity between source and target domains, as summarised in Table 2.43.

**Table 2.43** Choosing a transfer strategy by target-data size and domain similarity.

| Target data | Domain similarity | Recommended strategy | Risk |
|---|---|---|---|
| Small | High | Freeze backbone; train new head (feature extraction) | Under-fitting if head too simple |
| Small | Low | Freeze early layers; fine-tune upper layers with strong regularisation | Over-fitting |
| Large | High | Fine-tune whole network with a small learning rate | Compute cost |
| Large | Low | Fine-tune whole network, or train from scratch if domains are very different | Negative transfer |

### 2.44.2 Feature Extraction versus Fine-Tuning

In **feature extraction**, the pre-trained backbone is frozen and used as a fixed function that maps each input to a feature vector; only a new classifier head is trained. This is fast, needs little memory (features can even be pre-computed once) and is very resistant to over-fitting. In **fine-tuning**, some or all backbone weights are unfrozen and updated together with the head. Fine-tuning usually achieves higher accuracy but requires care, because large gradients from a randomly initialised head can destroy useful pre-trained features, a phenomenon sometimes called *catastrophic forgetting*. Algorithm 2.23 gives a robust two-stage recipe that combines both approaches.

**Algorithm 2.23** Two-Stage Fine-Tuning with Gradual Unfreezing

**Input:** Pre-trained backbone f_θ; target data D; new head g_φ; learning rates $\eta_{\text{head}} > \eta_{\text{body}}$; epochs $E_1$, $E_2$
**Output:** Adapted model g_φ ∘ f_θ

replace the source classifier with a new head g_φ (random init)
freeze all parameters θ of the backbone
**for** epoch = 1 **to** $E_1$ **do** // stage 1: train head only
    update φ on D with learning rate $\eta_{\text{head}}$
**for each** block b of f_θ, from top **to** bottom **do** // stage 2: gradual unfreezing
    unfreeze block b
    **for** epoch = 1 **to** $E_2$ **do**
        update φ with $\eta_{\text{head}}$ **and** unfrozen θ with $\eta_{\text{body}}$ (discriminative rates)
        keep BatchNorm statistics frozen **if** the batch size is small
    stop early **if** validation loss stops improving
**return** g_φ ∘ f_θ

Three practical refinements are common. **Discriminative learning rates** assign smaller rates to lower layers (for example, dividing by 2.6 per layer group, as in ULMFiT by Howard and Ruder, 2018). **Warm-up** followed by cosine decay of the learning rate, discussed in the section on learning-rate schedules, stabilises the first steps. Finally, inputs must be preprocessed exactly as during pre-training (same resolution, normalisation mean and standard deviation), otherwise the pre-trained features are evaluated out of distribution. Listing 2.37 shows the two-stage recipe in Keras.

**Listing 2.37** Feature extraction followed by fine-tuning of an ImageNet-pre-trained EfficientNet in Keras.

```
import tensorflow as tf
from tensorflow import keras

base = keras.applications.EfficientNetB0(include_top=False, weights="imagenet",
                                         input_shape=(224, 224, 3), pooling="avg")
base.trainable = False                                  # stage 1: frozen backbone
```

```
inputs = keras.Input((224, 224, 3))
x = base(inputs, training=False)                          # keep BN in inference mode
x = keras.layers.Dropout(0.3)(x)
outputs = keras.layers.Dense(5, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(keras.optimizers.Adam(1e-3), "sparse_categorical_crossentropy",
    ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=5)

base.trainable = True                                     # stage 2: fine-tune top
    layers
for layer in base.layers[:-30]:
    layer.trainable = False
model.compile(keras.optimizers.Adam(1e-5), "sparse_categorical_crossentropy",
    ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10,
          callbacks=[keras.callbacks.EarlyStopping(patience=3,
              restore_best_weights=True)])
```

### 2.44.3 Parameter-Efficient Fine-Tuning: Adapters and LoRA

For models with billions of parameters, full fine-tuning is costly: every task requires storing a complete copy of the weights, and the optimiser state (for Adam, two extra values per parameter) multiplies memory needs. **Parameter-efficient fine-tuning (PEFT)** methods update only a tiny fraction of parameters while keeping the pre-trained weights frozen. **Adapters** (Houlsby et al., 2019) insert small bottleneck MLPs, of the form $\mathrm{h} \leftarrow \mathrm{h} + \mathrm{W}_{\text{up}}\,\sigma(\mathrm{W}_{\text{down}}\,\mathrm{h})$, after the attention and feed-forward sub-layers of each transformer block. **Prefix tuning** and **prompt tuning** prepend learnable vectors to the input or to the keys and values of each layer.

**LoRA** (Low-Rank Adaptation; Hu et al., 2022) is now the most widely used PEFT method. It rests on the hypothesis that the weight change needed to adapt a model has low intrinsic rank. Each adapted weight matrix $\mathrm{W}_0 \in \mathbb{R}^{\mathrm{dxk}}$ is frozen, and the update is parameterised as the product of two thin matrices, as in Equation (2.119) and Figure 2.57.

$$\begin{aligned} \mathbf{h} &= W_0\,\mathbf{x} + \Delta W\,\mathbf{x} = W_0\,\mathbf{x} + \tfrac{\alpha}{r}\,BA\,\mathbf{x} \\ B &\in \mathbb{R}^{d\times r}, \quad A \in \mathbb{R}^{r\times k}, \quad r \ll \min(d,k) \end{aligned} \tag{2.119}$$

where $\mathrm{W}_0$ is the frozen pre-trained weight, A is initialised randomly and B with zeros (so $\Delta\mathrm{W} = 0$ at the start), r is the rank and $\alpha$ is a scaling constant. Only A and B are trained.

**E Worked Example 2.21 | Parameter Savings with LoRA**

Take a single attention projection with $\mathrm{d} = \mathrm{k} = 4096$ (as in a 7-billion-parameter LLM).

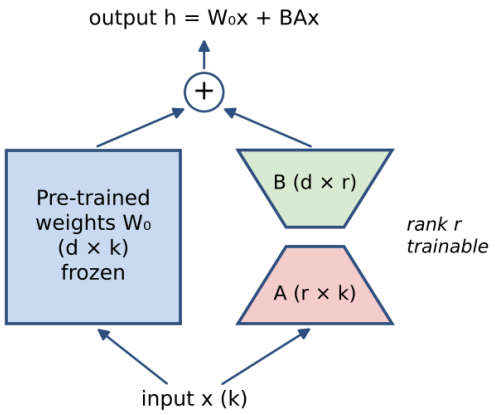



**Figure 2.57** LoRA: the frozen weight $W_0$ is augmented by a trainable low-rank path BA; after training, BA can be merged into $W_0$ so inference cost is unchanged.

- Full fine-tuning updates $d \cdot k = 4096 \times 4096 = 16{,}777{,}216$ parameters.
- LoRA with $r = 8$ trains $r(d + k) = 8 \times 8192 = 65{,}536$ parameters, i.e. 0.39% of the original.

Applied to the query and value projections of all 32 layers, LoRA trains $2 \times 32 \times 65{,}536 \approx 4.2$ million parameters instead of about 6.7 billion, and each task's adapter can be stored as a file of a few megabytes.

Combined with 4-bit quantisation of the frozen base model (**QLoRA**; Dettmers et al., 2023), LoRA makes it possible to fine-tune models with tens of billions of parameters on a single GPU. The Hugging Face `peft` library provides LoRA, adapters and prompt tuning behind a uniform interface.

**T Practice Tip | Practical Fine-Tuning Advice**

- Always establish a frozen-backbone baseline first; it is cheap and tells you how much fine-tuning gains.
- Use a learning rate 10–100 times smaller for pre-trained layers than for a new head.
- Watch for negative transfer: if fine-tuning is worse than training from scratch, the domains may be too different, or the preprocessing may not match.

## 2.45 Foundation Models and Pretrained Models

The term **foundation model** was introduced by Bommasani et al. (2021) at Stanford to describe models trained on broad data at scale, usually with self-supervision, that

can be adapted to a wide range of downstream tasks. BERT, GPT-3 and its successors, CLIP, DALL·E, Whisper and the Segment Anything Model are all foundation models. The name emphasises both their central role, as a base on which many applications are built, and the fact that any defects in the foundation propagate to everything built on top of it.

**Definition 2.9 | Foundation Model**

A large model pre-trained on broad, typically unlabelled data using self-supervised objectives, whose learned representations and capabilities can be adapted (by prompting, fine-tuning or parameter-efficient methods) to many downstream tasks, often including tasks not anticipated during training.

## 2.45.1 From Task-Specific Models to Pre-Train-then-Adapt

Before about 2018, most deep learning systems were trained from scratch or from ImageNet weights for each task. The shift to foundation models was driven by three converging factors: the transformer architecture, which scales efficiently on parallel hardware; self-supervised objectives (covered in the previous sections and in the sections on BERT, GPT and large language models), which turn raw text, images and audio into training signal; and the empirical **scaling laws** of Kaplan et al. (2020) and Hoffmann et al. (2022), which showed that test loss falls predictably as a power law in model size, data size and compute. Equation (2.120) gives the form fitted by Hoffmann et al. for language models.

$$L(N, D) \approx E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \tag{2.120}$$

where N is the number of parameters, D the number of training tokens, E the irreducible loss of the data, and A, B, $\alpha \approx 0.34$, $\beta \approx 0.28$ are fitted constants. Minimising L under a fixed compute budget $C \approx 6ND$ suggests scaling N and D in roughly equal proportion (about 20 tokens per parameter).

Figure 2.58 visualises this law: each curve corresponds to a model size, and larger models reach lower loss but need more data to do so. The practical consequence is that capabilities can be bought, to a degree, with compute, which has concentrated foundation-model development in well-resourced organisations.

## 2.45.2 Families of Pretrained Models

Pretrained models are distributed through model hubs (Hugging Face Hub, TensorFlow Hub, PyTorch Hub, timm) and can be loaded in a few lines of code, as Listing 2.38 shows. Table 2.44 lists widely used families across modalities; most are described in more detail elsewhere in this chapter.

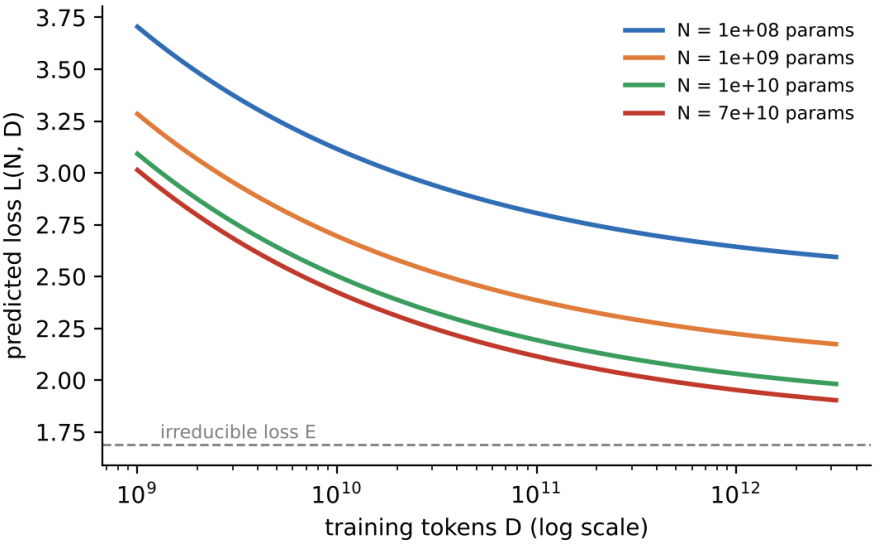



**Figure 2.58** Illustrative scaling-law curves computed from Equation (2.120): loss decreases as a power law in training tokens, and larger models achieve lower asymptotic loss.

**Table 2.44** Representative pretrained and foundation models by modality.

| Model family | Modality | Pre-training objective | Typical use |
|---|---|---|---|
| ResNet, EfficientNet, ConvNeXt | Images | Supervised ImageNet classification | Backbones for vision tasks |
| ViT, DINOv2, MAE | Images | Supervised or self-supervised | General visual features |
| BERT, RoBERTa | Text | Masked language modelling | Classification, extraction |
| GPT, LLaMA, Claude, Gemini | Text (+ images) | Next-token prediction + alignment | Generation, assistants |
| T5, BART | Text | Span corruption / denoising | Summarisation, translation |
| CLIP, SigLIP | Image + text | Contrastive image–text matching | Zero-shot classification, retrieval |
| wav2vec 2.0, Whisper | Speech | Masked contrastive / weakly supervised ASR | Speech recognition |
| Segment Anything (SAM) | Images | Promptable segmentation on 1B masks | Interactive segmentation |
| Stable Diffusion | Text → image | Latent denoising diffusion | Image generation and editing |

**Listing 2.38** Using pretrained foundation models through Hugging Face pipelines.

```python
from transformers import pipeline

# Zero-shot text classification with an NLI-fine-tuned BART model
clf = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
print(clf("The patient reports chest pain and shortness of breath.",
          candidate_labels=["cardiology", "dermatology", "orthopaedics"]))

# Zero-shot image classification with CLIP
img_clf = pipeline("zero-shot-image-classification",
    model="openai/clip-vit-base-patch32")
print(img_clf("cat.jpg", candidate_labels=["a photo of a cat", "a photo of a dog"]))

# Speech recognition with Whisper
asr = pipeline("automatic-speech-recognition", model="openai/whisper-small")
print(asr("lecture.wav")["text"])
```

### 2.45.3 Adaptation, Emergence and Homogenisation

Foundation models are adapted in several ways, in increasing order of cost: **prompting** and **in-context learning** (supplying instructions and examples in the input, with no weight updates); **retrieval augmentation** (grounding outputs in external documents); **parameter-efficient fine-tuning** such as LoRA, discussed in the previous section; and **full fine-tuning**. Large language models are additionally aligned with human preferences through instruction tuning and reinforcement learning from human feedback, as discussed in the section on large language models.

Bommasani et al. highlight two characteristic phenomena. **Emergence** refers to capabilities, such as in-context learning, that were not explicitly trained for and appear only at sufficient scale (although some apparent emergence reflects the choice of discontinuous evaluation metrics). **Homogenisation** refers to the convergence of many applications onto a few base models; this brings efficiency, since improvements propagate widely, but also creates single points of failure, since biases, security vulnerabilities or factual errors in a foundation model are inherited by all downstream systems.

> **Note | Limitations of Foundation Models**
>
> - **Hallucination:** generative models can produce fluent but false statements.
> - **Opacity of training data:** the composition of web-scale corpora is often undocumented, complicating assessment of bias, copyright and test-set contamination.
> - **Cost and environmental footprint:** training a frontier model requires thousands of accelerators for months; inference at scale also consumes substantial energy.
> - **Evaluation:** broad capabilities are hard to measure, and benchmark scores can overstate real-world reliability.

**K Key Points | Foundation Models**

- Foundation models are pre-trained once at scale and adapted many times.
- Scaling laws make performance predictable from parameters, data and compute.
- The default modern workflow is: choose a suitable pretrained model, then prompt, retrieve, or fine-tune with PEFT before considering training from scratch.

# 2.46 Multimodal Deep Learning

Humans perceive the world through several senses at once: we read a caption while looking at a picture, and we follow a conversation by combining speech with lip movements and gestures. **Multimodal deep learning** builds models that process and relate information from two or more *modalities*, such as images, text, audio, video, tabular records and sensor signals. Multimodal systems can be more accurate than unimodal ones, because the modalities provide complementary evidence, and more robust, because one modality can compensate when another is noisy or missing. They also enable entirely new tasks, such as describing an image in words or generating an image from a description.

**D Definition 2.10 | Modality and Multimodal Learning**

A **modality** is a particular way in which information is encoded and perceived (for example, pixels, word tokens or audio waveforms). **Multimodal learning** studies models that represent, align, fuse, translate or jointly reason over data from multiple modalities (Baltrušaitis, Ahuja and Morency, 2019).

## 2.46.1 Core Challenges and Fusion Strategies

Baltrušaitis et al. identify five core challenges. **Representation** asks how to encode each modality and their combination. **Alignment** finds correspondences between elements, such as which words refer to which image regions. **Fusion** combines information to make a prediction. **Translation** maps one modality to another (captioning, text-to-image, speech-to-text). **Co-learning** transfers knowledge between modalities, for example using text to supervise vision. Of these, fusion is the most basic design decision, and three strategies are illustrated in Figure 2.59.

**Early fusion** is simple and lets the model learn low-level interactions, but it requires the modalities to be synchronised and of compatible granularity. **Late fusion** trains independent models and combines their outputs, which is robust to a missing modality

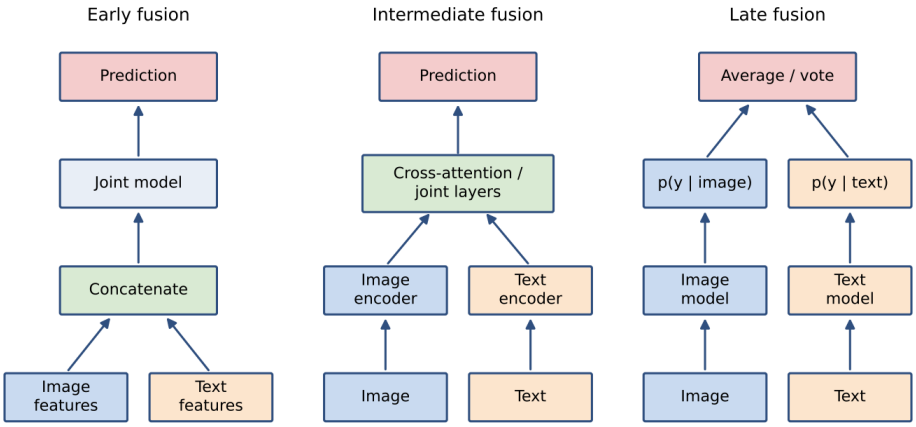



**Figure 2.59** Fusion strategies: early fusion concatenates raw or low-level features; late fusion combines independent unimodal predictions; intermediate (joint) fusion merges learned representations, for example with cross-attention.

and allows each model to be built separately, but it cannot capture fine-grained cross-modal interactions. **Intermediate fusion** merges learned representations, and in modern systems this usually takes the form of transformer **cross-attention**, where queries from one modality attend to keys and values from another. Table 2.45 summarises the trade-offs.

**Table 2.45** Comparison of multimodal fusion strategies.

| Strategy | Where modalities meet | Strengths | Weaknesses |
|---|---|---|---|
| Early | Input / low-level features | Captures low-level correlations; one model | Needs alignment; high-dimensional input |
| Intermediate | Hidden representations | Rich interactions (cross-attention) | More complex; needs paired data |
| Late | Decisions / scores | Modular; tolerant of missing modality | Misses cross-modal interactions |

## 2.46.2 CLIP: Contrastive Language–Image Pre-training

**CLIP** (Radford et al., 2021) applies the contrastive learning ideas of the previous sections across modalities. An image encoder (ResNet or ViT) and a text encoder (transformer) are trained jointly on 400 million image–text pairs collected from the web. In a batch of N pairs, the N matching pairs are positives and the $N^2 - N$ mismatched pairs are negatives. The embeddings are L2-normalised, and the $N \times N$ matrix of scaled cosine similarities is fed into a symmetric cross-entropy loss, as in Equation (2.121).

$$s_{ij} = \frac{\mathbf{u}_i^\top \mathbf{v}_j}{\tau}, \qquad \mathbf{u}_i = \frac{f_I(x_i)}{\|f_I(x_i)\|}, \quad \mathbf{v}_j = \frac{f_T(t_j)}{\|f_T(t_j)\|}$$

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2N}\sum_{i=1}^{N}\left[\log \frac{e^{s_{ii}}}{\sum_j e^{s_{ij}}} + \log \frac{e^{s_{ii}}}{\sum_j e^{s_{ji}}}\right] \tag{2.121}$$

where $f_I$ and $f_T$ are the image and text encoders (each followed by a linear projection to a shared space), and $\tau$ is a learned temperature. The first term classifies the correct caption for each image; the second classifies the correct image for each caption.

Once trained, CLIP performs **zero-shot classification**: each class name is inserted into a prompt such as "a photo of a {class}", the prompts are embedded by the text encoder, and an image is assigned to the class whose text embedding is most similar, as implemented in Listing 2.39. Without seeing any ImageNet training labels, CLIP ViT-L/14 matched the accuracy of a supervised ResNet-50 on ImageNet and proved far more robust under distribution shift. CLIP embeddings also underpin text-to-image retrieval and the text conditioning of diffusion models such as Stable Diffusion (see the section on diffusion models).

**Listing 2.39** Zero-shot image classification with CLIP via Hugging Face Transformers.

```python
import torch
from PIL import Image
from transformers import CLIPModel, CLIPProcessor

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").eval()
proc = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

labels = ["cat", "dog", "car", "x-ray of a chest"]
prompts = [f"a photo of a {c}" for c in labels]
image = Image.open("example.jpg")

inputs = proc(text=prompts, images=image, return_tensors="pt", padding=True)
with torch.no_grad():
    logits = model(**inputs).logits_per_image          # (1, num_labels) scaled similarities
probs = logits.softmax(dim=-1).squeeze()
for c, p in zip(labels, probs):
    print(f"{c:>18s}: {p:.3f}")
```

## 2.46.3 Visual Question Answering and Image Captioning

**Image captioning** translates an image into a natural-language description. Early neural captioners such as Show and Tell (Vinyals et al., 2015) paired a CNN encoder with an LSTM decoder; Show, Attend and Tell (Xu et al., 2015) added visual attention so that each generated word could focus on relevant image regions. Modern captioners use a vision transformer encoder and a transformer decoder, trained with cross-entropy on

datasets such as MS COCO and evaluated with BLEU, METEOR, CIDEr and SPICE. Generation at inference uses beam search or sampling, as described in the section on sequence modelling.

**Visual question answering (VQA)** (Antol et al., 2015) takes an image and a free-form question ("How many people are wearing helmets?") and produces an answer. VQA is a demanding test of joint reasoning because the question determines which parts of the image matter. Classical VQA models encode the question with an RNN or transformer, encode the image as region or patch features, fuse them with co-attention, and classify over the most frequent answers. A known pitfall is **language bias**: models learned to answer "yes" to most "Is there…" questions without looking at the image, which motivated the balanced VQA v2 data set.

Current **vision–language models** (VLMs) such as Flamingo (Alayrac et al., 2022), BLIP-2 (Li et al., 2023) and LLaVA (Liu et al., 2023) unify captioning, VQA and visual dialogue by connecting a pretrained image encoder to a pretrained large language model through a small learned bridge (a linear projection, a Q-Former, or gated cross-attention layers). Visual tokens are interleaved with text tokens, so any task can be posed as text generation conditioned on images.

**C Case Study 2.11 | Multimodal Models in Practice**

Assistive applications describe photographs aloud to blind and low-vision users; e-commerce search engines use CLIP-style embeddings so that a text query retrieves matching product images; and clinical decision-support research fuses medical images with electronic health records, where late or intermediate fusion of imaging and tabular data often outperforms either alone.

**N Note | Pitfalls in Multimodal Learning**

- **Modality imbalance:** a network may rely on the easier modality and under-train the other; balancing losses or modality dropout helps.
- **Missing modalities** at test time must be planned for during training.
- **Web-scale pairs are noisy** and carry social biases that propagate into zero-shot predictions.

## 2.47 Speech Recognition and Audio Processing

Audio is a one-dimensional signal sampled at high rate (16,000 samples per second is standard for speech; 44,100 for music). **Automatic speech recognition (ASR)** maps

such a waveform to a word sequence, and related tasks include speaker identification, keyword spotting, speech synthesis (text-to-speech), sound event detection and music tagging. For decades ASR relied on hidden Markov models with Gaussian mixture emission models (HMM–GMM). Deep neural networks first replaced the GMMs in hybrid DNN–HMM systems (Hinton et al., 2012), and today fully **end-to-end** neural models map audio directly to text.

## 2.47.1 From Waveform to Features: Spectrograms and MFCCs

Raw waveforms are long and their information is spread across frequency, so most systems first convert audio to a time–frequency representation. The pipeline in Figure 2.60 proceeds as follows. A **pre-emphasis** filter boosts high frequencies. The signal is cut into overlapping **frames** (typically 25 ms long with a 10 ms hop), each multiplied by a Hamming window. A **short-time Fourier transform (STFT)** gives the power spectrum of each frame. A bank of triangular **mel filters** pools the spectrum on the mel scale, which approximates human pitch perception (Equation (2.122)); taking the logarithm gives the **log-mel spectrogram**. Applying a discrete cosine transform (DCT) to each log-mel frame and keeping the first 13 coefficients yields **mel-frequency cepstral coefficients (MFCCs)**, which decorrelate the features.

$$m(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right)$$
$$c_n = \sum_{k=1}^{K} \log(E_k) \cos\left[\frac{\pi n}{K}\left(k - \tfrac{1}{2}\right)\right], \quad n = 1, \ldots, 13 \tag{2.122}$$

where f is frequency in hertz, $m(f)$ is the corresponding mel value, $E_k$ is the energy in the k-th of K mel filters and $c_n$ is the n-th MFCC.

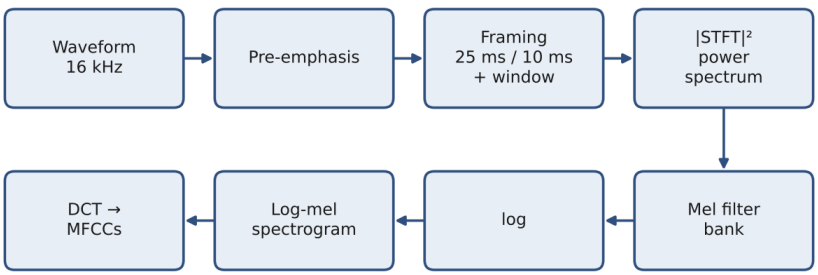



**Figure 2.60** Feature-extraction pipeline for speech: framing and windowing, STFT, mel filter bank and logarithm produce a log-mel spectrogram; a DCT further produces MFCCs.

Figure 2.61 plots the mel mapping and a synthetic log-mel spectrogram of a tone sweeping upward in frequency. Modern deep networks usually take log-mel spectrograms (80 or 128 bands) as input, since a CNN or transformer can learn its own decorrelation; MFCCs remain common for classical models and small devices. **SpecAugment** (Park et al., 2019), which masks random blocks of time steps and frequency bands, is the standard augmentation for spectrogram inputs.

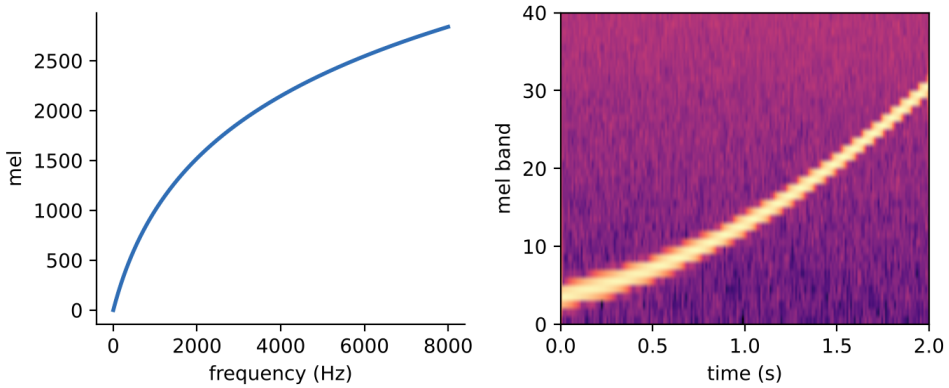



**Figure 2.61** Left: the mel scale compresses high frequencies, mirroring human hearing. Right: log-mel spectrogram of a synthetic chirp rising from 200 Hz to 4 kHz.

## 2.47.2 Connectionist Temporal Classification (CTC)

End-to-end ASR faces an alignment problem: an utterance with T = 500 acoustic frames might correspond to only U = 30 characters, and we do not know which frames belong to which character. **Connectionist temporal classification (CTC)** (Graves et al., 2006) solves this without frame-level labels. The network outputs, at each frame, a distribution over the vocabulary plus a special **blank** symbol ε. An *alignment* $\pi$ is any length-T sequence of such symbols; the collapsing function $\mathcal{B}$ removes repeated symbols and then blanks, so that, for example, $\mathcal{B}$("hh ε e ε ll ε l oo") = "hello". The probability of the label sequence y sums over all alignments that collapse to it, as in Equation (2.123).

$$p(\mathbf{y} \mid \mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} p_t(\pi_t \mid \mathbf{x}) \tag{2.123}$$

$$\mathcal{L}_{\text{CTC}} = -\log p(\mathbf{y} \mid \mathbf{x})$$

where $p_t(\pi_t \mid x)$ is the network's softmax output for symbol $\pi_t$ at frame t and $\mathcal{B}^{-1}(y)$ is the set of all alignments that collapse to y. The sum is computed efficiently in O(T·U) time with the forward–backward dynamic programme.

The blank symbol is what allows repeated characters (the double "l" in "hello" requires a blank between the two l's) and lets the network emit nothing during silences. CTC assumes outputs are conditionally independent given the input, so it is often combined with an external language model during beam-search decoding. Alternatives that remove this assumption include the **RNN-Transducer** (Graves, 2012), favoured for streaming on-device ASR, and **attention-based encoder–decoders** (Chan et al., 2016, *Listen, Attend and Spell*). Listing 2.40 shows CTC in PyTorch.

**Listing 2.40** Computing the CTC loss in PyTorch for a batch of acoustic-model outputs.

```
import torch
import torch.nn as nn

T, N, C = 120, 4, 29                       # frames, batch, classes (28 chars + blank=0)
log_probs = torch.randn(T, N, C, requires_grad=True).log_softmax(dim=2)
targets = torch.randint(1, C, (N, 20), dtype=torch.long)   # label indices (no blanks)
input_lengths = torch.full((N,), T, dtype=torch.long)
target_lengths = torch.tensor([20, 18, 15, 12])

ctc = nn.CTCLoss(blank=0, zero_infinity=True)
loss = ctc(log_probs, targets, input_lengths, target_lengths)
loss.backward()
print(f"CTC loss: {loss.item():.3f}")

def greedy_decode(lp, blank=0):          # best-path decoding for one utterance (T, C)
    best = lp.argmax(dim=-1).tolist()
    out, prev = [], None
    for s in best:
        if s != prev and s != blank:
            out.append(s)
        prev = s
    return out
```

## 2.47.3 Self-Supervised Speech: wav2vec 2.0 and Whisper

**wav2vec 2.0** (Baevski et al., 2020) brought self-supervised pre-training to speech. A convolutional feature encoder turns raw audio into latent vectors every 20 ms; spans of these latents are masked, and a transformer context network must identify the true quantised latent at each masked position among distractors, using a contrastive loss of the InfoNCE type discussed in the section on self-supervised learning. After pre-training on tens of thousands of hours of unlabelled audio, fine-tuning with a CTC head on as little as ten minutes of transcribed speech produced usable recognisers, a striking demonstration for low-resource languages.

**Whisper** (Radford et al., 2023) took the opposite route: *weak supervision at scale*. It is a standard transformer encoder–decoder trained on 680,000 hours of audio paired with

transcripts found on the internet, in nearly 100 languages. Special tokens in the decoder prompt specify the task (transcribe or translate to English), the language and whether to predict timestamps, so one model performs multilingual recognition, translation and language identification. Whisper is notably robust to accents, background noise and technical vocabulary, although, like other generative models, it can occasionally hallucinate text during long silences. Table 2.46 compares the main ASR approaches.

**Table 2.46** Evolution of speech-recognition approaches.

| Approach | Era | Acoustic modelling | Alignment handling | Notes |
|---|---|---|---|---|
| HMM–GMM | 1980s–2011 | Gaussian mixtures on MFCCs | HMM states | Needs pronunciation lexicon |
| Hybrid DNN–HMM | 2012–2016 | DNN predicts HMM states | HMM + forced alignment | Large accuracy jump |
| CTC models (Deep Speech) | 2014– | RNN/CNN/transformer | CTC blank symbol | Simple, non-autoregressive |
| RNN-Transducer | 2012– | Encoder + prediction network | Transducer lattice | Streaming, on-device |
| wav2vec 2.0 + CTC | 2020 | Self-supervised transformer | CTC fine-tuning | Few labels needed |
| Whisper | 2022–23 | Encoder–decoder transformer | Attention | Multilingual, robust, multitask |

Recogniser quality is reported as **word error rate (WER)** $= (\mathrm{S} + \mathrm{D} + \mathrm{I})/\mathrm{N}$, where S, D and I are the numbers of substituted, deleted and inserted words in the minimum-edit alignment with the N-word reference. Note that WER can exceed 100% when there are many insertions.

**E Worked Example 2.22 | Word Error Rate**

Reference: "the cat sat on the mat" ($\mathrm{N} = 6$). Hypothesis: "the cat sat in mat today".

- Substitution: on → in ($\mathrm{S} = 1$). Deletion: the second "the" ($\mathrm{D} = 1$). Insertion: "today" ($\mathrm{I} = 1$).
- WER $= (1 + 1 + 1)/6 = 0.50$, i.e. 50%.

## 2.48 Deep Learning for Medical Imaging and Healthcare

Healthcare is one of the most consequential application areas of deep learning. Medical imaging alone generates a substantial share of all clinical data, and the shortage of radiologists and pathologists in many countries makes automated assistance attractive. Deep learning has reached specialist-level performance on narrowly defined tasks

such as detecting diabetic retinopathy in retinal photographs (Gulshan et al., 2016) and classifying skin lesions (Esteva et al., 2017). Translating such results into safe and effective clinical use, however, requires attention to data, validation, regulation and workflow that goes well beyond model accuracy.

## 2.48.1 Imaging Modalities and Their Characteristics

Each imaging modality has distinct physics, dimensionality and artefacts, which influence architecture and preprocessing choices. Table 2.47 summarises the main ones. Volumetric modalities such as CT and MRI are usually processed with 3-D convolutions or as stacks of 2-D slices; digital pathology slides can exceed 100,000 × 100,000 pixels and are handled by tiling into patches and aggregating predictions with **multiple-instance learning**. Images are stored in the DICOM standard, which carries acquisition metadata and, frequently, patient identifiers that must be removed before research use.

**Table 2.47** Medical imaging modalities and typical deep-learning tasks.

| Modality | Data form | Typical tasks | Special considerations |
|---|---|---|---|
| X-ray (radiography) | 2-D greyscale | Pneumonia, fracture, TB detection | Projection overlap; label noise from reports |
| CT | 3-D volume, Hounsfield units | Nodule detection, organ segmentation | Windowing of intensity range; radiation dose |
| MRI | 3-D, multiple sequences (T1, T2, FLAIR) | Brain tumour segmentation | Scanner-dependent intensities; bias field |
| Ultrasound | 2-D/3-D video | Fetal biometry, cardiac function | Speckle noise; operator dependence |
| Histopathology (WSI) | Gigapixel RGB | Cancer grading, metastasis detection | Stain variation; tiling and MIL |
| Fundus / OCT | 2-D colour / 3-D | Diabetic retinopathy, AMD | Image quality grading |
| Dermoscopy | 2-D colour | Melanoma classification | Skin-tone diversity in data |

## 2.48.2 Segmentation and the Dice Loss

Many clinical tasks require delineating structures pixel by pixel, for example outlining a tumour to plan radiotherapy. The **U-Net** architecture (Ronneberger, Fischer and Brox, 2015), introduced in the section on object detection and segmentation, was originally designed for biomedical segmentation, and its self-configuring descendant **nnU-Net** (Isensee et al., 2021) remains a very strong baseline across many medical segmentation benchmarks. A key difficulty is **class imbalance**: a lesion may occupy less than 1% of the voxels, so a per-pixel cross-entropy loss is dominated by the background. The **Dice loss** (Milletari et al., 2016) directly optimises a soft version of the Dice similarity coefficient, which measures overlap, as in Equation (2.124).

$$\mathrm{DSC}(P, G) = \frac{2\,|P \cap G|}{|P| + |G|}$$

$$\mathcal{L}_{\mathrm{Dice}} = 1 - \frac{2\sum_i p_i\, g_i + \epsilon}{\sum_i p_i + \sum_i g_i + \epsilon} \tag{2.124}$$

where P and G are the predicted and ground-truth masks, $p_i \in [0, 1]$ is the predicted foreground probability of pixel i, $g_i \in \{0, 1\}$ is its label, and $\varepsilon$ is a small smoothing constant that avoids division by zero.

Because the Dice loss is computed over the whole image, it is insensitive to the number of background pixels. In practice it is often combined with cross-entropy ($\mathrm{L} = \mathrm{L}_{\mathrm{CE}} + \mathrm{L}_{\mathrm{Dice}}$), which gives smoother gradients early in training. Segmentation quality is also reported with the **Hausdorff distance**, which measures the worst-case boundary error. Listing 2.41 implements the soft Dice loss.

**E Worked Example 2.23 | Dice Coefficient**

A ground-truth tumour mask contains 400 pixels; the model predicts 360 pixels, of which 300 overlap the truth.

- DSC = 2 × 300 / (360 + 400) = 600 / 760 = 0.789.
- Intersection over union (Jaccard) = 300 / (360 + 400 − 300) = 300 / 460 = 0.652.

In general DSC = 2·IoU / (1 + IoU), so Dice is always at least as large as IoU.

**Listing 2.41** Soft Dice loss combined with binary cross-entropy for segmentation in PyTorch.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiceBCELoss(nn.Module):
    def __init__(self, eps=1.0, w_dice=0.5):
        super().__init__()
        self.eps, self.w = eps, w_dice

    def forward(self, logits, target):          # logits, target: (B, 1, H, W)
        p = torch.sigmoid(logits)
        dims = (1, 2, 3)
        inter = (p * target).sum(dims)
        dice = (2 * inter + self.eps) / (p.sum(dims) + target.sum(dims) + self.eps)
        bce = F.binary_cross_entropy_with_logits(logits, target)
        return self.w * (1 - dice.mean()) + (1 - self.w) * bce

loss_fn = DiceBCELoss()
logits = torch.randn(2, 1, 128, 128)
mask = (torch.rand(2, 1, 128, 128) > 0.95).float()   # sparse foreground
print(loss_fn(logits, mask).item())
```

### 2.48.3 Beyond Imaging: Clinical Data and Healthcare AI

Deep learning is also applied to electronic health records (EHRs), where sequence models and transformers predict outcomes such as readmission or sepsis from irregularly sampled visits; to physiological signals, where 1-D CNNs detect arrhythmias in ECGs; to genomics; and to protein structure, where AlphaFold 2 (Jumper et al., 2021) achieved near-experimental accuracy in predicting 3-D structure from amino-acid sequence. Clinical language models are used to summarise notes and extract structured information.

### 2.48.4 Clinical, Regulatory and Ethical Issues

High test accuracy is necessary but far from sufficient for clinical deployment. The workflow in Figure 2.62 highlights the stages at which medical AI projects most commonly fail.

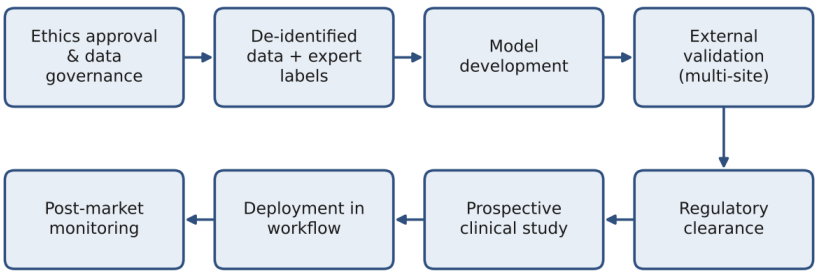



**Figure 2.62** Lifecycle of a clinical deep-learning system, from data governance to post-market surveillance; regulatory review and prospective validation sit between model development and routine use.

- **Dataset shift and generalisation.** Models trained at one hospital often degrade at another because of different scanners, protocols and patient populations. External, multi-site validation is essential.
- **Shortcut learning.** Networks can exploit spurious cues: a chest X-ray model learned to detect a hospital's portable-scanner markers, which correlated with sicker patients, rather than the pathology (Zech et al., 2018).
- **Label quality.** Labels mined from radiology reports are noisy; inter-reader variability bounds attainable accuracy.
- **Regulation.** Software that diagnoses or guides treatment is a medical device. In the USA it is reviewed by the FDA (510(k), De Novo or PMA pathways); in the

EU it falls under the Medical Device Regulation (MDR 2017/745) and the AI Act classifies it as high-risk.

- **Privacy.** Health data are protected by laws such as HIPAA in the USA and the GDPR in Europe; federated learning, discussed in the section on ethics and privacy, allows training without centralising data.
- **Reporting standards.** Guidelines such as TRIPOD+AI, CONSORT-AI and SPIRIT-AI specify what studies must report.
- **Human factors.** Automation bias, alert fatigue and unclear accountability can negate technical gains; systems should be designed as decision support, with calibrated uncertainty and explanations.

**Case Study 2.12 | Diabetic Retinopathy Screening**

Gulshan et al. (2016) trained an Inception-v3 network on about 128,000 retinal images graded by ophthalmologists and reported sensitivity and specificity above 90% for referable diabetic retinopathy. In 2018, IDx-DR became the first FDA-authorised autonomous AI diagnostic system, for use in primary care.

A subsequent field study in Thai clinics (Beede et al., 2020) found that poor lighting, image quality and internet connectivity caused many images to be rejected, showing that real-world performance depends on the whole socio-technical system, not only the model.

## 2.49 Deep Learning for Computer Vision

Computer vision is the field in which the modern deep learning era began, with AlexNet's 2012 ImageNet victory, and it remains one of its largest application areas. Earlier sections of this chapter covered convolutional networks and their classic architectures, and object detection and segmentation. This section takes a wider view: it organises vision tasks into a taxonomy, then examines two tasks not yet covered, **human pose estimation** and **object tracking**, and closes with a recap of the vision transformer and the convergence of vision with foundation models.

### 2.49.1 A Taxonomy of Vision Tasks

Vision tasks differ mainly in the granularity of their output: one label per image, one box per object, one label per pixel, or a set of keypoints or 3-D quantities. Table 2.48 organises the main tasks, and Figure 2.63 shows how they relate as a hierarchy from image-level to pixel-level to temporal and 3-D understanding.

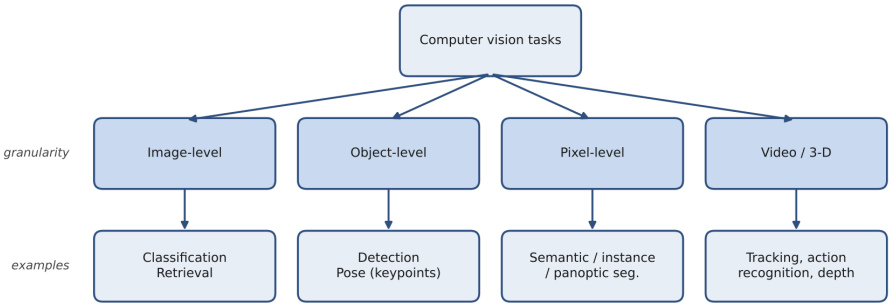



**Figure 2.63** Taxonomy of computer-vision tasks grouped by the granularity of the output, from image-level recognition to dense, temporal and 3-D understanding.

**Table 2.48** Computer-vision tasks, their outputs and representative deep models.

| Task | Output | Representative models | Typical metric |
|---|---|---|---|
| Image classification | Class label | ResNet, EfficientNet, ViT, ConvNeXt | Top-1 / top-5 accuracy |
| Object detection | Boxes + labels | Faster R-CNN, YOLO, DETR | mAP@[0.5:0.95] |
| Semantic segmentation | Label per pixel | U-Net, DeepLabv3+, SegFormer | mIoU |
| Instance segmentation | Mask per object | Mask R-CNN, SAM | Mask mAP |
| Pose estimation | Keypoints per person | OpenPose, HRNet, ViTPose | OKS-based AP, PCK |
| Multi-object tracking | Identities over time | SORT, DeepSORT, ByteTrack | MOTA, IDF1, HOTA |
| Action recognition | Label per clip | I3D, SlowFast, TimeSformer | Accuracy |
| Depth estimation | Depth per pixel | MiDaS, DPT | AbsRel, RMSE |

## 2.49.2 Human Pose Estimation

**Pose estimation** locates anatomical keypoints (the COCO standard uses 17: nose, eyes, ears, shoulders, elbows, wrists, hips, knees and ankles). The dominant formulation is **heatmap regression**: for each keypoint k, the network predicts a heatmap whose target is a Gaussian centred at the true location, as in Equation (2.125), and the keypoint is read off as the arg-max of the predicted map. Regressing heatmaps rather than raw coordinates preserves spatial structure and is easier for a fully convolutional network to learn.

$$H_k^*(\mathbf{p}) = \exp\left(-\frac{\|\mathbf{p}-\mathbf{p}_k\|^2}{2\sigma^2}\right)$$
$$\mathcal{L}_{\text{pose}} = \frac{1}{K}\sum_{k=1}^{K} v_k \sum_{\mathbf{p}} \big(H_k(\mathbf{p}) - H_k^*(\mathbf{p})\big)^2 \tag{2.125}$$

where p ranges over pixel positions, $\mathbf{p}_k$ is the true location of keypoint k, $\sigma$ controls the spread

(a few pixels), $H_k$ is the predicted heatmap and $v_k \in \{0, 1\}$ indicates whether the keypoint is labelled.

Multi-person systems follow one of two strategies. **Top-down** methods first detect each person with an object detector and then estimate a single pose within each box (e.g. HRNet, which maintains high-resolution feature maps throughout, and ViTPose). **Bottom-up** methods detect all keypoints in the image at once and group them into individuals; OpenPose (Cao et al., 2017) does this with *part affinity fields*, vector fields that encode the direction of each limb. Top-down methods are usually more accurate; bottom-up methods have a running time that is nearly independent of the number of people.

## 2.49.3 Object Tracking

**Multi-object tracking** (**MOT**) assigns consistent identities to objects across video frames. The dominant *tracking-by-detection* paradigm runs a detector on each frame and then associates detections with existing tracks, as in Algorithm 2.24. **SORT** (Bewley et al., 2016) predicts each track's next box with a Kalman filter and matches predictions to detections by IoU using the Hungarian algorithm. **DeepSORT** adds a deep appearance embedding (a re-identification network) so that identities survive occlusions, and **ByteTrack** (2022) improves recall by also associating low-confidence detections in a second matching pass.

**Algorithm 2.24** Tracking-by-Detection (SORT / DeepSORT style)

**Input:** Video frames $I_1 \ldots I_T$; object detector $D$; IoU threshold $\theta$; maximum age $A_{\max}$
**Output:** Set of tracks, each a sequence of (frame, box, identity)

tracks $\leftarrow \emptyset$
**for** $t = 1$ **to** $T$ **do**
  dets $\leftarrow D(I_t)$ // boxes + scores (+ appearance features)
  **for each** track $\tau$ **in** tracks **do** predict $\tau$.box with its Kalman filter
  $C \leftarrow$ cost matrix with $C[i, j] = 1 - \text{IoU}(\text{track_i}, \text{det_j})$ $(+ \lambda \cdot \text{appearance distance})$
  matches $\leftarrow$ Hungarian$(C)$, discarding pairs with IoU $< \theta$
  **for each** matched $(\tau, d)$ **do** update $\tau$'s Kalman filter with $d$; $\tau$.age $\leftarrow 0$
  **for each** unmatched detection $d$ **do** create a new track with a new identity
  **for each** unmatched track $\tau$ **do**
    $\tau$.age $\leftarrow \tau$.age $+ 1$
    **if** $\tau$.age $> A_{\max}$ **then** remove $\tau$ from tracks
**return** tracks

### 2.49.4 Vision Transformers: A Recap and Outlook

The **vision transformer (ViT)** (Dosovitskiy et al., 2021), introduced in the section on transformers, splits an image into $16 \times 16$ patches, linearly embeds each patch, adds position embeddings and a learnable [CLS] token, and processes the sequence with a standard transformer encoder. Lacking the locality and translation-equivariance biases of convolutions, ViTs need large pre-training sets (or strong augmentation and distillation, as in DeiT) but scale better than CNNs. Hierarchical variants such as the **Swin Transformer** restrict attention to shifted local windows to obtain multi-scale feature maps suitable for detection and segmentation, while **ConvNeXt** showed that CNNs modernised with transformer-era design choices remain competitive.

Computer vision is converging with the foundation-model paradigm described earlier in this chapter. Self-supervised backbones (DINOv2), promptable models (Segment Anything), open-vocabulary detectors built on CLIP-style embeddings (OWL-ViT, Grounding DINO) and vision–language models increasingly replace task-specific architectures trained from scratch. Listing 2.42 shows how easily a pretrained ViT can be used.

**Listing 2.42** Loading a pretrained vision transformer from the timm library and classifying an image.

```python
import timm
import torch
from PIL import Image

model = timm.create_model("vit_base_patch16_224", pretrained=True).eval()
cfg = timm.data.resolve_data_config({}, model=model)
transform = timm.data.create_transform(**cfg)       # correct resize + normalisation

img = transform(Image.open("dog.jpg").convert("RGB")).unsqueeze(0)
with torch.no_grad():
    probs = model(img).softmax(dim=-1)
top5 = probs.topk(5)
print(top5.indices.tolist(), top5.values.tolist())
```

**Key Points | Computer Vision**

- Vision tasks are distinguished by output granularity: image, object, pixel, keypoint, or temporal identity.
- Pose estimation is usually solved by Gaussian heatmap regression; tracking by detection plus association.
- Transformers and large pretrained backbones now underpin most state-of-the-art vision systems.

# 2.50 Deep Learning for Recommendation Systems

Recommendation systems decide which products, films, songs, news articles or posts each user sees, and they drive a large share of engagement and revenue on platforms such as Netflix, YouTube, Amazon and Spotify. The core problem is to estimate how much a user u will like an item i, given a history of interactions that is extremely sparse: a typical user has interacted with a tiny fraction of a catalogue of millions of items. Interactions may be **explicit** (star ratings) or, far more commonly, **implicit** (clicks, purchases, watch time), in which case an unobserved interaction is ambiguous: the user may dislike the item or simply never have seen it.

## 2.50.1 Matrix Factorisation as a Shallow Embedding Model

Classical **collaborative filtering** recommends items liked by similar users. Its most successful form, made famous by the Netflix Prize (2006–2009), is **matrix factorisation (MF)** (Koren, Bell and Volinsky, 2009). Each user and item is represented by a d-dimensional latent vector, and the predicted preference is their dot product plus biases, as in Equation (2.126). Viewed through the lens of this chapter, MF is a neural network with two embedding tables and no hidden layers, and it is the conceptual ancestor of all the deep models that follow.

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^{\top}\,\mathbf{q}_i$$

$$\min_{P,\,Q,\,b} \sum_{(u,\,i)\,\in\,\mathcal{K}} \left(r_{ui} - \hat{r}_{ui}\right)^2 + \lambda\left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2\right) \tag{2.126}$$

where $\mu$ is the global mean rating, $b_u$ and $b_i$ are user and item biases, $p_u, q_i \in \mathbb{R}^d$ are latent factors, $\mathcal{K}$ is the set of observed (user, item) pairs and $\lambda$ is the L2 regularisation strength.

For implicit feedback, the squared error is usually replaced by a ranking loss. **Bayesian personalised ranking (BPR)** (Rendle et al., 2009) samples, for each observed pair (u, i), a non-interacted item j and maximises $\log \sigma(\hat{y}_{ui} - \hat{y}_{uj})$, encouraging observed items to be ranked above unobserved ones. Alternatively, the problem is framed as binary classification with sampled negatives and binary cross-entropy.

## 2.50.2 Deep Recommendation Architectures

Deep learning extends MF in three directions: replacing the dot product with a learned interaction function, incorporating rich side features (user demographics, item text

and images, context such as time and device), and modelling sequences of user behaviour. Four influential architectures are compared in Table 2.49 and illustrated in Figure 2.64.

- **Neural collaborative filtering (NCF)** (He et al., 2017) feeds user and item embeddings into an MLP, and its NeuMF variant combines this with a generalised MF branch (element-wise product of embeddings). Rendle et al. (2020) later showed that a well-tuned dot product is hard to beat, a useful caution against assuming more layers always help.
- **Wide & Deep** (Cheng et al., 2016), developed for Google Play app recommendations, jointly trains a *wide* linear model on hand-crafted cross-product features, which **memorises** specific co-occurrences, and a *deep* MLP on embeddings, which **generalises** to unseen combinations. DeepFM and DCN replace the hand-crafted crosses with learned feature interactions.
- **Two-tower (dual encoder) models** (e.g. Yi et al., 2019 for YouTube) encode the user and the item with separate networks into the same space and score by dot product. Because item embeddings do not depend on the user, they can be precomputed and indexed for **approximate nearest neighbour (ANN)** search (e.g. FAISS, ScaNN), allowing retrieval from millions of items in milliseconds.
- **Sequential recommenders** such as GRU4Rec, SASRec and BERT4Rec treat a user's history as a sequence and predict the next item with RNNs or self-attention, as in the earlier sections on sequence modelling and transformers.

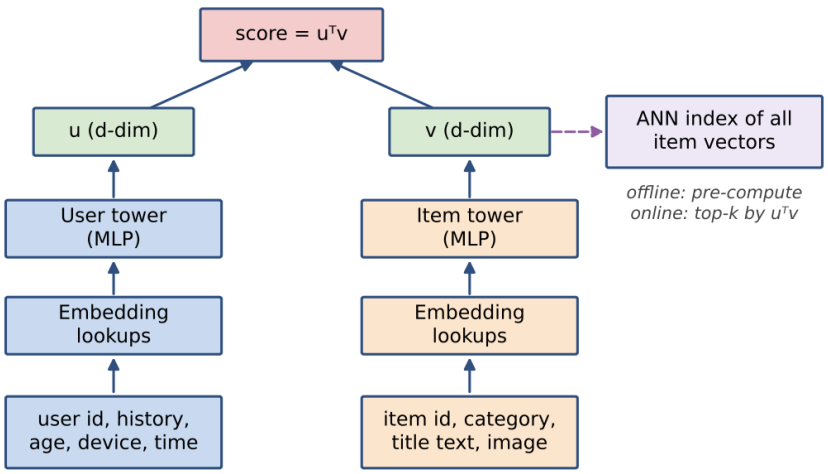



**Figure 2.64** Two-tower retrieval model: user and item towers map features to a shared embedding space, the score is a dot product, and item embeddings are pre-computed in an ANN index for fast candidate retrieval.

**Table 2.49** Comparison of recommendation model families.

| Model | Interaction function | Side features | Typical stage | Key strength |
|---|---|---|---|---|
| Matrix factorisation | Dot product | No (basic form) | Retrieval / ranking | Simple, strong baseline |
| NCF / NeuMF | MLP + GMF | Limited | Ranking | Learned non-linear interactions |
| Wide & Deep / DeepFM / DCN | Linear crosses + MLP | Yes | Ranking | Memorisation + generalisation |
| Two-tower | Dot product of tower outputs | Yes | Retrieval | Sub-linear serving via ANN |
| SASRec / BERT4Rec | Self-attention over history | Optional | Retrieval / ranking | Captures sequential intent |

Production systems are organised as a **multi-stage funnel**: a cheap *retrieval* (candidate generation) stage, often a two-tower model, selects a few hundred items from millions; a heavier *ranking* model, such as a Wide & Deep or DCN network with many features, scores those candidates; and a *re-ranking* stage enforces business rules, diversity and freshness. Two-tower training typically uses an in-batch softmax, the same contrastive idea as InfoNCE, in which the other items in the batch act as negatives; a log-frequency correction compensates for popular items appearing more often as negatives. Listing 2.43 implements a compact NCF model.

**Listing 2.43** A compact NeuMF-style neural collaborative filtering model in PyTorch.

```
import torch
import torch.nn as nn

class NeuMF(nn.Module):
    def __init__(self, n_users, n_items, d=32, hidden=(64, 32)):
        super().__init__()
        self.gmf_u, self.gmf_i = nn.Embedding(n_users, d), nn.Embedding(n_items, d)
        self.mlp_u, self.mlp_i = nn.Embedding(n_users, d), nn.Embedding(n_items, d)
        layers, inp = [], 2 * d
        for h in hidden:
            layers += [nn.Linear(inp, h), nn.ReLU(), nn.Dropout(0.2)]
            inp = h
        self.mlp = nn.Sequential(*layers)
        self.out = nn.Linear(d + hidden[-1], 1)

    def forward(self, u, i):
        gmf = self.gmf_u(u) * self.gmf_i(i)                       # generalised MF
        ↪ branch
        mlp = self.mlp(torch.cat([self.mlp_u(u), self.mlp_i(i)], dim=-1))
        return self.out(torch.cat([gmf, mlp], dim=-1)).squeeze(-1)  # logit

model = NeuMF(n_users=1000, n_items=5000)
u, i = torch.randint(0, 1000, (256,)), torch.randint(0, 5000, (256,))
y = torch.randint(0, 2, (256,)).float()                           # 1 = interacted, 0
↪ = sampled negative
```

```
loss = nn.BCEWithLogitsLoss()(model(u, i), y)
loss.backward()
```

### 2.50.3 Evaluating Recommenders: HR@k and NDCG@k

Because users see only a short list, recommenders are evaluated with **top-k ranking metrics** rather than RMSE. In the common *leave-one-out* protocol, each user's most recent interaction is held out, and the model ranks it among the full catalogue or among, say, 100 sampled negatives. **Hit rate (HR@k)** is the fraction of users whose held-out item appears in the top k. **Normalised discounted cumulative gain (NDCG@k)** additionally rewards placing relevant items near the top, using a logarithmic position discount, as in Equation (2.127).

$$\mathrm{HR@}k = \frac{1}{|U|}\sum_{u\in U} \mathbb{1}\left[\mathrm{rank}_u \le k\right] \qquad \mathrm{DCG@}k = \sum_{j=1}^{k} \frac{2^{\mathrm{rel}_j}-1}{\log_2(j+1)}, \qquad \mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k} \tag{2.127}$$

where rank_u is the position of user u's held-out item, rel_j is the graded relevance of the item at position j, and IDCG@k is the DCG of the ideal ordering, so that NDCG@k $\in [0,1]$.

**E Worked Example 2.24 | NDCG@5**

A model returns five items whose binary relevances are (1, 0, 1, 0, 0). With rel $\in \{0,1\}$, 2^rel − 1 = rel.

- DCG@5 = $1/\log_2 2 + 1/\log_2 4 = 1 + 0.5 = 1.5$.
- The ideal list (1, 1, 0, 0, 0) gives IDCG@5 = $1/\log_2 2 + 1/\log_2 3 = 1 + 0.631 = 1.631$.
- NDCG@5 = 1.5 / 1.631 = 0.920. HR@5 = 1 because at least one relevant item is in the top five.

Other common metrics include precision@k, recall@k, mean reciprocal rank (MRR) and mean average precision (MAP). Offline metrics are only proxies: final decisions are made with online **A/B tests** measuring engagement, retention and revenue. Beyond accuracy, practitioners monitor **coverage**, **diversity**, **novelty** and **fairness** to item providers, and must address the **cold-start** problem for new users and items (usually by relying on content features) and the **feedback loop** by which a recommender shapes the very data it is later trained on.

# 2.51 Explainable AI for Deep Learning

Deep networks with millions of parameters are often described as black boxes: they can be accurate, yet it is hard to say *why* a particular prediction was made. **Explainable AI** (**XAI**) develops methods to make model behaviour understandable to humans. Explanations are needed to debug models (for example, to discover shortcut learning), to build appropriate trust among users such as clinicians, to satisfy regulation (the EU GDPR and AI Act contain transparency provisions), and to generate scientific insight. This section focuses on post-hoc **attribution** methods for deep networks, which assign an importance score to each input feature for a given prediction.

**Definition 2.11 | Feature Attribution**

Given a model $f$, an input $x \in \mathbb{R}^d$ and an output of interest (for example the logit of class $c$), an attribution method returns a vector $a \in \mathbb{R}^d$ in which $a_i$ quantifies the contribution of feature $x_i$ to $f_c(x)$. For images, the attribution map is displayed as a heatmap over the pixels, often called a **saliency map**.

## 2.51.1 A Taxonomy of Explanation Methods

Explanation methods can be classified along several axes, summarised in Table 2.50: whether they explain a single prediction (**local**) or the model as a whole (**global**); whether they require access to gradients and internals (**model-specific**, white-box) or only to inputs and outputs (**model-agnostic**, black-box); and whether interpretability is built into the model (**intrinsic**, e.g. attention or prototype layers) or applied after training (**post-hoc**).

## 2.51.2 Gradient-Based Saliency and Grad-CAM

The simplest attribution is the **vanilla gradient** (Simonyan, Vedaldi and Zisserman, 2014): the saliency of pixel $i$ is $|\partial f_c / \partial x_i|$, which measures how much a small change in that pixel would change the class score. Gradients are cheap but noisy, and they **saturate**: if a feature's effect has already pushed a ReLU or sigmoid into a flat region, its gradient is near zero even though the feature matters. **SmoothGrad** averages gradients over several noisy copies of the input to reduce visual noise.

**Grad-CAM** (Selvaraju et al., 2017) produces class-discriminative heatmaps for any CNN. It uses the feature maps $A^k$ of the last convolutional layer, which retain spatial layout and high-level semantics. The gradient of the class score with respect to each feature map is global-average-pooled to obtain an importance weight, and the heatmap

**Table 2.50** Common explanation methods for deep networks.

| Method | Type | Needs gradients? | Output | Main caveat |
|---|---|---|---|---|
| Vanilla gradient saliency | Local, post-hoc | Yes | Pixel heatmap | Noisy; gradient saturation |
| SmoothGrad | Local, post-hoc | Yes | Denoised heatmap | Extra forward/backward passes |
| Grad-CAM | Local, post-hoc | Yes (to last conv layer) | Coarse class heatmap | Low resolution |
| Integrated Gradients | Local, post-hoc | Yes | Pixel/feature attributions | Choice of baseline |
| SHAP (Deep/Gradient/Kernel) | Local → global | Deep: yes; Kernel: no | Additive attributions | Costly; feature independence |
| LIME | Local, model-agnostic | No | Sparse linear surrogate | Unstable across samples |
| Attention maps / rollout | Intrinsic | No | Token/patch weights | Attention ≠ explanation |
| Concept methods (TCAV) | Global | Yes | Concept sensitivity | Needs concept examples |

is the ReLU of the weighted combination of maps, as in Equation (2.128). The ReLU keeps only features with a positive influence on the class. The coarse map (e.g. $7 \times 7$ for ResNet-50) is upsampled to the image size and overlaid, as in Figure 2.65. Listing 2.44 computes Grad-CAM for a pretrained ResNet-50 using forward and backward hooks.

$$\alpha_k^c = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$
$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \tag{2.128}$$

where y^c is the pre-softmax score for class c, A^k$_{ij}$ is activation (i, j) of the k-th feature map in the chosen convolutional layer, Z is the number of spatial positions, and α^c$_k$ is the importance of map k for class c.

**Listing 2.44** Computing a Grad-CAM heatmap for a pretrained ResNet-50 with PyTorch hooks.

```
import torch
import torch.nn.functional as F
from torchvision.models import resnet50, ResNet50_Weights

model = resnet50(weights=ResNet50_Weights.DEFAULT).eval()
feats, grads = {}, {}
layer = model.layer4                                    # last convolutional stage
layer.register_forward_hook(lambda m, i, o: feats.update(a=o))
layer.register_full_backward_hook(lambda m, gi, go: grads.update(g=go[0]))

def grad_cam(x, cls=None):
    logits = model(x)                                   # x: (1, 3, 224, 224),
    ↪  normalised
```

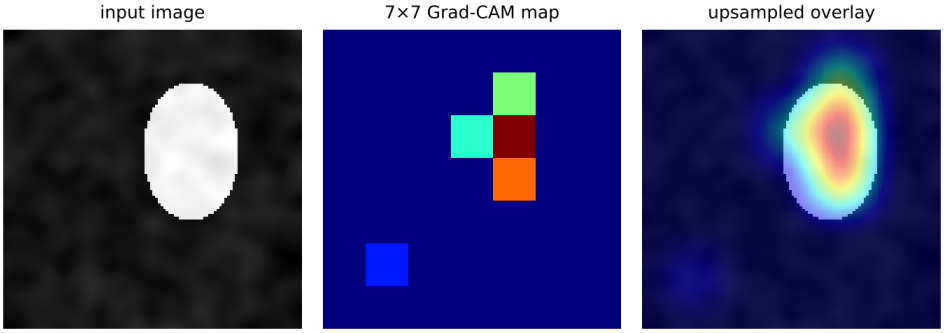



**Figure 2.65** Illustration of Grad-CAM on a synthetic image: the class-weighted combination of last-layer feature maps yields a coarse heatmap that is upsampled and overlaid on the input, highlighting the object region.

```
    cls = logits.argmax(1).item() if cls is None else cls
    model.zero_grad()
    logits[0, cls].backward()
    alpha = grads["g"].mean(dim=(2, 3), keepdim=True)   # GAP of gradients -> (1, K, 1, 1)
    cam = F.relu((alpha * feats["a"]).sum(dim=1, keepdim=True))
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
    return (cam / (cam.max() + 1e-8)).squeeze().detach(), cls
```

## 2.51.3 Integrated Gradients

**Integrated Gradients (IG)** (Sundararajan, Taly and Yan, 2017) addresses gradient saturation on a principled basis. It defines attribution through two axioms: *sensitivity* (if changing one feature changes the output, that feature receives non-zero attribution) and *implementation invariance* (functionally identical networks receive identical attributions). IG accumulates gradients along the straight-line path from a **baseline** $\mathbf{x}'$ (for images, often a black image) to the input $\mathbf{x}$, as in Equation (2.129), and satisfies **completeness**: the attributions sum to $f(\mathbf{x}) - f(\mathbf{x}')$. Algorithm 2.25 gives the Riemann-sum approximation used in practice.

$$\begin{aligned} \mathrm{IG}_i(\mathbf{x}) &= (x_i - x_i') \int_0^1 \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i}\, d\alpha \\ \sum_i \mathrm{IG}_i(\mathbf{x}) &= f(\mathbf{x}) - f(\mathbf{x}') \end{aligned} \tag{2.129}$$

where $\mathbf{x}'$ is the baseline input and $\alpha \in [0, 1]$ parameterises the path from $\mathbf{x}'$ to $\mathbf{x}$.

**Algorithm 2.25** Integrated Gradients (Riemann approximation)

**Input:** Model f (output for target class); input x; baseline $x'$; number of steps m (e.g. 50)
**Output:** Attribution vector $IG \in \mathbb{R}^d$

$g_{sum} \leftarrow 0$
**for** $k = 1$ **to** m **do**
  $\alpha \leftarrow k / m$
  $\tilde{x} \leftarrow x' + \alpha\,(x - x')$ // point on the straight-line path
  $g_{sum} \leftarrow g_{sum} + \nabla_x f(\tilde{x})$
$IG \leftarrow (x - x') \odot g_{sum} / m$
check completeness: $\Sigma_i IG_i \approx f(x) - f(x')$; increase m **if** the gap is large
**return** IG

## 2.51.4 Attention Visualisation and SHAP for Deep Networks

For transformers, it is tempting to read the attention weights as explanations. Attention maps are useful for visualising which tokens or patches a head attends to, and **attention rollout** (Abnar and Zuidema, 2020) multiplies attention matrices across layers (adding the identity for residual connections) to estimate how information flows from inputs to the output. However, Jain and Wallace (2019) showed that attention weights can often be altered substantially without changing predictions, so attention should be treated as one view of the computation rather than a faithful explanation; gradient-weighted attention methods combine the two.

**SHAP** (SHapley Additive exPlanations; Lundberg and Lee, 2017) attributes a prediction to features using **Shapley values** from cooperative game theory: each feature's attribution is its average marginal contribution over all orderings in which features could be added, as in Equation (2.130). Shapley values are the unique additive attribution satisfying efficiency, symmetry, dummy and additivity axioms. Exact computation is exponential in the number of features, so approximations are used: **KernelSHAP** (model-agnostic weighted regression), **DeepSHAP** (which combines DeepLIFT-style back-propagation rules with Shapley averaging over background samples) and **GradientSHAP** (an expected-gradients method closely related to IG with a distribution of baselines).

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\;(|F| - |S| - 1)!}{|F|!} \Big[ f_{S \cup \{i\}}(\mathbf{x}_{S \cup \{i\}}) - f_S(\mathbf{x}_S) \Big] \tag{2.130}$$

where F is the set of all features, S a subset not containing feature i, and $f_S$ the model's expected output when only the features in S are known (the others marginalised over a background distribution).

**Note | Evaluating Explanations**

- **Faithfulness** asks whether an explanation reflects what the model actually computes. A common test is deletion: remove the most important pixels first and check that the class score falls quickly.
- **Sanity checks** (Adebayo et al., 2018): some saliency methods produce nearly identical maps even after the model's weights are randomised, so they reveal image edges rather than model reasoning.
- Explanations can be manipulated adversarially, and different methods often disagree; use several and treat them as hypotheses to be tested.

# 2.52 Adversarial Attacks and Model Robustness

In 2013, Szegedy et al. made a startling discovery: adding a carefully computed perturbation, imperceptible to humans, to an image correctly classified by a deep network can cause it to be misclassified with high confidence. Such **adversarial examples** exist for essentially every type of deep model, including image classifiers, speech recognisers, malware detectors and language models. They expose a gap between human and machine perception and are a serious concern for safety-critical and security-sensitive applications.

**Definition 2.12 | Adversarial Example**

Given a classifier $f$, an input $x$ with true label $y$ and a perturbation budget $\varepsilon$, an adversarial example is an input $x' = x + \delta$ with $\|\delta\|_p \le \varepsilon$ such that $f(x') \neq y$ (untargeted attack) or $f(x') = t$ for a chosen target class $t \neq y$ (targeted attack). Common norms are $\ell_\infty$ (maximum per-pixel change, e.g. $\varepsilon = 8/255$) and $\ell_2$.

## 2.52.1 Threat Models

An attack is characterised by its **threat model**: what the attacker knows and can do. In a **white-box** attack the adversary has full access to architecture and weights and can compute gradients. In a **black-box** attack only queries are possible, either returning scores or only the final label. Remarkably, adversarial examples **transfer**: perturbations crafted on one model often fool another trained on similar data, enabling black-box attacks via a surrogate model. Attacks are also distinguished by timing: **evasion** attacks perturb inputs at test time, whereas **poisoning** and **backdoor** attacks corrupt the training data. Physical-world attacks, such as printed stickers that cause a stop sign to be misread (Eykholt et al., 2018), show that the threat is not merely digital.

### 2.52.2 FGSM and PGD

Goodfellow, Shlens and Szegedy (2015) argued that adversarial vulnerability arises largely from the locally *linear* behaviour of networks in high dimensions: many tiny per-pixel changes aligned with the gradient sum to a large change in the output. This insight yields the **fast gradient sign method (FGSM)**, a single-step attack given in Equation (2.131). Its multi-step generalisation, **projected gradient descent (PGD)** (Madry et al., 2018), repeatedly takes small signed gradient steps and projects back onto the $\varepsilon$-ball, as in Equation (2.132) and Algorithm 2.26. PGD (implemented in Listing 2.45) with random starts is regarded as the standard strong first-order attack for evaluating robustness.

$$\mathbf{x}' = \mathrm{clip}_{[0,1]}\big(\mathbf{x} + \epsilon \cdot \mathrm{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y))\big) \tag{2.131}$$

where $\mathcal{L}$ is the training loss (e.g. cross-entropy), $\varepsilon$ is the $\ell\infty$ budget and $\mathrm{sign}(\cdot)$ is applied element-wise.

$$\mathbf{x}^{(t+1)} = \Pi_{\mathcal{B}_\varepsilon(\mathbf{x})}\Big(\mathbf{x}^{(t)} + \alpha \cdot \mathrm{sign}\big(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}^{(t)}), y)\big)\Big) \tag{2.132}$$

where $\alpha$ is the step size (e.g. $\varepsilon/4$), and $\Pi$ projects onto the $\ell\infty$ ball $\mathcal{B}_\varepsilon(x) = \{x' : \|x' - x\|\infty \le \varepsilon\}$ intersected with the valid pixel range.

**Algorithm 2.26** Projected Gradient Descent (PGD) $\ell\infty$ Attack

**Input:** Model f_θ; input x; label y; budget ε; step size α; iterations T; random start flag
**Output:** Adversarial example $x_{adv}$

**if** random start **then** $x_{adv} \leftarrow x + U(-\varepsilon, \varepsilon)$ **else** $x_{adv} \leftarrow x$
$x_{adv} \leftarrow \mathrm{clip}(x_{adv}, 0, 1)$
**for** t = 1 **to** T **do**
  $g \leftarrow \nabla_x \mathcal{L}(f_\theta(x_{adv}), y)$
  $x_{adv} \leftarrow x_{adv} + \alpha \cdot \mathrm{sign}(g)$ // ascend the loss
  $x_{adv} \leftarrow \min(\max(x_{adv}, x - \varepsilon), x + \varepsilon)$ // project onto ε-ball
  $x_{adv} \leftarrow \mathrm{clip}(x_{adv}, 0, 1)$ // stay a valid image
**return** $x_{adv}$

**Listing 2.45** FGSM and PGD attacks in PyTorch (inputs scaled to $[0, 1]$).

```python
import torch
import torch.nn.functional as F

def fgsm(model, x, y, eps=8/255):
    x = x.clone().detach().requires_grad_(True)
    loss = F.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

def pgd(model, x, y, eps=8/255, alpha=2/255, steps=10):
```

```
    x_adv = (x + torch.empty_like(x).uniform_(-eps, eps)).clamp(0, 1)
    for _ in range(steps):
        x_adv.requires_grad_(True)
        loss = F.cross_entropy(model(x_adv), y)
        grad, = torch.autograd.grad(loss, x_adv)
        x_adv = x_adv.detach() + alpha * grad.sign()
        x_adv = torch.min(torch.max(x_adv, x - eps), x + eps).clamp(0, 1)
    return x_adv.detach()

# robust accuracy: (model(pgd(model, x, y)).argmax(1) == y).float().mean()
```

Other important attacks include the **Carlini–Wagner (C&W)** attack, which solves an optimisation problem to find minimal $\ell_2$ perturbations; **DeepFool**; query-based black-box attacks such as Square Attack; and **AutoAttack** (Croce and Hein, 2020), a parameter-free ensemble that has become the standard benchmark on RobustBench. For language models, adversarial prompts and *jailbreaks* play an analogous role.

### 2.52.3 Defences: Adversarial Training and Certified Robustness

Many proposed defences were later broken; Athalye, Carlini and Wagner (2018) showed that defences relying on **obfuscated gradients** (non-differentiable preprocessing, randomisation) give a false sense of security. The most reliable empirical defence is **adversarial training**, which Madry et al. framed as the min–max (saddle-point) problem in Equation (2.133): the inner maximisation finds the worst-case perturbation (approximated by PGD), and the outer minimisation trains the model on it.

$$\min_{\theta} \; \mathbb{E}_{(\mathbf{x},\,y)\sim\mathcal{D}}\Big[\max_{\|\delta\|_\infty \le \epsilon} \mathcal{L}\big(f_\theta(\mathbf{x}+\delta),\,y\big)\Big] \tag{2.133}$$

where $\mathcal{D}$ is the data distribution and $\delta$ the perturbation chosen adversarially for each example.

Adversarial training is expensive (each step requires a multi-step attack) and typically reduces clean accuracy: on CIFAR-10, robust models reach roughly 60–70% accuracy under $\ell\infty$ attacks with $\varepsilon = 8/255$, while their clean accuracy drops to around 85–90% compared with over 95% for standard training. **TRADES** (Zhang et al., 2019) explicitly trades off clean accuracy against a robustness regulariser. Figure 2.66 shows the typical shape of the accuracy-versus-budget curves.

Empirical defences can be broken by stronger future attacks. **Certified defences** instead *prove* that no perturbation within the budget changes the prediction. **Interval bound propagation** and convex relaxations propagate bounds through the network layer by layer. **Randomised smoothing** (Cohen, Rosenfeld and Kolter, 2019) classifies

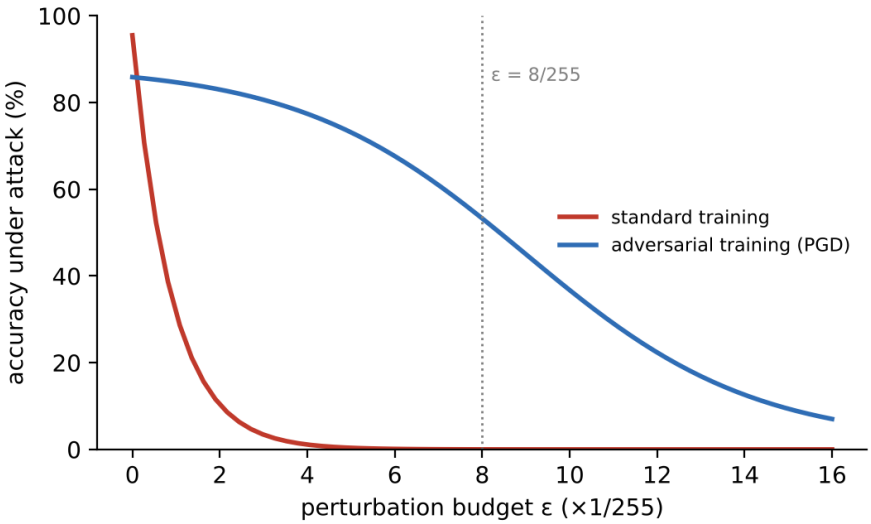



**Figure 2.66** Illustrative accuracy under PGD attack as the $\ell\infty$ budget $\varepsilon$ grows: a standard model collapses almost immediately, while an adversarially trained model trades some clean accuracy for much greater robustness.

by majority vote of the base classifier under Gaussian noise; if the top class has probability $p_A$ and the runner-up $p_B$, the smoothed classifier is certifiably robust within the $\ell_2$ radius in Equation (2.134).

$$R = \frac{\sigma}{2}\left(\Phi^{-1}(p_A) - \Phi^{-1}(p_B)\right) \tag{2.134}$$

where $\sigma$ is the standard deviation of the Gaussian noise and $\Phi^{-1}$ is the inverse standard normal CDF.

**K Key Points | Robustness**

- Always evaluate defences with strong adaptive attacks (PGD with restarts, AutoAttack), never only FGSM.
- Adversarial training is the most reliable empirical defence but costs compute and clean accuracy.
- Certified methods give guarantees, currently for smaller radii and models than empirical defences.
- Robustness to natural distribution shift (corruptions, new domains) is a related but distinct property.

## 2.53 Ethical Issues, Bias, Privacy, and Security

Deep learning systems now influence decisions about credit, hiring, policing, healthcare and what information people see. Their errors and side effects therefore carry ethical weight. This section surveys four intertwined concerns: **bias and fairness**, **privacy**, **security** (beyond the adversarial examples of the previous section) and broader **societal and governance** issues. Many of these problems are not unique to deep learning, but the scale, opacity and data-hunger of deep models amplify them.

## 2.53.1 Sources of Bias

Bias can enter at every stage of the pipeline, as Table 2.51 summarises. A widely cited audit, *Gender Shades* (Buolamwini and Gebru, 2018), found that commercial gender-classification systems had error rates below 1% for lighter-skinned men but up to about 35% for darker-skinned women, reflecting unrepresentative training data. Word embeddings trained on news text were shown to encode gender stereotypes (Bolukbasi et al., 2016), and Obermeyer et al. (2019) found that a widely used health-risk algorithm under-estimated the needs of Black patients because it used past healthcare *cost* as a proxy label for *need*.

**Table 2.51** Sources of bias in the machine-learning pipeline.

| Stage | Type of bias | Example |
|---|---|---|
| Data collection | Representation / sampling bias | Face data sets dominated by lighter-skinned subjects |
| Labelling | Historical and annotator bias | Past hiring decisions encode discrimination |
| Problem formulation | Measurement (proxy) bias | Using healthcare cost as a proxy for health need |
| Modelling | Aggregation bias | One model for groups with different feature–outcome relations |
| Evaluation | Benchmark bias | Reporting only overall accuracy hides subgroup failures |
| Deployment | Feedback loops / automation bias | Predictive policing directs patrols that generate new arrests |

## 2.53.2 Formal Fairness Criteria

Fairness can be made precise with statistical criteria relating the prediction $\hat{Y}$, the true label $Y$ and a protected attribute $A$ (such as sex or ethnicity). Three of the most important are given in Equation (2.135). **Demographic parity** requires equal positive-prediction rates across groups; **equalised odds** (Hardt, Price and Srebro, 2016) requires equal true-positive *and* false-positive rates; and **equal opportunity** relaxes this

to equal true-positive rates only. **Predictive parity** (calibration within groups) requires $\mathrm{P}(\mathrm{Y}=1 \mid \hat{\mathrm{Y}}=1, \mathrm{A}=\mathrm{a})$ to be equal across groups.

$$\begin{aligned} &\text{Demographic parity}: \quad P(\hat{Y}=1 \mid A=0)=P(\hat{Y}=1 \mid A=1) \\ &\text{Equalised odds}: \quad P(\hat{Y}=1 \mid A=0, Y=y)=P(\hat{Y}=1 \mid A=1, Y=y), \quad y \in\{0,1\} \\ &\text{Equal opportunity}: \quad P(\hat{Y}=1 \mid A=0, Y=1)=P(\hat{Y}=1 \mid A=1, Y=1) \end{aligned} \tag{2.135}$$

where A is the protected attribute, Y the true outcome and $\hat{\mathrm{Y}}$ the model's decision. In practice the absolute differences between groups (e.g. the demographic-parity difference) are reported and constrained.

**E Worked Example 2.25 | Checking Fairness Metrics**

A loan model is evaluated on two groups of 1,000 applicants each.

- Group A: 400 truly creditworthy; the model approves 360 of them and 60 of the 600 others. Approval rate = 420/1000 = 0.42; TPR = 360/400 = 0.90; FPR = 60/600 = 0.10.
- Group B: 200 truly creditworthy; the model approves 150 of them and 40 of the 800 others. Approval rate = 190/1000 = 0.19; TPR = 150/200 = 0.75; FPR = 40/800 = 0.05.
- Demographic-parity difference = 0.42 − 0.19 = 0.23; equal-opportunity (TPR) difference = 0.15; FPR difference = 0.05.

The model violates all three criteria; note that some of the approval-rate gap reflects different base rates (0.40 vs 0.20).

A fundamental result, the **impossibility theorem** of fairness (Kleinberg, Mullainathan and Raghavan, 2017; Chouldechova, 2017), shows that when base rates differ between groups, a non-trivial classifier cannot satisfy calibration and equalised odds simultaneously. Choosing a fairness criterion is therefore a value judgement that depends on context, not a purely technical decision. Mitigation methods act at three points: **pre-processing** (re-sampling or re-weighting data), **in-processing** (adding fairness constraints or adversarial debiasing to the training objective) and **post-processing** (group-specific decision thresholds). Toolkits such as Fairlearn and AIF360 implement these metrics and methods.

## 2.53.3 Privacy Risks and Privacy-Preserving Learning

Deep networks can memorise their training data. Carlini et al. (2021) extracted verbatim personal information, including names and phone numbers, from GPT-2 by sampling and filtering its outputs. In a **membership inference attack** (Shokri et al., 2017), an adversary determines whether a specific record was in the training set, which for a medical model would itself reveal sensitive information. The attack exploits the fact that models are usually more confident, and have lower loss, on training examples than

on unseen ones. The simplest version is a loss threshold test, as in Equation (2.136); Shokri et al. trained **shadow models** on similar data to learn the decision rule.

$$\mathcal{M}(\mathbf{x}, y) = \mathbb{1}[\mathcal{L}(f_\theta(\mathbf{x}), y) < \gamma] \tag{2.136}$$

where $\mathcal{M} = 1$ predicts that $(x, y)$ was a training member and $\gamma$ is a threshold calibrated, for example, on shadow models or held-out data. The larger the generalisation gap, the more successful the attack.

Defences include regularisation (which narrows the generalisation gap) and, most rigorously, **differential privacy (DP)**. A randomised training algorithm M is $(\varepsilon, \delta)$-differentially private if, for any two data sets D and D′ differing in one record and any set of outcomes S, $P[M(D) \in S] \leq e^{\varepsilon} P[M(D') \in S] + \delta$. **DP-SGD** (Abadi et al., 2016) achieves this by clipping each example's gradient to a maximum norm C and adding Gaussian noise before the update, with a privacy accountant tracking the cumulative $\varepsilon$. Libraries such as Opacus (PyTorch) and TensorFlow Privacy implement it. **Federated learning** (McMahan et al., 2017) keeps data on devices or within hospitals and shares only model updates, which are averaged by a server (FedAvg); because updates can still leak information, it is often combined with DP and secure aggregation.

### 2.53.4 Security: Model Stealing, Poisoning and Backdoors

Trained models are valuable intellectual property. In a **model stealing (extraction) attack** (Tramèr et al., 2016), an adversary queries a prediction API and uses the returned labels or probabilities to train a *substitute* model that mimics the victim, effectively performing knowledge distillation (see the next section) without permission. The stolen copy can then be used to avoid fees or to craft transferable adversarial examples. Mitigations include returning only labels rather than full probability vectors, rate-limiting queries, detecting anomalous query patterns and **watermarking** models so that copies can be identified.

**Data poisoning** inserts malicious examples into the training set to degrade accuracy or cause targeted errors. A **backdoor (trojan) attack** (Gu et al., 2017, *BadNets*) poisons training data so that inputs carrying a secret trigger (e.g. a small patch) are classified as the attacker's target while clean accuracy is unaffected; this is a real risk when using third-party data or pretrained weights. Large language models face additional threats such as **prompt injection**, in which instructions hidden in retrieved content hijack the model's behaviour. Table 2.52 summarises the attack landscape.

**Table 2.52** Attacks on deep-learning systems and representative defences.

| Attack | Attacker goal | When | Defences |
|---|---|---|---|
| Evasion (adversarial examples) | Cause misclassification | Inference | Adversarial training, certification |
| Membership inference | Learn if a record was used | Inference | DP-SGD, regularisation |
| Model inversion / extraction of data | Reconstruct training data | Inference | DP, output limiting |
| Model stealing | Copy model functionality | Inference (API) | Rate limits, label-only output, watermarking |
| Poisoning | Degrade or bias model | Training | Data provenance, robust training, sanitisation |
| Backdoor / trojan | Hidden trigger behaviour | Training / supply chain | Trusted sources, trigger detection, fine-pruning |
| Prompt injection (LLMs) | Hijack instructions | Inference | Input isolation, privilege separation |

### 2.53.5 Governance and Responsible Practice

Technical tools must be complemented by governance. **Model cards** (Mitchell et al., 2019) and **datasheets for datasets** (Gebru et al., 2021) document intended uses, evaluation across subgroups and known limitations. The EU AI Act (adopted in 2024) imposes risk-based obligations, with strict requirements for high-risk systems in areas such as employment, credit and medical devices, and transparency requirements for general-purpose AI models. Broader concerns include the environmental cost of large-scale training (Strubell, Ganesh and McCallum, 2019), labour conditions of data annotators, copyright in training data, deepfakes and misinformation, and the concentration of capability in a few organisations.

> **T Practice Tip | A Responsible-AI Checklist**
>
> - Report performance disaggregated by relevant subgroups, not only overall accuracy.
> - Document data provenance, consent and licensing; publish a model card.
> - Choose and justify a fairness criterion with stakeholders; monitor it after deployment.
> - Assess privacy risk (membership inference) and consider DP or federated training for sensitive data.
> - Threat-model the deployment: who could attack it, how, and what would the harm be?

## 2.54 Model Compression, Quantization, and Pruning

State-of-the-art networks are large: ResNet-50 has about 25 million parameters and needs roughly 4 GFLOPs per image, and large language models have tens or hundreds of billions of parameters. Deploying them on mobile phones, microcontrollers or at high request rates in the cloud requires reducing memory, latency and energy. **Model compression** techniques achieve this, often with little loss in accuracy, because trained networks are heavily over-parameterised. The four main families are **knowledge distillation**, **quantisation**, **pruning** and **low-rank factorisation**, together with the design of efficient architectures such as MobileNet and EfficientNet (depthwise-separable convolutions and compound scaling).

### 2.54.1 Knowledge Distillation

**Knowledge distillation (KD)** (Hinton, Vinyals and Dean, 2015) trains a small **student** network to mimic a large **teacher**. The insight is that the teacher's full output distribution carries *dark knowledge*: the relative probabilities of incorrect classes (a picture of a 3 resembling an 8) reveal how the teacher generalises. Softening both distributions with a temperature $T > 1$ exposes this information. The student minimises a weighted sum of the usual cross-entropy with hard labels and the KL divergence between softened teacher and student outputs, as in Equation (2.137).

$$p_i^{T} = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

$$\mathcal{L}_{\mathrm{KD}} = (1-\lambda)\,\mathcal{L}_{\mathrm{CE}}\big(y, p^{S,1}\big) + \lambda\, T^2\, \mathrm{KL}\big(p^{\mathrm{teacher},T} \,\|\, p^{S,T}\big) \tag{2.137}$$

where z are logits, T is the temperature, p^{S,T} is the student's softened distribution, $\lambda \in [0,1]$ balances the two terms, and the factor $T^2$ keeps the soft-target gradients on the same scale as the hard-label gradients.

DistilBERT (Sanh et al., 2019) used distillation to obtain a model 40% smaller and 60% faster than BERT-base while retaining about 97% of its language-understanding performance. Variants distil intermediate features (FitNets) or attention maps rather than only outputs. Listing 2.46 shows the loss in PyTorch.

**Listing 2.46** Knowledge-distillation loss in PyTorch.

```
import torch
import torch.nn.functional as F

def distillation_loss(student_logits, teacher_logits, labels, T=4.0, lam=0.7):
    hard = F.cross_entropy(student_logits, labels)
    soft = F.kl_div(F.log_softmax(student_logits / T, dim=1),
                    F.softmax(teacher_logits / T, dim=1),
                    reduction="batchmean") * (T * T)
```

```
    return (1 - lam) * hard + lam * soft

# inside the training loop
# with torch.no_grad():
#     t_logits = teacher(x)
# loss = distillation_loss(student(x), t_logits, y)
```

## 2.54.2 Quantisation

**Quantisation** represents weights and activations with fewer bits, typically 8-bit integers (INT8) instead of 32-bit floats (FP32). This reduces model size by 4×, and integer arithmetic is faster and more energy-efficient on most hardware. The standard **affine (asymmetric) quantisation** scheme maps a real value r to an integer q with a scale s and zero-point z, as in Equation (2.138); **symmetric** quantisation fixes $z = 0$.

$$q = \text{clip}(\text{round}(\tfrac{r}{s}) + z,\ q_{\min},\ q_{\max}), \qquad \hat{r} = s\,(q - z)$$
$$s = \tfrac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \qquad z = \text{round}(q_{\min} - \tfrac{r_{\min}}{s}) \tag{2.138}$$

where $[r_{\min}, r_{\max}]$ is the observed (calibrated) range of the tensor, $[q_{\min}, q_{\max}]$ is the integer range (e.g. $[-128, 127]$ for INT8), and $\hat{r}$ is the de-quantised approximation of r.

**E Worked Example 2.26 | INT8 Quantisation**

Weights lie in $[-1.0, 2.0]$; quantise to unsigned 8-bit $[0, 255]$.

- $s = (2.0 - (-1.0)) / 255 = 0.011765$; $z = \text{round}(0 - (-1.0)/0.011765) = \text{round}(85.0) = 85$.
- $r = 0.5$: $q = \text{round}(0.5/0.011765) + 85 = \text{round}(42.5) + 85 = 43 + 85 = 128$ (ties rounded up for this example).
- De-quantised: $\hat{r} = 0.011765 \times (128 - 85) = 0.5059$, a rounding error of 0.0059 (at most $s/2$ for in-range values).

**Post-training quantisation (PTQ)** quantises a trained FP32 model using a small calibration set to estimate activation ranges; it needs no retraining and usually loses under 1% accuracy at INT8 for CNNs. **Quantisation-aware training (QAT)** inserts *fake-quantisation* operations during training (with the straight-through estimator for gradients through rounding) so the network learns to tolerate the noise; it is needed for aggressive (4-bit and below) quantisation. For LLMs, weight-only methods such as GPTQ and AWQ quantise weights to 4 bits while keeping activations in 16-bit, and per-channel or per-group scales handle outlier values.

### 2.54.3 Pruning

**Pruning** removes redundant parameters. **Unstructured pruning** zeroes individual weights, producing sparse matrices; it can remove 80–90% of weights with little accuracy loss (Han et al., 2015) but yields speed-ups only on hardware or libraries that exploit sparsity. **Structured pruning** removes whole filters, channels, attention heads or layers, producing a smaller dense network that is faster on any hardware, usually at a lower compression ratio. The classic criterion is weight **magnitude**, applied iteratively with fine-tuning, as in Algorithm 2.27. The **lottery ticket hypothesis** (Frankle and Carbin, 2019) posits that dense networks contain sparse sub-networks that, when trained from their original initialisation, match the full network's accuracy.

**Algorithm 2.27** Iterative Magnitude Pruning with Fine-Tuning

**Input:** Trained network with weights $W$; target sparsity $s_{final}$; pruning rounds $R$; fine-tuning epochs $E$
**Output:** Sparse network $W \odot M$

$M \leftarrow$ all-ones mask with the shape of $W$
**for** $r = 1$ **to** $R$ **do**
  $s_r \leftarrow s_{final} \cdot r / R$ // gradually increase sparsity
  $\tau \leftarrow$ the $s_r$-quantile of $\{ |w| : w \in W, M = 1 \}$ (global **or** per layer)
  $M \leftarrow M \odot \mathbb{1}[\, |W| > \tau \,]$ // remove smallest-magnitude weights
  $W \leftarrow W \odot M$
  fine-tune $W$ **for** $E$ epochs, re-applying the mask after **each** update
**return** $W \odot M$

Figure 2.67 shows the typical relationship between sparsity and accuracy: accuracy is almost flat up to high sparsity when pruning is iterative with fine-tuning, but collapses much earlier with one-shot pruning and no retraining. Table 2.53 compares the techniques, which are complementary: Han, Mao and Dally's (2016) *Deep Compression* combined pruning, quantisation and Huffman coding to shrink VGG-16 by 49×.

**Practice Tip | Choosing a Compression Strategy**

- Start with PTQ to INT8: it is nearly free and often sufficient.
- If a much smaller architecture is required, distil into an efficient student (e.g. a MobileNet).
- Use structured pruning when you need speed on commodity hardware; unstructured pruning when size matters most.
- Always measure latency on the target device; FLOP counts and parameter counts are imperfect proxies.

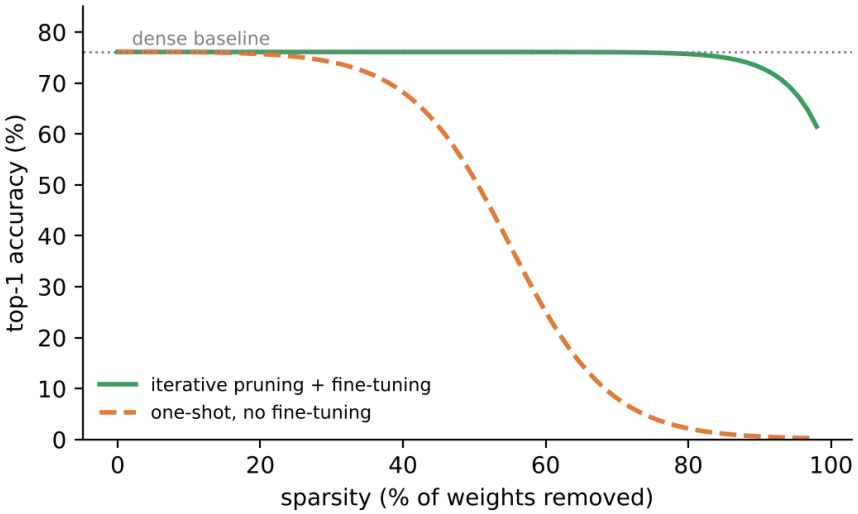



**Figure 2.67** Illustrative accuracy versus sparsity for magnitude pruning of an image classifier: iterative pruning with fine-tuning preserves accuracy to much higher sparsity than one-shot pruning without retraining.

**Table 2.53** Comparison of model-compression techniques.

| Technique | Typical reduction | Retraining needed | Hardware support | Main trade-off |
|---|---|---|---|---|
| Knowledge distillation | 2–10× fewer parameters | Yes (train student) | Any (dense model) | Training cost; teacher needed |
| Post-training quantisation | 4× size (INT8) | No (calibration only) | CPUs, GPUs, NPUs, TPUs | Accuracy drop at low bits |
| Quantisation-aware training | 4–8× size (INT8/INT4) | Yes | Same as PTQ | Longer training pipeline |
| Unstructured pruning | 5–10× non-zeros | Usually (fine-tune) | Needs sparse kernels | Little real speed-up on dense hardware |
| Structured pruning | 1.5–4× FLOPs | Yes | Any | Lower compression ratio |
| Low-rank factorisation | 2–5× in large layers | Usually | Any | Rank selection per layer |

## 2.55 Distributed Training and Parallel Computing

Training a modern deep network on a single accelerator is often too slow or simply impossible. A ResNet-50 on ImageNet takes days on one GPU, and a model with 70 billion parameters stored in 16-bit precision needs 140 GB just for its weights, more than any single GPU's memory, before counting gradients, optimiser states and activations. **Distributed training** spreads the computation across many devices, possibly

on many machines, to reduce wall-clock time (by processing more data in parallel) and to fit models that exceed one device's memory (by partitioning the model). The main strategies are illustrated in Figure 2.68 and compared in Table 2.54.

## 2.55.1 Data Parallelism and All-Reduce

In **data parallelism**, every one of the K workers holds a full replica of the model. Each mini-batch is split into K shards; each worker computes the forward and backward pass on its shard, the gradients are averaged across workers, and every replica applies the same update, so the replicas stay identical. Because the loss is an average over examples, the averaged gradient equals the gradient of the full global batch, as in Equation (2.139).

$$\mathbf{g} = \frac{1}{K}\sum_{k=1}^{K} \mathbf{g}_k = \frac{1}{K}\sum_{k=1}^{K} \frac{1}{|B_k|}\sum_{i \in B_k} \nabla_\theta \ell(\mathbf{x}_i, y_i; \theta) \tag{2.139}$$

where $B_k$ is the shard of the global batch processed by worker $k$ (all shards of equal size) and $g_k$ is that worker's local gradient.

Gradient averaging is implemented with the **all-reduce** collective operation, which leaves every worker holding the sum (or mean) of all workers' vectors. A naive implementation that sends everything to one parameter server creates a bottleneck. The **ring all-reduce** algorithm, popularised for deep learning by Baidu and by Horovod (Sergeev and Del Balso, 2018), arranges the K workers in a ring and splits each gradient vector into K chunks; it consists of a *reduce-scatter* phase followed by an *all-gather* phase, as in Algorithm 2.28. Each worker sends and receives $2(K-1)/K$ times the gradient size in total, which is nearly independent of K, so the communication time is bandwidth-optimal. NVIDIA's NCCL library implements ring and tree all-reduce over NVLink and InfiniBand.

**Algorithm 2.28** Ring All-Reduce

**Input:** K workers arranged in a ring; each worker k holds a vector $g_k$ split into K equal chunks $g_k[0..K-1]$
**Output:** Every worker holds the summed vector $\Sigma_k g_k$

```
// phase 1: reduce-scatter (K − 1 steps)
for s = 0 to K − 2 do
    for each worker k in parallel do
        send chunk c = (k − s) mod K to worker (k + 1) mod K
        receive chunk c′ = (k − s − 1) mod K from worker (k − 1) mod K and add it to its own chunk c′
// now worker k holds the complete sum of chunk (k + 1) mod K
// phase 2: all-gather (K − 1 steps)
```

**for** s = 0 **to** K − 2 **do**
  **for each** worker k **in** parallel **do**
    send its most recently completed summed chunk **to** worker (k + 1) mod K
    receive a completed chunk from worker (k − 1) mod K **and** overwrite its copy
**return** the full summed vector on every worker

Data parallelism scales the effective batch size with the number of workers, which affects optimisation. Goyal et al. (2017) trained ResNet-50 on ImageNet in one hour on 256 GPUs with a batch of 8,192 using the **linear scaling rule** (multiply the learning rate by k when the batch grows by k) together with a gradual warm-up; layer-wise adaptive optimisers such as LARS and LAMB extend this to even larger batches. Communication can be overlapped with computation by launching the all-reduce of each layer's gradients as soon as they are ready during the backward pass, which is what PyTorch's `DistributedDataParallel` (DDP) does with gradient *buckets*.

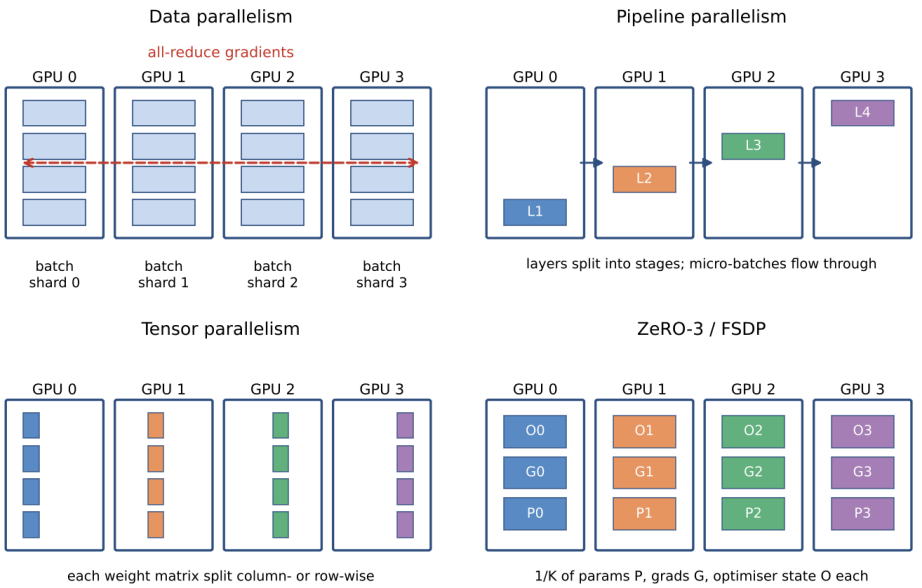



**Figure 2.68** Four forms of parallelism: data parallelism replicates the model and splits the batch; pipeline parallelism splits layers across devices; tensor parallelism splits individual weight matrices; ZeRO/FSDP shards parameters, gradients and optimiser states across data-parallel workers.

## 2.55.2 Model Parallelism: Pipeline and Tensor

When the model does not fit on one device, it must be partitioned. **Pipeline parallelism** assigns consecutive groups of layers (stages) to different devices. Naively, only one device would work at a time; GPipe (Huang et al., 2019) splits each mini-batch into

**micro-batches** that flow through the stages like an assembly line, so that devices work concurrently. Idle time remains at the start and end of each batch (the pipeline **bubble**); with p stages and m micro-batches, the bubble fraction is approximately $(p-1)/(m+p-1)$, so m should be several times p. Schedules such as 1F1B (PipeDream) interleave forward and backward passes to reduce activation memory.

**Tensor parallelism**, introduced for transformers by Megatron-LM (Shoeybi et al., 2019), splits individual matrix multiplications across devices. In a transformer MLP block $Y = \mathrm{GeLU}(XA)B$, the first matrix A is split by columns and the second B by rows, so each device computes its part independently and a single all-reduce combines the results; attention heads are partitioned similarly. Tensor parallelism requires very fast interconnects (NVLink within a node), whereas pipeline parallelism tolerates slower links between nodes. Training the largest models combines all three in **3-D parallelism**: tensor parallelism within a node, pipeline parallelism across nodes, and data parallelism over replicas of the whole arrangement.

**Table 2.54** Comparison of parallelism strategies for deep-learning training.

| Strategy | What is split | Communication | Memory per device | Best for |
|---|---|---|---|---|
| Data parallel (DDP) | Batch | All-reduce of gradients each step | Full model + optimiser | Models that fit on one GPU |
| ZeRO-1/2 | Optimiser state (+ gradients) | Reduce-scatter + all-gather | Reduced by up to ~8× | Medium-large models |
| ZeRO-3 / FSDP | Params, grads, optimiser state | All-gather params per layer | $\approx 1/K$ of model states | Large models, many GPUs |
| Pipeline parallel | Layers (stages) | Activations between stages | $1/p$ of layers | Very deep models across nodes |
| Tensor parallel | Weight matrices | All-reduce inside each layer | $1/t$ of each layer | Huge layers within a node |

## 2.55.3 ZeRO and Fully Sharded Data Parallelism

Plain data parallelism is memory-wasteful because every worker stores identical copies of the parameters, gradients and optimiser states. With mixed-precision Adam, each parameter needs 2 bytes (FP16 weights) + 2 bytes (FP16 gradients) + 12 bytes (FP32 master weights, momentum and variance) = 16 bytes, so a 7.5-billion-parameter model needs 120 GB of model state per GPU. The **Zero Redundancy Optimizer (ZeRO)** (Rajbhandari et al., 2020) removes this redundancy in three stages, as quantified in Equation (2.140): stage 1 shards the optimiser states across the K workers, stage 2 also shards the gradients, and stage 3 also shards the parameters themselves, gathering each layer's full weights just in time for its forward and backward computation and discarding them afterwards.

$$M_{\text{DDP}} = (2 + 2 + 12)\,\Psi = 16\,\Psi$$
$$M_{\text{ZeRO 3}} = \frac{(2+2+12)\,\Psi}{K} = \frac{16\,\Psi}{K} \tag{2.140}$$

where $\Psi$ is the number of parameters and K the number of data-parallel workers; activation memory is additional. With $\Psi = 7.5 \times 10^9$ and K = 64, ZeRO-3 needs about 1.9 GB of model state per GPU instead of 120 GB.

PyTorch's **Fully Sharded Data Parallel** (**FSDP**) implements the ZeRO-3 idea natively, and DeepSpeed provides all ZeRO stages plus CPU and NVMe offloading. Memory for activations can be reduced further with **activation checkpointing** (gradient checkpointing), which stores only some activations in the forward pass and recomputes the rest during back-propagation, trading roughly 30% extra compute for a large memory saving.

### 2.55.4 Mixed-Precision Training

**Mixed-precision training** (Micikevicius et al., 2018) performs most computation in 16-bit floating point while keeping a 32-bit master copy of the weights. Tensor cores on modern GPUs execute FP16/BF16 matrix multiplications several times faster than FP32, and halving activation size halves memory traffic. FP16 has a narrow exponent range, so small gradients can underflow to zero; **loss scaling** multiplies the loss by a factor S (e.g. $2^{16}$) before back-propagation and divides the gradients by S before the update, with *dynamic* loss scaling reducing S when overflow is detected. **BF16** (bfloat16) keeps FP32's 8-bit exponent with a shorter mantissa, so it rarely needs loss scaling and is now the default on TPUs and recent NVIDIA GPUs. Listing 2.47 combines DDP with automatic mixed precision.

**Listing 2.47** Data-parallel training with PyTorch DDP and automatic mixed precision (launch with torchrun).

```
import os, torch, torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, DistributedSampler

dist.init_process_group("nccl")                       # one process per GPU
rank = int(os.environ["LOCAL_RANK"]); torch.cuda.set_device(rank)

model = build_model().cuda(rank)
model = DDP(model, device_ids=[rank])                 # gradients all-reduced
↪ automatically
sampler = DistributedSampler(train_set)               # disjoint shard per process
loader = DataLoader(train_set, batch_size=64, sampler=sampler, num_workers=4)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
scaler = torch.cuda.amp.GradScaler()                  # dynamic loss scaling for FP16

for epoch in range(10):
```

```
    sampler.set_epoch(epoch)                                # reshuffle shards each epoch
    for x, y in loader:
        x, y = x.cuda(rank, non_blocking=True), y.cuda(rank, non_blocking=True)
        with torch.autocast("cuda", dtype=torch.float16):
            loss = torch.nn.functional.cross_entropy(model(x), y)
        opt.zero_grad(set_to_none=True)
        scaler.scale(loss).backward()
        scaler.step(opt); scaler.update()
dist.destroy_process_group()
# run: torchrun --nproc_per_node=4 train.py
```

**Practice Tip | Scaling Training in Practice**

- Before distributing, make one GPU efficient: mixed precision, an input pipeline that keeps the GPU busy, and the largest batch that fits.
- Use DDP when the model fits on one device; move to FSDP/ZeRO when it does not; add tensor and pipeline parallelism only for very large models.
- When increasing the global batch, scale the learning rate and add warm-up; check that validation accuracy matches the single-GPU baseline.

# 2.56 GPU and TPU Computing for Deep Learning

The deep learning revolution was as much a hardware story as an algorithmic one. AlexNet (2012) was trained on two NVIDIA GTX 580 GPUs, and the subsequent growth in model size has tracked advances in specialised accelerators. Deep networks are dominated by dense linear algebra (matrix multiplications and convolutions) in which the same operation is applied to millions of numbers independently, exactly the kind of **data-parallel**, high-arithmetic workload that throughput-oriented hardware handles well. Understanding the hardware helps practitioners choose batch sizes, data types and model shapes that run efficiently.

## 2.56.1 CPUs, GPUs and TPUs

A **CPU** has a few powerful cores optimised for low-latency execution of sequential, branch-heavy code, with large caches and sophisticated control logic. A **GPU** devotes most of its silicon to thousands of simpler arithmetic units organised into *streaming multiprocessors* (*SMs*), executing groups of 32 threads (*warps*) in lock-step; it hides memory latency by switching between many warps rather than by large caches. Since the Volta generation (2017), NVIDIA GPUs include **tensor cores** that perform small matrix

multiply–accumulate operations in FP16, BF16, TF32, FP8 or INT8 in a single instruction. Google's **Tensor Processing Unit** (**TPU**) (Jouppi et al., 2017) is an application-specific chip built around a large **systolic array**, a grid of multiply–accumulate cells (128 × 128 in recent generations) through which operands flow rhythmically so that each value fetched from memory is reused many times. TPUs are connected in *pods* by high-speed interconnects and programmed through the XLA compiler from TensorFlow, JAX or PyTorch/XLA. Table 2.55 compares the three.

**Table 2.55** Comparison of CPU, GPU and TPU for deep-learning workloads.

| Property | CPU | GPU | TPU |
|---|---|---|---|
| Design goal | Low latency, general purpose | High throughput, parallel | Matrix-multiply throughput for ML |
| Cores / units | Tens of complex cores | Thousands of simple cores + tensor cores | Large systolic arrays (MXUs) |
| Memory | DDR, large caches | HBM (up to ~80–192 GB), ~2–8 TB/s | HBM, high bandwidth |
| Precision | FP64/FP32 (some BF16/INT8) | FP32, TF32, FP16, BF16, FP8, INT8 | BF16, INT8 (FP32 accumulate) |
| Programming | Any language | CUDA, ROCm; via PyTorch/TF/JAX | XLA via JAX, TensorFlow, PyTorch/XLA |
| Strengths | Preprocessing, small models, flexibility | Flexible, mature ecosystem | Efficiency at large scale |
| Weaknesses | Low throughput for dense maths | Power, cost, memory limits | Less flexible; cloud-only access |

## 2.56.2 CUDA and the GPU Programming Model

**CUDA**, released by NVIDIA in 2007, exposes the GPU as a massively parallel co-processor. A programmer writes a **kernel**, a function executed by many threads; threads are grouped into **blocks** that run on one SM and can share fast on-chip memory, and blocks form a **grid**. Each thread computes its global index from its block and thread coordinates and processes the corresponding data element. Deep learning frameworks rarely require users to write CUDA directly: they call optimised libraries such as **cuBLAS** (dense linear algebra) and **cuDNN** (convolutions, normalisation, attention). Listing 2.48 shows a simple element-wise kernel written in Python with Numba's CUDA support, and how PyTorch places tensors on a GPU.

**Listing 2.48** A CUDA kernel written in Python with Numba, and GPU tensors in PyTorch.

```
import numpy as np
from numba import cuda

@cuda.jit
def saxpy(a, x, y, out):                # out = a * x + y, one thread per element
    i = cuda.grid(1)                    # = blockIdx.x * blockDim.x + threadIdx.x
```

```
    if i < x.size:
        out[i] = a * x[i] + y[i]

n = 1_000_000
x = cuda.to_device(np.random.rand(n).astype(np.float32))
y = cuda.to_device(np.random.rand(n).astype(np.float32))
out = cuda.device_array_like(x)
threads = 256
blocks = (n + threads - 1) // threads
saxpy[blocks, threads](np.float32(2.0), x, y, out)
print(out.copy_to_host()[:5])

# In PyTorch the same idea is one line; the kernel is launched for you
import torch
dev = "cuda" if torch.cuda.is_available() else "cpu"
a = torch.rand(4096, 4096, device=dev); b = torch.rand(4096, 4096, device=dev)
c = a @ b                                  # dispatched to cuBLAS (tensor cores if
↪ eligible)
```

### 2.56.3 Memory Hierarchy and the Roofline Model

On modern accelerators, arithmetic is cheap and moving data is expensive. Figure 2.69 shows the GPU memory hierarchy: a small number of very fast registers per thread, on-chip shared memory and L1 cache per SM (hundreds of kilobytes), a shared L2 cache (tens of megabytes) and off-chip **high-bandwidth memory (HBM)** of tens of gigabytes. Each level down is larger but slower; host memory, reached over PCIe, is slower still. Efficient kernels **tile** computations so that data loaded from HBM into shared memory is reused many times before being evicted.

Whether an operation is limited by computation or by memory traffic is captured by the **roofline model** (Williams, Waterman and Patterson, 2009) in Equation (2.141). The **arithmetic intensity** $I$ of a kernel is the number of floating-point operations performed per byte moved from memory. Large matrix multiplications have high intensity and are *compute-bound*; element-wise operations such as activations, normalisation and residual additions have low intensity and are *memory-bound*. This is why **kernel fusion** (combining several element-wise operations into one kernel, as done by `torch.compile` and XLA) and IO-aware algorithms such as **FlashAttention** (Dao et al., 2022), which computes attention in tiles held in shared memory without materialising the full attention matrix in HBM, yield large speed-ups.

$$P_{\text{attainable}} = \min\left(P_{\text{peak}},\; I \times \beta\right), \qquad I = \frac{\text{FLOPs}}{\text{bytes moved}} \tag{2.141}$$

where $P_{\text{peak}}$ is the peak arithmetic throughput (FLOP/s), $\beta$ is the memory bandwidth (bytes/s) and $I$ is the arithmetic intensity (FLOP/byte).

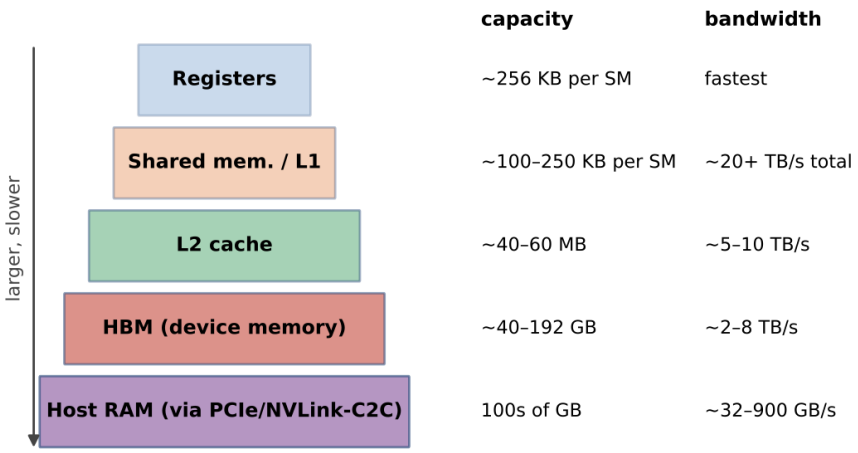



**Figure 2.69** GPU memory hierarchy: capacity increases and bandwidth decreases from registers to host memory; efficient kernels keep data in the upper levels as long as possible.

**E Worked Example 2.27 | Compute- or Memory-Bound?**

Consider a GPU with $P_{\text{peak}} = 300$ TFLOP/s (BF16) and $\beta = 2$ TB/s. The *ridge point* is $300/2 = 150$ FLOP/byte.

- Multiplying two $4096 \times 4096$ BF16 matrices: FLOPs $= 2 \cdot 4096^3 \approx 1.37 \times 10^{11}$; bytes $\approx 3 \times 4096^2 \times 2 \approx 1.0 \times 10^8$; $I \approx 1365$ FLOP/byte $> 150$, so it is compute-bound.
- Adding two vectors of $10^8$ BF16 values: $10^8$ FLOPs; $3 \times 10^8 \times 2 = 6 \times 10^8$ bytes; $I \approx 0.17$ FLOP/byte, deeply memory-bound, attaining at most $0.17 \times 2$ TB/s $\approx 0.33$ TFLOP/s.

**T Practice Tip | Getting Good Hardware Utilisation**

- Use mixed precision (BF16/FP16) so tensor cores are used; keep matrix dimensions multiples of 8 (or 64).
- Make sure the data loader is not the bottleneck: use multiple workers, pinned memory and prefetching.
- Profile (PyTorch Profiler, NVIDIA Nsight) before optimising; watch GPU utilisation with `nvidia-smi`.
- Try `torch.compile` or XLA to fuse memory-bound operations.

## 2.57 Deep Learning Frameworks: TensorFlow, Keras, and PyTorch

Deep learning frameworks provide three essential services: **tensor operations** on CPUs and accelerators, **automatic differentiation** (the reverse-mode backpropagation described in the section on backpropagation), and **building blocks** such as layers, losses, optimisers, data pipelines and model serialisation. The history of frameworks runs from Theano (2007) and Caffe (2013) through Google's **TensorFlow** (2015) and the high-level **Keras** API (Chollet, 2015) to Facebook's **PyTorch** (2016, building on the Lua-based Torch), with Google's **JAX** (2018) as a newer functional alternative. Today PyTorch dominates research and much of industry, TensorFlow remains strong in production and mobile deployment, and Keras 3 (2023) is multi-backend, running on TensorFlow, JAX or PyTorch.

### 2.57.1 Static and Dynamic Computation Graphs

The most important historical design difference concerns the **computation graph**. TensorFlow 1.x used a **define-and-run** (static graph) model: users first built a symbolic graph and then executed it in a session, which enabled whole-graph optimisation but made debugging awkward. PyTorch introduced **define-by-run** (eager, dynamic graph) execution: operations run immediately as ordinary Python, and the autograd engine records them on the fly so that control flow such as loops and conditionals works naturally. The two approaches have since converged: TensorFlow 2 is eager by default and compiles functions to graphs with `tf.function`, while PyTorch offers graph capture and compilation through TorchScript and, since PyTorch 2.0, `torch.compile`. Table 2.56 compares TensorFlow/Keras, PyTorch and JAX across the main dimensions.

**Table 2.56** Comparison of the main deep-learning frameworks.

| Aspect | TensorFlow 2 / Keras | PyTorch | JAX |
|---|---|---|---|
| Developer / release | Google, 2015 (TF2 2019) | Meta, 2016 | Google, 2018 |
| Execution | Eager + tf.function graphs | Eager + torch.compile | Functional; jit via XLA |
| API style | High-level Keras (Sequential, Functional, subclassing) | Pythonic nn.Module + explicit loop | Pure functions; Flax/Equinox for layers |
| Training loop | model.fit() built in | Written by user (or Lightning) | Written by user (Optax) |
| Deployment | TF Serving, TFLite, TF.js | TorchScript, ExecuTorch, ONNX, TorchServe | Export via TF or ONNX |
| Distributed | tf.distribute strategies | DDP, FSDP, DeepSpeed | pmap / pjit, sharding |
| Ecosystem strengths | Production, mobile, TPU | Research, Hugging Face, community | Large-scale TPU research |

### 2.57.2 The Same Model in Keras and PyTorch

The difference in style is best seen by implementing the same model twice. Listing 2.49 and Listing 2.50 both define a multilayer perceptron with two hidden layers of 256 and 128 units, ReLU activations, dropout of 0.2 and a 10-class output, and train it on MNIST with Adam for five epochs. In Keras, compiling and fitting are handled by the framework; in PyTorch the training loop is explicit, which is more verbose but gives full control over every step.

**Listing 2.49** An MLP classifier for MNIST in Keras.

```
import tensorflow as tf
from tensorflow import keras

(x_tr, y_tr), (x_te, y_te) = keras.datasets.mnist.load_data()
x_tr = x_tr.reshape(-1, 784).astype("float32") / 255.0
x_te = x_te.reshape(-1, 784).astype("float32") / 255.0

model = keras.Sequential([
    keras.Input(shape=(784,)),
    keras.layers.Dense(256, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10),                      # logits
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
model.fit(x_tr, y_tr, batch_size=128, epochs=5, validation_split=0.1)
print(model.evaluate(x_te, y_te, verbose=0))
```

**Listing 2.50** The same MLP classifier for MNIST in PyTorch, with an explicit training loop.

```
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

dev = "cuda" if torch.cuda.is_available() else "cpu"
tf = transforms.Compose([transforms.ToTensor(), transforms.Lambda(lambda t:
    t.view(-1))])
train = DataLoader(datasets.MNIST(".", train=True, download=True, transform=tf),
    batch_size=128, shuffle=True)
test = DataLoader(datasets.MNIST(".", train=False, download=True, transform=tf),
    batch_size=512)

model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.2),
                      nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.2),
                      nn.Linear(128, 10)).to(dev)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
```

```
for epoch in range(5):
    model.train()
    for x, y in train:
        x, y = x.to(dev), y.to(dev)
        opt.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        opt.step()
    model.eval(); correct = 0
    with torch.no_grad():
        for x, y in test:
            correct += (model(x.to(dev)).argmax(1) == y.to(dev)).sum().item()
    print(f"epoch {epoch+1}: test acc = {correct / len(test.dataset):.4f}")
```

Several details deserve attention. Both versions output raw **logits** and use a loss that applies the softmax internally, which is numerically more stable. In PyTorch, `model.train()` and `model.eval()` switch dropout and batch normalisation between training and inference behaviour, and `torch.no_grad()` disables gradient tracking during evaluation to save memory; Keras handles these automatically inside `fit` and `evaluate`. Gradients in PyTorch accumulate by default, so `zero_grad()` must be called each step.

### 2.57.3 Choosing a Framework

For most learners and practitioners the choice matters less than it once did: concepts transfer directly, models can be exchanged via ONNX, and the Hugging Face ecosystem supports both. Keras is excellent for rapid prototyping and teaching; PyTorch is the default in research and for state-of-the-art open models; TensorFlow offers mature mobile and browser deployment; JAX suits large-scale numerical research on TPUs. Higher-level libraries such as PyTorch Lightning, fastai and Hugging Face `Trainer` reduce boilerplate in PyTorch.

**Key Points | Frameworks**

- All major frameworks provide tensors on accelerators, reverse-mode autodiff and reusable layers.
- Eager execution aids debugging; graph compilation (tf.function, torch.compile, jax.jit) aids speed and deployment.
- Keras trades flexibility for brevity; PyTorch's explicit loop gives full control.

## 2.58 Deploying Deep Learning Models

A model delivers value only when it is used. **Deployment** is the process of making a trained model available to make predictions (inference) within a product or business process, under real constraints on latency, throughput, cost, memory, privacy and reliability. Deployment differs from training in important ways: inference must often respond within milliseconds, run on hardware very different from the training cluster, and operate continuously for months while data changes around it (the subject of the next section). This section covers model export formats, serving systems, edge versus cloud deployment and a minimal REST API.

### 2.58.1 Exporting Models: TorchScript, SavedModel and ONNX

Training code in Python is not ideal for production, where one may want to run without a Python interpreter, on a mobile phone, or in an optimised runtime. Models are therefore **exported** to a serialised format that captures both the computation graph and the weights. **TorchScript** converts a PyTorch model to an intermediate representation either by *tracing* (recording the operations executed for an example input, which misses data-dependent control flow) or by *scripting* (compiling the Python source); newer PyTorch versions provide `torch.export` for the same purpose. TensorFlow's **SavedModel** format stores graphs and variables for TF Serving and conversion to TFLite. **ONNX** (Open Neural Network Exchange, 2017) is a framework-neutral open format; an ONNX model exported from PyTorch can run in **ONNX Runtime**, NVIDIA **TensorRT**, OpenVINO and other optimised engines on a variety of hardware. Listing 2.51 exports a model to TorchScript and ONNX.

**Listing 2.51** Exporting a PyTorch model to TorchScript and ONNX and running it with ONNX Runtime.

```python
import torch, torchvision
import numpy as np
import onnxruntime as ort

model = torchvision.models.resnet18(weights="DEFAULT").eval()
example = torch.randn(1, 3, 224, 224)

ts = torch.jit.trace(model, example)          # TorchScript (tracing)
ts.save("resnet18.pt")

torch.onnx.export(model, example, "resnet18.onnx",
                  input_names=["image"], output_names=["logits"],
                  dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}},
                  opset_version=17)

sess = ort.InferenceSession("resnet18.onnx", providers=["CPUExecutionProvider"])
out = sess.run(None, {"image": example.numpy()})[0]
print(np.abs(out - model(example).detach().numpy()).max())   # should be ~1e-5
```

### 2.58.2 Serving Patterns: Cloud and Edge

Inference may happen in the cloud or on the edge. In **cloud deployment** the model runs on servers behind an API; this allows large models, centralised updates and powerful GPUs, at the cost of network latency, per-request cost and the need to send user data off-device. Dedicated **model servers** such as TensorFlow Serving, TorchServe, NVIDIA Triton Inference Server and Ray Serve provide versioning, batching of concurrent requests (*dynamic batching*), GPU sharing, metrics and gRPC/REST endpoints. Predictions may be served **online** (one request at a time, low latency) or computed in **batch** (for example, nightly scoring of all customers).

In **edge deployment** the model runs on the user's device or local hardware: phones, browsers, cameras, cars, microcontrollers. This offers low latency, offline operation and privacy, but the model must be small and efficient, which is where the compression techniques of the earlier section become essential. **TensorFlow Lite** (now LiteRT) converts models into a compact FlatBuffer format with optional INT8 quantisation and hardware delegates for mobile GPUs and NPUs; **Core ML** targets Apple devices; **ExecuTorch** and PyTorch Mobile target on-device PyTorch; **TensorFlow.js** and ONNX Runtime Web run models in the browser. Table 2.57 contrasts the two settings, and Figure 2.70 shows a typical cloud serving architecture.

**Table 2.57** Edge versus cloud deployment.

| Criterion | Cloud deployment | Edge deployment |
|---|---|---|
| Latency | Network round trip (tens–hundreds of ms) | Very low; no network needed |
| Model size | Large models feasible | Must be small (MBs); quantised/pruned |
| Privacy | Data leaves the device | Data can stay on device |
| Updates | Instant, centralised | Requires app/firmware updates |
| Cost | Pay per server/GPU hour | Uses customers' hardware |
| Typical tools | TF Serving, TorchServe, Triton, FastAPI, Kubernetes | TFLite/LiteRT, Core ML, ExecuTorch, ONNX Runtime Mobile |

### 2.58.3 Serving a Model with FastAPI

For many projects, a lightweight Python web service is sufficient. Listing 2.52 wraps an image classifier in a **FastAPI** application: the model is loaded once at start-up rather than per request, inputs are validated, inference runs under `torch.inference_mode()`, and a health-check endpoint supports orchestration systems. The service would typically be packaged in a **Docker** container and run behind a production server such as Uvicorn with several workers, or deployed on Kubernetes or a managed platform.

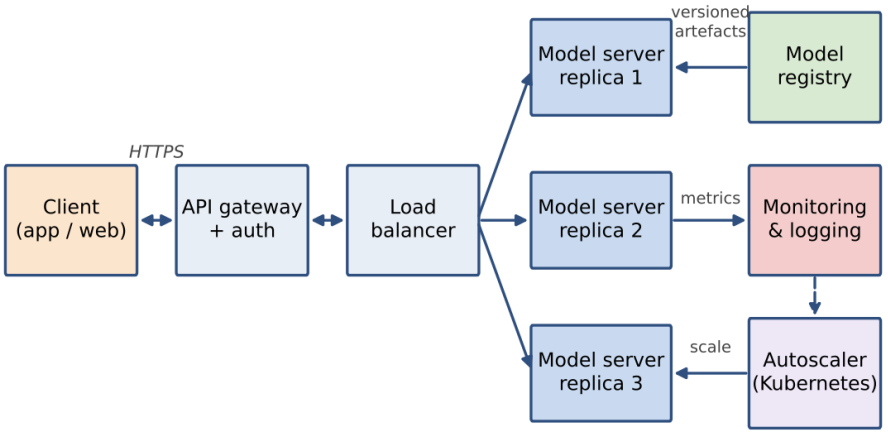



**Figure 2.70** A typical cloud serving architecture: requests pass through a gateway and load balancer to replicated model servers, which load versioned models from a registry; monitoring collects metrics that drive autoscaling.

**Listing 2.52** A minimal FastAPI inference service for an image classifier.

```python
import io, torch, torchvision.transforms as T
from fastapi import FastAPI, File, UploadFile, HTTPException
from PIL import Image

app = FastAPI(title="Image classifier")
model = torch.jit.load("resnet18.pt").eval()            # load once at start-up
CLASSES = [l.strip() for l in open("imagenet_classes.txt")]
prep = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToTensor(),
                  T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])])

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    try:
        img = Image.open(io.BytesIO(await file.read())).convert("RGB")
    except Exception:
        raise HTTPException(status_code=400, detail="invalid image")
    with torch.inference_mode():
        probs = model(prep(img).unsqueeze(0)).softmax(1)[0]
    p, idx = probs.topk(3)
    return {"top3": [{"label": CLASSES[i], "prob": round(v, 4)} for v, i in
    ↪  zip(p.tolist(), idx.tolist())]}

# run with: uvicorn app:app --host 0.0.0.0 --port 8000 --workers 2
```

**Note | Training–Serving Skew**

A frequent cause of silent failure is a mismatch between preprocessing at training and at serving time: a different image resize method, normalisation constants, tokenizer version or feature definition. Package preprocessing with the model (for example, as layers inside a Keras model or in the exported graph), and test the deployed endpoint on held-out examples whose expected outputs are known.

**Practice Tip | Deployment Checklist**

- Define latency, throughput and cost targets before choosing the serving stack.
- Export, then verify numerical agreement with the original model.
- Load-test the endpoint; use dynamic batching on GPUs.
- Version models and roll out gradually (shadow, canary or A/B deployment) with the ability to roll back.

# 2.59 MLOps, Model Monitoring, and Model Maintenance

Surveys of industrial practice repeatedly find that most machine-learning projects never reach production, and that those which do often degrade silently afterwards. Sculley et al. (2015) described the problem memorably as **hidden technical debt in machine learning systems**: the model code is a small fraction of a real system, surrounded by much larger infrastructure for data collection, feature extraction, verification, serving, monitoring and configuration. **MLOps** (machine-learning operations) applies the principles of DevOps (automation, version control, continuous integration and delivery, monitoring) to this whole system, with the additional complication that an ML system's behaviour depends on data as well as code.

**Definition 2.13 | MLOps**

A set of practices, tools and organisational roles for reliably and efficiently building, deploying, monitoring and maintaining machine-learning models in production, treating data, code and models as versioned artefacts within automated, reproducible pipelines.

## 2.59.1 The ML Lifecycle

Unlike conventional software, an ML model is never finished. The lifecycle in Figure 2.71 is a loop: data is collected and validated, models are trained and evaluated, the best model is registered and deployed, its behaviour in production is monitored, and

monitoring signals (drift, performance decay, new labelled data) trigger retraining. Each stage produces artefacts that must be versioned so that any prediction can be traced back to the exact data, code, hyperparameters and environment that produced the model.

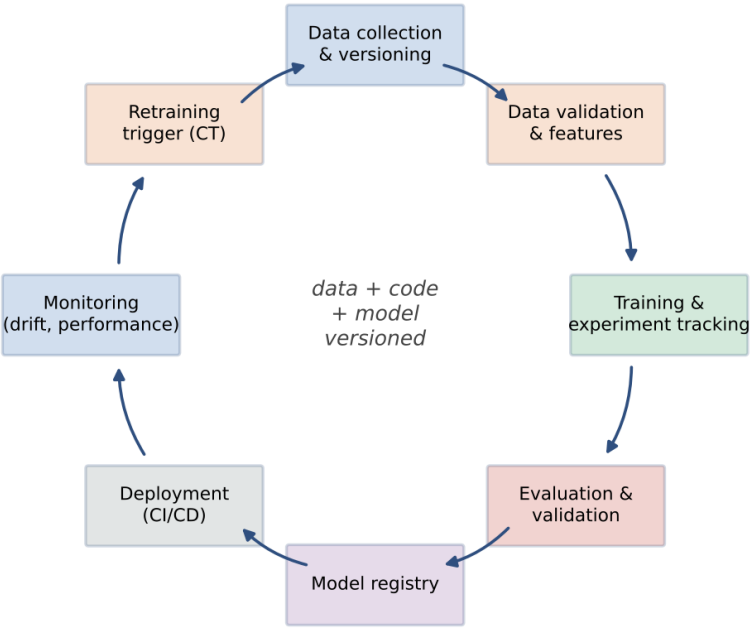



**Figure 2.71** The MLOps lifecycle: a continuous loop in which production monitoring feeds back into data collection and retraining.

## 2.59.2 Data Drift, Concept Drift and Their Detection

A deployed model assumes that future data resembles its training data. When this assumption fails, accuracy decays. It is useful to distinguish types of change in the joint distribution $P(x, y) = P(y \mid x)\,P(x)$. **Data drift** (covariate shift) is a change in the input distribution $P(x)$, for example a new camera model producing brighter images or a shift in the age profile of customers. **Concept drift** is a change in the relationship $P(y \mid x)$ itself: fraudsters change tactics, so the same transaction features now indicate a different risk. **Label (prior) shift** is a change in $P(y)$, such as a rise in disease prevalence during an epidemic. Drift may be sudden, gradual, incremental or recurring (seasonal).

Because true labels often arrive late or never, monitoring relies on several signals: model performance where labels are available; the distribution of predictions (a proxy for label shift); and the distribution of each input feature compared with a reference

(training) window. Common statistics include the Kolmogorov–Smirnov test for continuous features, the $\chi^2$ test for categorical ones, the Jensen–Shannon divergence and, very widely in industry, the **population stability index (PSI)** defined in Equation (2.142). For images and text, drift is usually measured on embeddings from the model or a pretrained encoder, for example with the maximum mean discrepancy. Listing 2.53 computes PSI with quantile bins and illustrates its response to shifts in mean and variance.

$$\mathrm{PSI} = \sum_{b=1}^{B} (a_b - e_b) \ln \frac{a_b}{e_b} \tag{2.142}$$

where the feature's range is divided into B bins (often deciles of the reference data), $e_b$ is the proportion of reference (expected) observations in bin b and $a_b$ the proportion of current (actual) observations. A common rule of thumb: PSI $< 0.1$ indicates no significant shift, 0.1–0.25 a moderate shift, and $> 0.25$ a major shift requiring investigation.

**E Worked Example 2.28 | Population Stability Index**

A model input is binned into four ranges. Reference proportions e $= (0.25, 0.25, 0.25, 0.25)$; current proportions a $= (0.10, 0.20, 0.30, 0.40)$.

- Bin 1: $(0.10 - 0.25)\ln(0.10/0.25) = (-0.15)(-0.916) = 0.137$.
- Bin 2: $(0.20 - 0.25)\ln(0.20/0.25) = (-0.05)(-0.223) = 0.011$.
- Bin 3: $(0.30 - 0.25)\ln(0.30/0.25) = (0.05)(0.182) = 0.009$.
- Bin 4: $(0.40 - 0.25)\ln(0.40/0.25) = (0.15)(0.470) = 0.071$.

PSI $= 0.137 + 0.011 + 0.009 + 0.071 = 0.228$: a moderate shift, close to the 0.25 action threshold.

**Listing 2.53** Computing PSI for a feature with NumPy, using reference-data quantile bins.

```python
import numpy as np

def psi(reference, current, bins=10, eps=1e-6):
    edges = np.quantile(reference, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf                # capture out-of-range values
    e = np.histogram(reference, edges)[0] / len(reference)
    a = np.histogram(current, edges)[0] / len(current)
    e, a = np.clip(e, eps, None), np.clip(a, eps, None)   # avoid log(0)
    return float(np.sum((a - e) * np.log(a / e)))

rng = np.random.default_rng(0)
ref = rng.normal(50, 10, 10_000)                         # training-time feature
print(f"no drift:   {psi(ref, rng.normal(50, 10, 5_000)):.3f}")
print(f"mean shift: {psi(ref, rng.normal(56, 10, 5_000)):.3f}")
print(f"var change: {psi(ref, rng.normal(50, 16, 5_000)):.3f}")
```

### 2.59.3 CI/CD/CT Pipelines and Tooling

MLOps extends software **continuous integration** (**CI**) and **continuous delivery** (**CD**) with **continuous training** (**CT**). CI for ML tests not only code but also data (schema checks, value ranges, missing values) and models (minimum accuracy on a golden test set, fairness checks, latency budgets). CD automatically packages a validated model, typically as a container, and rolls it out with safe strategies: **shadow deployment** (the new model receives real traffic but its predictions are not used), **canary release** (a small percentage of traffic first) or **A/B testing**. CT automatically retrains the model on a schedule or when monitoring detects drift, then passes the candidate through the same validation gates before promotion. Algorithm 2.29 outlines such a pipeline.

**Algorithm 2.29** Continuous Training with Drift-Triggered Retraining

**Input:** Production model $M_{prod}$; reference data $D_{ref}$; drift threshold $\kappa$; performance floor $a_{min}$; schedule S
**Output:** Possibly updated production model

loop every monitoring window W **do**
  $D_{cur} \leftarrow$ inputs (**and** any delayed labels) logged during W
  drift $\leftarrow$ max over features f of PSI($D_{ref}[f]$, $D_{cur}[f]$)
  perf $\leftarrow$ accuracy of $M_{prod}$ on newly labelled data **in** W (**if** available)
  **if** drift $> \kappa$ **or** perf $< a_{min}$ **or** schedule S is due **then**
    D_train $\leftarrow$ versioned snapshot of recent labelled data
    validate D_train (schema, ranges, label distribution)
    $M_{new} \leftarrow$ train(D_train) with tracked hyperparameters
    **if** $M_{new}$ beats $M_{prod}$ on the hold-out set **and** passes fairness/latency tests **then**
      register $M_{new}$; deploy as canary; monitor
      **if** canary metrics are acceptable **then** $M_{prod} \leftarrow M_{new}$; $D_{ref} \leftarrow$ D_train
    **else** raise an alert **for** human review

A rich tool ecosystem supports these practices: **DVC** and lakeFS for data versioning; **MLflow** and Weights & Biases for experiment tracking and model registries; **Kubeflow**, TFX, Airflow and ZenML for pipeline orchestration; **Feast** for feature stores that guarantee consistent features at training and serving time; and Evidently, WhyLabs or Prometheus with Grafana for monitoring. Managed platforms such as Amazon SageMaker, Google Vertex AI and Azure Machine Learning bundle many of these capabilities.

### 2.59.4 MLOps Maturity Levels

Google's widely cited MLOps guidance describes three maturity levels, summarised in Table 2.58. Most organisations begin at level 0, where data scientists hand a trained model file to engineers; the goal is to reach level 1, where the *training pipeline* itself,

rather than a model, is deployed and runs automatically; and eventually level 2, where the pipelines are also built, tested and deployed through CI/CD.

**Table 2.58** MLOps maturity levels (after Google Cloud's MLOps guidance).

| Level | Characteristics | Deployment unit | Retraining | Typical risk |
|---|---|---|---|---|
| 0: Manual process | Notebook-driven, manual hand-off, no monitoring | A model file | Rare, manual | Silent decay; irreproducible results |
| 1: ML pipeline automation | Automated training pipeline, feature store, metadata tracking, monitoring | Training pipeline | Continuous training on triggers | Pipeline changes still manual |
| 2: CI/CD pipeline automation | Pipelines themselves built, tested and deployed automatically | Pipeline via CI/CD | Automated, frequent | Organisational and tooling complexity |

**T Practice Tip | What to Monitor in Production**

- **Service health:** latency percentiles (p50, p95, p99), throughput, error rates, resource usage.
- **Data quality:** schema violations, missing values, out-of-range values, feature drift (PSI, KS).
- **Model quality:** prediction distribution, confidence, accuracy or business KPIs when labels arrive, and performance per subgroup for fairness.
- Set alert thresholds and define, in advance, who responds and how a model is rolled back.

## 2.60 Practical Deep Learning Projects Using Python and Google Colab

This section brings together many ideas from the chapter in a complete, end-to-end project that readers can run for free in **Google Colaboratory** (**Colab**), a hosted Jupyter notebook environment that provides Python, the major deep-learning libraries preinstalled and time-limited access to GPUs and TPUs. The project is to build an image classifier for the **CIFAR-10** data set (60,000 colour images of $32 \times 32$ pixels in ten classes: aeroplane, car, bird, cat, deer, dog, frog, horse, ship and truck), using PyTorch. We follow the workflow in Figure 2.72: set up the environment, load and augment the data, define a CNN, train it with good practices, evaluate it thoroughly, and save it for deployment.

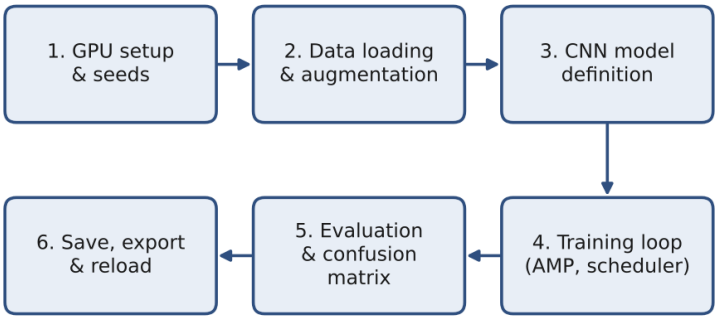



**Figure 2.72** Workflow of the end-to-end Colab project, with the listing that implements each stage.

## 2.60.1 Step 1: Environment and GPU Setup

In Colab, select *Runtime* → *Change runtime type* → *GPU* before running any code. Listing 2.54 checks that a GPU is visible, prints its type, mounts Google Drive so that checkpoints survive the end of the session (Colab virtual machines are reset after inactivity or a maximum lifetime), and fixes random seeds for reproducibility. Full determinism on GPUs also requires deterministic cuDNN algorithms, which can be slower, so it is usually enabled only when exact reproducibility is essential.

**Listing 2.54** Colab environment setup: GPU check, Google Drive mount and seeding.

```
!nvidia-smi --query-gpu=name,memory.total --format=csv   # shell command in Colab
import os, random, numpy as np, torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch", torch.__version__, "| device:", device,
      "|", torch.cuda.get_device_name(0) if device.type == "cuda" else "")

from google.colab import drive                # persist checkpoints across sessions
drive.mount("/content/drive")
SAVE_DIR = "/content/drive/MyDrive/cifar10_project"
os.makedirs(SAVE_DIR, exist_ok=True)

def set_seed(seed=42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True      # fastest conv algorithms
    ↪ (non-deterministic)

set_seed(42)
```

### 2.60.2 Step 2: Data Loading and Augmentation

Listing 2.55 downloads CIFAR-10 with torchvision, splits off 5,000 training images as a validation set, and builds data loaders. Training images are augmented with random crops (after 4-pixel padding) and horizontal flips, the standard recipe for CIFAR, which acts as a regulariser as described in the section on regularisation. All images are normalised with the per-channel mean and standard deviation of the training set. Validation and test images are *not* augmented. Note the subtlety that the validation subset must use the non-augmented transform, which is why the data set is loaded twice.

**Listing 2.55** Loading, splitting and augmenting CIFAR-10 with torchvision.

```
import torchvision, torchvision.transforms as T
from torch.utils.data import DataLoader, Subset

MEAN, STD = (0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)
train_tf = T.Compose([T.RandomCrop(32, padding=4), T.RandomHorizontalFlip(),
                      T.ToTensor(), T.Normalize(MEAN, STD)])
test_tf = T.Compose([T.ToTensor(), T.Normalize(MEAN, STD)])

full_aug = torchvision.datasets.CIFAR10("data", train=True, download=True,
    transform=train_tf)
full_plain = torchvision.datasets.CIFAR10("data", train=True, download=True,
    transform=test_tf)
test_set = torchvision.datasets.CIFAR10("data", train=False, download=True,
    transform=test_tf)

idx = torch.randperm(len(full_aug), generator=torch.Generator().manual_seed(0))
train_set = Subset(full_aug, idx[5000:].tolist())         # 45,000 augmented
val_set = Subset(full_plain, idx[:5000].tolist())         # 5,000 clean
CLASSES = full_aug.classes

kw = dict(num_workers=2, pin_memory=True)
train_loader = DataLoader(train_set, batch_size=128, shuffle=True, drop_last=True,
    **kw)
val_loader = DataLoader(val_set, batch_size=512, **kw)
test_loader = DataLoader(test_set, batch_size=512, **kw)
print(len(train_set), len(val_set), len(test_set), CLASSES)
```

### 2.60.3 Step 3: Defining the Model

Listing 2.56 defines a compact VGG-style CNN with residual connections, applying the design principles from the section on CNN architectures: stacks of $3 \times 3$ convolutions with batch normalisation and ReLU, down-sampling by max-pooling while doubling the channel count, residual blocks to ease optimisation, and global average pooling before a linear classifier. With about 6.5 million parameters it trains in a few minutes per epoch on a Colab T4 GPU and can exceed 90% test accuracy.

**Listing 2.56** A compact residual CNN for CIFAR-10.

```
import torch.nn as nn

def conv_bn(c_in, c_out, pool=False):
    layers = [nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
              nn.BatchNorm2d(c_out), nn.ReLU(inplace=True)]
    if pool:
        layers.append(nn.MaxPool2d(2))
    return nn.Sequential(*layers)

class Residual(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.body = nn.Sequential(conv_bn(c, c), conv_bn(c, c))
    def forward(self, x):
        return x + self.body(x)

class ResNet9(nn.Module):
    def __init__(self, n_classes=10):
        super().__init__()
        self.net = nn.Sequential(
            conv_bn(3, 64), conv_bn(64, 128, pool=True), Residual(128),     # 16x16
            conv_bn(128, 256, pool=True), conv_bn(256, 512, pool=True),    # 4x4
            Residual(512), nn.AdaptiveAvgPool2d(1), nn.Flatten(),
            nn.Dropout(0.2), nn.Linear(512, n_classes))
    def forward(self, x):
        return self.net(x)

model = ResNet9().to(device).to(memory_format=torch.channels_last)
print(f"parameters: {sum(p.numel() for p in model.parameters()) / 1e6:.2f} M")
```

## 2.60.4 Step 4: Training with Good Practices

The training loop in Listing 2.57 combines several techniques discussed earlier in the chapter: the SGD optimiser with Nesterov momentum and weight decay; a **one-cycle learning-rate schedule** (Smith, 2018), which warms up then anneals the learning rate within a single run; **label smoothing** in the cross-entropy loss (Equation (2.143)); automatic **mixed precision** to use tensor cores; **gradient clipping** for stability; and checkpointing of the best model by validation accuracy, which implements early stopping in its simplest form. Training history is recorded for plotting learning curves, of the kind shown in Figure 2.73.

$$\tilde{y}_k = (1-\varepsilon)\, y_k + \frac{\varepsilon}{C}, \qquad \mathcal{L} = -\sum_{k=1}^{C} \tilde{y}_k \log p_k \tag{2.143}$$

where $y_k$ is the one-hot target, $C = 10$ is the number of classes, $\varepsilon = 0.1$ is the smoothing factor and $p_k$ is the predicted probability of class $k$. Smoothing discourages over-confident logits and usually improves calibration slightly.

**Listing 2.57** Training loop with one-cycle schedule, mixed precision, clipping and best-model checkpointing.

```
import time, torch.nn.functional as F

EPOCHS = 30
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True,
↪  weight_decay=5e-4)
sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=0.1, epochs=EPOCHS,
↪  steps_per_epoch=len(train_loader))
scaler = torch.cuda.amp.GradScaler()
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

def evaluate(loader):
    model.eval(); correct, total, loss_sum = 0, 0, 0.0
    with torch.no_grad(), torch.autocast("cuda", dtype=torch.float16):
        for x, y in loader:
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            out = model(x)
            loss_sum += F.cross_entropy(out, y, reduction="sum").item()
            correct += (out.argmax(1) == y).sum().item(); total += y.numel()
    return loss_sum / total, correct / total

history, best_acc = [], 0.0
for epoch in range(EPOCHS):
    model.train(); t0, run_loss = time.time(), 0.0
    for x, y in train_loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        with torch.autocast("cuda", dtype=torch.float16):
            loss = criterion(model(x), y)
        opt.zero_grad(set_to_none=True)
        scaler.scale(loss).backward()
        scaler.unscale_(opt); nn.utils.clip_grad_norm_(model.parameters(), 2.0)
        scaler.step(opt); scaler.update(); sched.step()
        run_loss += loss.item()
    val_loss, val_acc = evaluate(val_loader)
    history.append((run_loss / len(train_loader), val_loss, val_acc))
    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), f"{SAVE_DIR}/best.pt")
    print(f"ep {epoch+1:2d} | train {history[-1][0]:.3f} | val {val_loss:.3f} "
          f"acc {val_acc:.4f} | {time.time() - t0:.0f}s")
```

**Note | Why Is the Training Loss Higher Than the Validation Loss?**

In Figure 2.73 the training loss stays above the validation loss. This is expected here, not a bug: the training loss includes label smoothing (which prevents it from approaching zero), is measured on augmented images, and is averaged over an epoch while the weights were still changing, whereas the validation loss uses clean images, hard labels and the end-of-epoch weights.

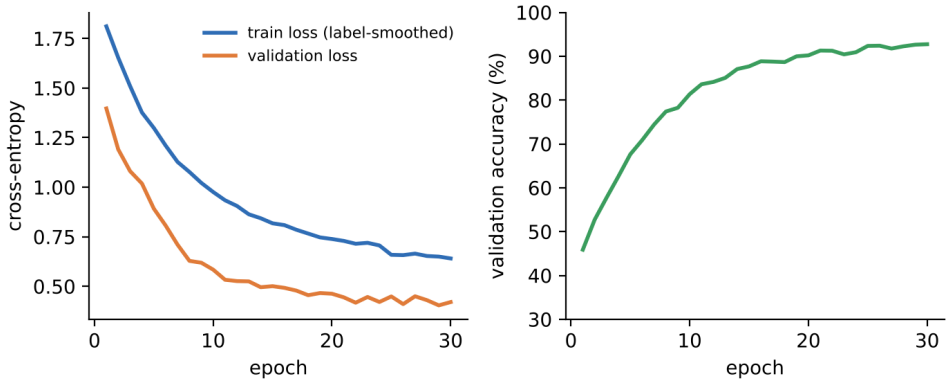



**Figure 2.73** Typical learning curves for the CIFAR-10 project under a one-cycle schedule (illustrative): validation accuracy rises steadily as the learning rate anneals, and the small train–validation gap indicates well-controlled over-fitting.

## 2.60.5 Step 5: Evaluation and Error Analysis

A single accuracy figure hides much useful information. Listing 2.58 reloads the best checkpoint, computes test accuracy only once (the test set must never guide model selection), and produces a per-class report and a **confusion matrix** with scikit-learn. On CIFAR-10 the most frequent confusions are typically between cats and dogs and between cars and trucks, which suggests where more data or capacity would help. Inspecting the most confidently wrong images, and applying Grad-CAM from the section on explainable AI, often reveals label errors or shortcuts.

**Listing 2.58** Final test evaluation, classification report and confusion matrix.

```
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

model.load_state_dict(torch.load(f"{SAVE_DIR}/best.pt", map_location=device))
model.eval(); preds, labels = [], []
with torch.no_grad():
    for x, y in test_loader:
        preds.append(model(x.to(device)).argmax(1).cpu()); labels.append(y)
preds, labels = torch.cat(preds).numpy(), torch.cat(labels).numpy()

print(f"test accuracy: {(preds == labels).mean():.4f}")
print(classification_report(labels, preds, target_names=CLASSES, digits=3))
ConfusionMatrixDisplay.from_predictions(labels, preds, display_labels=CLASSES,
                                        normalize="true", values_format=".2f",
                                        xticks_rotation=45, cmap="Blues")
plt.tight_layout(); plt.show()
```

### 2.60.6 Step 6: Saving, Exporting and Reusing the Model

Finally, Listing 2.59 saves the model in three forms: a `state_dict` checkpoint (the recommended PyTorch format, which requires the class definition to reload), a TorchScript file that runs without the Python class, and an ONNX file for other runtimes, as discussed in the section on deployment. It then reloads the TorchScript model and predicts a single image, verifying the round trip. Saving the preprocessing constants and class names alongside the weights avoids training–serving skew. Table 2.59 lists common Colab pitfalls and how to avoid them.

**Listing 2.59** Saving, exporting and reloading the trained model.

```
import json

torch.save({"model": model.state_dict(), "classes": CLASSES,
            "mean": MEAN, "std": STD}, f"{SAVE_DIR}/cifar10_resnet9.pth")

model_cpu = ResNet9().eval()
model_cpu.load_state_dict(torch.load(f"{SAVE_DIR}/best.pt", map_location="cpu"))
example = torch.randn(1, 3, 32, 32)
torch.jit.trace(model_cpu, example).save(f"{SAVE_DIR}/cifar10_resnet9_ts.pt")
torch.onnx.export(model_cpu, example, f"{SAVE_DIR}/cifar10_resnet9.onnx",
                  input_names=["image"], output_names=["logits"],
                  dynamic_axes={"image": {0: "batch"}}, opset_version=17)
json.dump({"classes": CLASSES, "mean": MEAN, "std": STD}, open(f"{SAVE_DIR}/meta.json",
↪  "w"))

reloaded = torch.jit.load(f"{SAVE_DIR}/cifar10_resnet9_ts.pt")
img, label = test_set[0]
pred = reloaded(img.unsqueeze(0)).argmax(1).item()
print("true:", CLASSES[label], "| predicted:", CLASSES[pred])
```

**Table 2.59** Common Colab pitfalls and remedies.

| Pitfall | Symptom | Remedy |
|---|---|---|
| Runtime disconnects | Lost model and variables | Checkpoint to Google Drive each epoch; resume from checkpoint |
| CPU runtime selected | Very slow epochs | Runtime → Change runtime type → GPU; check torch.cuda.is_available() |
| Data loader bottleneck | Low GPU utilisation in nvidia-smi | num_workers=2, pin_memory=True; cache data locally, not on Drive |
| Out-of-memory (CUDA OOM) | RuntimeError during forward pass | Reduce batch size; use mixed precision; free unused tensors |
| Non-reproducible results | Different accuracy each run | Fix seeds; record library versions; report mean ± std over runs |
| Test-set leakage | Optimistic final accuracy | Use a separate validation split for all tuning decisions |

**T Practice Tip | Extending the Project**

- Replace the CNN by a pretrained ResNet or ViT and fine-tune it (see the section on transfer learning); compare accuracy and training time.
- Add stronger augmentation (RandAugment, CutMix, Mixup) and measure the effect.
- Quantise the exported model to INT8 and compare size, latency and accuracy.
- Wrap the TorchScript model in the FastAPI service from the deployment section and test it with real photographs.

# 2.61 Challenges, Limitations, and Future Directions of Deep Learning

This chapter has traced deep learning from the perceptron to foundation models serving billions of users. Its achievements are real: superhuman performance on many perception benchmarks, protein-structure prediction, practical machine translation and speech recognition, and conversational systems capable of writing code. Yet deep learning also has well-documented limitations, and many of the most active research directions aim to address them. This closing section surveys these challenges and the directions that are likely to shape the field over the coming decade; Table 2.60 summarises them.

## 2.61.1 Data, Compute and Efficiency

Deep networks are **data-hungry**: supervised models typically need thousands of labelled examples per class, and foundation models are trained on trillions of tokens. High-quality public text may be approaching exhaustion for the largest models, raising interest in synthetic data, data curation and more sample-efficient learning. **Computational cost** has grown by orders of magnitude: the compute used in the largest training runs has historically doubled every few months, with corresponding financial and environmental costs and a concentration of frontier research in a few well-funded laboratories. Efficiency research (sparse mixture-of-experts models, quantisation, distillation, efficient attention, better hardware) is therefore central rather than peripheral.

## 2.61.2 Robustness, Generalisation and Reasoning

Deep networks generalise remarkably well within their training distribution but can fail unpredictably outside it. Adversarial examples, sensitivity to distribution shift,

**shortcut learning** (Geirhos et al., 2020) and over-confident predictions on unfamiliar inputs all point to the same gap: models learn statistical associations that do not always coincide with the causal structure humans rely on. Research directions include **causal representation learning**, **uncertainty quantification** (Bayesian deep learning, deep ensembles, conformal prediction) and **out-of-distribution detection**. Large language models show impressive but brittle reasoning, and can hallucinate; approaches such as chain-of-thought training, tool use, retrieval and verification aim to make reasoning more reliable, while **neuro-symbolic** methods combine learned perception with explicit symbolic inference.

Over-confidence can be quantified with the **expected calibration error** (**ECE**) in Equation (2.144), which compares a model's confidence with its accuracy across confidence bins; Guo et al. (2017) showed that modern deep networks are often poorly calibrated, and that simple temperature scaling of the logits substantially reduces ECE.

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right| \tag{2.144}$$

where predictions are grouped into M bins $B_m$ by their confidence (maximum softmax probability), n is the total number of predictions, and $\mathrm{acc}(B_m)$ and $\mathrm{conf}(B_m)$ are the accuracy and mean confidence in bin m. A perfectly calibrated model has ECE = 0.

## 2.61.3 Interpretability, Trust and Alignment

As deep models take on consequential roles, understanding and controlling them becomes critical. Post-hoc attribution methods, discussed in the section on explainable AI, give only partial insight. **Mechanistic interpretability** attempts to reverse-engineer the algorithms implemented by network weights, identifying circuits and features within transformers. **AI alignment** research studies how to ensure that increasingly capable systems pursue the goals their designers and users intend, using techniques such as reinforcement learning from human feedback, constitutional approaches, scalable oversight and red-teaming. These sit alongside the fairness, privacy and security concerns covered earlier and the regulatory frameworks emerging worldwide.

## 2.61.4 Emerging Directions

Several research directions are especially active. **Multimodal and embodied AI** extends foundation models to vision, audio, video and robot actions, with *world models* that learn to predict the consequences of actions. **Agentic systems** combine language models with planning, memory and tools to carry out multi-step tasks. **Continual (lifelong) learning** seeks models that keep learning without catastrophic forgetting.

**Scientific machine learning** applies deep learning to physics, chemistry, biology, climate and materials, often embedding known physical laws (physics-informed neural networks). **New architectures** such as state-space models (e.g. Mamba) challenge the transformer's dominance for long sequences, and **neuromorphic** and analogue hardware explore radically more energy-efficient computation. Figure 2.74 organises these directions around the limitations they address.

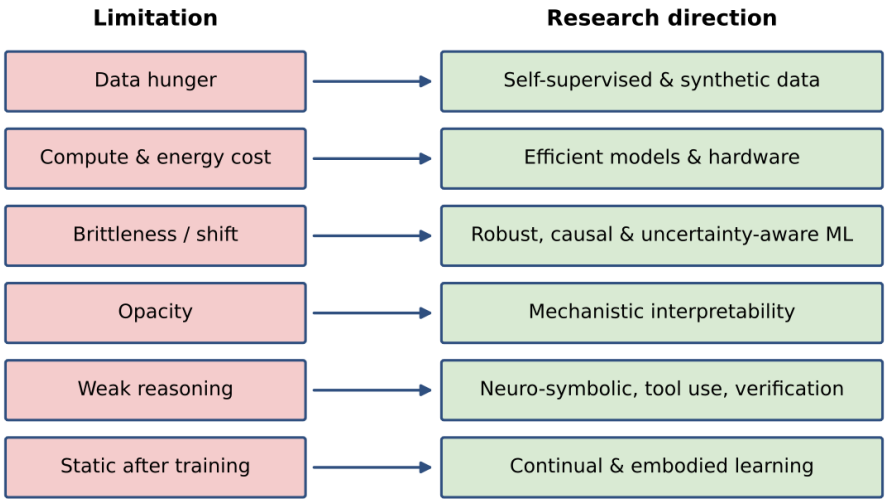



**Figure 2.74** Map of current limitations of deep learning (left) and the research directions that address them (right).

**Key Point | Looking Ahead**

- Deep learning's main strengths are representation learning at scale and transfer across tasks.
- Its main weaknesses are data and compute demands, brittleness, opacity and unreliable reasoning.
- Progress increasingly depends on efficiency, evaluation, safety and responsible deployment as much as on accuracy.

# Chapter Summary

This chapter has presented deep learning from its foundations to its frontiers. We began by situating deep learning within artificial intelligence and machine learning: it is the branch of machine learning that learns hierarchical representations with multilayer neural networks. Its history runs from the McCulloch–Pitts neuron (1943) and

**Table 2.60** Key challenges of deep learning and promising responses.

| Challenge | Why it matters | Promising responses |
|---|---|---|
| Data requirements | Labels are costly; quality data is finite | Self-supervision, transfer learning, synthetic data, active learning |
| Compute and energy | Cost, carbon footprint, access inequality | Efficient architectures, compression, MoE, specialised hardware |
| Robustness | Failures under shift or attack | Adversarial training, certification, augmentation, OOD detection |
| Uncertainty | Over-confident wrong predictions | Deep ensembles, Bayesian methods, conformal prediction |
| Interpretability | Trust, debugging, regulation | XAI methods, mechanistic interpretability, concept models |
| Fairness and privacy | Harm to individuals and groups | Fairness constraints, DP-SGD, federated learning, audits |
| Reasoning and factuality | Hallucination, brittle logic | Retrieval, tool use, verification, neuro-symbolic methods |
| Continual adaptation | World changes after training | Continual learning, MLOps retraining, test-time adaptation |

Rosenblatt's perceptron (1958), through the first AI winter, the popularisation of back-propagation (Rumelhart, Hinton and Williams, 1986) and the 2012 ImageNet breakthrough of AlexNet, to today's era of transformers and foundation models. Its recent success rests on three pillars: large data sets, GPU and TPU computation, and algorithmic advances in architectures and optimisation.

The core machinery was developed in detail. Artificial neurons compute weighted sums followed by non-linear **activation functions** (sigmoid, tanh, ReLU and its variants, GELU, softmax); stacking them gives **multilayer perceptrons**, whose depth and width define the architecture. **Forward propagation** computes predictions, **loss functions** (mean squared error, cross-entropy) measure their quality, and **back-propagation** applies the chain rule to obtain gradients efficiently. **Gradient descent** in its batch, stochastic and mini-batch forms, enhanced by **optimisers** such as momentum, RMSProp and Adam, by careful **weight initialisation** (Xavier/Glorot, He) and by **learning-rate schedules**, turns these gradients into learning. We examined the **vanishing and exploding gradient** problems and their remedies, and the tools that control over-fitting: **regularisation**, **dropout**, **batch normalisation**, early stopping and data augmentation, together with principled **training/validation** practice and **hyperparameter tuning**.

We then studied the major architecture families. **Convolutional neural networks** exploit local connectivity and weight sharing for images, evolving from LeNet and AlexNet through VGG, Inception and ResNet to EfficientNet, and extend to **object detection** (R-CNN family, YOLO) and **segmentation** (FCN, U-Net, Mask R-CNN).

**Recurrent networks**, **LSTMs** and **GRUs** model sequences, and sequence-to-sequence models with **attention** paved the way for the **transformer**, whose self-attention underlies **BERT**, **GPT** and today's **large language models**. For NLP we covered word **embeddings** from word2vec and GloVe to contextual representations. Generative modelling was treated through **autoencoders**, **variational autoencoders**, **generative adversarial networks** and **diffusion models**, and **deep reinforcement learning** showed how neural networks can learn to act from reward, from DQN to AlphaGo and policy-gradient methods.

The final part of the chapter broadened the view. **Graph neural networks** extend deep learning to relational data through message passing. **Self-supervised and contrastive learning** and **transfer learning**, including parameter-efficient fine-tuning such as LoRA, explain how large pretrained **foundation models** are built and adapted. Applications in **multimodal learning**, **speech and audio**, **medical imaging**, **computer vision** and **recommendation systems** illustrated how general techniques are specialised to domains with their own data, metrics and constraints.

Finally, we addressed what it takes to use deep learning responsibly and at scale. **Explainable AI** methods such as Grad-CAM, Integrated Gradients and SHAP make predictions more interpretable; **adversarial robustness** research exposes and mitigates vulnerabilities; and the study of **bias, fairness, privacy and security** provides formal criteria and defences such as differential privacy. **Model compression**, **distributed training** and an understanding of **GPU and TPU hardware** make large models trainable and deployable; **frameworks** such as TensorFlow, Keras and PyTorch make them programmable; and **deployment** and **MLOps** practices keep them reliable in production. An end-to-end Colab project tied these threads together, and the chapter closed by reviewing the limitations of deep learning and the research directions that seek to overcome them.

# Key Terms

Table 2.61 collects the principal terms introduced across all sections of this chapter, with concise definitions for revision.

**Table 2.61** Key terms introduced in Chapter 2.

| Term | Definition |
|---|---|
| Deep learning | Machine learning with multi-layer neural networks that learn hierarchical representations from data. |
| Artificial neuron | Unit computing a weighted sum of inputs plus bias, followed by a non-linear activation. |
| Perceptron | Single-layer linear threshold classifier trained by the perceptron learning rule (Rosenblatt, 1958). |
| Multilayer perceptron (MLP) | Feedforward network of fully connected layers with non-linear activations. |
| Activation function | Non-linearity applied to a neuron's pre-activation, e.g. ReLU, sigmoid, tanh, GELU. |
| Forward propagation | Computation of a network's outputs layer by layer from its inputs. |
| Loss function | Scalar measure of prediction error minimised during training, e.g. cross-entropy or MSE. |
| Back-propagation | Efficient application of the chain rule to compute gradients of the loss with respect to all weights. |
| Gradient descent | Iterative optimisation that updates parameters in the direction of the negative gradient. |
| Mini-batch SGD | Gradient descent using gradients estimated from small random subsets of the training data. |
| Adam | Adaptive optimiser combining momentum with per-parameter learning rates from squared-gradient averages. |
| Learning-rate schedule | Rule that changes the learning rate during training, e.g. step decay, cosine, warm-up, one-cycle. |
| Vanishing gradient | Exponential shrinking of gradients through many layers or time steps, stalling learning. |
| Dropout | Regulariser that randomly zeroes activations during training to prevent co-adaptation. |
| Batch normalisation | Normalises layer inputs over a mini-batch, then rescales them with learned parameters. |
| Hyperparameter | Setting chosen before training (e.g. learning rate, depth) and tuned on validation data. |
| Convolutional neural network (CNN) | Network using convolutions with shared local filters, suited to grid data such as images. |
| Residual connection | Skip connection adding a block's input to its output, easing optimisation of very deep networks. |
| Recurrent neural network (RNN) | Network with a hidden state carried across time steps for sequential data. |
| LSTM / GRU | Gated recurrent units that control information flow to capture long-range dependencies. |
| Word embedding | Dense vector representation of a word learned so that similar words have similar vectors. |
| Attention | Mechanism computing a weighted combination of values, with weights from query–key similarity. |

| Term | Definition |
|---|---|
| Transformer | Architecture built from multi-head self-attention and feedforward layers, without recurrence. |
| Large language model (LLM) | Very large transformer language model pre-trained on massive text corpora. |
| Generative adversarial network (GAN) | Generator and discriminator trained in a minimax game to produce realistic samples. |
| Variational autoencoder (VAE) | Latent-variable generative model trained by maximising the evidence lower bound. |
| Diffusion model | Generative model that learns to reverse a gradual noising process. |
| Deep reinforcement learning | Reinforcement learning with deep networks approximating policies or value functions. |
| Graph neural network (GNN) | Network that learns on graphs by passing and aggregating messages between neighbouring nodes. |
| Self-supervised learning | Learning from unlabelled data using targets derived from the data itself. |
| Contrastive loss (InfoNCE) | Loss pulling positive pairs together and pushing negatives apart in embedding space. |
| Transfer learning / fine-tuning | Reusing a pretrained model and adapting its weights to a new task. |
| LoRA | Parameter-efficient fine-tuning that learns low-rank updates to frozen weight matrices. |
| Foundation model | Large model pre-trained on broad data that can be adapted to many downstream tasks. |
| CTC loss | Alignment-free sequence loss that sums over all alignments using a blank symbol. |
| Grad-CAM | Class activation heatmap from gradient-weighted convolutional feature maps. |
| Adversarial example | Input with a small crafted perturbation that causes a model to err. |
| Differential privacy | Formal guarantee that an algorithm's output changes little when one record changes. |
| Knowledge distillation | Training a small student model to mimic a larger teacher's softened outputs. |
| Quantisation | Representing weights and activations with low-precision numbers such as INT8. |
| Data parallelism | Distributed training in which replicas process different data shards and average gradients. |
| Data / concept drift | Change in the input distribution $P(x)$ / in the relationship $P(y \mid x)$ after deployment. |
| MLOps | Practices and tools for reliably deploying, monitoring and maintaining ML models in production. |

# Review Questions

1. Explain the relationship between artificial intelligence, machine learning and deep learning. What distinguishes deep learning from classical machine learning in terms of feature engineering?
2. Outline three milestones in the history of neural networks and explain why deep learning became successful after 2012.
3. Why can a single perceptron not solve the XOR problem, and how does adding a hidden layer overcome this?
4. Compare the sigmoid, tanh and ReLU activation functions with respect to output range, gradient behaviour and computational cost. Why is a non-linear activation essential?
5. Describe forward propagation and back-propagation in an MLP. Which quantities must be stored during the forward pass, and why?
6. When would you use cross-entropy rather than mean squared error as a loss function? Justify your answer.
7. Compare batch, stochastic and mini-batch gradient descent, and explain how momentum and Adam improve on plain SGD.
8. Why does weight initialisation matter? State the idea behind Xavier and He initialisation.
9. Explain the vanishing-gradient problem and list three architectural or training techniques that mitigate it.
10. How do dropout and batch normalisation each act during training and at inference time?
11. Describe a principled procedure for hyperparameter tuning that avoids information leakage from the test set.
12. Explain how local connectivity, weight sharing and pooling make CNNs suitable for images. What problem did ResNet's residual connections solve?
13. Distinguish image classification, object detection, semantic segmentation and instance segmentation, naming one representative architecture for each.
14. How do LSTM gates address the limitations of simple RNNs? Compare LSTMs and GRUs.
15. Explain scaled dot-product self-attention and why transformers replaced RNNs for most sequence tasks.
16. Contrast the pre-training objectives of BERT and GPT and the kinds of tasks each is best suited to.

17. Compare GANs, VAEs and diffusion models in terms of training objective, sample quality, diversity and stability.
18. Explain the roles of the policy, value function and reward in deep reinforcement learning, using DQN as an example.
19. What is message passing in graph neural networks, and why must the aggregation function be permutation invariant?
20. Explain how contrastive self-supervised learning and LoRA fine-tuning together enable effective use of foundation models with limited labelled data.
21. Define demographic parity and equalised odds. Why can these criteria conflict with calibration?
22. Describe the differences between data, pipeline and tensor parallelism, and explain what ZeRO adds to data parallelism.
23. What is data drift, how can PSI be used to detect it, and how should an MLOps pipeline respond?
24. Identify three major limitations of current deep learning and a research direction addressing each.

# Exercises

1. **Forward and backward pass.** A neuron has inputs $x = (1, 2)$, weights $w = (0.5, -0.3)$, bias $b = 0.1$ and sigmoid activation. Compute its output; then, for target $y = 1$ and binary cross-entropy loss, compute $\partial L/\partial w_1$, $\partial L/\partial w_2$ and $\partial L/\partial b$.
2. **Parameter counting.** Compute the number of trainable parameters in (a) an MLP 784–256–128–10 with biases; (b) a convolutional layer with 64 input channels, 128 output channels, $3 \times 3$ kernels and bias; (c) a LoRA adapter of rank 16 on a $1024 \times 1024$ weight matrix.
3. **Convolution arithmetic.** An input of size $64 \times 64 \times 3$ passes through a $5 \times 5$ convolution with 32 filters, stride 1, padding 2, followed by $2 \times 2$ max-pooling with stride 2. Give the output shape after each step.
4. **Optimisers.** Starting from $w = 2$ on $f(w) = w^2$, perform three updates of (a) plain gradient descent with $\eta = 0.1$ and (b) momentum with $\eta = 0.1$, $\beta = 0.9$. Compare the trajectories.
5. **Attention by hand.** For queries, keys and values $Q = K = V = [[1, 0], [0, 1], [1, 1]]$, compute the scaled dot-product attention output for the first token.

6. **GCN layer.** For the star graph with centre node 1 connected to nodes 2, 3 and 4, scalar features $x = (4, 1, 1, 1)$ and $W = 1$, compute one GCN layer (with self-loops and symmetric normalisation, identity activation).

7. **Evaluation metrics.** A recommender returns a top-5 list with relevances (0, 1, 0, 1, 1) for a user who has three relevant items in total. Compute precision@5, recall@5, HR@5 and NDCG@5. A segmentation model predicts 500 foreground pixels, 420 of which overlap a 480-pixel ground-truth mask; compute Dice and IoU.

8. **Quantisation and PSI.** Quantise the values $\{-0.8, 0.0, 0.35, 1.2\}$ to unsigned 8-bit using the range $[-0.8, 1.2]$, then de-quantise them and report the errors. Separately, compute the PSI between reference bins (0.2, 0.3, 0.3, 0.2) and current bins (0.1, 0.3, 0.3, 0.3).

9. **Programming: MLP from scratch.** Implement a two-layer MLP with NumPy only (forward pass, back-propagation and mini-batch SGD) and train it on scikit-learn's `make_moons` data. Plot the decision boundary and the loss curve.

10. **Programming: transfer learning.** In Colab, fine-tune an ImageNet-pretrained ResNet-18 on a small data set (e.g. a 5-class flowers data set). Compare (a) feature extraction, (b) full fine-tuning and (c) training from scratch in accuracy and training time, and visualise three Grad-CAM heatmaps.

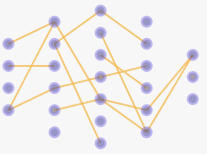
11. **Programming: robustness.** Train a small CNN on MNIST, then implement FGSM and PGD ($\varepsilon = 0.1, 0.2, 0.3$). Plot accuracy against $\varepsilon$, then adversarially train the model with PGD and repeat the plot.

12. **Programming: compression and deployment.** Take the CIFAR-10 model from the Colab project, apply post-training INT8 quantisation and 50% structured pruning, and tabulate size, CPU latency and accuracy for each variant. Export the best variant to ONNX and serve it through a FastAPI endpoint.

# Further Reading


- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press.
- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Prince, S. J. D. (2023). *Understanding Deep Learning*. MIT Press.
- Zhang, A., Lipton, Z. C., Li, M. and Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press.
- Chollet, F. (2021). *Deep Learning with Python*, 2nd edn. Manning.

- LeCun, Y., Bengio, Y. and Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.
- Rumelhart, D. E., Hinton, G. E. and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- Krizhevsky, A., Sutskever, I. and Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
- He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Vaswani, A. et al. (2017). Attention is all you need. *NeurIPS*.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL*.
- Goodfellow, I. et al. (2014). Generative adversarial nets. *NeurIPS*.
- Ho, J., Jain, A. and Abbeel, P. (2020). Denoising diffusion probabilistic models. *NeurIPS*.
- Kipf, T. N. and Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *ICLR*.
- Chen, T., Kornblith, S., Norouzi, M. and Hinton, G. (2020). A simple framework for contrastive learning of visual representations. *ICML*.
- Radford, A. et al. (2021). Learning transferable visual models from natural language supervision. *ICML*.
- Hu, E. J. et al. (2022). LoRA: Low-rank adaptation of large language models. *ICLR*.
- Bommasani, R. et al. (2021). On the opportunities and risks of foundation models. arXiv:2108.07258.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D. and Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. *ICLR*.
- Selvaraju, R. R. et al. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *ICCV*.
- Sculley, D. et al. (2015). Hidden technical debt in machine learning systems. *NeurIPS*.



CHAPTER 2

# Deep Learning

Deep learning is the branch of machine learning that builds models from many stacked layers of simple, differentiable processing units and trains them end to end from data. In little more than a decade it has moved from a niche research topic to the engine behind image recognition, machine translation, speech assistants, protein-structure prediction and large language models. This chapter develops the subject from first principles: what a neural network computes, how it is trained, why training sometimes fails, and how the major architectural families (convolutional, recurrent, attention-based and generative networks) extend the basic ideas to images, sequences, language and data synthesis.

## AT A GLANCE

| | | |
|---|---|---|
| **61** Sections | **74** Figures | **61** Tables |
| **29** Algorithms | **144** Equations | **59** Code Listings |

## IN THIS CHAPTER YOU WILL LEARN TO

- ✓ explain what distinguishes deep learning from classical machine learning and place it within the wider field of artificial intelligence.
- ✓ describe the key historical milestones of neural network research and the factors behind the deep learning revival after 2006–2012.
- ✓ compute the output of an artificial neuron, a perceptron and a multilayer perceptron, and explain why hidden layers with non-linear activations are needed.
- ✓ select appropriate activation and loss functions for regression, binary, multi-class and multi-label problems.
- ✓ derive and implement forward propagation and backpropagation for a fully connected network in matrix form.

ABOUT THE AUTHOR

**Luxshi Karunakaran**
B.Sc. (Hons) in Computer Science and Technology
M.Sc. in Data Science and Artificial Intelligence (Reading)
Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka